Bioinformatics

Databases and Algorithms

N Gautham



Alpha
Science

Bioinformatics

Databases and Algorithms

Bioinformatics

Databases and Algorithms

N Gautham

NATIONAL INSTITUTES OF HEALTH
NIH LIBRARY
AUG 26 2006
BLDG 10, 10 CENTLR DR.
BETHESDA, MD 20892-1150



Alpha Science International Ltd.
Oxford, U.K.

N Gautham
Department of Crystallography and Biophysics
University of Madras
Chennai, India

Copyright © 2006

Alpha Science International Ltd.
7200 The Quorum, Oxford Business Park North
Garsington Road, Oxford OX4 2JZ, U.K.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without prior written permission of the publisher.

Printed from the camera-ready copy provided by the Author.

ISBN 1-84265-300-8

Printed in India

Dedicated to my wife, Sethumathy
With love

Certainly no subject or field is making more progress on so many fronts at the present moment, than biology, and if we were to name the most powerful assumption of all, which leads one on and on in an attempt to understand life, it is that all things are made of atoms*, and that everything living things do can be understood in terms of the jigglings and wigglings of atoms.*

Richard Feynman, 1963

In this class, I hope you will learn not merely results or formulae applicable to cases that may possibly occur in our practice afterwards, but the principles on which those formulae depend, and without which the formulae are mere mental rubbish.

James Clerk Maxwell, 1860

Preface

Long years ago I made my own tryst with destiny, promising my well-wishers, and myself, that I would write a book on bioinformatics. The volume you now hold is the result of my attempts to redeem this pledge. It began in the early 1990s when I got involved in a project of the Universities Grants Commission to conduct a certificate course on 'Computer Applications in Biology' at the University of Madras. At that time the subject was just beginning to burgeon in the country, and there were no textbooks I could follow right away. I had to first educate myself by reading the primary literature, then formulate a suitable course, prepare the lecture notes and finally deliver the lectures. It was during this period that I first thought of arranging the notes, with a good deal supplementary writing, into a textbook that would be useful to students of biology seeking acquaintance with bioinformatics. In the context of the scenario of that time, this meant that the emphasis would be on, first of all, explaining the use of computers in general, since biology students were not (as yet) expected to be deeply aware of computers and their uses. (Though this was some time after Bill Gates, John Sculley, Steven Jobs and others 'stopped selling coloured sugar water, and changed the world'; the revolution was slow in coming to India.) The rest of the book, as then planned, would consist of detailed explanations of data, databases and how to access the latter to retrieve the former. There would be cursory reference to sequence analyses, but no details, and almost no mention of structural analyses. This was also the plan of the course that my colleagues and I delivered in the University.

But times changed rather rapidly, and the plan for the book became obsolete faster than the latest computer technology. The course expanded to include the other hitherto un-addressed aspects of bioinformatics, not only because these aspects were important for the students to know, but also because we found them more interesting than simple descriptions of databases. Several books became available, talking to their readers at different levels. Bioinformatics courses, leading to certificates, diplomas or degrees, were introduced in many Institutes and Universities. It was then that I decided on the present plan. The emphasis now is on algorithms, though databases have not been entirely neglected, and they have a whole chapter to themselves. Also, the attention given to structural bioinformatics here was not part of the first plan. Thus, the present book has two features that I hope will help to distinguish it from others in this genre. First, it seeks to explain some of the basic algorithms in bioinformatics to readers who have little previous exposure to computer science and mathematics. Second, it deals with structural bioinformatics in greater depth than most other books. In the hierarchy of textbooks, it falls between a very elementary introduction to the subject, and an advanced monograph on the topic. I could name (and, indeed, do so at the end of this book) several

texts that fall into one or the other of these two categories. I hope the present book would serve as a bridge between them.

There are of course several topics in bioinformatics that I have not addressed here, chiefly because I lack the expertise. For example I would dearly have liked to include chapters on micro-array analyses, drug discovery, and systems biology, all of which fall within the broad ambit of bioinformatics. But the book, I think, is too long already, and has taken too much of my time. Maybe I will write on those topics some other time.

The book has benefited from the courses and lectures I have delivered over the past decade to post-graduate students, both in our department at the University of Madras, as well as in several seminars in colleges in Chennai and elsewhere. To all my patient listeners and critics, I express my gratitude. The writing of this book has received a great deal of encouragement and help from my colleagues and friends. Mrs Vasantha Jothi, who is the departmental librarian, has not only helped with the reference material I frequently sought, but has also encouraged me at every stage. I thank her for all her help. Professor Vasantha Pattabhi and Professor S. S. Rajan have been generous with their time, academic and moral support, and with the numerous cups of tea (black and without sugar, for preference) I consumed throughout the writing. I thank them for all they have done, and seek their continued friendship.

Dr. S Krishnaswamy, of Madurai-Kamaraj University, merits a separate paragraph of thanks. He is a friend, 'there for me' at the beginning, in the middle and, now, at the end of this project. He helped design the book, made several suggestions throughout its writing, and has even written bits of it, though modestly he refuses any credit. If there is one person about whom I can say, 'But for him this book would not have been written', it is Dr. Krishnaswamy. I am grateful for this chance to acknowledge all he has done. Thanks, Kicha. My thanks also to his wife Dr. R. Usha, and son K.U. Amudhan, for affectionate hospitality over the years.

My student, Arun Prasad, helped me to prepare the index and the table of contents. My thanks go to him. My publishers, the Mehras, father and son, have been immensely supportive of this endeavour, and have been very patient throughout. I express my gratitude to them. I can only hope they will decide that the final result has been worth the wait.

My family has had to undergo a fair amount of nervous strain, especially over the last few months, due to my bad temper and late nights during the course of this writing, my wife most of all. I dedicate this book to her, but I must also mention, with love and gratitude, the constant support of my daughter Chitra, and my mother Sarasvathy.

Finally a disclaimer on behalf of everyone mentioned above. All mistakes that remain in the book are of course mine. I hope these are few and far between.

Chennai N Gautham

Contents

Preface vii

1. Introduction 1

1.1 An Elementary Introduction to Modern Molecular Biology *2*
- 1.1.1 The reductionist programme in biology *2*
- 1.1.2 A cell is the smallest unit of Life *3*
- 1.1.3 Biological information pathways *4*
- 1.1.4 Storage and representation of genetic information *6*
- 1.1.5 Transfer of genetic information: Replication *8*
- 1.1.6 Transfer of genetic information: Transcription *9*
- 1.1.7 Transfer of genetic information: Translation and protein folding *10*
- 1.1.8 Genome organization *12*
- 1.1.9 Genome sequencing projects and computers *13*

1.2 A Brief Overview of Information Technology and Science *14*
- 1.2.1 History *14*
- 1.2.2 Hardware and Software *15*
- 1.2.3 Computerised Databases and Database Management Systems *17*
- 1.2.4 The Internet *19*

1.3 An Introduction to Bioinformatics and Sequence Analyses *22*
- 1.3.1 What is bioinformatics? *22*
- 1.3.2 Elemental tasks of bioinformatics analyses *23*
- 1.3.3 An overview of the rest of the book *24*

1.4 Summary *25*

2. Data Types and Databases in Molecular Biology 26

2.1 Data types in Molecular biology *27*
- 2.1.1 Gene expression data *27*
- 2.1.2 Metabolic pathways and molecular interactions *28*
- 2.1.3 Mutations and polymorphisms *30*
- 2.1.4 Miscellaneous – genetic maps, physicochemical properties *31*
- 2.1.5 A note on the software used *32*

2.2 Sequence databases *33*
2.2.1 Primary nucleotide sequence repositories – GenBank, EMBL, DDBJ *33*
2.2.2 Primary protein sequence repositories *36*
2.2.3 Derived or Secondary Databases of Nucleotide Sequences *40*
2.2.4 Derived or Secondary Databases of Amino Acid Sequences: Subcollections *42*
2.2.5 Derived or Secondary Databases of Amino Acid Sequences: Patterns and Signatures *44*
2.3 Structure databases *46*
2.3.1 The Primary Structure Databases – PDB and CSD *46*
2.3.2 Derived or Secondary Databases of Biomolecular Structures *51*
2.4 Summary *53*

3. Sequence Alignment 55
3.1 Introduction *56*
3.1.1 Why align sequences? *56*
3.1.2 Similarity vs Homology *57*
3.1.3 Homologs, heterologs, analogs, orthologs, paralogs, xenologs *58*
3.1.4 The significance of an alignment *60*
3.2 Dot matrices and Hash coding *62*
3.2.1 Comparing sequences using dot matrices *62*
3.2.2 Pattern searching using hash coding *66*
3.3 Dynamic programming in sequence alignment *68*
3.3.1 The Needleman-Wunsch algorithm *69*
3.3.2 Scoring schemes – briefly *76*
3.3.3 The Needleman-Wunsch algorithm – part II *76*
3.3.4 The Smith-Waterman algorithm *81*
3.4 BLAST and FASTA *83*
3.4.1 BLAST *83*
3.4.2 FASTA *87*
3.5 Summary *89*

4. Multiple Alignment, Substitution Matrices and Phylogenetic Trees 91
4.1 Multiple sequence alignment *91*
4.1.1 Goals of multiple sequence alignment *91*
4.1.2 Representation of a multiple sequence alignment *93*
4.1.3 Scoring a MSA *95*
4.1.4 Dynamic programming for MSA *95*
4.1.5 Progressive or hierarchical alignment *97*
4.2 Substitution matrices *100*
4.2.1 What are substitution matrices? *100*
4.2.2 Evolutionary models *101*
4.2.3 PAM substitution matrices *103*
4.2.4 BLOSUM substitution matrices *107*
4.2.5 Gap penalties *108*
4.3 Phylogenetic trees *109*
4.3.1 Introduction *109*

4.3.2 Distance matrix methods – UPGMA *112*
4.3.3 Distance matrix methods – Neighbour joining *114*
4.3.4 Maximum parsimony *115*
4.3.5 Maximum likelihood I *118*
4.4 Summary *120*

5. Protein and DNA Sequence Analyses 123
5.1 Pattern Representations and Characterisation *123*
5.1.1 Deterministic patterns – regular expressions *123*
5.1.2 Probabilistic patterns – sequence logos *124*
5.1.3 Pattern characterisation and classification *126*
5.2 Pattern Discovery and Sequence Classification in Proteins *127*
5.2.1 General methods *127*
5.2.2 General methods for proteins *128*
5.2.3 Hidden Markov models *129*
5.2.4 HMM – Application to analyses of protein sequences *131*
5.2.5 Other analyses of protein sequences *132*
5.3 Pattern Discovery and Sequence Classification in Nucleic Acids *133*
5.3.1 Gene discovery *133*
5.3.2 Gene discovery using HMM - GeneMark *135*
5.3.3 Gene discovery using ANN - GRAIL *136*
5.3.4 Gene discovery using Fourier analysis – GeneScan *137*
5.3.5 Other analyses of DNA sequences *139*
5.4 Summary *141*

6. Determination and Analysis of Molecular Structures 143
6.1 Experimental structure determination techniques *144*
6.1.1 X-ray crystallography *144*
6.1.2 Nuclear magnetic resonance *147*
6.1.3 Electron microscopy, image reconstruction and electron diffraction 150
6.2 Visualisation and representation of molecular structure - computer graphics *151*
6.2.1 Methods of representing a three-dimensional object on a two-dimensional surface *151*
6.2.2 Methods of representing biological molecules *154*
6.3 Geometrical analyses of structures *157*
6.3.1 Coordinate transformations *157*
6.3.2 Bond lengths *158*
6.3.3 Bond angles *158*
6.3.4 Hydrogen bonds *158*
6.3.5 Torsion angles *158*
6.3.6 Calculations of planes *159*
6.3.7 Pseudorotation parameters *159*
6.3.8 The Ramachandran map *161*
6.3.9 Protein surface and solvent accessibility *162*
6.3.10 Nucleic acid structural parameters *163*
6.3.11 Software *164*
6.4 Structure comparisons *164*
6.4.1 Superposition techniques *164*

6.4.2 Distance matrices *167*
6.4.3 Dynamic programming using distance matrices *172*
6.4.4 Generalised dynamic programming using a set of residue properties *174*
6.4.5 Other structure comparison methods *175*
6.5 Summary *178*

7. Protein Structure Prediction and Protein Folding 180
7.1 Protein secondary structure prediction *182*
7.1.1 Measures of prediction accuracy *183*
7.1.2 Statistical techniques – the CF algorithm *184*
7.1.3 Statistical techniques – the GOR I algorithm *186*
7.1.4 Use of segment statistics – the GOR III algorithm *187*
7.1.5 Neural networks in secondary structure prediction – PHD *187*
7.1.6 Nearest neighbour methods – PREDATOR and NNSSP *189*
7.1.7 Consensus methods – JPRED, NPS *190*
7.1.8 What is the best accuracy attainable – pitfalls, precautions and prognosis *191*
7.2 Protein tertiary structure prediction *191*
7.2.1 Homology modeling *193*
7.2.2 Threading *195*
7.2.3 *Ab initio* structure prediction *198*
7.3 CASP, Levinthal paradox, protein folding *202*
7.3.1 CASP *202*
7.3.2 Levinthal paradox *203*
7.3.3 Protein folding *205*
7.4 Summary *206*

8. Nucleic Acid Structure 208
8.1 RNA structure prediction *209*
8.1.1 Dynamic programming – the Nussinov algorithm *211*
8.1.2 Dynamic programming – the Zuker algorithm *217*
8.1.3 Structure prediction by analysis of sequence covariance *220*
8.1.4 RNA tertiary structure prediction and RNA folding *223*
8.2 DNA Structural Polymorphism *225*
8.2.1 Regular DNA helices, A, B and Z types *226*
8.2.2 Sequence Specific DNA structure *230*
8.2.3 DNA bending *233*
8.2.4 DNA supercoiling *234*
8.2.5 Unusual DNA structures *235*
8.3 Summary *238*

Further Reading 240
Index 243

1

Introduction

The word 'bioinformatics' is of recent origin, but already it has acquired several, though overlapping, meanings. It is generally understood to refer to the application of information technology (IT) in biology, and the following words and phrases also refer to approximately similar areas of study: computational (molecular) biology, biocomputing or biocomputation, computational genomics, *in silico* biology, and systems biology. Seen narrowly, it may appear that a special word to refer to the use of computers in biology is unnecessary, especially when we consider that there is no such word when we talk about the application of IT in, for example, physics ('physinformatics?'). Just as virtually any endeavour in modern life has been enhanced by the application of IT, so have biologists always been helped in all their tasks by computers. Therefore, the need for a special word to describe this particular application requires explanation. It arises from the fact that *the most important biological information is digital*. This makes it not only possible, but also necessary, to use all the methods of information science and technology in a most intimate way in order to 'do' modern biology. Just as we may consider biochemistry as dealing with metabolic pathways, so we may consider bioinformatics to comprise the study of information pathways in biology. And just as metabolic pathways are an essential description of biological systems, so are information pathways crucially required for the existence of any organism.

Of the types of data that are used in modern molecular biology since the last decade or so, DNA and protein sequences form the major proportion. Both these are represented as strings of discrete symbols chosen from an alphabet of four for DNA, and twenty for proteins. Analysis of these sequences is therefore tantamount to analyses of strings of symbols, a topic that lies at the centre of computer science. Thus all the tools and techniques that have been developed to process digital information may be directly applied to the sequences, without having to go through an intermediate step, common in many other disciplines, of converting analogue data to a digital form. The theory underlying bioinformatics, thus, is intimately connected with theoretical computer science, especially such topics as natural language processing, machine learning, computational linguistics and digital pattern recognition. Ideas and methods have been freely borrowed from these sciences and effectively applied in bioinformatics to obtain useful biological information. And, as often happens in such cases, the necessity for obtaining such biological information has been the mother of several inventions which could find application in other sciences.

In this introductory chapter we explain in an elementary way the central paradigms of modern molecular biology and show exactly what we mean by the phrase 'digital nature of biological information'. We briefly describe information technology and then introduce the reader to the tasks

and techniques that we consider to make up bioinformatics. The last part of this chapter is an overview of the rest of the book.

Note: There are subtle but well marked differences between the terms 'computer science' and 'information technology'. In this book however we will use them interchangeably.

1.1 An Elementary Introduction to Modern Molecular Biology

1.1.1 The reductionist programme in biology

Modern biology takes as its starting point the principle that biological systems are in no essential respect different from physical or chemical systems, though they are much more complex than the latter. All the laws of physics and chemistry that are used to describe, say, planetary systems or chemical reactions in a test tube, may be used to describe how a single living cell functions and how it grows into a complex multicellular organism. Furthermore, and this is the crucial point, no *new* laws are required, laws that may be applicable to living systems alone and not to non-living matter. To restate this important point, biology is an extension of physics and chemistry, albeit far more complex. Before the advent of modern molecular biology, approximately half a century ago, biological systems were thought to be based upon an unknown principle that set them apart from non-living matter. This principle was known by the Latin name *vis vitalis* (*prana* Sanskrit or *chi* in Chinese). However even at this time questions regarding the exact nature of elementary life forms such as viruses blurred the boundaries between Life and Non-life. Darwinian principles of evolution, together with related theories on prebiotic systems showed it possible to think of Life as consisting of complex interactions among inert chemicals. This realisation led to the 'reductionist programme' in biology. According to this programme, biological systems are studied as component parts. These parts are subjected to analysis by physical and chemical techniques and theories. The understanding of the function of each separate portion so gained is put together, bit by bit, to build an understanding of the entire biological system.

The reductionist programme begins by reducing a living organism to its component tissues and organs. These are made of differentiated cells, and the next reductionist step is to understand cell function. A cell is a collection of a large number of molecules that are in intimate interaction with each other. To understand cell function therefore one needs to understand these molecules and their interactions. Molecular biology, as the name indicates, works out biological function on the basis of interactions between molecules. Some of these molecules are found also in non-living matter. But most are specific to biological systems. The macromolecules, in particular, belong to this category, namely proteins, DNA and polysaccharides.

Another reductionist path also exists from organism to molecule. Here the emphasis is on function. A living system is characterised by many specific functions such as reproduction, metabolism, motion, etc. One may study each of these functions in molecular terms and put them together to obtain a molecular explanation of the entire biological system. In either case the essential idea is the same and is expressed as 'the whole is the sum of its parts'. It may be noted that this epigram is not by any means a proven idea, and future studies may overturn this dogmatic representation of the living state. However, so far, the programme has been enormously successful, and there does not seem to be any end to it in view. The development of the discipline of bioinformatics is just one manifestation of this success. In practical terms, many new drugs, new crops and other biotechnological products are direct outcomes of the reductionist programme, apart from, of course, a much deeper and more satisfactory understanding of the mysteries of Life.

1.1.2 A cell is the smallest unit of life

For the purposes of this book, we will go down two steps into the reductionist programme and start with a description of a single living cell. Figure 1.1 is a sketch indicating the important organelles and structures inside a typical eukaryotic cell. Of the components listed in the figure, the important ones from the point of view of the information transfer mechanisms are the nucleus, the ribosomes, the endoplasmic reticulum and the Golgi apparatus.

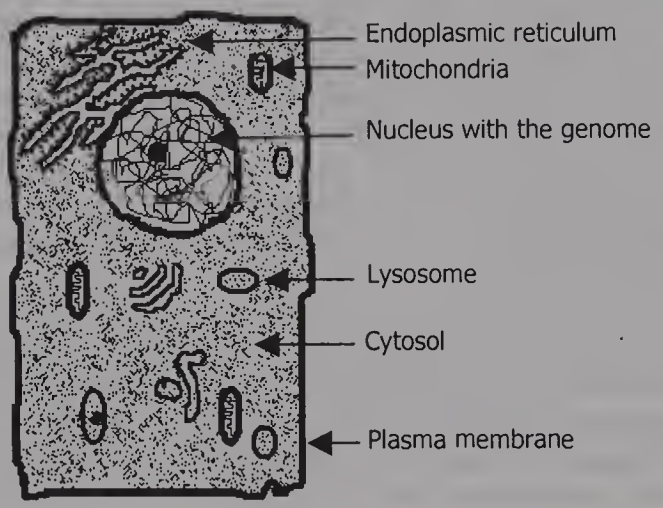


Figure 1.1 The eukaryotic cell. Note this is only a schematic two-dimensional representation of a complex three-dimensional object.

- The cell is contained within a membrane consisting of two layers of molecules that have hydrophilic heads and hydrophobic tails. The membrane gives unity and integrity to the cell. The largely aqueous medium within the cell, containing all the molecules of Life, cannot easily pass out through this membrane. Neither can nutrients and other essential molecules pass into the cell through it. There are thus several channels, mainly made of proteins or long peptides, which help the transport of all the polar substances that need to go in and come out. Embedded in the membrane are also proteins that functions as signal transducers, to pass messages from outside the cell to the inside and vice versa. In addition, molecules or molecular complexes are found partially buried in the membrane coats of some of the organelles, and they carry out their function from that location. An example is ATP synthase that sits on the chloroplast or mitochondrial membrane envelopes in eukaryotic cells, and is crucial in converting energy to ATP (Adenosine Tri Phosphate – ATP is the energy currency in the cell, delivering the energy where needed.) In prokaryotic or bacterial cells ATP synthase is bound to the outer plasma membrane. Other examples of such 'molecular machines' bound to the outer cell membrane include the bacterial photosynthetic reaction centre in plant cells.
- Contained within the cell membrane are several organelles, some of which possess membrane skins of their own. The nucleus is one such structure and is present in all eukaryotes. Inside the membrane wall of the nucleus are thousands of different types of molecules, the chief of them being DNA that carry the entire genetic inheritance of the cell. The DNA is complexed with several different types of proteins and is tightly packaged into chromosomes. The number of chromosomes depends on the size of the genome, which in turn is very approximately related to the complexity of the organism, with several important exceptions. (This lack of perfect correlation between complexity and genome size is called the 'C value paradox' and will be dealt

with in greater detail later on.) Just like the plasma cell membrane, the nuclear membrane is again the site of several receptor and channel proteins that convey messages and molecules across the barrier. In prokaryotic cells, i.e. the single-celled eubacteria and archea, there is no structure that can be identified as the nucleus. However, the DNA that constitutes the genome of these organisms is usually found condensed at the centre of the cell.

- Mitochondria are large organelles, next in size only to the nucleus, which are found in large numbers in most eukaryotic cells. They are the sites of ATP production and are considered the energy factories of the cell. Mitochondria have their own complement of genomic DNA, apart from that found in the nucleus. The mitochondrial genes encode a relatively small set of proteins, which are synthesised within the organelle. Mitochondria migrate from place to place within the cell and in many other respects function autonomously from the rest of the cell. These features have led to the theory that mitochondria are the remnants of bacteria that have been absorbed by eukaryotes through a process called 'endocytosis'. Analysis and comparison of the mitochondrial DNA with nuclear DNA, using the techniques of bioinformatics, has helped in establishing this theory on a firm foundation.
- Chloroplasts are large organelles found in plant cells and in green algae. The function of chloroplasts is similar to that of mitochondria, namely they convert the energy in sunlight to ATP. Thus they are the chief organelles involved in photosynthesis. Another important point of similarity between a chloroplast and a mitochondrion is that the former too has its own small complement of genomic DNA. Chloroplasts, too, are very likely to be endosymbionts, i.e. they too are bacterial cells that have been incorporated into ancestral eukaryotes at a very early stage of evolution.
- The endoplasmic reticulum (ER) is a tightly interconnected network of membrane bound vesicles that is the site of synthesis of a wide variety of molecules. There are two types of ER. The first, or smooth ER, is the site of synthesis of fatty acids and phopholipids. The second, or rough ER, lies more directly in the information pathway of the cell. It gets its name from the multitudes of ribosomes that stud the membrane surface. The ribosomes are large complexes of proteins and RNA that synthesize all the proteins required for the functioning of the cell. These proteins then move to the Gogli apparatus where they are sorted and prepared for transport to the positions where they are required. Secretory proteins are prepared for secretion by enclosing them in vesicles, which later bud out from the plasma membrane surface.
- Lysosomes are acidic organelles containing degradative enzymes, and perform a scavenging function in the cell. They degrade the polymeric proteins and nucleic acids into their constitutive monomers, which are then recycled. Peroxisomes perform a similar function for small organic molecules.
- The cytosol is the protein-rich substance of the cell that remains after all the organelles are removed. It contains soluble enzymes as well as filamentous proteins that form structural scaffolding for the cell and assist in cell motion.

These are only some of the more important structures within the cell. Figure 1.1 may give an inaccurate idea of the nature of the cell. See Table 1.1 for some physical specifications.

1.1.3 Biological information pathways

Information contained in one generation of cells is passed on to the succeeding generation in two ways. When a cell divides, it passes about half its complement of molecules and structures to each of the two daughter cells. This is the first pathway. In the second pathway, each daughter cell receives from its parent an identical set of genes. This set of genes is present in the DNA that the cell receives. The genes are the instructions required by the cell to build new molecules and structures, as and when

Table 1.1 Approximate physical specifications of a cell

Volume = 1 micron^3 = 10^4 x 10^4 x 10^4 Å^3
Protein carrying capacity = 2 - 3 million molecules (not including solvent and small molecules)
Typical protein volume = 10,000 Å^3
Total protein volume = 10,000 x 1,000,000 Å^3
Mean Free Path between proteins = 100 Å
Average density = 1000 kg/m^3 = 1mg/ml
Average density of proteins and RNA in cell = 300 mg/ml
Average viscosity = 50 Poise = 500 N-s/m^2
Doubling time
- in minimal media = 1 hour
- in rich media = 20 minutes

required in its life cycle. The principal molecular process involved in this pathway of information transfer is the replication of the DNA.

Traffic of information also occurs within the cell and during the cell cycle. The 'Central Dogma of Molecular Biology' specifies this pathway. The principle states that information transfer always occurs from DNA to RNA to protein, never in the other direction (Figure 1.2). The discovery of retroviruses and their reverse transcriptase enzyme has led to a modification of the principle to include also the possibility of information going from RNA to DNA. The title 'Central Dogma'[1] has been given to this principle to emphasize the fact that it is as yet only a strong belief based on experiment, and is not a principle or idea that is self-evidently true. The central dogma may also be viewed as a restatement of the Darwinian principle that changes in the genotype lead to changes in the phenotype, and never the other way round. To digress a little from our main line of study in order to expand on this statement, Darwinian evolution postulates that random mutations in the genes lead to random changes in the abilities of the organisms to adapt to the environment. Most of these changes may be harmful, and the individuals that suffer these mutations are unable to produce enough live and healthy progeny to propagate the genes. A few of the mutations, however, may render the individual better suited to the environment, and these would be propagated, to eventually lead to the rise of new species. The key point here is that the modifications in the genes, i.e. the mutations, arise randomly due to natural causes, such as cosmic radiation, natural radioactivity etc., and are not controlled by whether they benefit the organism or not. In direct contrast to this theory, Lamarck postulated that when an organism modifies itself to better suit its environment, it directly passes on the changes to its progeny. An oft-quoted example is the giraffe. According to Lamarck's theory, the first generation of the animal would have stretched its neck to reach farther up the tree for fresh food. This would have led to a very slight

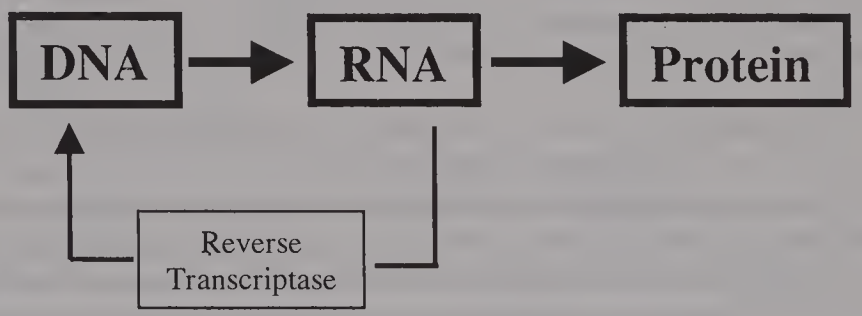


Figure 1.2 The transfer of information within a cell.

[1] The word 'dogma' means a principle or a set of principles accepted on faith alone, and is usually applied to religious beliefs.

increase in the length of its neck, which would have been passed on to the next generation. The neck would have grown longer and longer with each succeeding generation, arriving finally at the present-day physiology of the giraffe. Needless to say, while there is no *a priori* logical argument against this principle, in fact, experimental observations have shown that the Darwinian explanation is the correct one and not the Lamarckian one. The cellular version of the Darwinian principle is the central dogma. A cellular version of Lamarckism would not render the central dogma false. However the path it specifies, i.e. DNA to protein, would not be exclusively correct, and the more important information transfer pathway would then be in the reverse direction, i.e. from protein to DNA. To reiterate, the central dogma assures us that Darwinian principles of information transfer hold even at the cellular level, if we make the correspondence that the gene, as coded in DNA, is the genotype and proteins represent the phenotype.

1.1.4 Storage and representation of genetic information

The chief molecules involved in the information transfer pathway are deoxyribonucleic acids or DNA, ribonucleic acids or RNA, and proteins. All three are macromolecules with average molecular weights ranging from a few thousands for the smallest proteins and RNA, to about a billion for the largest DNA molecule. Another common feature shared by all three is that they are polymeric molecules. In the case of DNA and RNA, the monomeric units are nucleotides, while proteins are built from amino acids. The difference between RNA and DNA is the presence, in RNA, of an extra oxygen atom on the sugar ring. All three molecules possess a backbone of atoms that repeats monotonously over the entire length. In the case of the nucleic acids, DNA and RNA, this is the sugar-phosphate backbone. In the case of proteins this is the polypeptide backbone. Branching out from the backbone at precise intervals are the nucleic acid bases in the case of DNA and RNA, and the amino acid side chains in the case of proteins. Thus we have a portion of the molecule that is constantly and exactly repetitive, and another that represents a variation or modulation of this underlying constancy. Of the three, DNA is the molecule that stores genetic information and the following discussion indicates how well suited it is for this task.

More than fifty years ago, the great quantum physicist Erwin Schrödinger suggested in his book 'What is Life?' that genetic information must be stored in the cell as a 'quasi-periodic crystal'. A crystal is an exact repetition of some basic unit. For information to be represented however, an exact repetition is useless. There must be a modulation of this repetition, which would then be a representation of the information. Different modulations would be different pieces of information. All the same, if there were no underlying constancy, there would be no standard or reference level with which to compare the representation and obtain the information. Putting these two requirements together leads to the concept of a quasi-periodic crystal.

DNA is quasi-periodic crystal, not only in its chemical structure as described above, but also in its three-dimensional structure. It is a double-stranded molecule, consisting of two nucleic acid strands that run in opposite directions, and are wound around each other to form a double helix. (See Chapter 8.) Owing to the fact that the two ends of a nucleotide do not have exactly the same atomic groups, there is directionality in the nucleic acid backbone. One end is referred to as the 5' end and the other as the 3' end. The two strands of DNA are arranged with the 5' end of one strand on the same side as the 3' end of the other, and vice versa. In other words, the two strands are arranged anti-parallel to each other. The negatively charged phosphate-sugar backbones of the two strands are on the outside of the double helix, while the planar, nitrogenous and hydrophobic bases are on the inside of the double helix, away from the aqueous solvent molecules. The two strands stick to each other through hydrogen bonds that form between the bases in a highly specific manner. In three dimensions, the planar bases are arranged as a virtually infinite stack of flat boards at the centre of the helix, while the two backbone chains twist around the outside. If the long DNA molecule is placed in a beam of X-rays, a diffraction

pattern is obtained, which reflects the quasi-periodic nature of the molecule[2]. Genetic information is represented by the sequence of bases in the DNA chain. Since there are four types of bases, one may think of the information as being represented by four symbols, namely A, T, G and C. Therefore, a sequence of DNA bases is a genetic message, or a portion of one. Thus there are sequences that represent proteins and each such sequence is a gene. Also there are sequences that represent control elements such as promoters, operons, enhancers, etc. For every activity of the cell that requires proteins or other such molecules for its operation, there is a sequence or a set of sequences that pertains to it on the DNA molecule (Figure 1.3). The entire set of genes and control elements, etc. is called the

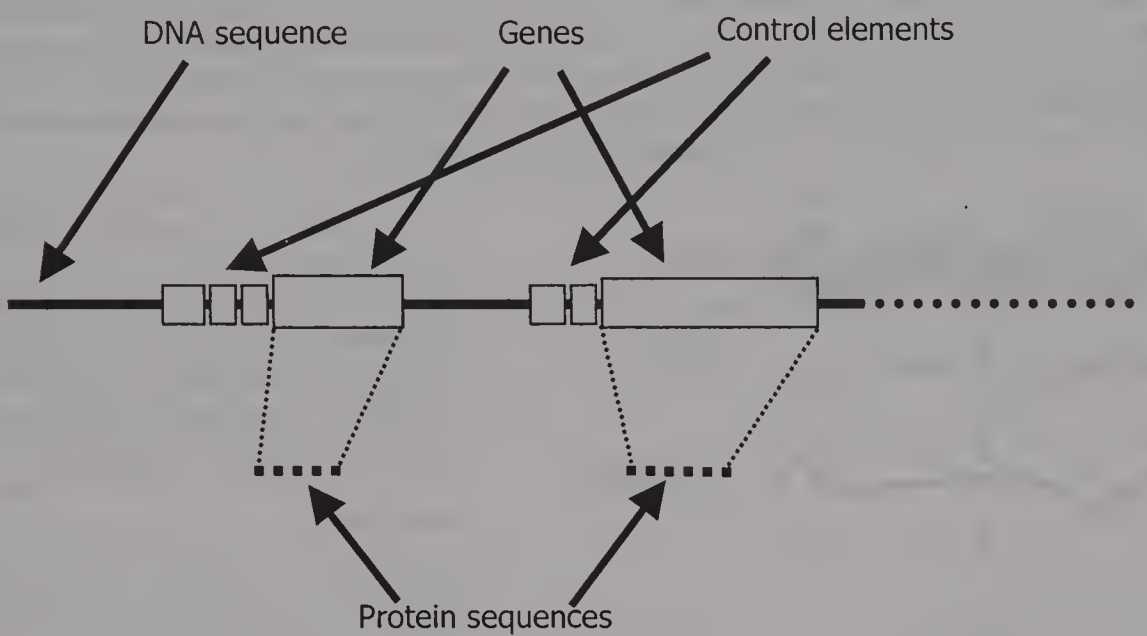


Figure 1.3 Schematic sketch of genome structure. The diagram does not indicate several important fine structural details, such as introns and exons, operons, etc. The figure also does not indicate the fact that in the higher organisms, the portion of the genome without assigned function (black line) is up to 10 times larger than the portion with known function (boxes).

genome of the organism. We may also use the term genome to refer to the entire DNA contained in the cell. In multi-cellular organisms the DNA in each and every one of the possibly billions of cells of the organism is identical. In such organisms, different cells have different functions not because the cellular DNA is different, but because different genetic information is expressed in each cell. In other words, different portions of the DNA molecule are active at different times and in different cells.

It is now easy to understand that genetic information is digital in nature. By this phrase we mean that the most natural representation of this information is as a string of symbols chosen from an alphabet of four. It must be emphasized that by 'digital' we do not mean 'binary'. Internally, in computers, information is represented by binary digits, and this is not the case with genetic information. The essential idea is that the representation is not an analogue signal such as a voltage or a picture, but a string of digits.

---

[2] The occurrence of a diffraction pattern always indicates that the sample has some periodic structure.

A similar digital representation of information occurs in both RNA and proteins. In the former, again the sequence of nucleic acid bases corresponds to the information carried by the molecule. However, while in DNA the alphabet consists of the four bases A, T, G and C, in RNA the base T is not found and instead we have the base U. RNA has three different functions in the process of information storage and transfer. Its three-dimensional structure may be compact and folded, like a protein, or it may be extended, without any particular structure, depending on which role it plays.

A protein chain is represented again by a string of symbols, this time chosen from an alphabet of 20 letters, representing the 20 different amino acids. In itself, a protein sequence is not a piece of genetic information, but is an expression of it. According to the central dogma, this information cannot be passed on directly to the succeeding generation of cells. (The exception to this was mentioned at the beginning of Section 1.1.3, i.e. when the cell divides, it passes on about half its proteins to each daughter cell.) Within the cell, the expression of the genetic information proceeds further as the correct three-dimensional folding of the protein chain, which depends crucially on the sequence. Different sequences lead to different structures. The ultimate step in the chain of expression is the function of the protein that arises from the three-dimensional structure (Figure 1.4).

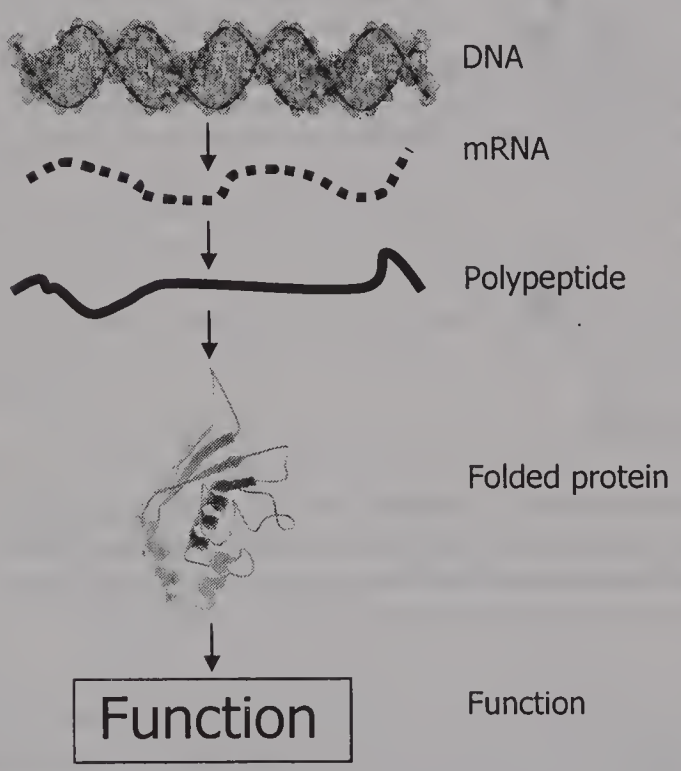


Figure 1.4 The transfer of information from DNA to cellular function. mRNA and the polypeptide are shown as lines to indicate their normal lack of a specific three-dimensional structure.

1.1.5 Transfer of genetic information: Replication

Let us now look in greater detail at the mechanisms by which information is transferred between a cell and its daughter. The process requires exact replicas of the DNA to be made. This is achieved through a process known as replication.

The key concept underlying the immensely successful Watson-Crick double-helical model of DNA structure is that of base pairing. Of the four types of bases found in DNA, two are purines, namely Adenine, A and Guanine, G. The other two, Cytosine, C and Thymine, T, are pyrimidines. The principle of base pairing states that, if the base on one of the two strands of the DNA double helix is C or T, the corresponding base on the other strand, and forming specific hydrogen bonds to the first must be G or A, respectively. In other words, the only possible base pairing schemes are A-T and C-G. This complementarity in base pairing immediately suggests a way in which DNA can replicate and produce exact copies of itself. The two strands come apart, and each acts as a template to produce another version of the complementary strand. This results in two copies of the original DNA double helix (see Figure 8.1), which are then sorted one to each daughter cell upon cell division. Experiments have shown that this scheme to transfer genetic information to the next generation is generally correct. Several details of the process are also now known.

- Replication is semi-conservative. This means that each of the two resulting double helices that are produced after one round of replication contain one strand from the original double helix and one strand that is newly synthesized.
- Most DNA replication is bi-directional. A 'transcription bubble' is formed in the double helix, where the hydrogen bonds between the two strands are broken, and for a short stretch there are two single strands of DNA. The formation of the complementary strand occurs on both. Since the strands run anti-parallel, and since the formation of the new strand always occurs in its 5'-3' direction corresponding to the 3'-5' direction of the template, therefore the replication is bi-directional.
- Replication always begins at a particular sequence of bases on the DNA, called the 'replication origin'. The precise pattern of bases at the replication origin may vary from species to species.
- Each transcription bubble has two forks, one at each end. The two forks each have two anti-parallel strands. On one of them, the 3'-5' strand (also called the 'leading strand'), replication proceeds continuously. But on the other 'lagging' strand, the process requires the formation of an RNA primer, and then elongation of this primer in discontinuous steps, to produce the so-called Okazaki fragments, which are then ligated or joined together to form the complementary replica of the lagging strand.
- In the circular bacterial chromosomes, replication terminates normally when the two forks meet. In linear eukaryotic chromosomes, termination of replication occurs at the 'telomeres'. These are special sequences that occur at the ends of the chromosomes, and contain particular sequences that repeat several, sometimes several thousand, times. These sequences, and a special protein called telomerase, prevent the incomplete replication of the lagging strand that would occur due to the discontinuous process, and thus also prevent the shortening of the chromosome at each replication cycle.

Genetic information passed on to the next generation is not always an exact copy of the parent. Modification and diversity in the information may occur due to many events such as, for example, mistakes in the replication process. Specialised repair enzymes correct most of these mistakes. If we consider the thermodynamics of base pairing and estimate how often errors would occur in the base recognition and pairing process, we obtain a figure of about 1 out of 1000 bases replicated. Due to the efficiency of the repair mechanisms this number is actually about 1 in a billion in bacteria. This corresponds to a single mistake in the entire bacterial genome after 1000 cell divisions. The amount of genetic diversity that this process can yield is thus small. Therefore there is another mechanism, called 'homologous recombination', that can produce the required diversity in the genomes of all organisms. In this process, two duplex (i.e. double stranded) DNA molecules break at homologous sites. These are sites at which the sequences are similar. The broken strands are then exchanged. This results in two duplexes with new sequences, thus generating the genetic diversity required for the proper operation of the Darwinian principles of evolutionary selection.

1.1.6 Transfer of genetic information: Transcription

Transcription refers to the synthesis of messenger RNA molecules, using the DNA molecule as a template. Messenger RNA or mRNA is one of the three main avatars of RNA, the other two being transfer RNA or tRNA and ribosomal RNA or rRNA. (The recent discovery of small RNA molecules, 10 to 50 nucleotides long, that perform a variety of control functions in the cell, has increased the number of different types of RNA). The Central Dogma, which dictates the information pathway within the cell, states that the first step in the pathway is the transfer of information from DNA to RNA. Accordingly information in the DNA is transcribed to mRNA. Unlike as in replication, only those portions of the genome that specify the required proteins are transcribed, not the entire DNA. A sequence of DNA that codes for a protein is called a gene. Thus only the genes are transcribed and this

process is known as expression. Indeed, only some of the genes are expressed at any stage in the cell cycle. Recent estimates have shown that only about 10% or less of an eukaryotic genome actually codes for proteins. Much of the rest of the DNA is of unknown function. There are, for example, large portions of the genome that consist of thousands of repeats of very simple sequence patterns. Even the coding regions are not continuous from the beginning of the gene to the end, but are broken up by intervening sequences, or introns. The portions that code for the proteins are called exons. After transcription, the newly synthesized mRNA is processed to remove the introns and splice together a continuous RNA chain consisting only of the coding portions.

Transcription begins when a protein complex called the RNA polymerase binds to a section of DNA just in front of the gene. This section of the DNA is called a promoter. Promoters are highly specific sequences and are responsible for a variety of controls exerted on transcription. The regulation of gene expression is a critical activity. Specific proteins need to be expressed in specific quantities at specific times in the normal life cycle of the cell. Promoters, or rather, more specifically, certain elements within the promoter sequences play a major role in the regulation. These elements include the '-10 sequence' or 'TATA box' and the '-35 sequence'. These names represent short (in these cases hexameric) sequences present 10 and 35 bases respectively ahead of the transcription initiation point. A 'consensus' sequence may be defined for each of these elements by aligning all known promoters, and then by choosing the most commonly occurring base at each position of the short sequence. The consensus sequence for the TATA box is $T_{80}A_{95}T_{45}A_{60}A_{50}T_{96}$, where the subscripts represent the percent occurrence of each base at its position in the set of aligned promoter sequences. Other regulatory elements include enhancers and operator sequences. Though each of these may be spread over several hundred bases, specificity of action is always due to the existence of short specific sequences placed at precise positions.

After the initiation step, transcription continues with the elongation step, during which the mRNA corresponding to the gene being expressed is synthesized. This mRNA has the same sequence as one of the strands on the double-stranded DNA, called the coding or sense strand. The other strand acts as the template to the mRNA synthesis, and is thus called the template or antisense strand. It is complementary, both to the mRNA and to the coding strand. At the end of the gene, 'terminator' sequences bring the transcription to a halt, releasing the newly synthesized mRNA. There are many different types of terminators, varying widely in sequence, strength or the ability to induce termination, and in their dependence on other sequence elements, proteins or other factors. Termination also depends on the transient structures taken up by the mRNA that has been synthesized, and not just on the DNA sequence.

Post-transcriptional 'editing' of the mRNA may also be considered a part of the transcription process. Protein-RNA complexes called spliceosomes assist in the maturation of mRNA. Splicing or the process of cutting out the introns and joining the exons together requires the presence of specific short sequences or splice sites at either end of the intron.

1.1.7 Transfer of genetic information: Translation and protein folding

After synthesis and maturation, the mRNA is transported to a large protein-RNA complex called the ribosome, in which the nucleic acid sequence is translated into the corresponding protein sequence and the appropriate polypeptide chain is synthesized. There is a specific relationship between the nucleic acid sequence and the protein sequence, and this is called the genetic code (Table 1.2). The code associates each amino acid with a nucleic acid 'codon' consisting of three bases, taken together. It is necessary to take three bases at a time because there are only 4 different bases available, but twenty amino acids. Taking one base at a time will allow the specification of only 4 amino acids. Taken two bases at a time, there will be only 16 different combinations or dinucleotides. Taking three bases at a

Table 1.2 The Genetic Code. The codons are written 5' to 3' as seen in the mRNA sequence (and not in the DNA sequence). Hence U is used, not T.

AAA	*Lys*	UAA	*STOP*	GAA	*Glu*	CAA	*Gln*
AAU	*Asn*	UAU	*Tyr*	GAU	*Asp*	CAU	*His*
AAG	*Lys*	UAG	*STOP*	GAG	*Glu*	CAG	*Gln*
AAC	*Asn*	UAC	*Tyr*	GAC	*Asp*	CAC	*His*
AUA	*Ile*	UUA	*Leu*	GUA	*Val*	CUA	*Leu*
AUU	*Ile*	UUU	*Phe*	GUU	*Val*	CUU	*Leu*
AUG	*Met*	UUG	*Leu*	GUG	*Val*	CUG	*Leu*
AUC	*Ile*	UUC	*Phe*	GUC	*Val*	CUC	*Leu*
AGA	*Arg*	UGA	*STOP*	GGA	*Gly*	CGA	*Arg*
AGU	*Ser*	UGU	*Cys*	GGU	*Gly*	CGU	*Arg*
AGG	*Arg*	UGG	*Trp*	GGG	*Gly*	CGG	*Arg*
AGC	*Ser*	UGC	*Cys*	GGC	*Gly*	CGC	*Arg*
ACA	*Thr*	UCA	*Ser*	GCA	*Ala*	CCA	*Pro*
ACU	*Thr*	UCU	*Ser*	GCU	*Ala*	CCU	*Pro*
ACG	*Thr*	UCG	*Ser*	GCG	*Ala*	CCG	*Pro*
ACC	*Thr*	UCC	*Ser*	GCC	*Ala*	CCC	*Pro*

time gives 64 possibilities to build the codons, which are, however, too many. This problem of plenty is solved by associating more than one codon with most of the amino acids. This is termed the degeneracy of the genetic code. The degeneracy gives a choice of different mRNA sequences to represent the same protein sequence. This choice is made in different ways in different organisms, leading to the possibility of distinguishing phylogenetic relationships on the basis of codon usage. The genetic code itself is almost exactly the same in all living systems, from unicellular bacteria to complex multicellular mammals. The only differences are to be found in mitochondria, which have their own translation apparatus, independent of the cellular machinery. Even here the differences are very small.

Owing to the triplet nature of the code, the message contained in each mRNA may be interpreted in three different ways, depending on which 'reading frame' is used. A mutation in the sequence that leads to a shift in the reading frame could have disastrous consequences, since the entire protein message would then be wrong. Elaborate mechanisms are therefore present in the translation apparatus to ensure that the correct reading frame is recognised in the ribosome.

A mediator or adaptor molecule called transfer RNA (tRNA) is involved in deciphering the code. These are polyribonucleotides, about 75 bases in length, that fold up into an approximately L shaped three-dimensional structure. The anti-codon, i.e. the three bases complementary to a particular codon, is at the end of one arm of the L. The corresponding amino acid is at the end of the other arm. Each tRNA molecule is specific for a given codon-amino acid pair. The genetic code thus actually resides in these molecules. Special proteins called aminoacyl-tRNA synthetases 'read' this code and bind the appropriate, or cognate, amino acid on to each tRNA.

The tRNA molecules charged with the amino acids then travel to the ribosome, where the mRNA has already arrived. Now the translation process begins, with the appropriate tRNA binding to the mRNA through base pairing between its anticodon and the codon on the messenger. The process is

reminiscent of a magnetic tape or punched tape being read by a reading head. At the other end of the tRNA corresponding amino acid binds covalently to the previous one by forming a peptide bond, thus adding itself to the growing polypeptide chain.

When one of the three 'stop' codons is encountered on the mRNA, synthesis is terminated and the polypeptide is released. The newly synthesized chain now folds into its functional three-dimensional shape. A different sequence of folding events occurs in different proteins. Some fold spontaneously into the tertiary structure. Anifinsen's experiment on denaturation and renaturation of proteins showed that the information required to form the three-dimensional structure is present in the polypeptide chain. The search for the exact mechanisms by which this information is expressed has become one of the most important lines of study now being undertaken, and the problem is generally called the protein folding problem or the protein structure prediction problem. There are many proteins that do not assume the correct tertiary structure spontaneously, but require the assistance of the so-called chaperone proteins. In other cases, such as for secretory proteins, chaperones are required to maintain the extended structure of the polypeptide until it can be transported through a narrow channel across the membrane, after which the protein folds spontaneously. Thus the events of protein folding depend also on the final destination of the protein. Signal or leader sequences present at one end of the polypeptide chain are used by various translocation factors (i.e. other proteins or protein-RNA complexes) to determine the correct position of the protein in the cell and take it there. A hierarchy of such sequences indicates the location of the proteins within the organelles.

1.1.8 Genome organisation

The organisation of the genome may be considered from the functional or from the structural viewpoint. Several functional elements such as genes, promoters, introns, exons and splice sites have been mentioned in the previous sections. In this section we present a brief review of genome organisation from the functional viewpoint.

- The word 'genome' refers to the entire DNA sequence or sequences present in every cell of an organism. In prokaryotes, the entire genome consists of genes, where the word 'gene' means not just the portion coding for a protein, but also the regulatory sequences such as promoters, etc. In eukaryotes, apart from the genes, the genome also contains large amounts of DNA with no clear function known currently.
- The genes in prokaryotes are generally contiguous DNA sequences. In eukaryotes, especially in multicellular organisms, most genes are comprised of exons or coding regions, interrupted by introns or intervening sequences. One of the reasons for such an arrangement could be the additional genetic diversity that arises when the exons are shuffled to create proteins with new functions.
- The total amount of DNA present in each cell is specific for a given organism and is called its C value. It is measured as the number of base pairs present. There is vast variation in the C value across phyla and the values range from 10^6 base pairs in the smallest genomes of the eukaryotic mycoplasma, to about 5 times that in most bacteria, to about 10^7 in fungi, 10^8 in worms, 10^{10} in amphibians, to 10^9 in mammals. The C value paradox expresses the discrepancy between genetic complexity and genome size – greater complexity does not always correspond to a larger C value. Even within a phylum there is sometimes large variations in the C value. An example is amphibians, in which the range is from 10^9 to 10^{11}
- The variation in the C value is accounted for by several factors. The number of genes present or the number of proteins coded in each genome varies, but this accounts for a only a small portion of the variation. There are about 2350 genes in *E. coli* with a genome size of 4.2×10^6 base pairs, whereas recent estimates give a number between 30,000 and 60,000 genes in humans, with a genome size of about 3×10^9 base pairs. Differences in size of the large, repetitive, non-coding,

simple sequences of as-yet-unknown function account for a greater part of the variation. Such sequences may add up to 75% of some of the genomes.

- Another reason for the variation is the difference in the sizes and numbers of the introns. Changes in the introns, both as to sequence and size, are not fatal to the organism, since the protein is coded in the exons. Due to this, such changes may be tolerated to a far greater degree than mutations in the exons.
- Clusters of genes arise from gene duplication events. The number of copies in a cluster may range from just two to thousands. The functional significance of the copies varies. In some cases every copy is expressed since large numbers of the product are required. In other cases many of the copies may have fallen 'silent' due to mutations, i.e. they are 'pseudogenes' that are no longer expressed.

1.1.9 Genome sequencing projects and computers

Due to the central role played by the genome in the life cycle of the cell, it is essential to know and understand the sequence of the DNA it contains. The sequence represents a blueprint for the life of the organism. To use a different metaphor, this time taken from quantum physics, the genome for an organism is like a wave function for a physical system. If we know the wave function Ψ, we are able to predict the future development of the system as well as its responses to different physical perturbations. In a similar manner, if we know the genome, we may be able to map out the complete future history of an individual organism, including its responses to environmental stresses such as drought or temperature variations.

This view of the genome has lead to the setting up of large sequencing projects by governments and commercial organisations. The most visible of these is the Human Genome project, which aims at nothing less than the complete, error-free and fully annotated sequence of the entire human genome, all 3 billion or so base pairs of it. Continuous improvements in technology since the project was first mooted about 15 years ago, and better understanding of the benefits that would accrue, rapidly brought forward the completion date. A report on the initial sequencing and analysis of the first draft sequence of the complete genome was published in February 2001. The amount of this data is so vast, however, that new analysis and insights will continue to be made for many years to come. Even the sequence data available for analysis will continue to increase by several orders of magnitude, first, as the human genome is made more accurate, and then as the genome sequences of individuals start being available. Also studies are underway on the genome of other organisms ranging from bacteria to plants to other mammals. Already the complete genome sequences of more than 100 different organisms are known[3]. The sequences alone amount to several gigabytes of digital data. It is obvious that the sheer volumes involved make the use of information technology absolutely necessary, quite apart from any other consideration. To put into perspective the quantities involved, consider that we print out the sequence in the same format as a telephone book, say about 1500 letters to each page. The genome of *E. coli* would require about 3000 pages. An insect genome, such as that of *D. melanogaster* would require 10,000 pages, whilst the human genome sequence would require 2 million pages or 2000 books of the size and format of a telephone directory! It is not possible even to *read* through this information in a lifetime, let alone understand it.

Apart from sequence data, several other types of biological data are also being collected, and need to be stored, retrieved, analysed and understood. Biological data types will be discussed later on

---

[3] At the first writing of this chapter, we planned to give a table of completely sequenced organisms. But the number is expanding at so fast a rate that any number we give will be obsolete in a few weeks, if not days. We refer the interested reader to several websites that give constantly updated information, for example *www.ncbi.nlm.nih.gov*

in the book. Here it is sufficient to give a few examples, such as metabolic pathways and expression profiles. While we have not considered biological data that may not be molecular, such as information related to biodiversity, the environment and so on, eventually, in order to understand the biosphere in a way that is satisfying as well as useful, it is necessary to integrate these vast amounts of information into one seamless whole. This is impossible without the hardware and software of IT. It is not therefore surprising that computers are now considered essential instruments in biological research, and that computational biology or bioinformatics is now a part of any good biology course.

1.2 A Brief Overview of Information Technology and Science

1.2.1 History

The computer revolution started out in earnest in the late 1970s and early 1980s, when the personal computer (or PC) made its appearance. Computers began being used not only for computation (which original purpose in fact led to the name), but also for a variety of other purposes, such as entertainment, which had not been earlier envisaged. A computer has become a 'Universal Machine' that is now at the heart of almost every appliance or instrument, both those found commonly in homes and commercial establishments, and the specialised ones used in laboratories. A detailed history of computers is out of place here. However we list below some of the landmarks in the development of this technology.

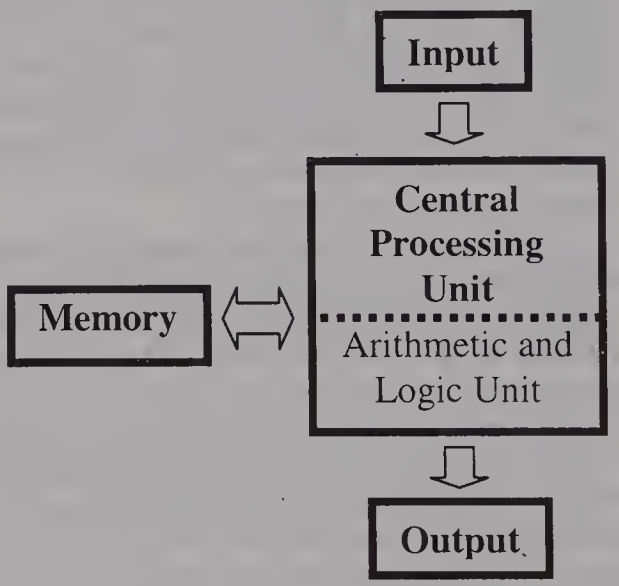


Figure 1.5 The Von Neumann computing model. The input consists of both the data and the operations to be carried out on it. A computer that follows this model may be used to carry out a variety of calculations instead of being restricted to just one.

- In 1812 Charles Babbage, an English mathematician and inventor, developed the concept of a stored program for computation and implemented it on a mechanical machine. The same machine could be asked to perform widely different computations simply by changing the program.
- Digital computers, first with electro-mechanical components, and then with purely electronic components were developed in the middle of the 20th century.
- At about the same time, John von Neumann, an American mathematician developed the theory underlying the design and architecture of digital computers (Figure 1.5). To a great extent these principles are in use even today.
- In the quarter century between 1945 and 1970, computer technology went through four generations. The first used vacuum tubes as the electronic components, the second used discrete solid state devices such as transistors and diodes, the third used integrated circuits with several devices on a small silicon chip, and, finally, the fourth generation computers were built of very

large scale integrated (VLSI) chips that packed thousands of devices onto silicon wafers the size of a thumbnail.

- It is no longer possible to talk of generations of computers, since advances are so rapid. The number of devices packed into each chip keeps increasing according to Moore's law (Figure 1.6). In the foreseeable future this law will continue to operate, at least until the level at which the effects of quantum mechanics become noticeable.

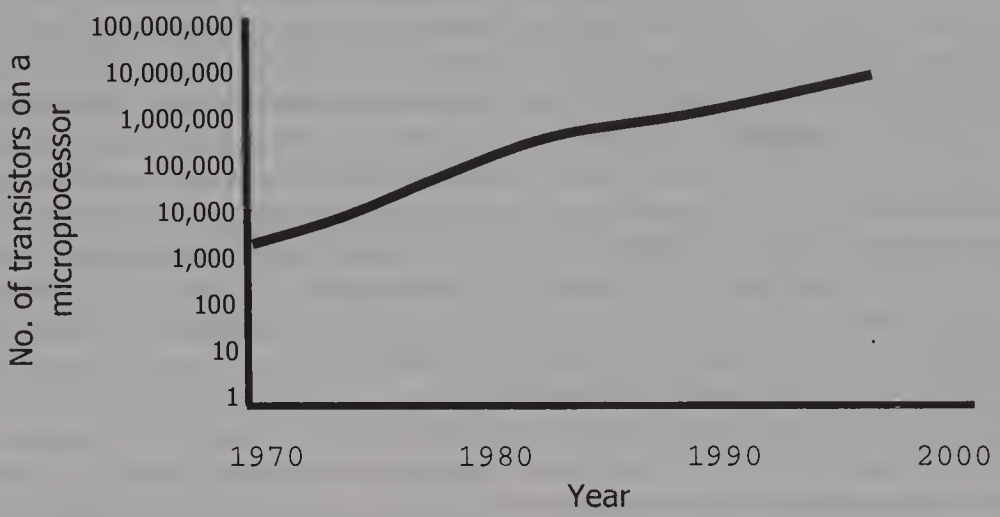


Figure 1.6 Gordon Moore, co-founder of Intel Corporation, stated in 1971, in effect, that at then current rate of technological development and advances in the semiconductor industry, the complexity of integrated circuits doubles every 18 months. This is called Moore's Law. The diagram above shows the actual rate of increase of the complexity in the last 30 years, vindicating Moore's law.

- This increase in computing power has resulted in a wide variety of different tasks being performed by computers, and has made computers the cheapest and most widely used machines. A key concept responsible for this is that the same piece of equipment may be used to perform myriad tasks, by the simple process of changing the program. A computer may be thus understood as an instrument in which the 'body' is the hardware and the 'soul' or the 'spirit' is the software.

1.2.2 Hardware and software

The heart (or perhaps more correctly, the brain) of the computer is the CPU or Central Processing Unit. This, together with the memory, constitutes the computer – the rest are peripherals that enable the machine to accept instructions, programs, data and so on and to give out the results. According to the von Neumann model given in Figure 1.5, the CPU is separate from the memory and both are in turn separate from the peripherals. The program and the data are loaded into the memory, from where the instructions that make up the program are executed one by one, obtaining the data as required from the same source.

The CPU consists mostly of a single silicon chip, called a microprocessor, on which all the necessary circuits are present. Commonly used present-day microprocessors include the Pentium series, the AMD series and the Power series. The memory consists also of silicon chips placed in close proximity to the CPU on a printed circuit board (PCB), called the motherboard. Other PCBs, carrying circuits to control the peripherals, are connected to the motherboard. Commonly used peripherals include monitors, keyboards, mouse and printers to input and output data and results, and magnetic

hard disks and floppy disks to store and transfer data. The storage capacity of hard disks has been increasing as fast as processing speed, and both are approximately keeping pace with the increasing requirements in bioinformatics. So much so, that the level of investment required by an organisation in order to have its own copy of, say, the complete DNA sequence database is approximately the same as a decade earlier, despite the enormous increase in the volume of data.

One of the more recent trends aimed at increasing computation speed, storage volume etc. is the use of computer 'clusters' or 'grids'. These may be thought of as tens, hundreds, or even thousands of individual computers tightly connected together so that they could all work on the same program. The cluster may share some of peripherals such as printers or even storage disks, but the essential idea is that all the processors in the cluster will be working on the same problem. There are many tasks in computational biology, which can be broken down into several subtasks, each of which may be performed independently of the other. Cluster computers are ideally suited to address such jobs, since each of the tasks may be run simultaneously on a different processor, instead of running them sequentially on a single processor, thereby speeding up the calculations enormously. One of the major advantages of such a machine in the context of bioinformatics is that all the processors may be programmed to access the same single copy of the large database, instead of having multiple copies of it. This results not only in greater processing speed, but also greater ease of database management, since a single copy is easier to handle than several copies.

There are several layers of programs (or software) that are required in a computer system. The lowest level of interaction with the computer is through the operating system (OS), apart from low-level machine code and instructions, which are usually hidden from all but the most technical of users. Commonly used operating systems include the Unix family, of which Linux is now a very popular member, and the Windows family of software, which is manufactured by Microsoft Corporation. OS is the program that helps the user instruct the computer to perform various tasks such as creating and managing files of data or text or programmes, executing programs written in a high level language to carry out specific computations or other tasks, etc. High level languages include FORTRAN, which is very popular in the scientific community, and C and C++, which are now among the most commonly used programming languages. A language that has been specially designed for use over the Internet is Java.

At the next level of sophistication are the application packages and programs that are used to carry out specific tasks. A widely used example is software for the preparation of documents, such as MSWord. These packages are the usually the level at which the readers of this book would approach computers. Most of these packages come with a 'graphical user interface' or GUI. These interfaces lay out all the functionalities of the program in the form of buttons and dials on the computer screen in such a way that the user conveys her commands to the computer by moving the pointer over the appropriate button and clicking on the mouse. In effect the screen of the computer monitor is transformed into the front panel of a machine, with knobs and buttons. Several packages designed for biological sequence and structure analysis are now available, both commercially, and freely in the context of academic exchange of software. Among the commercial packages we can name the Biosym suite, the GCG package, the Tripos set of programs and the Vector NTI suite. As far as the freely available software are concerned, one may mention the EMBOSS software package, which is obtained from the European Molecular Biology Laboratory and installed locally on the user machine. Several other useful programs are available as services. In these cases the user submits her data to a remote computer situated in the laboratory of the program developer. The data is processed and the results returned to the user. Such access has become very convenient due to the Internet. A vivid example of the utility of this approach is the group of sequence alignment programs under the name BLAST, which has become an icon of bioinformatics.

1.2.3 Computerised databases and database management systems

A database is a collection of data that have some common element or theme. In this sense a dictionary is a database, as is an encyclopaedia. Computerised databases, however, are more than just a simple transfer of such a database on to a computer system. They offer facilities and utilities that were simply not possible otherwise. Some of these are listed below.

- A computerised database is easy to search in order to obtain the required information. Of course, the degree of ease depends on the particular type of arrangement of the data.
- Redundancy of the data can be reduced. It is usually sufficient to store a piece of data only once in the database. This also avoids inconsistencies in the data, since any changes to the data need not be carried out at several places in the database.
- The data can be shared more easily. Especially, given current day computer technology, a database may be accessed by several users simultaneously.
- The data can be authenticated and standards can be enforced more easily.

As mentioned earlier, the large amounts of biological data that are being generated make it imperative to use computers to store, retrieve and analyse them. Computerised database systems are therefore employed. The arrangement of the data in such systems is based on database models, such as the hierarchical model or the network model or the relational model. A hierarchical model is conceptually the simplest one (Figure 1.7).

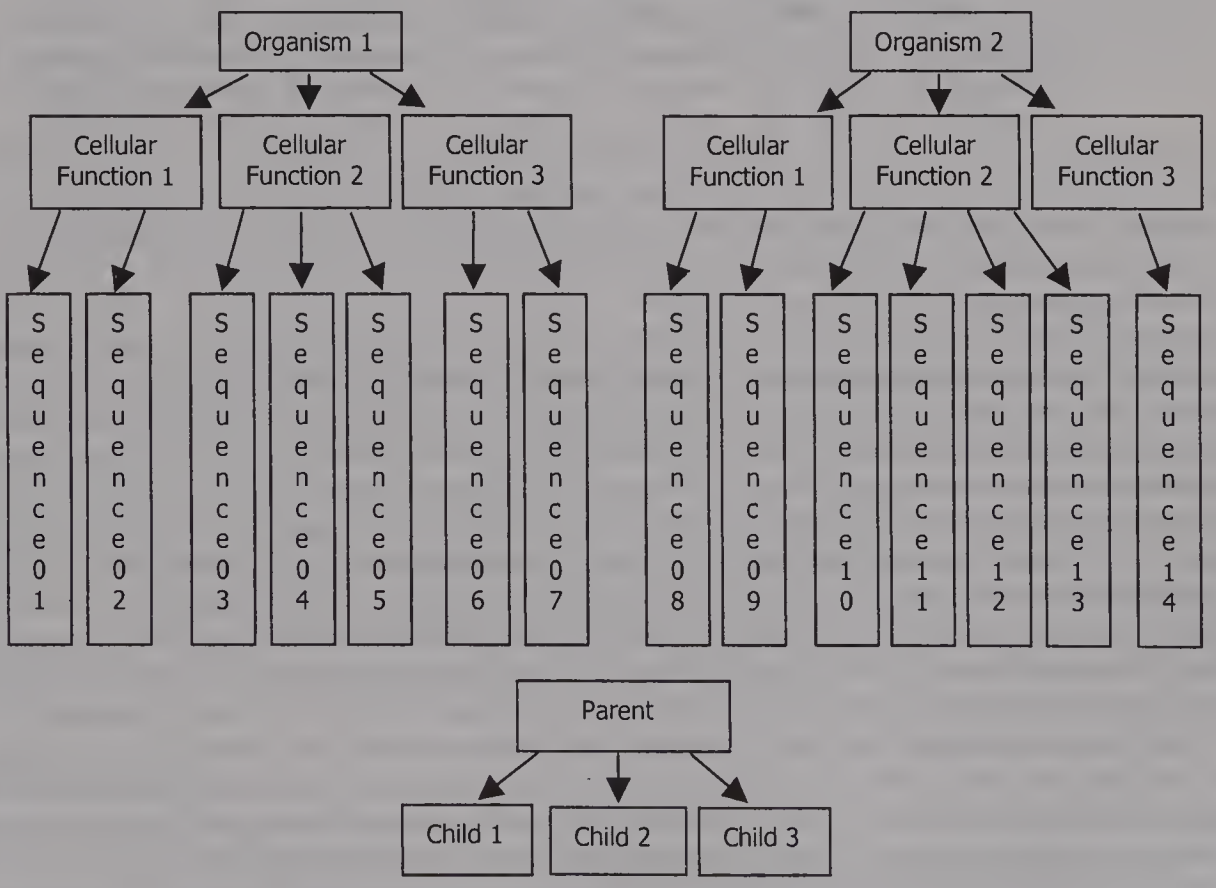


Figure 1.7a A sequence database arranged on the hierarchical model. The boxes indicate data or categories of data, and the arrows indicate their relationships. The lower diagram indicates the relationship between 'parent' data and 'child' data.

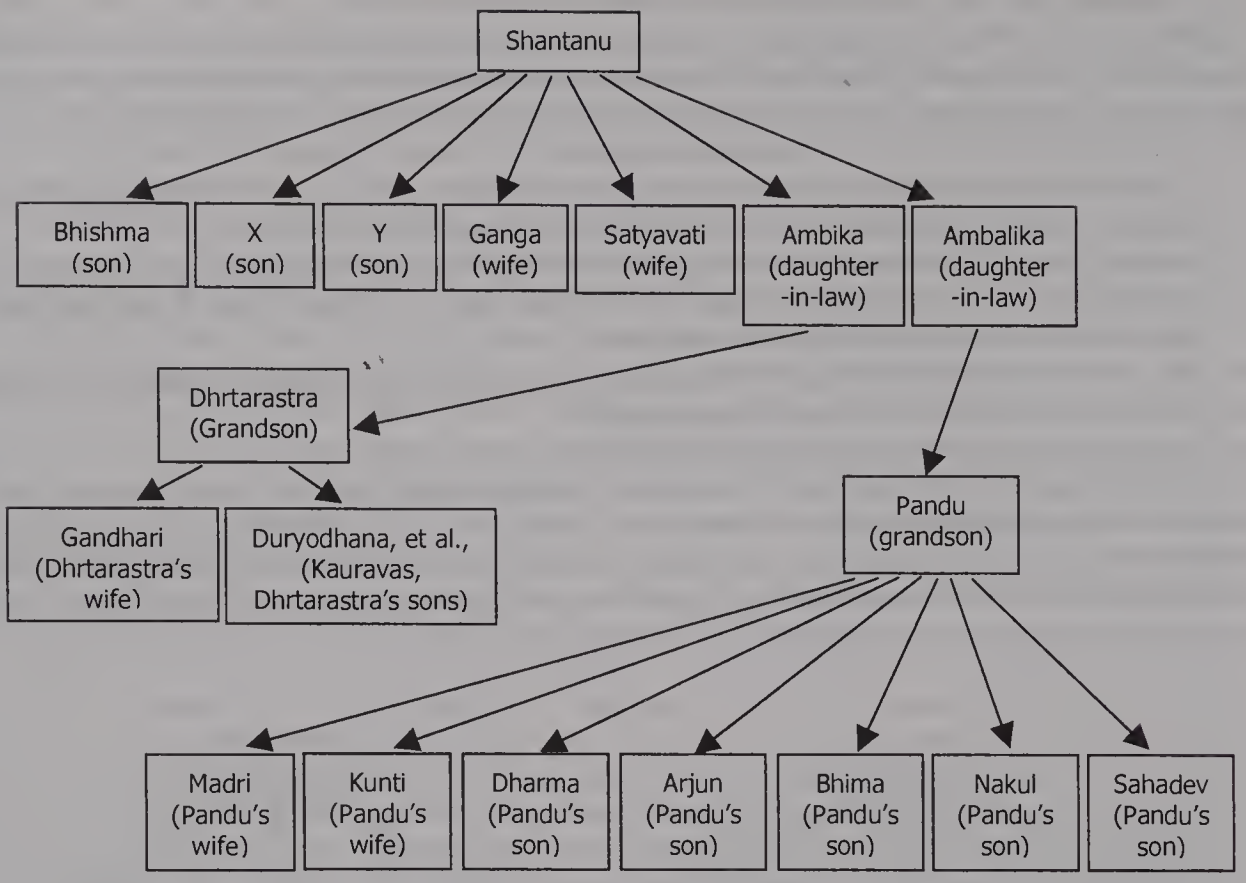


Figure 1.7b The principal characters in the *Mahabharata* arranged in a hierarchical structure, starting from Shantanu.

Many biological databases, such as the PDB, use this model. Searching through such a database is extremely fast for one particular way of doing it. Thus in the PDB, which is a database of protein structures, any particular structure can be retrieved easily if we search by its PDB code, since the data are arranged in alphabetical order of the code. However most uses of the data would require other ways of searching, such as, for example, by the molecular weight of the protein, or by the author's name. In such cases, the hierarchical arrangement of data is not the best one. The network model of a database (Figure 1.8) is better. Here pointers that can be read by the software are placed in the data to indicate many different relationships. Thus one may have pointers that point to all the structures solved by a particular scientist, making it easy to prepare such a list. In addition there may be other pointers, such as to all the structures that fall in a particular molecular weight class. However, again only a limited number of such relationships can be networked before the database structure becomes overloaded.

The best and nowadays the most commonly used database model is the relational model (Figure 1.9). The data are all placed in several tables, each one constructed specifically according to the relationships between the data. It therefore becomes equally easy to retrieve and arrange the data according to any relationship. If a table for a particular relationship does not alrcady exist, a new table is easily constructed. Such a relational database management system (or RDBMS) is constructed according to a set of twelve rules first stated by E.F. Codd. As the name suggest, the pride of place in a relational database is given to the relations between the data. The word relation, here, is used as a mathematical concept, which in ordinary language is a table of data. Thus a relation may consist of two

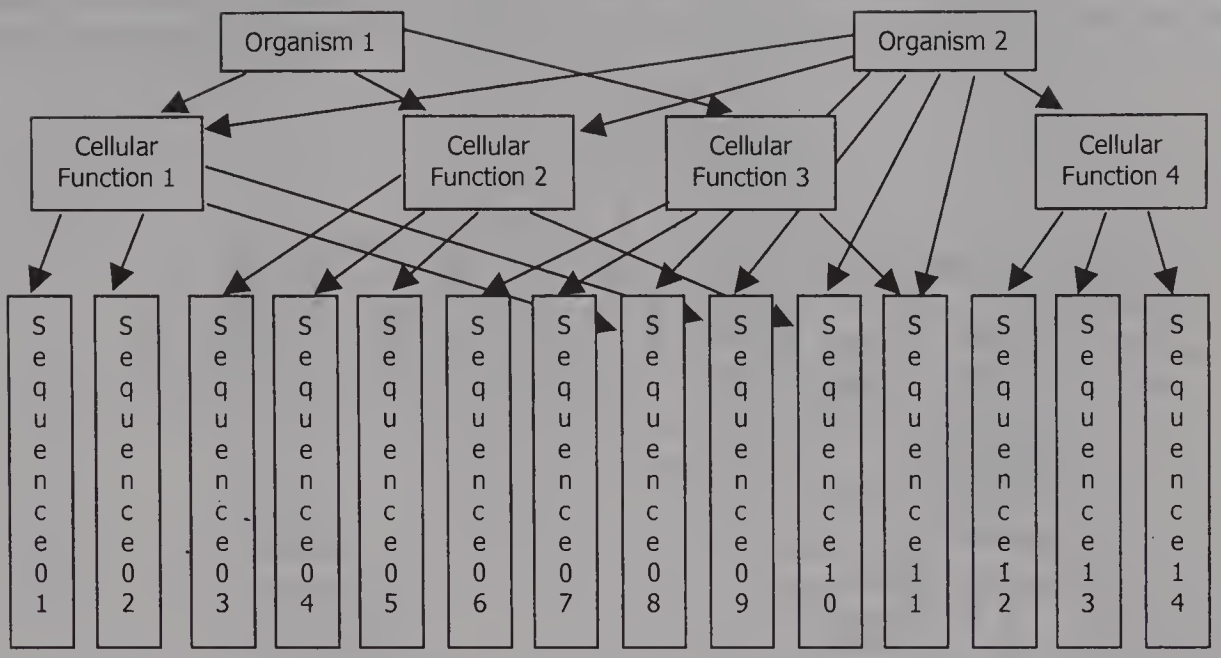


Figure 1.8a A sequence database arranged on the network model. As in Figure 1.7a the boxes indicate data or categories of data, and the arrows indicate their relationships. Note that now there is only one non-redundant set of data corresponding to 'cellular function', with relationships to both organisms.

or more columns. Each column has a column-name. The column-names are called the attributes of the table. Each row of the table is called a tuple and specifies the connections between a set of data. A more detailed explanation of the concepts underlying an RDBMS will not be attempted here. However even this elementary description will make clear the enormous utility of the relational method of arranging the data. Apart from concepts such as minimizing redundancy, maximizing data integrity and so on, it is evident that almost any question regarding the data may be answered with equal ease. In more formal words, *querying* such a database is easy. Among the several commercial relational database packages are Oracle, Sybase, etc. These packages come with not only the set of rules on how to arrange the data, but also software that will allow the user to rearrange any data in the relational format. Also included in the package is software to create reports, software to manage the data, i.e. to update, insert or delete data as required, and software to create user interfaces that make it easy to use a particular implementation of the database. An important part of such any such RDBMS is the query language, or the software that allows all the questions to the framed. Many of the biological databases are implemented as relational models, though this may not always be apparent, since the query language acts as an interface to retrieve the required data in the required format. With the introduction of GUIs the distance between the user and the data is increased one further step.

1.2.4 The Internet

It is probably true (and awe-inspiring) to say that the Internet, with the World Wide Web (the hypertext and hyperlinks based data presentation system), is a repository of MOST, if not ALL of human knowledge! Even if we consider only computation and computer technology, no description of their present-day state is complete without a discussion of the Internet. To many scientists, in fact, the Internet *is* the computer system, with the machine on their desks being merely a way of connecting up

to it. The paradigm of computation is now shifting from one of a large independent machine 'crunching' numbers at blinding speeds to one where millions of computers connected through the

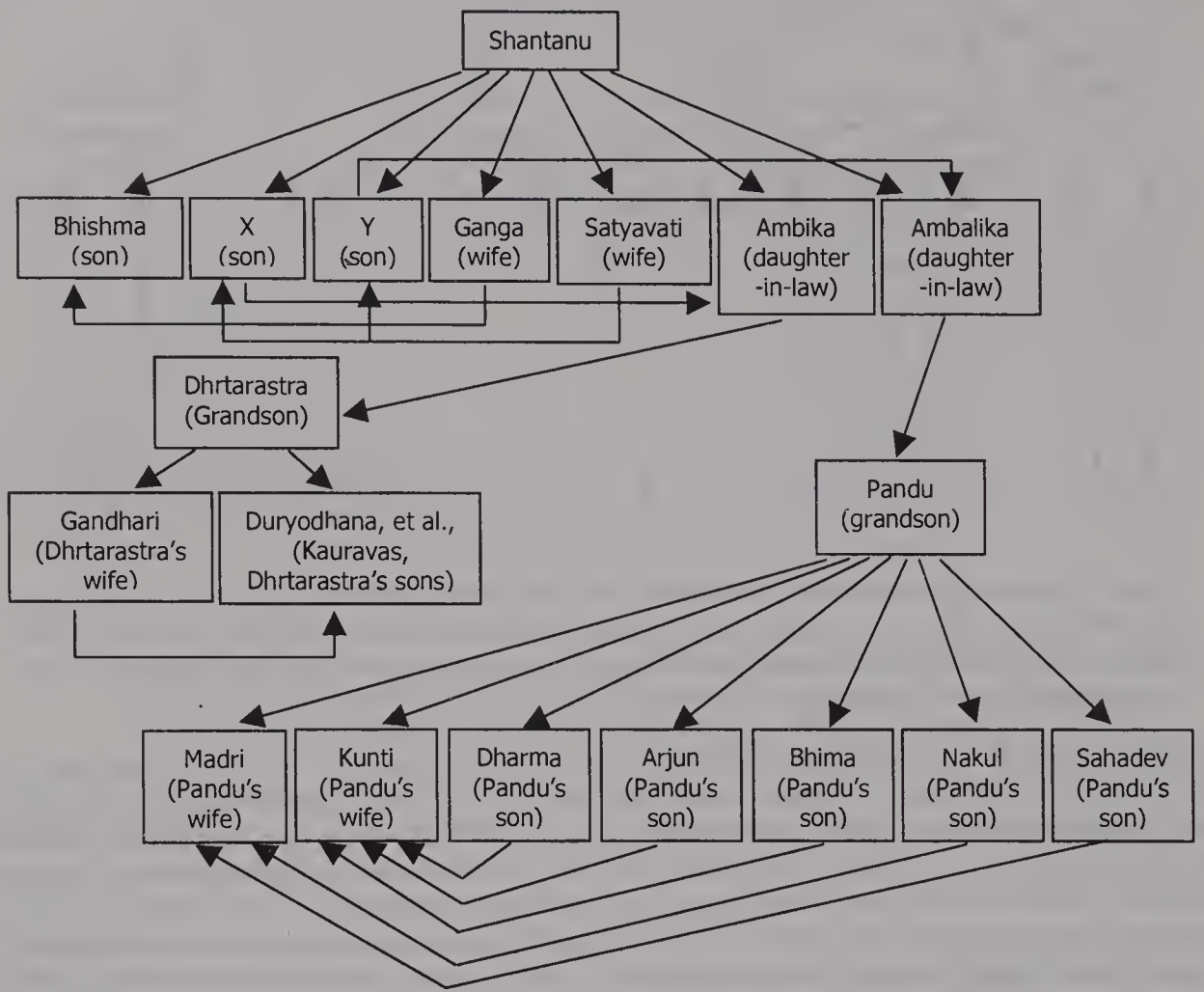


Figure 1.8b The principal characters in the *Mahabharata* arranged in a network structure.

Internet are all made to work on the same problem. The Internet is therefore more than just computers all over the world exchanging messages and data with each other. It can be made to behave like a single mammoth computer and also seem like one to the user. This idea of computation has been especially useful in the context of bioinformatics. First and foremost, the problem of distributing massive quantities of data is now converted into the much simpler one of distributing access to the data, since, with the Internet, its actual physical location is immaterial. For example, in the early days of GenBank, the DNA sequence database, the sequences were distributed on floppy disks. At 330 kilobytes of data per disk, a database containing about 10 million bases (as in the late 1980s) required more than 30 floppy disks. Today the number of bases in GenBank is of the order of 100 billion. Distribution even on CDs with a capacity of about 650 MB each is cumbersome. Also considering that the updating of the database is on a daily basis and that the rate of increase in the data is large, it is clear that without the Internet only a few scientists in a few labs would have had access to the sequence data. Most of the important databases required in bioinformatics are available over the Internet in the public domain. In addition there are many commercial value-added databases that are also accessible

Sl. No.	Identity code	Organism	Function	Sequence
1	AAA111	Organism 1	Function 1	Sequence 1
2	FFR014	Organism 1	Function 1	Sequence 2
3	ABC111	Organism 1	Function 2	Sequence 3
4	ABB111	Organism 1	Function 2	Sequence 4
5	BAA113	Organism 1	Function 2	Sequence 5
6	AAA316	Organism 1	Function 3	Sequence 6
7	XYZ117	Organism 1	Function 3	Sequence 7
8	AXA118	Organism 2	Function 1	Sequence 8
9	BBB312	Organism 2	Function 1	Sequence 9
10	TGH763	Organism 2	Function 2	Sequence 10
11	YER608	Organism 2	Function 3	Sequence 11
12	HSE005	Organism 2	Function 4	Sequence 12
13	GHY997	Organism 2	Function 4	Sequence 13
14	YHT249	Organism 2	Function 4	Sequence 14

Figure 1.9a The sequence database in Figure 1.8, now arranged on the relational model. Only one table is shown, though in fact there would be many such tables.

Sl. No.	Name	Son
1	Shantanu	Bhishma
2	Shantanu	X
3	Shantanu	Y
4	Pandu	Dharma
5	Pandu	Arjun
6	Pandu	Bhima
7	Pandu	Nakhula
8	Pandu	Sahadeva
9	Kunti	Dharma
10	Kunti	Arjun
11	Kunti	Bhima
12	Madri	Nakhula
13	Madri	Sahadeva

Sl. No.	Name	Wife
1	Shantanu	Ganga
2	Shantanu	Satyavati
3	X	Ambika
4	Y	Ambalika
5	Dhrtarastra	Gandhari
6	Pandu	Madri
7	Pandu	Kunti

Figure 1.9b The principal characters in the *Mahabaratha*, now arranged on the relational model. Only two of the relations are shown, and that incompletely.

through the Internet, but for a fee. Perhaps the best-known example of this is the human genome database made available by the company Celera Genomics[4].

The second important utility of the Internet in bioinformatics is as a medium through which services are delivered. The best-known example of this is the BLAST server maintained by the NCBI,

[4] As of May 2005, however, Celera has decided that a commercial genome database is not a viable business model. It has closed these operations and has turned over all its data to the public databases.

a US government agency. This is accessible free of charge to researchers 24 hours a day all over the world. Such access obviates the need to set up massive computational resources locally at each user centre. There are such Application Service Providers (or ASPs) in all areas of bioinformatics including sequence and structure retrieval and alignment. Most of these are in the public domain and usually no fee is charged for the service, but there are now also many commercial ASPs. The latter usually offer more comprehensive and reliable services, including sometimes access to privately owned databases. Academics, who have to consider costs, would tend to use public domain data and software, while those for whom reliability rather than innovation is important, such as for commercial users, tend to patronize the commercial databases, software and services.

1.3 An Introduction to Bioinformatics and Sequence Analyses

1.3.1 What is bioinformatics?

As mentioned at the beginning of the chapter, the word 'bioinformatics' has come to mean different things to different people. It was originally coined in the context of biological sequence analysis. When it was applied to the analysis of biomolecular structures, an additional word was prefixed to give rise to the topic of 'structural bioinformatics'. Currently the prefix 'structural' appears overly specialized and the title 'bioinformatics' by itself also encompasses the study of structures. In fact the word now has a very broad definition and can be applied to any study of biological systems using computer technology or science. The following are some examples

- All the text above and indeed this entire book have implicitly assumed the primacy of DNA and protein sequence analyses in any discussion of bioinformatics.
- The analysis of the three dimensional structures of biological molecules is a part of bioinformatics.
- Many of the techniques used in obtaining sequences and structures fall well within the ambit of bioinformatics. This includes the methods used, for example, in shotgun sequencing or in protein structure prediction.
- The construction and maintenance of computerised databases of all types of biological data is also bioinformatics.
- The analysis of gene expression profiles, especially when based on DNA chip technology, is bioinformatics.
- Analyses of medically important data such as single nucleotide polymorphisms or restriction fragment length polymorphisms (or DNA fingerprinting) use the methods and technologies of bioinformatics.
- Bioinformatics includes computer simulation of individual metabolic processes, as well as the more ambitious simulation of a whole cell or even a whole organism
- Many of the processes involved in the development of new drugs, such as lead discovery, lead optimisation through molecular modelling, design and analysis of the laboratory and clinical trials, are all considered a part of bioinformatics.
- On an even more liberal interpretation, bioinformatics may also mean construction, maintenance, use and analyses of databases not directly related to molecular biology, such as environmental data, biodiversity data, data on environmental or other toxicology, etc.

In this book we will deal mainly with the primary uses of bioinformatics in molecular biology, namely analyses of the sequences and structures of biological molecules. In this definition, bioinformatics may be broken down into certain elemental tasks that we describe in the next section.

1.3.2 Elemental tasks of bioinformatics analyses

The analyses of DNA, RNA and protein sequences and structures may generally be broken down into the following the elemental tasks.

- Searching for patterns within a sequence
- Obtaining statistical information on a sequence
- Searching for similarities between two sequences, or performing sequence alignment for a pair of sequences
- Searching for similarities among many sequences, or performing multiple sequence alignment
- Constructing phylogenetic trees based on sequences
- Predicting and analysing the secondary structures on basis of the sequence
- Predicting and analysing tertiary structure and folding for protein and RNA sequences

The first task, i.e. searching for patterns, is the one that is the most difficult, as well as the one most often required. It is becoming easier and cheaper to obtain the sequence of the genome of almost any organism. But to annotate it so as to get a comprehensive picture of the function of every bit of it, is not as yet an automatic task. Consider the different patterns to be found in a DNA sequence. Even at the beginning, in the sequencing project itself, we need to be able to identify overlaps among the fragments of the genome sequenced, in order to build contiguous sequences. Also we need to identify restriction sites and design primers for PCR amplification. Once the sequence is obtained and validated, coding regions or genes need to be identified along with various transcription signals such as promoters, enhancers, etc. Such gene annotation includes the identification of exons and introns. Gene expression levels may be predicted from promoter patterns and from the patterns of codon usage in the gene. We need to identify ribosomal RNA or tRNA genes, ribosomal binding sites on the mRNA and nucleosome positioning sequences. Several other patterns, sometimes with unknown function also need to be identified such as simple repeating sequences and inverted repeats. In the case of protein sequences, we need to look for sequence motifs connected to the secondary and tertiary structure and to the function. More subtle signals such as post-translational modification (e.g. glycosylation/ phosphorylation) sites and active site motifs (e.g. catalytic triad) need also to be identified.

The second task in the list above relates to the statistical information on a single sequence, such as the base or amino acid composition. Also, codon usage statistics vary from species to species and are required especially for finding genes. Other statistical signals in DNA sequences include CpG islands, polypurine-polypyrimidine stretches, alternating sequences, etc.

The third elemental task is searching for sequence similarities. This is one of most common tasks performed by bioinformaticians. (A bioinformatician is a practitioner of bioinformatics.) Sequence comparison and alignment programs such as BLAST and FASTA are among the most frequently used for this purpose. The discovery of a similarity in the sequence does not automatically imply biological homology between the sequences or between the organisms from which the sequences were obtained. Other information and algorithms are required to verify the relationship. This kind of analysis is used often for functional annotation of new sequences.

The fourth task in the list, multiple sequence comparison and alignment, is also very important for functional annotation. Further, it usually precedes the fifth task, viz. phylogenetic analysis, which explores relationships between the sequences and therefore between the parent organisms. The analysis helps to identify possible evolutionary relationships among the organisms.

The last two tasks mentioned in the list relate to the structures of the molecules. One of the major principles of science is that *structure implies function.* In other words, to understand completely the function of any physical system, in this case the biologically active molecule, it is necessary to know its structure. In the case of proteins, the single polypeptide chain takes up different three-dimensional arrangements such as alpha helices and beta strands, according to sequence. The next or tertiary level of structure refers to the arrangement of these secondary structures in space. The final three-

dimensional structure of the protein is its full functional form. RNA molecules, too, have secondary and tertiary structures, which bestow the correct function on them. The double stranded DNA molecule can adopt one of many different types of double-helical structures, depending on sequence. This too is related to the function of DNA. Within the ambit of structure analyses, we also include molecular modelling, docking, *in silico* mutation analyses and other tasks required in topics such as drug discovery or protein engineering.

The list of tasks given above corresponds to sequence and structure analysis. Other areas of bioinformatics have other associated tasks. For example, to compare gene expression profiles between, say, a normal and a diseased cell, it is necessary to compare the patterns of fluorescence on two different DNA chips. Such comparisons of two-dimensional maps require very different algorithms. Similarly analyses of restriction fragment length polymorphisms or single nucleotide polymorphisms are different from sequence analyses.

1.3.3 An overview of the rest of the book

The next chapter, Chapter 2, deals with the data types and databases used in modern molecular biology. As already mentioned, apart from sequences and structures, this includes the large amounts of textual and numerical data relating to the characteristics of the biological molecules such as molecular weight or tissue specificity, expression profiles obtained through DNA chip technology or 2-D gels, genome maps, metabolic pathways, chemical information on ligands, inhibitors and other small molecules, and so on. Attention is focussed on sequences and structures, and the chapter contains descriptions of databases of these. Data formats used in these databases are described along with discussion on access to them.

The third chapter discusses general algorithms for comparing strings of symbols. It introduces and explains several important concepts like dynamic programming and the use of different types of matrices. At the heart of this chapter is a detailed working out of the Needleman and Wunsch algorithm for sequence comparisons. Others used for the same purpose, such as the hashing method and the Smith-Waterman method are also dealt with comprehensively. The chapter includes a description of some software commonly used in sequence comparisons.

The fourth chapter clubs together a few related topics, though they do not always follow one from the other. It begins with a description of techniques of multiple sequence analysis. It then discusses substitution matrices especially the PAM and the BLOSUM matrices. This is followed by a detailed description of methods used to construct phylogenetic trees. Both substitution matrices and phylogenetic trees are often obtained by application of the techniques of multiple sequence analyses.

The fifth chapter deals with methods of structural analyses. It begins with brief descriptions of the experimental techniques of X-ray crystallography and NMR that are used to obtain the three-dimensional structures of molecules. The emphasis is to enable a non-specialist to appreciate and judge the biological relevance of any particular structure determination. This is followed by descriptions of methods to analyse the structures geometrically, as well as to compare structures both by conventional superposition techniques and by dynamic programming methods.

Chapter 6 focuses on protein and DNA sequences and methods specific to their analyses. Protein sequence motifs are described with examples and methods. Statistical properties of protein sequences such as amino acid composition and hydropathicity plots are discussed. It then discusses the analyses of nucleic acid sequences and the patterns and motifs that they contain. In particular it deals with the identification of coding sequences.

The following chapter deals with protein structures. The prediction of the secondary and tertiary structure of proteins from their sequence alone is a problem that is not completely solved. However substantial progress has been made and, though the topic deserves a whole book or maybe even several books all for itself, it is discussed here briefly in a few pages.

The final chapter (the eighth) deals with nucleic acid structure, specifically RNA secondary and tertiary structure and DNA secondary structure. It presents worked out examples of the Nussinov algorithm for RNA secondary structure prediction and also discusses other algorithms available. The chapter continues with the description of folding methods to obtain the three-dimensional structure of RNA. The final portion of the chapter is given over to DNA secondary structure and deals with polymorphism in DNA structure, DNA bending and other sequence-specific aspects of the double helix.

1.4 Summary

- The word bioinformatics has several slightly different meanings, but primarily means the analyses of nucleic acid and protein sequences and structures.
- The reductionist approach to biology seeks to divide a biological system into several subsystems, understand the subsystems in detail, and then put them together to derive an understanding of the entire system.
- Information pathways are important in biological systems.
- The Central Dogma of molecular biology states that, within the cell, information is transferred from DNA to RNA to protein, and never in the reverse direction.
- The most important biological information, namely DNA, RNA and protein sequences, may be represented as a string of symbols, i.e. in digital form.
- Genetic information is stored as the sequence of nucleotides in DNA.
- During replication this sequence of nucleotides is faithfully copied into the daughter cells.
- During the process of transcription, the information is transferred to mRNA, again as a string of nucleotides.
- The message in the mRNA is translated into the sequence of amino acids in the protein. The polypeptide chain that is formed in this process folds automatically into its functional three-dimensional structure.
- The amount of information in biological systems is very large. For example in humans, the size of the genome is about a few billion nucleotides. Computers and information technology is required to make sense of all this information
- Being based on the manipulation of digital data, computer technology is well suited to deal with DNA and protein sequences
- The Internet has made it not just convenient, but at all possible to access and analyse the large amounts of data generated by the genome sequencing projects.
- Bioinformatics deals with a variety of subjects, but in this book we restrict the meaning to analysis of DNA and protein sequences and structures.
- Bioinformatics, in this context, may be split up into some elemental tasks that include pattern recognition, sequence and structure comparisons, statistical analysis of sequences, and constructing phylogenetic trees.

2

Data Types and Databases in Molecular Biology

Bioinformatics is a rapidly developing field, and it is quite likely that some of the information contained in this chapter is outdated by the time it reaches the reader. This change however is most likely to affect the databases, of which there will probably be many more, and more user-friendly. The major types of data in use, however, are not likely to change to any great extent. Nevertheless a textbook like this is of no use if there is any uncertainty in what is written. Therefore, while every effort is made to be current and include all the recent developments, databases are dealt with as if they were finished products, and the current status with respect to content, format and access is described as if this would be the same when the reader actually tries to access them. This is not a serious problem, since changes in the databases are usually evolutionary, rather than revolutionary. The format of a database and the information it contains does not change so much that it becomes unrecognisable within a few years, and most of the descriptions given here would still be valid for some time. One must however bear in mind that many of the databases, especially the more specialised ones, often depend very much on the situation and interests of their authors. Sometimes if the author takes up a different professional position or starts following a different scientific interest the database she initially authored would first of all stop being updated, and then very soon fall into disuse and become obsolete. This hazard however does not attach to the larger, more general, well-funded and professionally managed databases such as GenBank.

In this chapter databases related to sequences and structures are treated in the greatest detail, though all major biological data types have their own databases. There are of course also databases of databases, such as the Molecular Biology Database Collection at http://www.nar.oupjournals.org, or the one at the National Center for Biotechnology Information, USA (http://www.ncbi.nlm.nih.gov). Also, given that most, if not all, of the databases are available over the Internet, by following the hyperlinks in any one database one can access all the others, one after the other. In this sense, the Internet itself is the largest 'database of databases', except that it does not limit itself to databases in molecular biology, but covers all of human knowledge.

Figure 2.1 is a network diagram representing some of the databases available and the interrelationships. Pride of place here is given to the nucleic acid and protein sequence and structure databases. This is an arbitrary choice, but to a certain extent reflects the current state of the art. With advancing knowledge, no doubt, other data will become more important, such as, for example, proteome data. Again the relationships shown are only the most obvious ones. These are the logical or biological relationships, and not the actual physical connections. (On the Internet, virtually everything

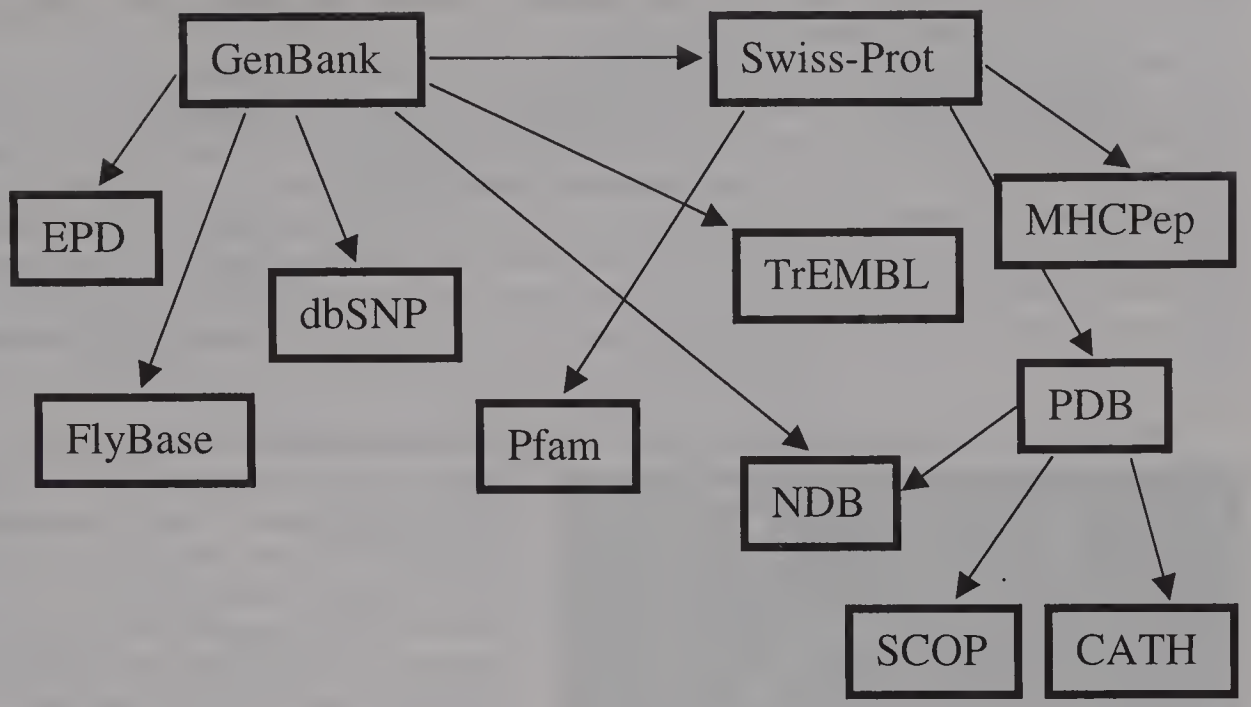


Figure 2.1 A portion of the network of molecular biology databases available.

is linked to everything else.) Clearly even these relationships are bound to change. In fact, one may define the discovery of new relationships as one of the major tasks of bioinformatics.

2.1 Data Types in Molecular Biology

Sequences and structures are dealt with in later sections of their own. In this section we describe the other types of data used in molecular biology, as well the relevant databases.

2.1.1 Gene expression data

After obtaining the genome sequence, the next major step in the programme to understand the life of an organism is to obtain the gene expression profiles. Such studies may be limited to a single gene, or the genes that constitute a single operon. But with the availability of DNA microarrays, it is possible now to obtain the expression profiles for entire genomes. Expression data may be represented in different ways. The GEO or the Gene Expression Omnibus at the NCBI site is a database for expression data obtained using a variety of methods including gene chips and Serial Analysis of Gene Expression or SAGE. As such it links up a variety of data types.

Single gene profiles are very often represented as a table or a graph giving a time series corresponding to the expression level of the gene. Thus, one may follow the course of expression of any particular gene as a function of, say, the evolution of the organism under conditions of glucose limitation. The *Saccharomyces* Genome Database or the SGD (http://genome-www.stanford.edu/Saccharomyces) is an address where data regarding yeast genome expression may be accessed in this format. The SGD itself does not contain expression profiles in this representation.

However it provides software tools that search through its own data and through other databases and present the information in the format described above.

Another way of representing expression data for single genes is as a textual report of studies of reporter genes. This is the format used in WormBase (http://www.wormbase.org), containing expression data on the *C.elegans* genome. Numerical and textual reports and tables are also used in more precise ways. Thus the Mouse Gene Expression Database (GXD: http://www.informatics.jax.org) contains all types of expression data and comes with browser software that enables the user to obtain answers to questions such as 'In which part of the anatomy and at what periods in the developmental cycle has a specified gene been detected?' or 'What genes have been detected by a specified technique used to assay expression levels at a particular time of development?" GXD, like many other expression databases, also stores and accesses expression microarrays. These may be displayed directly as graphical images, or searched for specific information.

The Stanford Microarray Database (SMD: http://genome-www.stanford.edu/microarray) is a special repository of the microarray images. These images (Figure 2.2) are represented as a square or rectangular array of thousands of spots of varying intensities, with each spot representing one specific gene that is being expressed. The most important information at each spot is the ratio of its intensity in the experimental array to that in the control array. By using different coloured gene markers, it is possible to use the same microarray for both the experiment and the control, or for more than one experiment. In this case, depending on the expression level, the colour of the spot will vary. The images are indexed and annotated with the biological information, so that while viewing them on the computer screen, one may use the mouse to click on any portion of the image, i.e. on any particular spot, to obtain all the corresponding data. A single microarray contains up to 20,000 spots and each spot may have several tens of parameters associated with it. This adds up to more than a million data points for a single microarray. This information however is an essential part of the database, for the microarrays are not meaningful unless set in the appropriate biological context.

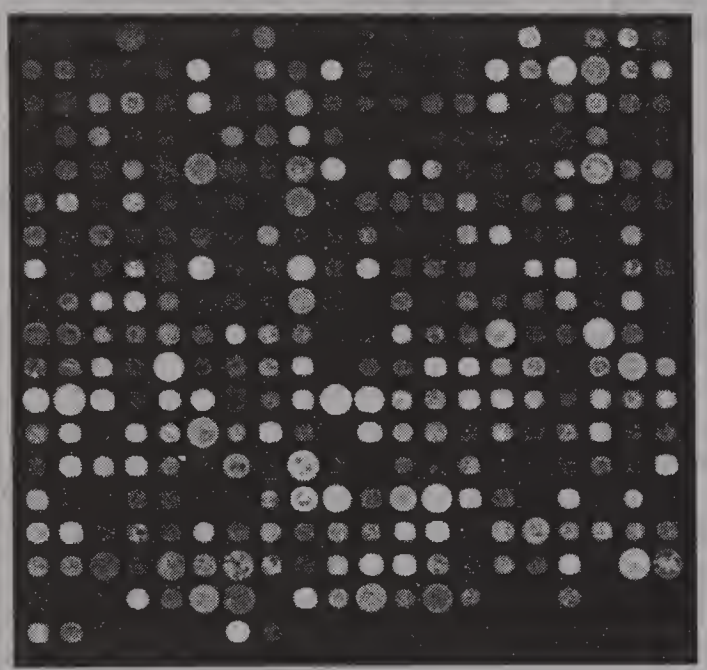


Figure 2.2 A small portion (1/16) of one microarray retrieved from SMD. The experiment is salicylic acid-dependent coupled regulation of cell growth and cell death

2.1.2 Metabolic pathways and molecular interactions

Metabolic pathways assume great importance in the context of genome annotation. The pathways are best represented as schematic diagrams featuring enzymes, their substrates, cofactors and products. The Kyoto Encyclopedia of Genes and Genomes (KEGG: http://www.genome.ad.jp/kegg) is one of the best-known databases for metabolic and regulatory pathways. Figure 2.3 shows one of the pathways present in KEGG. The numbers refers to the enzymes by their Enzyme Classification or E.C. number. The other molecules involved in the pathway are also shown, as are links to other pathways. The diagrams contain appropriate hyperlinks, and one may click on, for example, an enzyme to obtain all

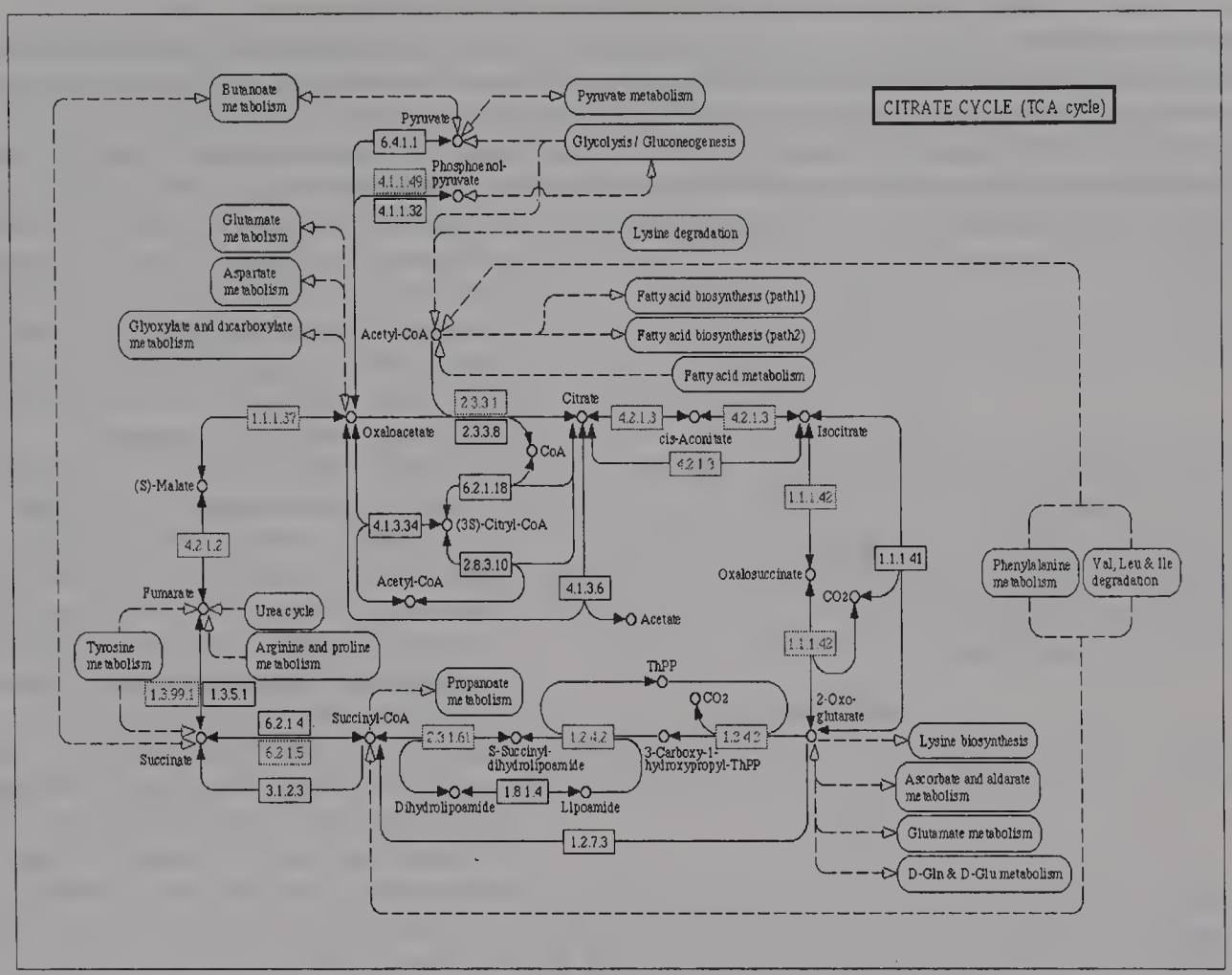


Figure 2.3 The TCA cycle as represented the KEGG database

information pertaining to it. Enzyme nomenclature and classifications are stored in the ENZYME database (http://www.expasy.ch/enzyme) and the links in KEGG ensure the correct cross-references.

A related data type is information on transcription regulation. This helps in studies of the transcriptome, i.e. the entire population of transcripts from the genome at a particular developmental stage. The information is itself normally numerical or textual and is usually to be found so in published literature or literature databases such as MEDLINE or PubMed. But it may be also represented diagrammatically for ease of understanding. RegulonDB is a database of transcriptional regulation and operon organisation for *E.coli* at http://www.cifn.unam.mx/Computational_Biology/regulondb. The database is accessed via a browser that represents the entire genome as a circle with the operons marked. Each operon can be magnified by 'zooming-in' to obtain complete details.

Another database of metabolic pathways focuses on the enzymes involved. The University of Minnesota Biocatalysis/Biodegradation Database (UM-BBD: http://umbbd.
ahc.umn.edu) lists out microbial catabolic enzymes. Each data page has links to various other enzyme databases, as well as to other pages each with a diagram of a reaction catalysed by the enzyme. The reaction may be used as a starting point to generate a microbial metabolic pathway, which is then displayed graphically.

A metabolic pathway may also be considered as a catalogue of interactions between biological molecules, particularly proteins. The Biomolecular Interaction Network Database (BIND: http://binddb.org) contains full descriptions of molecular interactions, molecular complexes and the

interaction pathways. An extensive data description method has been devised and implemented, such that the database can accept, store, search and analyse all the relevant types of data, including textual descriptions, images, three-dimensional structures, molecular interactions, molecular complexes, networks of interactions, physical locations within the cell, etc. There are now several sensitive methods to detect molecular interactions, such as the surface plasmon resonance technique and fluorescence resonance energy transfer (FRET). Information on post-translational modifications is also obtained easily and sensitively by mass spectrometry. BIND has been designed to accept the large volumes of data that are expected to arise out of these techniques and store them with proper annotations and links. Access to the data is through graphical interfaces that allow visualisation of the networks, from large molecular complexes to small molecules, with links to detailed information on all aspects of the network.

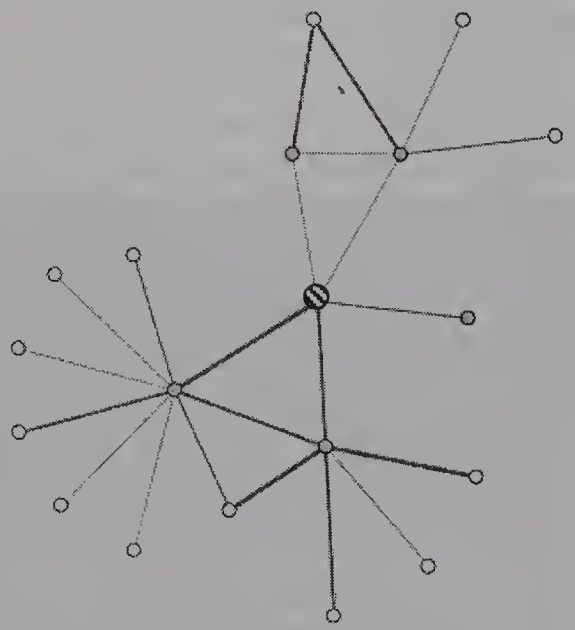


Figure 2.4 One of the sub-networks in the DIP database. This one is centred at the mitochondrial inner membrane protein, shown as a hatched circle. Other circles are proteins that interact with it. The colour of the line corresponds to the strength of the interaction.

Another database of protein-protein interactions is called the Database of Interacting Proteins (DIP: http://dip.doe-mbi.ucla.edu). This database documents experimentally determined protein-protein interactions. While the data are stored as tables of text and numbers, the visualisation of the data may be through appealing network graphs. Different level of interactions may be chosen for display and Figure 2.4 shows the network at a maximum of two interactions from the reference protein.

2.1.3 Mutations and polymorphisms

The study of mutations in genes and single nucleotide polymorphisms in the genome, and the medical consequences thereof is an important part of medical genetics. Databases of such mutations and polymorphisms are therefore essential in the study of human diseases. The Online Mendelian Inheritance in Man (OMIM: http://www.ncbi.nlm.nih.gov/Omim) is a computerised catalogue of human genes and the genetic mutations and changes that lead to clinical disorders. The database contains copious text that summarizes, for each mutation, the clinical features of the consequent disorder, the possible methods of diagnosis, possible therapies including gene therapy, any animal models that may help to understand the disorder further, and details regarding the history of the discovery of the mutation and its consequences. Apart from this text, OMIM also contains pictures, mainly maps of the chromosome, on which the locus of a gene responsible for a specific disorder is indicated. The database is extensively linked to the corresponding sequences, and to the alleles and variants that cause the disease. As with all databases at the NCBI site, this one too is linked to the literature on MEDLINE and PUBMED.

Single nucleotides polymorphisms, or SNPs, are substitutions or deletions/insertions or single nucleotides in the gene, which are associated with inheritable phenotypes, not necessarily diseases. SNPs could be responsible for such individual characteristics of an organism as eye-colour or resistance to disease, and they occur every 100 to 300 bases. dbSNP (http://www.ncbi.nlm.nih.gov/SNP) is a database of SNPs, defined, in spite of the name, both as

changes of a single base nucleotide, as well as short deletion and insertion polymorphisms. A very small portion of the database is given over to invariant regions of sequences, microsatellite repeats and named variants. The database records are in the form of tables giving the polymorphism name, the phenotype if known, the frequency of the allele, and other standard annotations such as the references. Links to the PubMed, sequences in genome project databases, and to other sequence databases within the NCBI site make it possible to obtained complete and detailed information about the SNP. The large part of the data, consists of information on humans, though some data on SNPs in other animals, particularly primates, are also available. dbSNP is also linked to other important SNP databases including the database on intragenic sequence polymorphisms called HGBASE (http://hgbase/cgr.ki.se) and the database maintained by the SNP consortium, a group of commercial organisations which includes drug companies such as Bayer AG and infotech companies such as Motorola.

The frequency of occurrence of different alleles of a gene in different ethnic populations or geographical locations is the subject matter of the database ALFRED (An Allele Frequency Database: http://alfred.med.yale.edu/alfred/). The data incorporated include the allele name, symbol and related information, the frequency of the allele in the population including the possible range of error in the measurement, the characteristics of the population including geographical location and the language spoken, the methods used for obtaining the information, locus of the allele including the chromosome and the chromosome band on which it is found, the corresponding references, and finally details regarding the contributors of the data.

The mutations and polymorphisms referred to above occur mostly in the genes, and the disorders arise when they are expressed to produce mutant proteins. Specific amino acid changes at specific positions in the protein chain may bring about structural and/or functional changes. The Protein Mutant Database (PMD: http://pmd.ddbj.nig.ac.jp) is a collection of information on mutant proteins that includes natural as well as artificial mutants, and random and site-directed ones, for all proteins except members of the globin and immunoglobulin families. Each entry in the database contains information on whether the mutation is natural or artificial, the name and other characteristics of the protein, the structure and stability of the mutant if known, and the related bibliography. The entries are linked to the sequence records and it is possible to retrieve a set of aligned protein sequences for a given entry, with the amino acids at the mutant positions alone highlighted.

Besides these, there are several databases of mutations, both in the genome as well as in the proteome, that refer to specific proteins, to a specific locus in the chromosome or to specific diseases. For example KinMutBase (http://www.uta.fi/imt/bioinfo/KinMutBase) is a database of mutations in protein kinases that cause diseases. Similarly the Human PAX2 Allelic Variant Database (http://www.hgu.mrc.ac.uk/Softdata/PAX2) contains tables and statistics of allelic variations at the PAX2 locus, associated with developmental anomalies in the eye, kidney and central nervous system. And the Asthma Gene Database (http://cooke.gsf.de/asthmagen) deals with asthma and allergy, and studies on the mutations that cause these disorders.

2.1.4 Miscellaneous – genetic maps, physicochemical properties...

- Genetic maps give the chromosomal locations of various genes. This data is most conveniently represented in the form of diagrams at various levels of resolution. One may thus start with a picture of the entire set of chromosomes of the organism. When a particular chromosome is selected, a larger picture of that chromosome alone appears on the screen, with the positions of the most important marker genes indicated. Selecting any one marker would then present a larger image of that region of the chromosome, with all the known genes marked. Selecting any one gene gives detailed information about that gene, including the sequence. Such representations of genetic maps are a part of many different databases, including some mentioned in the sections above, such

as the OMIM. There are databases specifically given over to genetic maps, such as the Genome Database (GDB: http://www.gdb.org), which has human genes and genomic maps, and INE (http://rgp.dna.affrc.go.jp/giot/INE.html), which contains genetic and physical maps and sequence data for the rice genome.

- Physicochemical properties and thermodynamic data for nucleic acid sequences are stored in the Thermodynamic Database for Nucleic Acids (NTDB: http://ntdb.chem.cuhk.edu.hk). Each entry contains information on the sequence, the structural type, enthalpy, entropy, Gibbs free energy, and melting temperature. Such a resource is also available for proteins and mutants, as well as for peptides and amino acids (Aaindex: http://www.genome.ad.jp/dbget)
- Images of two-dimensional polyacrylamide electrophoretic gels are stored in the database called SWISS-2DPAGE (http://www.expasy.ch/ch2d), along with annotations. This database is extensively used in proteome analysis to identify and classify expressed proteins.
- Data on restriction endonucleases and methylases are available at REBASE (http://rebase.neb.com/rebase/rebase.html). Not only the enzyme name and the restriction site sequence, but also details of the biochemistry are available here. Information on methylase sensitivity of the restriction enzymes is an added feature. Data on related proteins, such as nicking enzymes, control proteins and methyl-directed restriction enzymes are also included.
- Databases of nomenclature are available for various objects, such as for enzymes at http://www.expasy.ch/enzyme (called ENZYME).
- A database of glycan structures derived from glycoproteins is accessible at the address http://www.glycosuite.com. It contains information concerning glycan type, linkage, anomeric configuration, mass and composition, as well as about the source of the glycan, i.e. tissue and cell type, cell line, strain and disease state.
- The Tree of Life (http://phylogeny.arizona.edu/tree/phylogeny.html) is a large interconnected database that provides access to information on phylogeny and biodiversity. Rather more than a database, the Tree of Life is a network of databases, some of them mentioned in the list above, internetworked so as to provide information at various levels, from entire biosystems to molecular properties.

2.1.5 A note on the software used

It must be immediately clear to the reader that some very sophisticated database management techniques and software are required to store and manage the large amounts of data discussed above. In fact, most of the data, especially the numerical and textual data, are stored in well-designed relational database management systems, very often obtained from commercial sources, but sometimes developed in-house. An example of the former is SMD, which uses Oracle as its RDBMS. Another example is ALFRED, which is implemented using Microsoft Access. This is microcomputer-based relational database package, which may be queried using the SQL query language. The interface between the ALFRED database and the user on the World Wide Web has been built using Active Server Pages or ASP, which is a language or a set of protocols for this purpose. The user interface programs are written in Visual Basic scripts and database access is implemented using Open Database Connectivity (ODBC), all components of the Microsoft repertoire of DBMS utilities. These technical specifications are given here to indicate the levels of expertise needed to establish and maintain these databases.

An example of an RDBMS designed in-house for a specialized purpose occurs in the GID. This image database is implemented as a client-server system. When a web browser such as Netscape is used to access the database, a Java applet starts running on the user's computer, called the client. This applet acts as the interface between the client and the server, which is the system containing the images and the annotations. Another Java program runs on the server to receive the instructions given by the

client, such as searching through the database. The actual search is carried out by yet another program, this time written in Perl. The results are then sent back, first to the manager program running on the server, and through that to the client applet. This brief description of the software system is not detailed or complete, but it should make clear that modern biology requires personnel well trained in computer science to manage the databases.

2.2 Sequence Databases

Nucleic acid and protein sequences and structures form the core raw material of Bioinformatics as defined in this book. Sequence and structure databases may be classified into two types, viz. primary or raw databases and secondary or derived databases. The former consist of just the sequences and structures, and the value addition, if any, is in the annotation and arrangement. Derived databases are usually subsets of the primary databases, and are constituted either to serve a specific subtopic in biology, such as cancer, or have some added value, for example multiple alignment of a set of sequences or protein domain structures grouped according to structural similarity. Secondary databases are greatly dependent on the algorithms used to make the arrangements as well as on the curator of the database. It is not surprising, therefore, that often there is more than one database dealing with the same biological information, for instance the relationship between protein sequence and structure, but yet have different points of view and different results.

In this section we first describe the primary sequence databases, both for nucleic acids and for proteins. In the second portion of the section we describe the derived databases of nucleotides and of amino acids. Structure databases are dealt with in the next section.

2.2.1 Primary nucleotide sequence repositories – GenBank, EMBL, DDBJ

These are the three chief databases that store and make available raw nucleic acid sequences. GenBank is physically located in the USA and is accessible through the NCBI portal over the Internet. EMBL (stands for European Molecular Biology Laboratory) is in UK, at the European Bioinformatics Institute, and DDBJ (DNA DataBank of Japan) is in Japan. Historically the three started out as separate databases, with independent means of collecting the data, and with different formats for arranging it. Now the three together form the International Nucleotide Sequence Database Collaboration. They have uniform (but not identical) formats and exchange data on a daily basis, with the result that it is sufficient to connect to any one of them in order to access the sequences present in all three. Here we will describe only one of the data formats, GenBank, in detail, assuming that the differences with the other two are insignificant. In particular, we will describe the implementation of this database at the NCBI site. The access to GenBank, as to all databases at NCBI, is through the Entrez search program. This front-end search interface allows a great variety of search options. More importantly, it enables searching more than one database at the same time, interpreting the search parameters in an appropriate way for each database. Thus if the user specifies the name of a protein and asks for a 'nucleotide' search, Entrez will look for the corresponding gene sequence. The keyword 'nucleotide' identifies GenBank, as well as other nucleic acid sequence databases. The search therefore occurs across different database formats. This means that the formats in which the data are stored in the databases are necessarily transparent to the user, who would otherwise have to change her query to suit each database. Nevertheless, it would be instructive to study the data types and identifiers used in GenBank. This is especially so for the following two reasons. One, the result of a search would usually be a sequence record arranged to display all the database information, and it is necessary to know what the structure of this record is. And two, users often prefer to download the entire database on to their own local computer systems. The data is made available at NCBI (and the other sites) for download as flat

files[5], and again it is necessary to know just what data is obtained through such a download. Figure 2.5 shows one result of a search for the keyword 'chymotrypsin' through the 'nucleotide' database at NCBI. The format of the data is given as plain text, and contains a lot of information besides the gene sequence of chymotrypsin. All the data are grouped under titles, which not only describe the information under that heading, but also serve to define keyword fields that can be used in search programs. Thus the word 'ACCESSION' defines a field containing unique identification numbers. The sequence and other information may be retrieved from the database simply by searching for a given accession number. Taking these field names in order, we have first of all the word 'LOCUS'. This is a GenBank title that names the sequence entry. Apart from the accession number, it also specifies the number of bases in the entry, the nucleic acid type, a codeword PRI that indicates that the sequence is from a primate, and the date on which the entry was made. PRI is one of 17 such words that are used to classify the data (Table 2.1). They were chosen in the early years of GenBank to fairly represent the data then available. Subsequently other codes such as EST were added to list. However their utility is now not very great, and most uses of the GenBank do not have anything to do with them. The next line of the file contains the definition of the entry, giving the name of the sequence. The unique accession number comes next, followed by a version number in case the entry has gone through more than one version. The next item is a list of specially defined keywords that used to index the entries (Note that in this particular example, no keywords have been specified). Next come a set of SOURCE records, which describe the organism from which sequence was extracted. The complete scientific classification is given. This is followed by publication details. In the beginning, sequences were

Table 2.1 GenBank Keywords

1.	PRI	-	primate sequences
2.	ROD	-	rodent sequences
3.	MAM	-	other mammalian sequences
4.	VRT	-	other vertebrate sequences
5.	INV	-	invertebrate sequences
6.	PLN	-	plant, fungal, and algal sequences
7.	BCT	-	bacterial sequences
8.	VRL	-	viral sequences
9.	PHG	-	bacteriophage sequences
10.	SYN	-	synthetic sequences
11.	UNA	-	unannotated sequences
12.	EST	-	EST sequences (expressed sequence tags)
13.	PAT	-	patent sequences
14.	STS	-	STS sequences (sequence tagged sites)
15.	GSS	-	GSS sequences (genome survey sequences)
16.	HTG	-	HTGS sequences (high throughput genomic sequences)
17.	HTC	-	HTC sequences (high throughput cDNA sequences)

[5] Flat files are collections of data and other information arranged so that any general text-processing program can read them. They consist only of the characters found on any computer keyboard (i.e. the ASCII set of characters) and are devoid of special fonts, bold characters, italics and so on.

```
LOCUS       NM_007272               894 bp    mRNA    linear   PRI 17-DEC-2004
DEFINITION  Homo sapiens chymotrypsin C (caldecrin) (CTRC), mRNA.
ACCESSION   NM_007272
VERSION     NM_007272.1  GI:11321627
KEYWORDS
SOURCE      Homo sapiens (human)
  ORGANISM  Homo sapiens
            Eukaryota; Metazoa; Chordata; Craniata; Vertebrata; Euteleostomi;
            Mammalia; Eutheria; Primates; Catarrhini; Hominidae; Homo.
REFERENCE   1  (bases 1 to 894)
  AUTHORS   Tomomura,A., Akiyama,M., Itoh,H., Yoshino,I., Tomomura,M.,
            Nishii,Y., Noikura,T. and Saheki,T.
  TITLE     Molecular cloning and expression of human caldecrin
  JOURNAL   FEBS Lett. 386 (1), 26-28 (1996)
  MEDLINE   96221265
   PUBMED   8635596
COMMENT     PROVISIONAL REFSEQ: This record has not yet been subject to final
            NCBI review. The reference sequence was derived from S82198.1.
FEATURES             Location/Qualifiers
     source          1..894
                     /organism="Homo sapiens"
                     /mol_type="mRNA"
                     /db_xref="taxon:9606"
                     /chromosome="1"
                     /map="1p36.21"
     gene            1..894
                     /gene="CTRC"
                     /note="synonym: CLCR"
                     /db_xref="GeneID:11330"
                     /db_xref="LocusID:11330"
                     /db_xref="MIM:601405"
     CDS             1..807
                     /gene="CTRC"
                     /note="caldecrin (serum calcium decreasing factor,
                     elastase IV); OTTHUMP00000044526;
                     go_function: trypsin activity [goid 0004295] [evidence
                     IEA];
                     . . . . .
                     /codon_start=1
                     /product="chymotrypsin C (caldecrin)"
                     /protein_id="NP_009203.1"
                     /db_xref="GI:11321628"
                     . . . . .
                     /translation="MLGITVLAALLACASTCGVPSFPPNLSARVVGGEDARPHSWPWQ
                     ISLQYLKNDTWRHTCGGTLIASNFVLTAAHCISNTRTYRVAVGKNNLEVEDEEGSLFV
                     GVDTIHVHKRWNALLLRNDIALIKLAEHVELSDTIQVACLPEKDSLLPKDYPCYVTGW
                     GRLWTNGPIADKLQQGLQPVVDHATCSRIDWWGFRVKKTMVCAGGDGVISACNGDSGG
                     PLNCQLENGSWEVFGIVSFGSRRGCNTRKKPVVYTRVSAYIDWINEKMQL"
ORIGIN
        1 atgttgggca tcactgtcct cgctgcgctc ttggcctgtg cctccacctg tggggtgccc
       61 agcttcccgc ccaacctatc cgcccgagtg gtgggaggag aggatgcccg gccccacagc
       . . . . .
      841 caacagcaat aaacttcctt ctcctcgggc cacctgaaaa aaaaaaaaaa aaaa
//
```

Figure 2.5 The output of a search of the nucleotide database in NCBI using the keyword 'chymotrypsin'. (Dots indicate portions of the output deleted for compactness.)

extracted from published literature, and painstakingly entered in the database. Each entry was therefore associated with a publication. Every REFERENCE field would thus contain the title of the article, the names of the authors, and the name, volume, page numbers and publication year of the journal in which the article appeared. With the vast explosion of sequence data, it is now no longer feasible to either print the sequences in the journals, nor to manually enter the sequences into the database. The entries are all electronic, straight from the laboratory to the database. The REFERENCE field now points to the publication containing the biological information obtained from the sequencing project. The FEATURES table that follows is one of the most important pieces of information accompanying the sequence. It is an annotation of the sequence and describes whatever the submitters know about the sequence. The features (for nucleic acid sequences) include coding regions, exons, introns, promoters, alternate splice patterns, mutations, variations, and a translation into a protein sequence, if it codes for one. Each feature may be accompanied by a cross-reference to another database. After the features table, a single line gives the base count statistics for the sequence. Finally comes the sequence itself. The sequence is typed in the lowercase, and for ease of reading, each line is divided into six columns of ten bases each. A single number on the left numbers the bases.

The above description does not cover all the fields used in GenBank, but only the more important ones. While this is one of the chief formats used to deliver the sequences to the user, a variety of other display and file formats are also available. Finally it is worth noting that the GenBank is in fact a relational database system, and the data are actually stored as tables, each column corresponding to a field descriptor, and each row corresponding to a sequence entry. In addition extensive indices of keywords are also a part of the database, making it extremely fast to search through the mountains of data.

2.2.2 Primary protein sequence repositories

While the three 'mother' databases described above allow access to both nucleic acids and protein sequences, there are a few databases given over solely to protein sequences. Among them the chief ones are the PIR-PSD or Protein Information Resource - Protein Sequence Database, at the NBRF (National Biomedical Research Foundation, USA), and the SWISS-PROT at the SBI (Swiss Biotechnology Institute), Switzerland.

The PIR-PSD is a collaborative endeavour between the PIR, the MIPS (Munich Information Centre for Protein Sequences, Germany) and the JIPID (Japan International Protein Information Database, Japan). This database grew out of the work of Margaret Dayhoff and her colleagues and collaborators at the NBRF. Beginning in the 1960's, before the advent of computerised databases, Dayhoff collected published protein sequences, and on the basis of alignments and sequence comparisons, classified and annotated the collection painstakingly by hand, and made it available to the scientific community in the form of the Atlas of Protein Sequence and Structure, a set of printed volumes issued periodically. (Dayhoff also made two other important contributions to the then nascent field of bioinformatics. The first was her extensive reworking of evolutionary trees on the basis of protein sequences, in particular the sequences of Cytochrome C. The other was the development of the PAM matrices that indicated which amino acid substitution in the sequence was harmless to the function of the protein, and which disastrous. Both these contributions will be discussed at length later on in the book.) The Atlas of Protein Sequence and Structure grew into the PIR-PSD, and continues the former's tradition of high-quality annotation, except that the classification is now carried out by well-validated automatic procedures. Also the database is now fully computerised and uses the Oracle object relational DBMS. The PIR-PSD is now a comprehensive, non-redundant, expertly annotated, fully classified and extensively cross-referenced protein sequence database in the public domain. It is available at http://pir.georgetown.edu/pirwww. A unique characteristic of the PIR-PSD is its classification of protein sequences based on the superfamily concept. Sequences in PIR-PSD are also

classified based on homology domains and sequence motifs. Homology domains may correspond to evolutionary building blocks, while sequence motifs represent functional sites or conserved regions. The classification approach allows a more complete understanding of sequence-function-structure relationships. Figure 2.6 shows the result of searching the PIR-PSD database using the keyword 'chymotrypsin'.

Protein Name Chymotrypsin C (Caldecrin)

Taxonomy
Danio rerio (zebra danio)
NCBI Taxon ID: 7955
Lineage: cellular organisms; Eukaryota; Fungi/Metazoa group; Metazoa; Eumetazoa; Bilateria; Coelomata; Deuterostomia; Chordata; Craniata; Vertebrata; Gnathostomata; Teleostomi; Euteleostomi; Actinopterygii; Actinopteri; Neopterygii; Teleostei; Elopocephala; Clupeocephala; Otocephala; Ostariophysi; Otophysi; Cypriniphysi; Cypriniformes; Cyprinoidea; Cyprinidae; Rasborinae; Danio

Source Organism Danio rerio (*Taxon ID:* 7955)

Bibliography View Bibliography information. Submit Bibliography.
PubMed: PMID:12477932

Sequence Database

Database	Protein ID	Accession	Taxon ID	Protein Name
TrEMBL	Q7SZ51	Q7SZ51	7955	Chymotrypsin C (Caldecrin)
GenPept	g32484299	AAH54136.1	7955	Chymotrypsin C (caldecrin)
RefSeq	g41054047	NP_956180	7955	chymotrypsin C (caldecrin)

Protein Sequence

```
MMKFVVLAVLVVGAYSCGLPTFPPIVTRVVGGVDVRPNSWPWQISLQYKSGSNWYHTCGG
SLIDKQWVLTAAHCISSSRTYRVFLGKHSLSQEENGSVAIGAGKIIVHEAWNSFTIRNDI
ALIKLETAVTIGDTITPACLPEAGYVLPHNAPCYVTGWGRLYTNGPLADILQQALLPVVD
HATCSKSDWWGSQVTTSMVCAGGDGVVAGCDGDSGGPLNCAGSDGAWEVHGIVSFGSGLS
CNYNKKPTVFTRVSAYSDWISKNMASY
```

Sequence Length 267

Figure 2.6 The output of a search of the PIR-PSD using the keyword 'chymotrypsin'.

The other well-known and extensively used protein sequence database is SWISS-PROT (http://www.expasy.ch/sprot). Like the PIR-PSD, this curated protein sequence database also provides a high level of annotation. The data in each entry can considered separately as core data and annotation. The core data consists of the sequence entered in the common single letter amino acid code, and the related references and bibliography. The taxonomy of the organism from which the sequence was obtained also forms part of this core information. The annotations contain information on the function or functions of the protein, post-translational modifications such as phosphorylation, acetylation, etc., functional or structural domains and sites, such as calcium binding regions, ATP-binding sites, zinc fingers, etc., known secondary structural features as for example alpha helix, beta sheet, etc., the quaternary structure of the protein, similarities to other proteins if any, and diseases that may be associated with deficiency of the protein or mutations. Any sequence conflicts or variants that may arise due to different authors publishing different sequences for the same protein, or due to mutations in different strains of an organism are also described as part of the annotations. This scheme

also works to reduce redundancy, with a given protein from a given organism having only one entry, as far possible. The data descriptions may be seen in easily readable full format in what is called a 'NiceProt' view of the data. In the actual database, however, the data are stored as shown in Figure 2.7.

```
ID   CLCR_HUMAN     STANDARD;      PRT;   268 AA.
AC   Q99895; O00765; Q9NUH5;
DT   16-OCT-2001 (Rel. 40, Created)
DE   Caldecrin precursor (EC 3.4.21.2) (Chymotrypsin C).
GN   Name=CTRC; Synonyms=CLCR;
OS   Homo sapiens (Human).
OC   Eukaryota; Metazoa; Chordata; Craniata; Vertebrata; Euteleostomi;
OC   Mammalia; Eutheria; Primates; Catarrhini; Hominidae; Homo.
OX   NCBI_TaxID=9606;
RN   [1]
RP   NUCLEOTIDE SEQUENCE, AND VARIANT TRP-80.
RC   TISSUE=Pancreas;
RX   MEDLINE=96221265; PubMed=8635596 [NCBI, ExPASy, EBI, Israel, Japan];
RA   Tomomura A., Akiyama M., Itoh H., Yoshino I., Tomomura M., Nishii Y.,
RA   Noikura T., Saheki T.;
RT   "Molecular cloning and expression of human caldecrin.";
RL   FEBS Lett. 386:26-28(1996).
CC   ---------------------------------------------------------------------
CC   This SWISS-PROT entry is copyright. It is produced through a collaboration
CC   between the Swiss Institute of Bioinformatics and the EMBL outstation -
CC   the European Bioinformatics Institute. There are no restrictions on its
CC   use by non-profit institutions as long as its content is in no way
CC   modified and this statement is not removed. Usage by and for commercial
CC   entities requires a license agreement (See http://www.isb-sib.ch/announce/
CC   or send an email to license@isb-sib.ch).
CC   ---------------------------------------------------------------------
DR   EMBL; S82198; AAB47104.2; ALT_SEQ. [EMBL / GenBank / DDBJ] [CoDingSequence]
DR   HSSP; P05805; 1PYT. [HSSP ENTRY / SWISS-3DIMAGE / PDB]
KW   Glycoprotein; Hydrolase; Polymorphism; Serine protease; Signal;
KW   Zymogen.
FT   SIGNAL        1     16       Potential.
SQ   SEQUENCE   268 AA;  29484 MW;  460BF33B4A96516F CRC64;
     MLGITVLAAL LACASSCGVP SFPPNLSARV VGGEDARPHS WPWQISLQYL KNDTWRHTCG
     GTLIASNFVL TAAHCISNTR TYRVAVGKNN LEVEDEEGSL FVGVDTIHVH KRWNALLLRN
     DIALIKLAEH VELSDTIQVA CLPEKDSLLP KDYPCYVTGW GRLWTNGPIA DKLQQGLQPV
     VDHATCSRID WWGFRVKKTM VCAGGDGVIS ACNGDSGGPL NCQLENGSWE VFGIVSFGSR
     RGCNTRKKPV VYTRVSAYID WINEKMQL
//
```

Figure 2.7 'Complete' view of the search results of Swiss-Prot with the keyword 'Chymotrypsin. (The view has been extensively edited.)

This closely follows the format of entries in EMBL nucleotide sequence database. Each line begins with a two-letter code that specifies the information to be found in that line, as well as the format of that line. The first one is the 'ID' line that identifies the entry. The subsequent line codes and the information each line contains are given in Table 2.2. The last of the two letter codes is SQ, which also begins sequence. The sequence lines do not have any two-letter code, but the first two columns are blank. The sequence is written in 6 groups of 10 amino acids each per line, a total of 60 amino acids per line. The entry is terminated by a double slash (//).

Both PIR-PSD and SWISS-PROT have software that enables the user to easily search through the database to obtain only the required information. SWISS-PROT has the SRS or the sequence retrieval system that searches also through the other relevant databases on the site, such as TrEMBL. In

Table 2.2 Line codes in SWISS-PROT database.

Code	Expansion	Remarks
ID	Identification	Occurs at the beginning of the entry. Contains a unique name for the entry, plus information on the status of the entry. If it has been checked and conforms to SWISS-PROT standards, it is called STANDARD.
AC	Accession numbers	This is a stable way of identifying the entry. The name may change but not the AC. If the line has more than one number, it means that the entry was constituted by merging other entries.
DT	Date	There are three dates corresponding to the creation date of the entry and modification dates of the sequence and the annotations respectively
DE	Description	Lines that start with this identifier contain general description about the sequence.
GN	Gene name	The name of the gene (or genes) that codes for the protein
OS,OG,OC	Organism name, Organelle, Organism classification	The name and taxonomy of the organism, and information regarding the organelle containing the gene, e.g. mitochondria or chloroplast, etc.
RN, RP, RC, RX, RA, RT, RL	Reference number, position, comments, cross- reference, authors, title and location	Bibliographic reference to the sequence. This includes information (following the code RP) on the extent of work carried out by the authors.
CC	Comments	These are free text comments that provide any relevant information pertaining to the entry.
DR	Database cross-reference	This line gives cross-references to other databases where information regarding this entry is also found. As for example to structural information for the protein in the PDB.
KW	Keywords	This line gives a list of keywords that can be used in indexes. Search programs very often simply go through such indices to identify required information.
FT	Features Table	These lines describe regions or sites of interest in the sequence, e.g. post-translational modifications, binding sites, enzyme active sites and local secondary structure.
SQ	Sequence header	This line indicates the beginning of the sequence data and gives a brief summary of its content.

addition users may download the entire database for use and manipulation on a local computer system, though of course this would require very large resources. As on December 2004, PIR-PSD contained 19,43,163 entries and SWISS-PROT contained 1,66,310 entries. The large difference in the number of entries between the two databases may be attributed to two causes. Firstly, SWISS-PROT does not include variants as different entries. Secondly SWISS-PROT includes only validated entries of protein sequences. Putative protein sequences translated from the DNA or mRNA sequence normally go to TrEMBL, until they are verified.

TrEMBL (for Translated EMBL) is a computer-annotated protein sequence database that is released as a supplement to SWISS-PROT. It contains the translations of all coding sequences present in the EMBL Nucleotide Sequence Database, which have not been fully validated. Thus it may contain sequences of proteins that are never expressed and never actually identified in the organisms. These sequences are themselves classified in two ways. SP-TrEMBL sequences are given a SWISS-PROT accession number and are expected to be validated and entered into the main database. REM-TrEMBL entries, on the other hand, may or may not be eventually supported as individual protein sequences by experiments and other analyses. These do not have accession numbers. The format of the TrEMBL entries follows that of SWISS-PROT, except for the changes mentioned above.

2.2.3 Derived or secondary databases of nucleotide sequences

Many of the secondary databases are simply sub-collections of sequences culled from one or the other of the primary databases such as GenBank or EMBL. There is also usually a great deal of value addition in terms of annotation, software, presentation of the information and the cross-references. There are other secondary databases that do not present sequences at all, but only information gathered from the sequence databases. Both these type of derived databases are discussed in the present section.

An example of the former type of database is the FlyBase or The Berkeley Drosophila Genome Project (http://www.fruitfly.org). A consortium of several institutions, mainly in the USA, has undertaken this project. The consortium sequenced the entire genome of the fruit fly *D. melanogaster* to a high degree of completeness and quality. The information obtained from the sequencing project is made available to the public after extensive validation and annotation. The sequencing project was divided between the consortium and the private company Celera Genomics. The consortium produced a physical map of the genome based on Bacterial Artificial Chromosomes (or BACs), and the company subjected the BACs to shotgun sequencing. The genome sequence is subjected to constant refinement, and the newer versions of the database are made available periodically.

Another database that focuses on a single organism is the ACeDB. More than a database, this is a database management system that was originally developed for the *C. elegans* (a nematode worm) genome project. The system does not use the standard, commercial models of data, and has been specifically designed to store and manipulate genome data. Though used originally for the genome of the worm, the system and its software tools are very flexible, and are now used for many different genomic databases. The *C. elegans* genome sequence itself is essentially complete and is available at http://wormbase.sanger.ac.uk, in a database called WormBase. It is the repository of not only the sequence, but also the genetic map as well as phenotypic information about the *C. elegans* nematode worm.

The Comprehensive Microbial Resource maintained by TIGR (The Institute for Genomic Research) at http://www.tigr.org allows access to a database called the Omniome. This contains all the microbial genomes that have been sequenced. Unlike FlyBase or WormBase, therefore, it does not focus on one organism. On the other hand it is not a general sequence database either. Omniome has not only the sequence and annotation of each of the completed genomes, but also has associated information about the organisms (such as taxon and gram stain pattern), the structure and composition of their DNA molecules (such as plasmid vs. chromosome and GC content), and many attributes of the

protein sequences predicted from the DNA sequence (such as pI and molecular weight). The presence of all microbial genomes in a single database facilitates meaningful multi-genome searches and analyses, for instance, alignment of entire genomes, comparison of the physical properties of proteins and genes in different genomes, etc. Each genome can also be accessed individually. Apart from the genome sequence, each such entry contains extensive annotations including a schematic sketch of all the DNA in the organism, its complete taxonomy, cross-references and links to other databases, and information obtained by analyses of the genome, i.e. the percentage GC content and a 2-D gel electrophoresis pattern generated computationally based on the gene content of the genome. Each identified gene also has associated annotation such as the molecular weight of the protein, its sequence, hydrophobicity, etc.

A database of the genomes of mitochondria and other such organelles is available at the Organelle Genome Database at the University of Montreal, Canada, and is called GOBASE (http://megasun.bch.umontreal.ca/gobase). GOBASE is different both from the organism-specific databases such as FlyBase and the primary sequence archives such as GenBank. GOBASE contains nucleotide and related protein sequences of organelles, in particular mitochondria. The database also has genetic and physical maps, RNA secondary structures, biochemical and organismal information, all of which are tightly bound together to make access straightforward. The annotation records are organised under nine headings specific to organelles, apart from the generalised headings available in the original GenBank format. This extra level of organisation and additional ways of classifying the information leads to better search results. Other types of data that have been integrated and cross-referenced in GOBASE include phylogenetic relationships, functions of the organelle enzymes and biochemical pathways and organismal biology

A different type of specialised database collects together sequences that have one particular function. The Eukaryotic Promoter Database (EPD: http://www.epd.isb-sib.ch) is a one such collection. It contains the sequences and annotations of eukaryotic promoters recognised by RNA polymerase II, i.e. POL II. The sequences in this collection are exclusively those that have been experimentally determined to be promoters, i.e. those that occur just before a well-established transcription start site. The database itself does not store the actual sequences. Instead it binds the experimental descriptions to the sequences via machine-readable pointers to specific positions within the appropriate sequences in the EMBL nucleotide sequence database. The pointers and other software are arranged so that it makes it easy to dynamically extract subsets of promoter sequences for comparative analyses. It is also possible to obtain sequence libraries consisting of the nucleotides from number –499 to +100 with respect to the experimentally determined transcription start site, in the EMBL or FASTA format. Another feature of the EPD are the weight matrix descriptions of four major eukaryotic promoter elements, namely TATA-box, initiator, GC-box and CCAAT-box. The matrices, which define the most likely sequences at the consensus sites, are updated annually, using an algorithm based on hidden Markov models.

Some of the secondary databases do not contain collections or sub-collections of sequences. Instead they are repositories of useful (often numerical) data that has been obtained by an analysis of the sequence databases. One such is the CUTG or the Codon Usage Table database (http://www.kazusa.or.jp/codon/). This is a compilation of codon usage statistics from known protein sequences in the primary databases. It is a well-known fact of molecular biology that the genetic code is degenerate, and that most of the twenty amino acids found in proteins are coded for by more than one codon of three nucleotide bases. The choice of a particular codon to express a given amino acid depends upon various factors. For example, different organisms use different codons for the same amino acid, and, indeed, the codon choice is one way of identifying genes, as well as specifying the taxonomy of the organism. Tables such as those found in CUTG are therefore very useful. The

database consists of lists of the codon usage classified with respect to genes as also with respect to the organism.

Another similar database, again with data obtained by analyses of the primary databases is ExInt (http://intron.bic.nus.edu.sg/exint/) or the Exon-Intron database. This consists of information on the exon-intron structure of eukaryotic genes. Data on the positions of the introns and the size of the exons are collected for all the proteins in GenBank. Data on the phasing of the intron is also placed in the entry. This refers to the precise positioning of the intron with respect to the three nucleotides in the triplet codon. This information is sufficient to reconstruct, in the correct reading frame, the mature mRNA that would be obtained when the gene is expressed.

2.2.4 Derived or secondary databases of amino acid sequences: Subcollections

Databases containing subcollections of proteins sequences abound. An example is the Aminoacyl-tRNA Synthetases Data Bank (http://rose.man.poznan.pl/aars/), which collects the sequences of this crucial protein in the translation pathway. Synthetases charge tRNA molecules with the cognate amino acid, and have mechanisms to recognize both the correct amino acid and the tRNA and its anti-codon. The recognition mechanism is subtle and involves also an editing function that removes a wrongly placed amino acid from the tRNA. These proteins evolved at a very early stage in biological history, which is why the genetic code is the same in all organisms. The database classifies the synthetases according to the specific amino acid and, for each amino acid, presents the sequences of all the corresponding synthetases from different organisms. An analysis of the sequences showed that there are two broad classes of synthetases, depending on the site of aminoacylation on the tRNA, as well on the precise three-dimensional disposition of the protein, the amino acid and the tRNA during the process. The database also classifies the sequences according to this scheme.

Another example of a database focussed on a particular family of proteins, namely the G-protein coupled receptors, is GPCRDB (http://www.cmbi.kun.nl/7tm). These are transmembrane proteins used by cells to communicate with the outside world. They are involved in vision, smell, hearing, taste, and feeling, viz. the five senses. A large number of diseases are caused by a GPCR malfunction, and the majority of commonly prescribed medicines act on a GPCR. GPCRDB is in fact more than simply a collection of sequences of this protein family. It includes additional data on multiple sequence alignments, ligands and ligand binding data, 3D models, mutation data, literature references, disease patterns, cell lines, protocols, vectors, etc. It is a fully integrated information system with data, and browsing and querying tools.

A database of the proteins sequences of one single organism, but with unknown functions is assembled at http://www.kazusa.or.jp/huge/. It is called HUGE for Human Unidentified Gene-Encoded large proteins. The sequences are obtained as translations of a library of cDNA from the human transcriptome at the Kazusa DNA Research Institute in Japan. Their focus is on proteins with molecular weight greater than 50 kDa. A typical entry in the database has information on the cDNA, including its physical map, restriction map and sequence. This is followed by a description of the features of the predicted protein sequence, including a BLAST search for homology against a non-redundant protein sequence dataset. This information, together with details of sequence motifs and sequence domains obtained by searching motif databases such as Pfam, often leads to a clear identification of the function of the protein. The database is planned to cover large numbers of human sequences with hitherto unknown function.

Similar to introns and exons in eukaryotic genes is the occurrence of intervening sequences in proteins, called inteins. One or more inteins may alternate with exteins in the protein sequence. A post-translational maturation process splices out the inteins to form the mature protein sequence that may then fold into its functional form. The inteins themselves are often enzymes in their own right. InBase

(http://www.neb.com/inteins) is a database that collects together sequences and other information on inteins.

MHCPep (http://wehih.wehi.edu.au/mhcpep/) is a database comprising over 13000 peptide sequences known to bind to the Major Histocompatibility Complex of the immune system. Each entry in the database contains not only the peptide sequence, which may be 8 to 10 amino acids long, but in addition has information on the specific MHC molecule to which it binds, the experimental method used to assay the peptide, the degree of activity and the binding affinity observed, the source protein that, when broken down gave rise to this peptide along with others, the positions along the peptide where it anchors on to the MHC molecule, and references and crosslinks to other related information.

A database of slightly longer peptides is the peptaibol database (http://www.cryst.bbk.ac.uk/peptaibol/home.shtml). There are typically between 15 and 20 amino acid residues in a peptaibol, and though the molecules occur naturally in certain species of fungi and bacteria, they contain a high proportion of non-standard amino acids, i.e. ones other than the 20 different types that occur in proteins. Their importance stems from their use as antibiotics. The entries are very brief and carry just the sequence, the source and references.

Several of the most commonly used databases are based upon grouping together related protein or peptides sequences and classifying them into families and superfamilies. Since some members of each family would have been exhaustively characterised in terms of their biochemistry, it becomes possible to extrapolate the same properties to all members of that particular sequence family. This in turn helps assign function to a new sequence purely on basis of the homology with one or other of the families. Thus such databases are greatly in use for annotating newly sequenced genomes. The different databases classify the proteins using different techniques, leading to a certain amount of variety in the classification schemes.

The CluSTr (Clusters of SWISS-PROT + TrEMBL proteins at http://www.ebi.ac.uk/clustr) database offers an automatic classification of the entries in the SWISS-PROT and TrEMBL databases into groups of related proteins. The clustering is based on analysis of all pair wise comparisons between protein sequences. There are two steps in forming the clusters. First all proteins sequences are compared against all other sequences and matrix of similarity scores is constructed. For pairs of sequences, say A and B, which have a similarity score higher than a certain threshold, the next step is taken. This is to compare A with a set of randomly generated sequences that have the same length and amino acid composition as B. An average similarity score M for this comparison is calculated along with the standard deviation σ. A Z(A,B) score may now be calculated as

$$Z(A,B) = (S(A,B) - M)/\sigma$$

where S(A,B) is the similarity score between A and B. Similarly a score Z(B,A) is calculated by comparing B with randomised sequences that have the same length and composition as A. The final Z score between A and B is taken as the minimum of these two. Based upon the Z score between pairs of sequences, a hierarchical scheme is used to cluster together proteins at different levels of similarity. Note that the clustering is based purely on the sequence. The clustering has been carried out for a few sets of protein sequences, such as the mammalian sequences or the plant proteins, as well as for complete genomes, such as that of yeast.

Similar to CluSTr is the COGS or Clusters of Orthologous Groups database that is accessible at http://www.ncbi.nlm.nih.gov/COG. An orthologous group of proteins is one in which the members are related to each other by evolutionary descent. Such orthology may not be just from one protein to another, and then to another, and so on down the line. It may involve one-to-many and many-to-many evolutionary relationships, and hence the term 'groups'. COGS is thus a database of phylogenetic relationships. The approximately 2500 groups have been divided into 17 broad functional categories. For example, there are groups of proteins, all involved in translation, other groups involved in

transcription, and so on. The utility of COGS, as of CluSTr, is that it helps assign function to new protein sequences without going through tedious biochemical discovery processes.

2.2.5 Derived or secondary databases of amino acid sequences: Patterns and signatures

A set of databases collects together patterns found in protein sequences rather than the complete sequences. The patterns are identified with particular functional and/or structural domains in the protein, such as, for example, ATP binding site or the recognition site of a particular substrate. The patterns are usually obtained by first aligning a multitude of sequences through multiple alignment techniques. This is followed by further processing by different methods, depending on the particular database.

PROSITE is one such pattern database, which is accessible at http://www.expasy.ch/prosite. The protein motifs or patterns are encoded as 'regular expressions'. A regular expression (See Chapters 4 and 6) summarizes the sequence pattern obtained through a multiple sequence alignment. It is more detailed than a consensus sequence, which specifies the most commonly occurring residue at each position in the pattern. Apart from naming the invariant amino acids along with their positions in the motif, a regular expression also specifies possible alternate residues each point where there occurs more than one amino acid in the multiple alignment. The information corresponding to each entry in PROSITE is of two forms – the pattern and the related descriptive text. The regular expression is placed in a format reminiscent of the SWISS-PROT entries, with a two-letter identifier at the beginning of each line specifying the type of information the line contains. The expression itself is placed on lines identified by 'PA'. The entry also contains references and links to all the protein sequences that contain that pattern. The related descriptive text is placed in a documentation file with the accession number making the connection to the expression data.

In the PRINTS database (http://www.bioinf.man.ac.uk/dbbrowser/PRINTS), the protein sequence patterns are stored as 'fingerprints'. A fingerprint is a set of motifs or patterns rather than a single one. Figure 2.8 illustrates the relationship between a motif and a fingerprint. The advantage of a fingerprint

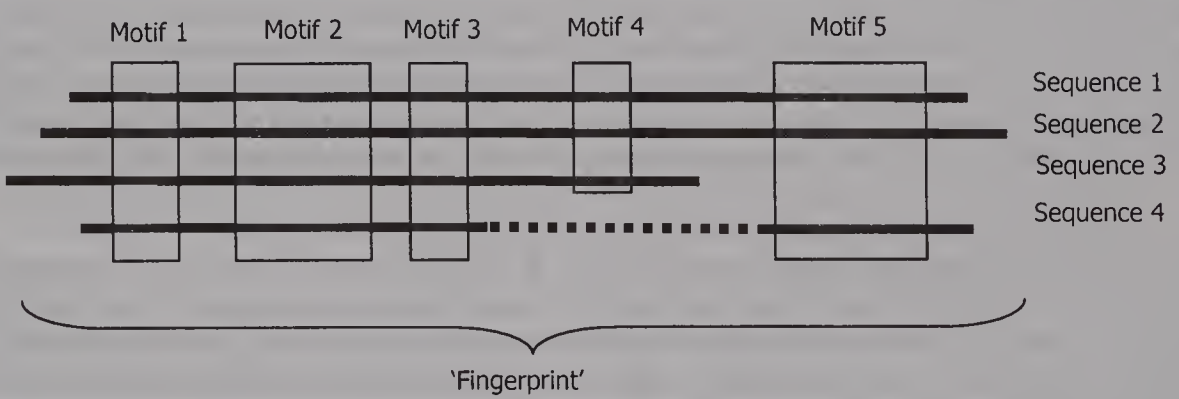


Figure 2.8 The relationship between a sequence motif and a fingerprint, as implemented in the PRINTS database.

over a single motif is that a sequence which is closely related to a given family of proteins would share all the motifs that define the fingerprint for that family, while more distantly related sequences or members of a subfamily would share one or more of the motifs, but not all. This makes it possible to quickly identify even distant relationships. Also it helps to quantify the relationships more precisely and thus deduce the probable function of the new sequence better. Fingerprints are built up in an iterative fashion, start with a few multiply aligned sequences and successively adding more and more sequences until the pattern does not change any more. The information contained in a PRINTS entry may be divided into three sections. In addition to the entry name, accession number and number of

motifs, the first section contains crosslinks to other databases that have more information about the characterised family. The second section provides a table showing how many of the motifs that make up the fingerprint occur in the how many of the sequences in that family. For example suppose a fingerprint consists of three motifs, namely A, B and C. Then the second section of the entry for this fingerprint may show that there are a total of say 50 sequences that fall within this family, of which all three motifs occur in 45 sequences, motifs A and B alone occur in 3 of them, motifs B and C occur in another 2 sequences, while motif A alone occurs in one sequence. This information is given in the form of a table called the composite fingerprint index. The last section of the entry contains the actual fingerprints that are stored as multiply aligned set of sequences, the alignment being made without gaps. There is therefore one set of aligned sequences for each motif.

Both PROSITE and PRINTS involve a great deal of manual intervention by their curators in their construction. In contrast, in the BLOCKS database (http://blocks.fhcrc.org/blocks) the process of identifying patterns is automatic. Here the motifs, called blocks, are created from the protein sequences already identified by PROSITE as belonging to one family. Note that the actual patterns are not taken from PROSITE, but only the protein IDs are obtained and then used to retrieve the complete sequences. These sequences are subjected to an automatic motif discovery process that includes identification of at least three conserved amino acids that occur in the same position in all the sequences in the family. This crude pattern is then refined by extending it over a larger portion of the sequence until the it converges. The newly discovered motif is validated by checking for a random or chance occurrence in the SWISS-PROT database. If this probability is low, the newly identified block is more likely to be correct. An entry in BLOCKS consists of the sequence fragments that make up the block. Each fragment is identified by the SWISS-PROT ID of the protein sequence of which it forms a part. Each sequence line ends with a number that specifies the relationship between that sequence and the others in that block, with 100 indicating that sequence is the most distant. Along with the sequences, the entry also has the block ID, data regarding the source and function, and numbers obtained from the validation process that indicate the reliability of the block. The BLOCKS database is the basis on which the BLOSUM amino acid substitution matrices have been constructed. This will be discussed in detail later in the book.

Instead of a regular expression, a profile[6] may be used to specify a pattern. In a profile, each position of the pattern is set to a particular amino acid. Each such specification is followed by twenty numbers, which refer to the substitution scores for the twenty amino acids. Note that insertions and deletions (or 'indels') are also specified. This allows the introduction of gaps or extra residues in the query sequence when making a match with the pattern. In this way sequence patterns that are distantly related are also identified. Profiles are usually constructed for whole sequences, and therefore are larger than regular expressions, blocks, etc. They also take into consideration the regions between highly conserved patterns, and thus incorporate all the information to be found in the sequence comparisons. Profiles may be built up in using many different methods. A database called Pfam contains profiles built using hidden Markov models (http://www.sanger.ac.uk/Software/Pfam). HMMs build the model of the pattern as a series of match, substitute, insert or delete states, with scores assigned for alignment to go from one state to another. (Hidden Markov models are discussed at length later on in the book.) Each family or pattern defined in Pfam consists of four elements. The first is the annotation, which has information on the source to make the entry (e.g. PROSITE or PRINTS, etc.), the method used, and some numbers that serve as figures of merit. The second is a seed alignment that is used to bootstrap the rest of the sequences into the multiple alignment, and then the family. The third is the HMM profile. The fourth element is the complete multiple alignment of all the sequences identified in that family. Pfam may be accessed in many ways. A new 'query' sequence is searched

[6] See Chapter 6 for more discussion on profiles.

against Pfam to find if it belongs to any one of the families already identified. If the user already knows the SWISS-PROT or TrEMBL ID for a protein sequence, she can directly access the family to which it may belong. In the third method of access, users may browse through the database and look for families by name.

The ProDom protein domain database (http://www.toulouse.inra.fr/prodom.html) is a compilation of homologous domains that have been automatically identified by sequence comparison and clustering methods using the program PSI-BLAST. No identification of patterns is made. The focus here is to look for complete and self-contained structural domains, and the search method includes signals for such features. A graphical user interface allows easy interactive analysis of structural and therefore functional homology relationships among protein sequences.

The above five databases, besides a few others not mentioned here are stores of protein patterns or signatures. Since they are all built from the same primary database of protein sequences, there is a great deal of overlap, and patterns identified in one have a good chance of being in another. However the methods are different and this leads to different emphasis in the data. Thus PROSITE contains regular expressions and profiles and focuses on functional sites; Pfam provides hidden Markov models that identify divergent domains; PRINTS consists of fingerprints, i.e. groups of aligned but unweighted motifs and helps classify sequences into hierarchically related families; BLOCKS contains aligned, weighted motifs or blocks; and finally ProDom identifies structural domains by clustering sequences together. The diversity of resources makes it somewhat difficult for confident pattern recognition in a completely new sequence, since it becomes necessary to run it through all the databases. InterPro (http://www.ebi.ac.uk.interpro) is a database that integrates all these resources into one, with aim to avoid duplication of efforts and make the task of pattern recognition far easier for the user. Each InterPro entry contains one pattern, together with information on the type of pattern (i.e. fingerprint, family, HMM, etc.), links to the original database, identification and annotation about the protein family, as well as matches of that pattern found in SWISS-PROT and TrEMBL databases. Owing to the difficulties of pattern recognition in proteins, the creators of InterPro expect that the whole, as represented by the integrated database, will soon turn out to be more productive than simply a sum of the parts, as represented by the individual databases PROSITE, PRINTS, etc.

2.3 Structure Databases

Structure databases, like sequence databases, come in two varieties, primary and secondary. Strictly speaking, there is only one database that stores primary structural data of biological macromolecules, namely the PDB. In the context of this database, the term 'macromolecule' stretches to cover three orders of magnitude of molecular weights from 1000 Daltons to 1000 kilo Daltons. Small (i.e. molecular weight approximately less than 1000 Daltons) biological and organic molecules have their structures stored in another primary structure database, viz. the CSD, which is also widely used in biological studies. This contains the three dimensional structures of, for example, drugs, inhibitors and fragments or monomers of the macromolecules. Apart from these there is no other primary structure database. In contrast, there are plenty of derived structural databases that store subsets of data extracted from the primary databases. However, most deal with protein structures, and there are very few secondary databases for nucleic acid structures.

2.3.1 The primary structure databases – PDB and CSD

PDB stands for Protein Data Bank. In spite of the name, PDB archives the three-dimensional structures of not only proteins but also all biologically important molecules, such as nucleic acid fragments, RNA molecules, large peptides such as the antibiotic gramicidin, and complexes of proteins and nucleic

acids. The database holds data derived from mainly three sources. Structures determined by X-ray crystallography form the large majority of the entries. In number this is followed by structures arrived at by NMR experiments. There are also several structures obtained by molecular modelling. The PDB was begun at the Brookhaven National Laboratory in USA, and was known for a long time as the Brookhaven protein data bank. In recent times its curation and maintenance has been taken over by a consortium of three organisations called the Research Collaboratory for Structural Bioinformatics, comprising of Rutgers University, the San Diego Supercomputer Centre and the National Institute of Standards and Technology, all of the USA. It is available on a set of CDs supplied free on request, or, more easily, over the Internet at http://www.rcsb.org.

Before we take a more detailed look at how the data are organised, let us consider how a three-dimensional molecular structure is stored in electronic form. While pictures and maps are more accessible to human understanding and also more pleasing, they do not reflect the level of accuracy reached by the experimental structure determination techniques. As will be described in a later chapter in this book, the methods of X-ray crystallography and NMR have been developed to an extent where the precise three-dimensional position of every single atom in the molecule can be determined. Each atomic position is represented as a set of three coordinates referred to a right-handed orthogonal system of axes, whose origin, i.e. the coordinate set (0.0, 0.0, 0.0) is chosen arbitrarily, or according to experimental convenience (Figure 2.9). Such a system of axes is known as the Cartesian coordinate system. Each three-dimensional structure in the PDB is therefore represented as a table of coordinates, each coordinate triple corresponding to one atom in that molecule or molecular complex. In order to represent the entire molecule, there will be as many triples as there are atoms. This way of representing a three dimensional structure has several advantages over pictures. Firstly, and most important, pictures are only two-dimensional projections of three-dimensional objects, and as such are only a small portion of the information present in the structure. Secondly, it is always possible to generate any picture of the molecule from the coordinates, either manually, or, as is common now, automatically with computers. This is equivalent to extracting a subset of the total three-dimensional information. In fact software for viewing molecules is designed so that the three-dimensional coordinates are directly loaded into the computer system, in order that any view of the molecule may be instantly displayed. Thirdly, coordinates allow the calculation of exact geometrical parameters such as bond lengths, bond angles, hydrogen bond distances and angles, and geometries of interaction between, for example, a protein and its substrate. And fourthly, coordinates allow scientists to use the structure as a model that can be manipulated and changed by rotating or moving parts of the molecule with respect to the rest, to see what the effect on the function would be. These advantages are in addition to the fact that the experimental techniques cited above naturally present their results in the form of atomic coordinates.

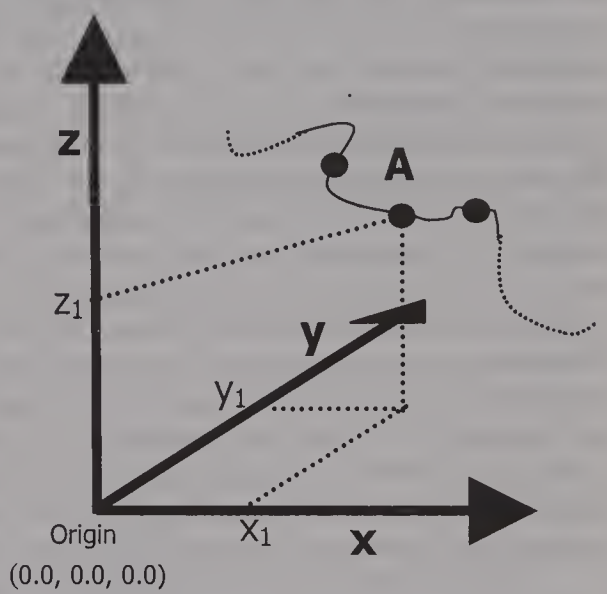


Figure 2.9 The Cartesian coordinate system. The coordinates of 'atom' A are (x_1, y_1, z_1).

The data in the PDB is organised as flat files, one to a structure, which usually means that each file contains one molecule, or one molecular complex. The files are named according to their PDB ID

code, which consists of four alphanumeric characters, which may very approximately be an abbreviation of the name of the molecule. All files that have the same characters at positions two and three of their names are placed together in a single directory named with that pair of characters. Thus the directory named '00' contains the files Pdb100d, Pdb200d, Pdb200l, Pdb300d and Pdb400d. Totally there are as many files as there are structures in the database. Within each file are to be found the coordinates of the structure, apart from other related information. This information is formatted in the well-recognised technique of placing a line identifier at the beginning of each line to specify the data contained in that line. Figure 2.10 gives an example of a structure file.

The data format of a PDB file is well established and is widely recognised by almost all software designed to read and manipulate structural data. As mentioned above, and just as in some of the sequence databases (e.g. GenBank), every line in a PDB coordinate file begins with a word that identifies the type of information present in that line. These keywords or identifiers may be grouped into many sections, viz. title, primary structure, heterogeneous, secondary structure, crystallographic and coordinate transformations, coordinates and connectivity. In the title section keywords point to information about the name, source, etc. of the compound, details of the authors, the references to published literature, and up to 1000 lines of general information about the structure, including all necessary experimental details required to interpret the structure correctly. The section on the primary structure contains keywords pointing to the protein or nucleic acid sequence of the structure in that file. References to sequence databases are also given here. Information on atoms that do not belong to the protein or nucleic acid, such as water, ions or prosthetic groups is given under keywords that are classified as heterogeneous. The section on the secondary structure describes secondary structure features in the molecule. In particular, it specifies the positions of alpha helices, beta sheets and turns in the protein structures. The connectivity annotation section specifies tertiary interactions such as disulphide bonds, hydrogen bonds and salt bridges. The section on crystallographic and coordinate transformations is, of course, specific to crystallographic structures. Here we find the parameters that are required if we wish to transform the coordinates from the Cartesian axial system in which they occur in the PDB to the crystallographic coordinate system in which they are specified in the original experimental structure determination. Such transformations are essential to study intermolecular interactions between any given molecule and its neighbours in the crystal.

The most important part of the entry is specified by keywords in the coordinate section. Of these the atomic coordinates of the molecule all begin with the word 'ATOM'. In contrast to the information up till now, which were in 'free format', i.e. had no particular arrangement within the line, each line that begins with 'ATOM' has the coordinates of one atom of the molecule arranged in a very specific format. In fact it is this format that is commonly referred to as the PDB format. The lines marked 'ATOM' in Figure 2.10 are in this format. Each 'ATOM' record starts with an identifier of the atom that includes a serial number, atom name, residue name, polypeptide or nucleic acid chain name and residue number, all between columns 7 and 26. Between columns 31 to 54 are given the three Cartesian coordinates of the atom in Angstrom units[7] from the origin, each coordinate occupying 8 columns. The next number between columns 55 and 60 gives the value of the occupancy of the atom. In other words it specifies in what proportion of the molecules that particular atom is in that position instead of in another. The occupancy is usually 1.0, but some portion of the molecule, such as the active site of an enzyme may have disordered atoms or atoms that occupy different positions at different times. These may have an occupancy value less than 1.0. The next number is the temperature factor and describes the degree of thermal vibration suffered by the atom. The larger the number, the greater the vibration of the atom about its mean position specified by the coordinates. The temperature factor is a positive number that occupies columns 61 to 66. The rest of the line has an

[7] 1 Angstrom unit = 10^{-10} metres

```
HEADER    DEOXYRIBONUCLEIC ACID                    25-AUG-04   1XA2
TITLE     COBALT HEXAMMINE INDUCED TAUTOMERIC SHIFT IN Z-DNA: THE
TITLE    2 STRUCTURE OF D(CGCGCA).D(TGCGCG) IN TWO CRYSTAL FORMS
COMPND    MOL_ID: 1;
COMPND   2 MOLECULE: 5'-D(*CP*GP*CP*GP*CP*A)-3';
...
SOURCE    MOL_ID: 1;
...
KEYWDS    DOUBLE HELIX, Z-DNA
EXPDTA    X-RAY DIFFRACTION
AUTHOR    S.THIYAGARAJAN,S.S.RAJAN,N.GAUTHAM
REVDAT   1   16-NOV-04 1XA2    0
JRNL        AUTH   S.THIYAGARAJAN,S.S.RAJAN,N.GAUTHAM
JRNL        TITL   COBALT HEXAMMINE INDUCED TAUTOMERIC SHIFT IN
...
REMARK   1
REMARK   2
REMARK   2 RESOLUTION. 1.71 ANGSTROMS.
REMARK   3
...
REMARK 290      SYMOP   SYMMETRY
REMARK 290     NNNMMM   OPERATOR
REMARK 290       1555   X,Y,Z
REMARK 290       2555   1/2-X,-Y,1/2+Z
...
SEQRES   1 A    6    C   G   C   G   C   A
SEQRES   1 B    6    T   G   C   G   C   G
HET    NCO     13       7
HETNAM     NCO COBALT HEXAMMINE ION
CRYST1   17.976   30.926   44.625  90.00  90.00  90.00 P 21 21 21    4
ORIGX1      1.000000  0.000000  0.000000        0.00000
ORIGX2      0.000000  1.000000  0.000000        0.00000
ORIGX3      0.000000  0.000000  1.000000        0.00000
SCALE1      0.055630  0.000000  0.000000        0.00000
SCALE2      0.000000  0.032335  0.000000        0.00000
SCALE3      0.000000  0.000000  0.022409        0.00000
ATOM      1  O5*   C A   1      -1.646  -3.103  15.385  1.00 21.58           O
ATOM      2  C5*   C A   1      -1.066  -3.909  14.359  1.00 17.63           C
ATOM      3  C4*   C A   1       0.416  -3.627  14.109  1.00 19.25           C
ATOM      4  O4*   C A   1       0.573  -2.252  13.622  1.00 17.08           O
...
ATOM    241  C4    G B  12       6.625   5.029  15.535  1.00 18.63           C
TER     242        G B  12
HETATM  243 CO   NCO    13      -0.358  -8.174   3.855  1.00 30.90          CO
...
HETATM  294  O   HOH    62      -4.345  -2.021  -8.846  1.00 43.77           O
CONECT  243  244  245  246  247
...
CONECT  249  243
MASTER      233    0    1    0    0    0    0    6  292    2    8    2
END
```

Figure 2.10 An example of a PDB file. The file has been extensively edited, wherever indicated by the ellipsis (...).

identifier for the segment of data, the symbol for the chemical element of the atom and the charge on the atom, if any.

The ATOM records are followed by the HETATM records that give the coordinates of the heteroatoms in exactly the same format. After the terminator keyword 'TER' come the connectivity records that describes the bonds that connect the atoms to one another. This information is sometime redundant since such information can always be calculated from the coordinates. If the distance between two atoms is less than the sum of their normal atomic radii, it is usually assumed that they are bonded, the nature of the bond corresponding to the actual distance between the two. The file ends with the keyword 'END'. As mentioned earlier, the PDB may be obtained on CDs and implemented on the local computer by the user. However the RCSB website at the address given above has software that helps to search through the software and obtain the coordinates of only the molecule of interest. The website also is linked to display software that shows pictures of the molecule, which may be turned around, or enlarged or reduced (zoomed in or zoomed out) to get a better appreciation of the three dimensional information. It also has software to make most geometrical calculations. Several other websites, such as the NCBI, also have the entire structural database. MMDB is the name of this database at NCBI, but it is not different from PDB, except that it is linked to different display and retrieval software.

The Cambridge Structural Database (CSD) was originally a project of the University of Cambridge, which set up the Cambridge Crystallographic Data Centre (CCDC) to collect together the published three-dimensional structures of small organic molecules. In the terms of molecular weight, 'small' refers to any molecule less about 1000 Daltons. This excludes proteins and medium sized nucleic acid fragments, but small peptides such as neuropeptides, and monomers and dimers of nucleic acids find a place in the CSD. In the beginning, the staff of CCDC would scan through the published material, in particular the journal Acta Crystallographica (where, even today, most of the structures are published), and painstakingly enter the coordinates and other data into the database. This was an expensive task and right from the beginning the database has not been open to free access. Users had to pay to obtain information from it. However, since most of the users were, at that time, from the academic community, the arrangement was that each country would pay a certain amount each year to Cambridge, which would then supply the annual update. The nominated institution in each country would then make the database available to all users in its country. In India, the Department of Crystallography and Biophysics at the University of Madras, Chennai, played the role of the nominated centre for a long time. Lately, however, there have been drastic changes in the profile of the users (pharmaceutical and chemical companies are the most interested users of the data), the technology for collecting and disseminating the data, and the organisational structure of the CCDC. Now any individual laboratory or department may obtain a copy of the database directly from CCDC on CDs on payment. Alone of all those discussed here, this database is not available over the Internet. Currently the CSD holds crystal structure information for about 2.5 lakh organic and metal organic compounds. All of these crystal structures have been obtained using X-ray or neutron diffraction techniques. For each entry in the CSD there are three distinct types of information stored. These are categorised as bibliographic information, chemical connectivity information and the three-dimensional coordinates. The annotation data fields incorporate all of the bibliographic material for the particular entry and summarise the structural and experimental information for the crystal structure. The text and numerical information includes the authors' names and the full journal reference, as well as the crystallographic cell dimensions and space group. Connectivity information is stored as a table, comprising atom and bond properties. The coordinates are referred to the crystallographic axial system. This system is the one that naturally describes the unit cell of the crystal and is the most useful one for not only encoding the molecular structures, but also the other molecules within the crystal that are related to the reference by symmetry transformations. The conversion to (and from) the Cartesian coordinate system is a

simple one. The CSD is accompanied by powerful search and display software called Quest, with a more recent version called ConQuest. Other software for retrieval, display and analysis of the structures are also supplied.

2.3.2 Derived or secondary databases of biomolecular structures

Just as for protein sequences, secondary databases of structures may also be classified as subcollections, such as the NDB, and collections of patterns and motifs, such as SCOP, PALI, etc.

NDB stands for Nucleic acid Data Base. It is a relational database of three-dimensional structures containing nucleic acids. This encompasses DNA and RNA fragments, including those with unusual chemistry such as phophothiorate backbones, RNA molecules, and complexes of nucleic acids and proteins or other molecules. The structures are the same as those found in the PDB, and therefore the NDB qualifies to be called a specialised subcollection. However a substantial amount of value addition has been carried out, and, unlike the PDB, the NDB is much more than just a collection of files. The structures of DNA have been classified into the A, B and Z polymorphic forms, based on information supplied by the authors. Other classes include RNA structures, unusual structures, and protein-nucleic acid complexes. These classes of structures are arranged in the form of an Atlas of Nucleic Acid Containing Structures, which can been browsed and searched to obtain the structure or structures required. Each entry in the atlas has information on the sequence, crystallisation conditions, references, and details of the parameters and figures of merit used in structure solution. The entry has links not only to the coordinates, but also to automatically generated graphical views of the molecule. NDB also has archives of structural geometries calculated for all the structures or for a subset of them. And finally, the database stores average geometrical parameters for nucleic acids, obtained by statistical analysis of the structures. These parameters are widely used in computer simulations of nucleic acids and their interactions The NDB may be accessed at http://ndbserver.rutgers.edu/NDB/.

The SCOP database (Structural Classification Of Proteins: http://scop.mrc-lmb.cam.ac.uk/scop/) is a manual classification of protein structures in a hierarchical scheme with many levels. The principal classes are the family, the superfamily and the fold. Proteins belong to the same family if there exists an evolutionary relationship between them that can be clearly demonstrated on the basis of similarity in the structure. While this usually also implies sequence similarity greater than 30%, some of families, for instance the globins, have a very high degree of similarity in structure and function concurrently with less than 15% similarity in sequence. The superfamily is a next larger level, and brings together proteins that again have low sequence identity, but with sufficient similarity in the structure and function to suggest a common ancestor. In some cases a domain of a protein (i.e. a structurally autonomous portion of a large protein) may be a member of a superfamily together with other domains or complete proteins. A superfamily is obviously a superset of families, with each superfamily subsuming many families. The third major class is the fold, which is a superset of superfamilies. Proteins that belong to the same fold have the same secondary structure elements, in the same three-dimensional arrangements with the same or similar connections between the secondary structures. No evolutionary relationship is implied between the members of the same fold. SCOP is a searchable and browsable database. In other words, one may either enter SCOP at the top of the hierarchy and examine the different folds and families as one pleases, or one may supply a keyword or a phrase to be used to search the database and retrieve corresponding entries. Once a structure, or a set of structures, has been selected, they may be obtained or viewed either as PDB entries or as graphical images. Each entry also has other annotation regarding function, etc., and links to other databases, including other structural classifications such as CATH.

CATH stands for Class, Architecture, Topology and Homologous superfamily. The name reflects the classification hierarchy used in the database. The structures chosen for classification are a subset of PDB, consisting of those that have been determined to a high degree of accuracy. This choice allows

greater reliance to be placed on the resulting classification. The next step in processing the data for classification consists of identifying domain structures in the proteins. Though the programs are automatic, multiple methods are used, and only those domains that have been identified by all of them are accepted as such. Each identified domain is considered an individual protein in the classification procedure. As mentioned above, there are four main levels in the CATH hierarchy. The first and most general of them is Class or the C level. This refers to the amount of a particular type of secondary structure present in the structure. Thus a protein may consist entirely of alpha helices (alpha class) or of beta sheets alone (beta class) or a mixture of both (alpha/beta class) or of low secondary structure (coil). The next level of the hierarchy is Architecture or A level. Structures classified as having the same architecture have same orientation of their secondary structures, but the order in which they occur along the polypeptide chain may be different. The third level consists of structures classified as having the same Topology or T level. This term refers to both the orientation and the connectivity of the secondary structures in the structure. The final level is called the Homologous superfamily or H level. Each group under this classification contains proteins that are closely related in structure, and therefore in function, implying a common evolutionary origin. There is yet another level of similarity in the hierarchy, the so-called S or Sequence level. Here the structures in each H group are clustered together on the basis of sequence similarity. CATH is accessible at http://www.biochem.ucl.ac.uk/bsm /cath for browsing and for searching. In addition the files of lists may be downloaded and implemented locally to suit any particular purpose. There are three main files of lists. The first one is the actual classification. Each item in this list consists of an ID that identifies the protein domain, and along with it specifies the identification number of the class, architecture, topology, homologous superfamily and sequence groups in which it belongs. The second list consists of a single protein in each of the different groups, and may therefore be considered as the file that specifies and describes the groups. The third file lists the domains considered in CATH and links them to the PDB entries.

FSSP (Fold classification based on Structure-Structure alignment of Proteins: http://www2.embl-ebi.ac.uk/dali/fssp/) uses an automatic classification system based on the results obtained from the program Dali. This program uses dynamic programming algorithms to make comprehensive comparisons of all structures in the PDB versus all others. As and when new structures are added to the database, the Dali program is run on them and, depending on the results, added to the appropriate family. The database can be browsed through a tree of relationships between the proteins that is available at the website. In addition searches for similar structures can be made using the Dali server.

The PALI database has been designed specifically as an aid in homology modelling of protein structures. Homology modelling is a technique used to predict the structure of a new protein based on the similarity of its sequence with those whose structures already have been experimentally determined. PALI stands for 'Phylogeny and ALIgnment of homologous protein structures'. It is available at http://pauling.mbu.iisc.ernet.in/~pali. The PALI database is a further refinement of SCOP and the proteins in the latter have been subjected to pair wise alignment of their structures for every possible pair. A distance is calculated for each pair called the Structural Distance Metric or SDM, which is a function of the deviations in the positions of the equivalent atoms when the two proteins are aligned on one another, as well as the lengths of the proteins. Multiple structural alignments of the members of family have also been made. Based on the results of this procedure, phylogenetic trees of relationships within the families have been constructed. The database is a compendium of homologous proteins as suggested by the phylogenies obtained from the structural alignments. One of the key features of this database is that it allows the extraction of a set of structures at a user-defined structural or sequence distance from each other.

All the above databases are rearrangements, in one way or another, of the experimentally determined protein structures. HSSP is a database of the secondary structures of proteins for which no experimental structure exists. The entries in this database have been derived by aligning sequences of

new proteins with sequences of proteins of known structure. A significant homology in the sequences is followed by the assignment of the known secondary structure to the new sequence. In this manner the database of Homology derived Secondary Structures of Proteins has been created and can be accessed at http://www.sander.ebi.ac.uk/hssp.

The final database discussed in this section focuses on the technique used to assign particular secondary structures to various parts of a folded polypeptide chain, based on the experimentally determined three dimensional structure. The database, called DSSP, and accessible at http://www.sander.embl-heidelberg.de/dssp/, is therefore not a subcollection or a rearrangement of the PDB. In fact it contains all the structures in the PDB together with an assignment of the secondary structural features on the protein, made using the program, also called DSSP. The database is extensively indexed and one may consider that the indices form an important part of the database. There are indices or lists of PDB IDs, names of the proteins, sources of the proteins, authors of the structures, intermolecular and intramolecular disulphide bridges, tertiary hydrogen bonds, accessible surface areas, etc.

2.4 Summary

- Sequences and structures are only two among the several different types of data required in the practice of modern molecular biology.
- Other important data types include metabolic pathways and molecular interactions, mutations and polymorphisms in molecular sequences and structures as well as in organelle structures and tissue types, genetic maps, physicochemical data, gene expression profiles, two dimensional DNA chip images of mRNA expression, two dimensional gel electrophoresis images of protein expression, data on restriction nucleases, nomenclature, video images and networks of biological data reflecting their inter-relationships.
- Sequence databases may be classified as primary databases and secondary or derived databases.
- The primary nucleotide sequence database is the trio GenBank/EMBL/DDBJ.
- The arrangement of data in each entry in the GenBank is self-explanatory. Besides the DNA sequence, it contains extensive annotations that give all details regarding the sequence.
- The primary protein sequence database is the duo SWISS-PROT/PIR.
- Again each protein sequence entry in the database is accompanied by extensive annotation.
- Derived or secondary databases may be divided into specialized subcollections of data and collections of information gathered by analysis of the primary databases
- Examples of subcollections of nucleotide sequences include the FlyBase, the Comprehensive Microbial Resource, the Eukaryotic Promoter Database, etc. Some of these also store information on patterns obtained by and analysis of the sequences, such as the consensus regions in the promoter sequences.
- Subcollections of protein sequences include, for example, the Amino-acyl tRNA Synthetases Data Bank.
- Patterns, signatures or motifs of protein sequences are to be found in databases such as PROSITE, PRINTS, BLOCKS, Pfam, etc., where they are stored as alignments, regular expressions, hidden Markov models, consensus sequences or profiles
- The primary structure database is the PDB.
- This contains all the experimentally determined structures of biological macromolecules, i.e. proteins, nucleic acids, and their complexes.

- Structures are stored as a list of the three-dimensional Cartesian coordinates of each atom in the molecule.
- The NDB is a value-added secondary structure database that collects together structures containing nucleic acids.
- Derived databases of protein motifs and patterns include SCOP and CATH.
- These databases cluster protein structures together in an increasingly distant hierarchy of structural similarity.
- Such databases are immensely useful in identifying the function of a newly sequenced protein.

3

Sequence Alignment

As already mentioned in this book, one of chief tasks of bioinformatics is to compare DNA and protein sequences and find similarities, or differences. The results of such analyses could have profound biological implications. For example, as early as 1984, at the dawn of the age of bioinformatics, an unexpected but important similarity was discovered, when scientists aligned the sequence of a human growth factor protein with an oncogene, i.e. a DNA sequence that appeared to help cause cancer when it was expressed. This suggested that oncogenes are mutated forms of proteins that have normal functions, such as helping cells to grow. Of course, not all sequence alignments lead to such dramatic discoveries. However, this example makes it clear why sequence comparison forms such an important part of bioinformatics. In this chapter we discuss some of generally used methods to compare sequences. The first section motivates the discussion, and explains why sequence comparison and alignment is not a trivial task, as might appear at a first, superficial approach. Indeed, this particular aspect of bioinformatics provided the first successful transplants of algorithms from the realm of computer science into biology. It continues to attract the attention of mathematicians, who try to devise ever newer algorithms to match strings of symbols, in general. This statement requires some amplification. We have pointed out, in Chapter 1, that one of the reasons for the excitement over bioinformatics is the digital nature of important biological information. This refers to the fact that DNA and protein sequences may be considered strings of symbols chosen from limited sets – four symbols in the case of DNA and twenty in the case of proteins. Various other forms of information we come across also may be considered as strings of symbols, the most immediate and potent being the English language sentences you are reading now. In fact any written language may be analysed by almost exactly the same methods used to analyse DNA and protein sequences, and vice versa. And as we stated above, comparison of the strings forms one important way of analysing such information. The second section talks about the use of dot matrices and hashing methods in sequence comparison. The next section explains, with worked out examples, dynamic programming techniques of sequence comparison, such as the Needleman-Wunsch and the Smith-Waterman methods. The final section details the use of BLAST and FASTA, perhaps the two most widely used bioinformatics programs

While the techniques explained here are more or less general in nature, and may be adapted to analyse any form of information that is expressible as linear strings, all examples used here are specifically DNA, RNA or protein sequences. Also this chapter deals only with sequence comparisons and alignments. There is another very important aspect of sequence analyses. We refer to the use of

pattern recognition techniques to annotate biological sequences, and this is held over for another chapter.

3.1 Introduction

3.1.1 Why align sequences?

The reason we align sequences is to look for a common or related pattern amongst them. The common pattern may be a short stretch of one sequence that is similar to a short stretch of the other sequence, or many short similar bits of the two sequences, or a part of one sequence matching a part of the other, or the similarity may be spread all across both sequences. If we discover such sequence similarities, we may infer biological similarity between the two sequences. This could be a structural, functional or evolutionary relationship. Every match discovered does not necessarily imply a biological relationship, and below we will describe the difference between finding a mere chance sequence similarity, and finding one that has biological meaning. First, however, we will try to explain why special techniques are required to align sequences.

Consider the following two sequences of (English language) letters:

Sequence 1: I AM WRITING THIS **A**RTICLE

Sequence 2: YOU ARE READING TH**A**T ARTICLE

Note that if we retain this way of placing the two sequences, or alignment, there is only one common letter noticeable between the two, viz. the letter A in ARTICLE in the first sequence matching with the same letter in THAT in the second sequence. Clearly this is not the only similarity between the two sequences. We need to shift one with respect to the other, i.e. to change the alignment, in order to bring out all the possible common letters. To help us do this, we will first ignore the spaces and collapse the two sequences, as follows.

Sequence 1: IAMW**R**ITINGTHISAR**T**ICLE

Sequence 2: YOUA**R**EREADINGTHA**T**ARTICLE

This arrangement obliterates, to some extent, the original English language meanings of the two sentences. But let us henceforth treat these not as English sentences, but simply as strings of symbols, chosen from a set of 26. When the sentences are arranged in this fashion, the number of matches, i.e. same symbols at the same positions, increases to two. Even this however is only fortuitous, since we have taken no pains at all to maximize the number of matches. Consider the following arrangement.

Sequence 1: IAMW**R**IT. . .**INGTH**IS**ARTICLE**

Sequence 2: YOUA**R**EREAD**INGTHA**T**ARTICLE**

Now the number of matches has increased substantially. But we have introduced a gap in one of sequences. If these were DNA or protein sequences, the gap implies that, if the two sequences were derived from a common ancestor, then there has been an insertion of three residues in Sequence 2, or equivalently a deletion of three residues in Sequence 1. Consider the following arrangement

Sequence 1:. .I**A**MW**R**.IT**INGTH**IS**ARTICLE**

Sequence 2:YOU**A**R**E**READ**INGTHA**T**ARTICLE**

Now we have two gaps, which make the arrangement worse than the previous one, but the total number of matching residues has increased by one, which is good. To decide overall which of the two alignments is better, we need a function that helps us to find some figure of merit or score for each alignment. Such a function may simply be a count of the number of matches minus a count of the number of gaps. In the above case, however, if we use this function, we are still unable to conclude which of the last two alignments is better, because the scores for both are exactly the same. If we are to decide between these two we need other criteria, or other functions. We will deal with scoring

functions later. Let us consider another aspect of the matter, viz. the method we use to find the alignment that yields the best value for the function. The above example was simple and we used a trial and error method to quickly arrive at what obviously is the correct alignment. However if the two sequences were much longer, or if they were less similar to each other, it would not be so simple. One may think that, all we have to do is to use a high speed computer to try out all possibilities, calculating the score each time, and at the end choosing the alignment with the best score. But this brute force approach simply leads to combinatorial explosion, and a concomitant explosion of the required computational time. We will not expand this statement further except to give the follow simple exercise. Try to find the best alignment for the following two sequences. The scoring scheme is simple – just give one point for every matching pair of residues. You can have as many gaps as you wish, and as many residue pairs that do not match, or mismatches, as you wish.

Sequence 3: ABCNJRQCLCRPM

Sequence 4: AJCJNRCKCRBP

Later in the chapter, we will show one way to quickly arrive at the correct answer.

3.1.2 Similarity versus Homology

The reason we align sequences is to discover new biological information. Similar sequences have similar structures and therefore similar functions. Biologists commonly use the word 'homologous sequences' when they refer to sequences that have a great deal of similarity, as for example in the phrase 'homologous recombination'. Thus, loosely speaking, homology and similarity appear to mean more or less the same thing. This is not so, and in bioinformatics in particular we need to make a clear distinction in the use of the two terms. We define them as follows. We say two sequences are similar to each other when, after the best alignment, identical (or similar) residues occur at identical (or similar) positions. The number of such matches indicates the degree of similarity. In this definition we say nothing at all about the reasons for the similarity. Similarity could arise by chance, or it could be a convergence towards a common sequence and structure and therefore function, through evolution. Or, the similarity could arise from divergent evolution of the two sequences from a common ancestral sequence. The similarity that rises from this last mechanism alone is called homology. Thus all sequences that are homologous will be similar, though this similarity could be very subtle and hidden by a number of other factors. The converse however is not true and all similar sequences are not necessarily homologous. To explain this further we take an example somewhat removed from sequence analysis.

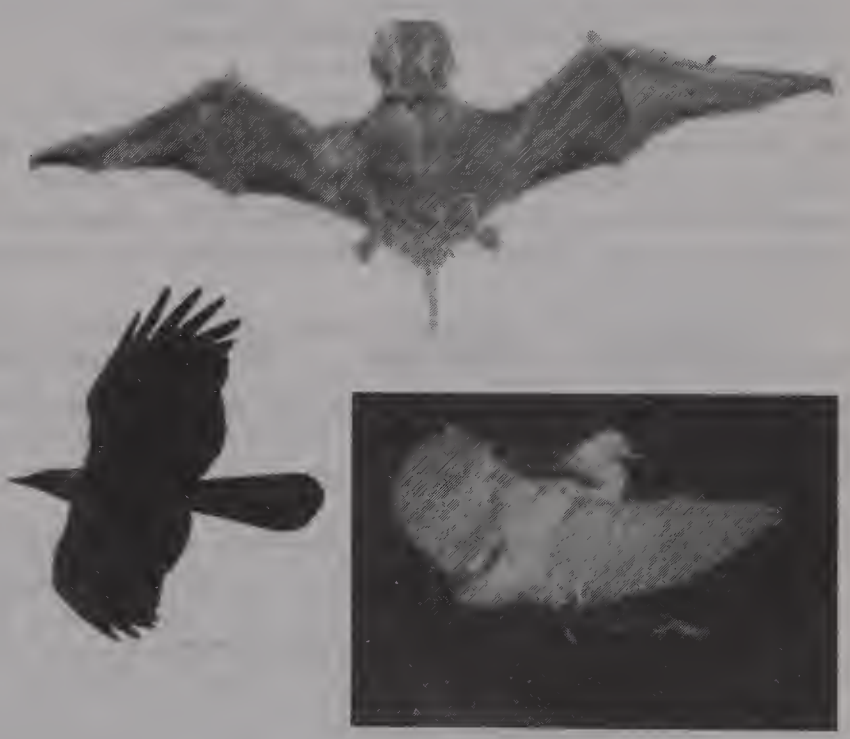


Figure 3.1 Similarity v/s Homology – the wings of a bat, a crow and a swan.

Consider the wings of a bat, of a crow and of a swan (Figure 3.1). All have very similar structures and function, i.e. they allow the animal to fly. But we know that crows and swans are birds, while bats are mammals. Thus while all three wings are similar, only those of the crow and the eagle are homologous. These two had a near common ancestor in evolution. The wings of the bat developed by convergent evolution into a shape very much like that of the other two. This was probably because this shape is the most efficient design for a flight system. To, reiterate, similarity and homology are two different concepts, and the presence of the former does not always imply the latter.

For sequences, similarity is usually measured, at least at a gross level, in terms of identity. This refers to the numbers of exact position matches between identical residues in the two sequences after the best alignment. Thus in the example above, the best match between Sequence 1 and Sequence 2 (the fourth one) shows an identity of 14/24 = 58%. While we cannot infer the presence of homology or otherwise directly from similarity, experience tells that if, in two complete, well aligned, protein sequences, more than 30 % of the residues are identical, then the two proteins probably have the same structure and, therefore, function. At the very least, if the structure of one of the proteins is known, the structure of the other may be built by homology modelling. If the level of identity is less than this, the similarity falls into what is called the 'twilight zone'. This is a zone of sequence similarity (approximately 10-20% identity, over approximately 300 amino acids) within which alignments are not statistically significant, and therefore homology cannot be directly ascertained. Homology can be inferred even in the twilight zone if there is a structural and/or functional similarity to support a low sequence similarity. When the identity is less than 10 %, we approach the 'midnight zone', a region of sequence identity where sequence comparisons fail completely to detect structural similarity.

3.1.3 Homologs, heterologs, analogs, orthologs, paralogs, xenologs

According to the definition given in the previous section, all homologous sequences are descended from some common ancestral sequence. The nature of the mechanism that caused two very similar sequences to exist, where only one existed previously, gives rise to the various terms in the title of this section.

'Homologs' is general term to indicate sequences with common origin, and subsumes within it the other expressions mentioned above. It is a qualitative term, and is somewhat arbitrary in the sense that it is up to the researcher to decide what level of similarity indicates homology. Apart from the numerical indicators of similarity determined by the sequence alignment programs, other parameters derived from biochemistry or other areas of biology may contribute to the decision on whether a pair of sequences is homologous.

The opposite of 'homologs' is 'heterologs'. Heterologous sequences may still be similar, but they do not have a common origin, and nor do they have a common function or activity. Again, sequence comparison alone may not be sufficient to decide if two sequences are heterologs, for sequences dissimilar by alignment may still be similar by function. The words 'homologs' and 'heterologs' describe the current relationship between the sequences, but do not indicate the mechanisms that lead to that relationship.

Sequences that have the same function but lack sufficient similarity to imply common origin are said to be 'analogs'. The implication is that analogous sequences followed evolutionary pathways from different origins to converge upon the same function and activity. Thus, analogs may be considered a product of convergent evolution. Analogs have homologous function but heterologous origins. Analogs are in general not considered to be homologs. Chymotrypsin and Subtilisin do not have a great deal of sequence similarity; yet have very similar protease functions. The structure of the active site and the 'catalytic triad' of residues in particular are very similar in both proteins. These two proteins are analogs.

'Orthologs' are homologs that arise by speciation. The implication is that the two sequences are currently taken from two different species, but they show similarity because they are both derived from the same ancestral sequence that was present in the ancestral organism. The amount of difference in two orthologous sequences may be taken as a rough indicator of the amount of time that has passed since the speciation event took place. Orthologs have homologous origin and homologous function. A good example of orthologs are haemoglobin sequences from, say, horse and zebra.

'Paralogs' are homologs that arise by gene duplication, without this duplication event being followed by a speciation event. That is to say, paralogs are homologous sequences that exist in the same organism, and that have different functions. As related to DNA sequences, the functional difference may simply be that one is a functional gene, while the other is a silent non-expressed gene, or a pseudogene. Paralogs have homologous origin but heterologous function. For example, sequences of myoglobin and haemoglobin from a given organism, say, humans, are paralogs.

'Xenologs' are homologs resulting from horizontal gene transfer. They are an exception to the rule that homologs are always descended from a common ancestor. With the discovery of bacterial plasmids, phages that transfer DNA, and transposons or mobile genes, it is now clear that horizontal or lateral exchange of genes may take place between different species. Thus one may find similar protein and DNA sequences in completely unrelated species. Of course the functions of the genes would be homologous, particularly if the transfer occurred relatively recently. In a sense xenologs are orthologs, i.e. similar sequences with same or similar function in different organisms, except that strictly speaking orthologs arise from a common ancestor by evolutionary descent and speciation, not by horizontal transfer. As may be expected, it is frequently very difficult to identify xenologs.

In fact, in general it is quite difficult to distinguish between the above categories on the basis of sequence similarity alone. Consider the events shown in Figure 3.2. A gene G1 has duplicated to form G11 and G12. These have evolved for some time, until the functions of the two have become somewhat different. Now a speciation event occurs, and G11 gives rise to G11A and G11B in two different species. Similarly G12 gives rise to G12A and G12B. These two species then evolve independently to reach the current stage, where the sequence analysis is to be carried out. Clearly all four genes will show a great deal of similarity with each other. However it would be difficult to clearly say that that gene duplication event that gave rise to G11A and G12A occurred not in the current organism, but in an ancestral organism. Similarly, the relationships between, say, G11A and G12B would be somewhat difficult to decipher. This example is in fact a relatively simple one. The real situation would of course be complicated by the presence analogs and xenologs.

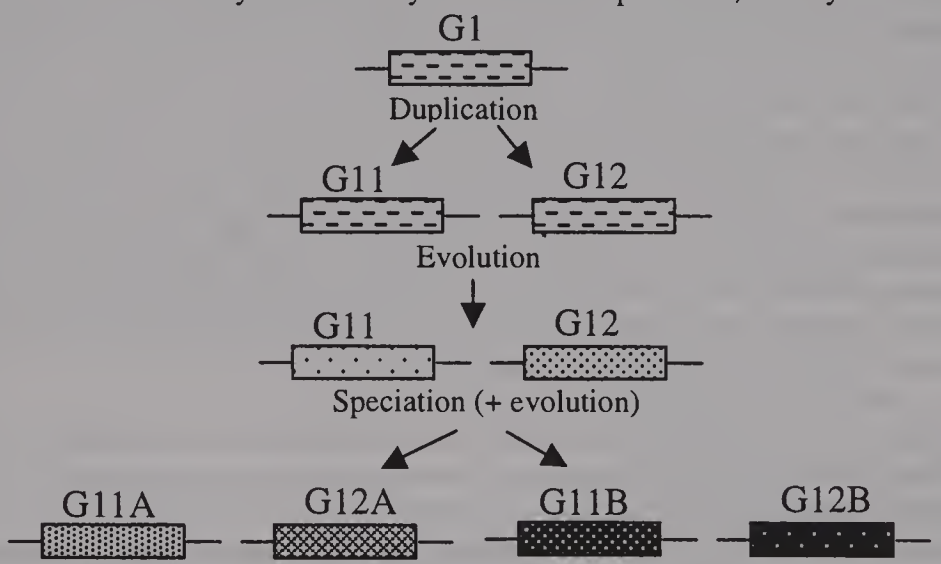


Figure 3.2 Sequence similarities could arise due to many different causes

For most purposes three classifications may be both necessary and sufficient to treat all the relationships between gene or protein sequences. They are heterologs, homologs, and analogs. These three classifications require low-level knowledge from similarity measurements and provide sufficient information to use in most applications. Many studies may benefit from further sub-classifications. However, although terms such as paralogs and orthologs are used frequently by researchers, is it difficult to know whether genes with similar sequence and different activity resulted from duplication and divergence within that organism/species, or arose by recombination, or evolved to different activity elsewhere and moved into the current space by horizontal transfer, etc. A word of caution against routinely sub-classifying sequences: more knowledge is required for accurate classification, and inaccurate classification can eventually lead wrong biological interpretations.

3.1.4 The significance of an alignment

How do we measure the significance of any similarity we detect when we deal with sequences consisting of a few residues, or small portions of a large sequence, and not entire protein or gene sequences? For example, consider the case where we have aligned two protein sequences with no known evolutionary, functional or structural relationship.

Sequence 5:. . . . . AMN**CHWKC**YW. . . . . .

Sequence 6:. . . . . NCY**CHWKC**ATF. . . . . .

There is a region in this alignment where five identical matches occur consecutively. The ellipses represent the rest of the two sequences, and we assume that nowhere in these other portions are there even two adjacent consecutive matches. Now the question is, how significant is this similarity? Clearly the fact that, in this region, the five matches occur adjacent to each other vests it with some significance. Equally clearly however, the significance of the entire alignment will depend on its length, and what portion of it the five matching residues above represent. In general, therefore, we may say that the significance of a match depends on the following – the number of matching residues, the number of consecutive matching residues and the length of the total alignment. If the portion of matching residues, more specifically the portions with a large number of consecutive matches, is a substantial fraction of the total alignment, the similarity is bound to be significant, if not evolutionarily, at least in terms of structure and function. These intuitive remarks may be put on a firm mathematical foundation through the use of extreme-value statistics.

Before we describe these statistics, we must first differentiate between 'global' and 'local' alignment. 'Global' alignment refers to the comparison of one complete protein or gene sequence with another. The consequence of global alignment would be an understanding of the biological relationship between the two entities that were compared. This is usually carried out when we already have some inkling that the two sequences are related. The alignment then helps to identify the portions that are the same and those that are different, and relate these features to the structure and function, and perhaps the phylogenetic relationship between the two. Local alignment, on the other hand, compares short portions of sequence or a whole library of sequences with short portions of another. This helps to identify local regions of a protein or gene that may have resemblances to other local regions in some other protein or gene. Sequence analysis often starts with a new sequence of unknown function and structure, one whose relationship with other sequences is also not known. Algorithms are then used to identify local matches between the new sequence and other sequences, perhaps those in a large database. In other words a global search for locally aligned segments is made, and the methods used are local alignment techniques. Once promising matches are identificd, a global alignment of the entire new sequence with a complete sequence that has been picked up is made to not only confirm the match, but also to identify the degree of similarity between the two sequences. The emphasis during local alignment is on similarity, or more stringently on identity, whereas during global alignment we have already identified the overall similarity between the sequences, and now wish to find the

differences. There are two classes of algorithms, one for local alignments and the other for global alignment. The significance and the degree of similarity of a global alignment are not difficult to estimate. We may simply calculate the number of identical matches, the number of similar residues that match, and add a penalty for any gaps that we may introduce. In fact, such a numerical score does not carry too much meaning, and it is more correct at this stage to use biological knowledge to estimate whether a given global alignment is significant, and if so to what degree. Local alignments however are a different matter. At this (early) stage of the analyses we cannot use biological knowledge to estimate the significance of any match we may find. We need to first of all estimate statistical significance before we proceed with the most significant matches to global alignment, and thence to biological knowledge.

The usual method of estimating the statistical significance of any event is to calculate the probability of its occurrence purely by chance. If this probability is very low, the event is statistically significant, if not, not. The problem therefore reduces to estimating this probability. In the case of sequence alignment, we estimate the significance of a given alignment by estimating the probability of an alignment with the same score occurring purely as a matter of chance. To take an extreme (and probably absurd) example, if two sequences of say 100 residues each were to be aligned, the probability of finding a chance alignment with at least one identical match is nearly 1.0, i.e. such a chance alignment is nearly certain. Thus if our alignment program were to output an alignment with a match score of 1, we would not consider that significant at all. On the other hand, we may estimate, by intuition, that an alignment that showed a match score of, say, 50 (i.e. 50 identical matches) would have a very, very low probability of occurrence, unless there was a special relationship between the two sequences. To replace our 'intuition' with statistical reasoning we may adopt the following method. After aligning the two sequences and calculating the score, we rearrange the residues in each sequence randomly. We now have two new 'sequences' each with the same residue composition as the respective original sequence, but with random order. If we now use the same algorithm to find the best alignment of these two random sequences, we would get a score that can be used to estimate the significance of the true score. Of course, since this is statistical, we cannot do this just once, but need to repeat the randomisation of the sequences many times, and each time find the best alignment score. We can now calculate the average random score, and its variance, and using these quantities we may estimate the probability of chance occurrence of the real score. The estimate of the probability is usually made on the assumption that the scores obtained by randomisation would follow a normal distribution about the mean value. That is, there would be a few scores much larger than the average, a few scores much smaller than the average, but the bulk of the scores would lie close to the average value. Such an assumption is however not correct for the problem of sequence alignment. It has been shown that the distribution of scores for random

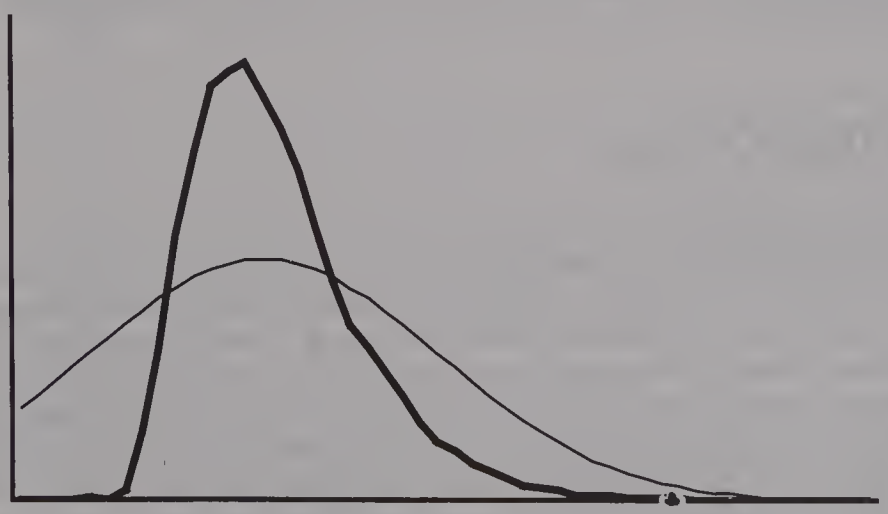


Figure 3.3 The thick line shows the 'extreme value' probability distribution curve. The thin line shows the 'normal' distribution for the same range.

alignments of two sequences does not follow the normal distribution, but follows an extreme value distribution. Both these distributions are shown in Figure 3.3. The expression used to calculate the probability is

$$P(S \geq x) = (Kmn)e^{-\lambda x}$$

This expression gives the probability that the score S is greater than or equal to the value x. In this expression K is a constant that depends on the scoring matrix used. (We will deal with scoring matrices later.) m and n are the lengths of the two sequences being compared. Note that they could be of widely different magnitudes – m could correspond to a small search sequence, and n could correspond to the entire sequence database. λ is the decay or scale parameter calculated from the standard deviation of the distribution. x, of course, is the score whose probability is being estimated. A very approximate version of the above expression is given below.

$$P(S \geq x) = e^{-x}$$

Using either formula we may estimate the significance of the alignment between Sequence 5 and Sequence 6, shown above. We will assume that the total lengths are 10 and 11 residues respectively, and that there are no other matches except the ones shown. An approximate value for K is 0.1 and for λ it is 0.23, irrespective of the scoring scheme we use. If we use the PAM120 scoring scheme, the score of the alignment shown above is 23. Plugging these values into the equation, we obtain

$$P(S \geq 23) = 0.1 \times 10 \times 11 \times \exp(-0.23 \times 23) = 0.05$$

In other words, the probability that, purely by chance, one obtains a match with a score better than the one above is less than 0.05. This number is also called the expectation value, or the e-value, and apart from the raw scores, is an effective way of deciding on the merit of an alignment. Alignments with e-values less than about 0.01 are usually considered acceptable, while values of order of 10^{-5} or less indicate biologically significant alignments.

The score of the alignment, measured in bits, is also used as an indicator of its utility. These are often indicated as Z values, or as normalised Z values. Z values are not the actual or raw scores, but indicate the difference between the score of the alignment and the average score for random sequences of the same composition as the sequences used in the alignment. The average and standard deviation may be calculated for such randomised sequences as $\langle x \rangle$ and σ. If x the score of the alignment then

$$Z = (x - \langle x \rangle)/\sigma$$

i.e. Z is the number of standard deviations between the mean and the score. In general, the higher the Z value, the more significant is the alignment.

3.2 Dot Matrices and Hash Coding

3.2.1 Comparing sequences using dot matrices

A 'dot matrix' is a visually appealing, intuitive, but qualitative tool for the comparison of sequences. The procedure to construct a dot matrix to compare two sequences is simple. We require, of course, that both sequences be of the same type, e.g. both are protein sequences. The two sequences are written in such a way as to form a matrix. One sequence is written on top, from left to right, each residue in the sequence labeling one column of the matrix. The second sequence is written on the left, from top to bottom, each residue of this sequence labeling one row of the matrix. Each element of the matrix is thus labeled by one residue from the first sequence (the column index), and one residue from the second sequence (the row index). The elements of the matrix are usually of two types – a dot, or a blank – chosen according to the following rules.

F(i,j) = 'dot' if $x_i = y_j$

F(i,j) = 'blank' if $x_i \neq y_j$

where F(i,j) is the matrix element being considered, and x_i and y_j are the residues corresponding to that matrix element from the first sequence and the second sequence, respectively. In other words, at each element of the matrix a dot is placed if the two corresponding residues are identical, else the element is left blank. Figure 3.4 shows a dot matrix constructed for comparing Sequence 3 with Sequence 4

	A	B	C	N	J	R	Q	C	L	C	R	P	M
A	*												
J					*								
C			*					*		*			
J					*								
N				*									
R						*					*		
C			*					*		*			
K													
C			*					*		*			
R											*		
B		*											
P												*	

A diagonal row of dots showing a small similar portion in the two sequences

Figure 3.4 A dot matrix.

(section 3.1.1, above). Inspection of the matrix shows that portions of one sequence that are similar to portions of the second sequence are indicated by diagonal rows of dots. This is more evident in the dot matrix (Figure 3.5) constructed to compare the following two DNA sequences.

Sequence 5: CGGGTGAAGGTGTGAAGGTATTG
Sequence 6: GTAAGGGTGCCCCTGTGAACCGGTA

	C	G	G	G	T	A	A	G	G	T	G	T	G	A	A	G	G	T	A	T	T	G
G		.	.	.				.	.		.		.			.	.					.
T					.					.		.						.		.	.	
A						.	.							.	.				.			
A						.	.							.	.				.			
G		.	.	.				.	.		.		.			.	.					.
G		.	.	.				.	.		.		.			.	.					.
G		.	.	.				.	.		.		.			.	.					.
T					.					.		.						.		.	.	
G		.	.	.				.	.		.		.			.	.					.
C	.																					
C	.																					
C	.																					
T					.					.		.						.		.	.	
G		.	.	.				.	.		.		.			.	.					.
T					.					.		.						.		.	.	
G		.	.	.				.	.		.		.			.	.					.
A						.	.							.	.				.			
A						.	.							.	.				.			
C	.																					
C	.																					
G		.	.	.				.	.		.		.			.	.					.
G		.	.	.				.	.		.		.			.	.					.
T					.					.		.						.		.	.	
A						.	.							.	.				.			

Figure 3.5 Another dot matrix.

	C	G	G	G	T	A	A	G	G	T	G	T	G	A	A	G	G	T	A	T	T	G
G				.							.						.					
T					.							.						.				
A						.	.							.					.			
A						.	.							.	.							
G		.						.	.							.						
G			.					.	.							.	.					.
G				.					.		.						.					
T					.					.		.						.				
G											.		.									
C																						
C	.																					
C																						
T										.												
G		.		.					.		.											
T					.					.		.									.	
G				.							.		.									.
A							.							.								
A						.									.							
C	.																					
C	.																					
G		.	.					.			.					.						
G			.	.					.		.						.					
T					.					.		.						.				
A						.								.					.			

Figure 3.6 The dot matrix in Figure 3.5, after applying a '3 out of 5' filter.

Non-identical pairs of residues that occur in between stretches of identical pairs are represented by breaks in the diagonal lines of dots. Insertions and deletions in the sequences make the corresponding lines of dots to be non-colinear. Such features are also indicated in Figure 3.5. While visual inspection reveals the similarity between the sequences in a qualitative way, quantitative measures of similarity may be obtained by analyzing the diagonal lines in terms of their numbers, their lengths, whether they are lined up along one line or are broken up, etc. Mathematically more rigorous analyses would include normalizing the lengths and numbers of the diagonal lines with respect to the possibility of their random or chance occurrence.

One of the problems with dot matrices, particularly when comparing DNA sequences, is that it is usually very difficult to separate the 'signal', i.e. the relatively long diagonal lines of dots, from the 'noise', or all the other dots. DNA sequences have only four different residues, and this implies that the matrix will have many dots, with a large proportion of them not part of significantly long identical portions. (In this respect proteins sequences are less 'crowded' since there are 20 different residues.) The best way to get over this problem is to apply filters so that only those dots that are significant are represented in the final matrix. A common filter used is the so-called 'n of m' filter. In this method of filtering, n and m are two integers, usually about 5, with n always less than or equal to m. The filter chooses only those dots that satisfy the following criterion. Out of every m consecutive diagonal elements in the matrix, at least n of them must be filled. This filter is applied to every possible (overlapping) set of m diagonal elements. This procedure may be also described as follows. A diagonal 'window' comprising m elements is inspected, and if n of those elements are filled, all the dots in that window are displayed. The window is then moved to the next, overlapping set of elements, and the process repeated to display additional dots. Of course, once the dots have passed the filter, they are not erased again, even if they do not pass the next application of the filter. Figure 3.6 shows a filtered version of Figure 3.5, where a '3 of 5' filter has been applied, i.e. n = 3 and m = 5. Figure 3.7 shows

an appropriately filtered dot matrix constructed for portions of two genomes, clearly showing the homology.

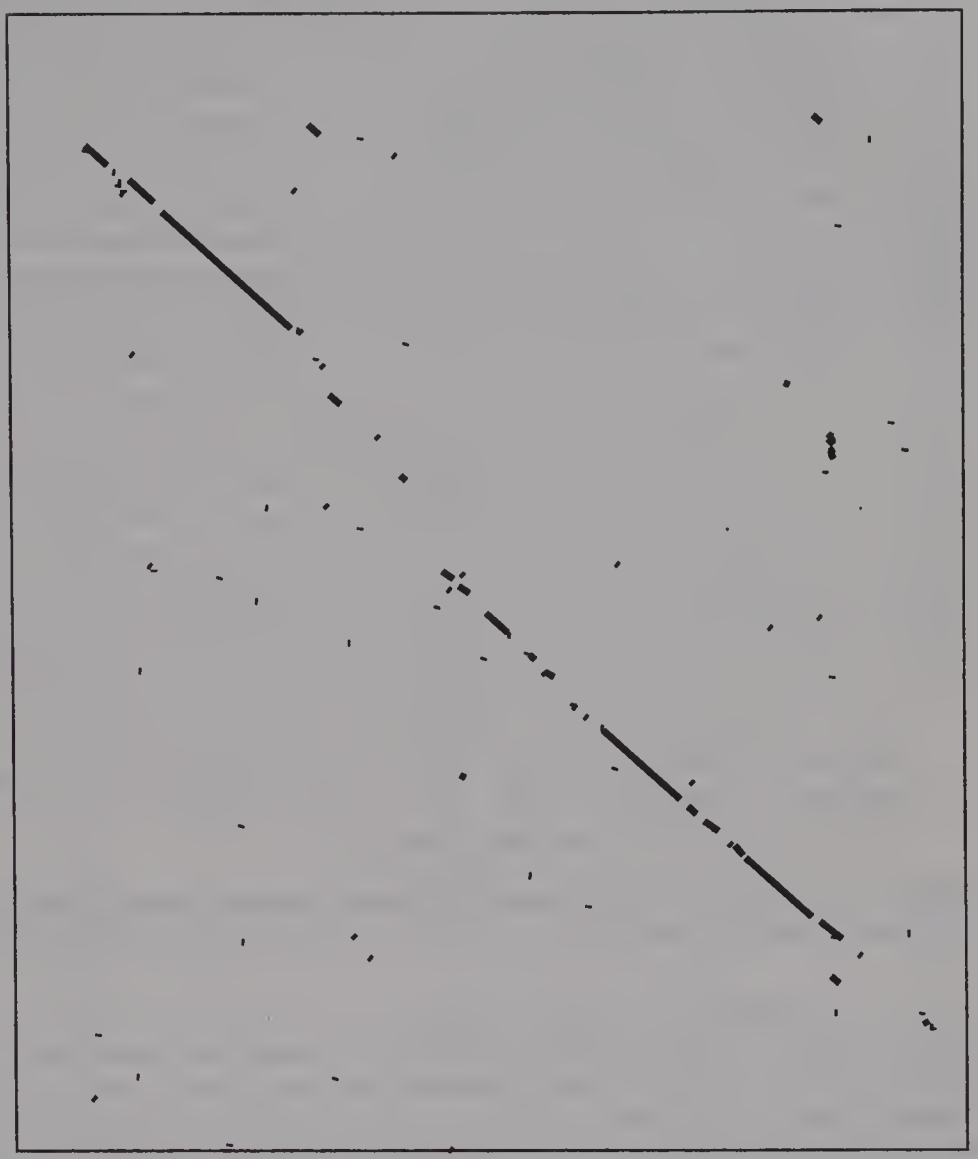


Figure 3.7 A well-filtered dot matrix plot of the comparison of portions of two bacterial genomes. The broken diagonal line indicates the large degree of similarity between the two.

Dot matrices may also be constructed in different ways. We may use one to compare a sequence with itself, for example, in order to detect repetitive regions. In this case the same sequence is used to label both the rows and the columns of the matrix. Note that in such cases, the matrix is always symmetrical (mirror symmetry) about the main diagonal, and in fact only one half of the diagram need be considered. Filters may be applied to remove the background noise. We may also use the matrix to look for self-complementary regions in nucleic acid sequences, in particular RNA sequences. Figure 3.8 shows such an application. Again the same sequence is used to label both the rows and the columns of the matrix. A dot is placed only when the residues labeling the element are complementary to each other. Here the lines of dots representing self-complementary portions are perpendicular to the main diagonal. Again the matrix is symmetrical about the main diagonal. The use of dot matrices in RNA secondary structure analyses is dealt with in more detail in Chapter 8.

Dot matrices are binary diagrams, with each element taking one of two values, here represented by a dot or a blank. Equally they could be represented as 1 or 0. When comparing protein sequences, in particular, it is difficult to classify all relationships between the amino acid residues simply as 'identical' or 'not identical'. The elements of the matrix then take up values that better reflect the

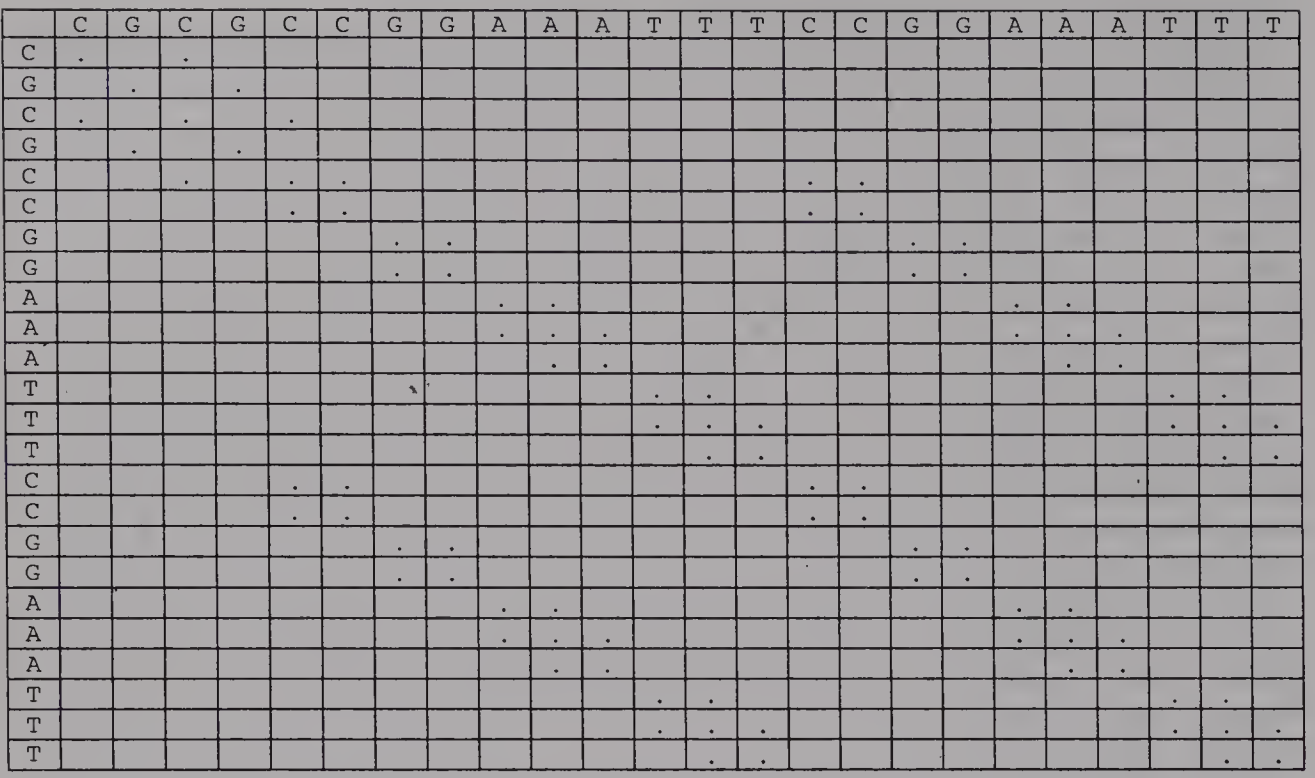


Figure 3.8 Using a dot matrix to compare a sequence with itself. A '3 out of 5' filter has been applied. Note the symmetry about the main diagonal. Also note that the main diagonal is completely filled. The repeating portions of the sequence, namely 'CCGGAAATTT' are clearly seen in the plot.

chemical or structural similarity between the pairs of residues. Such numbers are usually obtained from 'substitution matrices' described in brief below (Section 3.2.2), and in greater detail later in Chapter 4.

3.2.2 Pattern searching using hash coding

This is a method that is particularly suited for fast, ungapped searches of a small sequence, sequence pattern or motif, through a large database. The word 'hash' derives from the name 'hash table' commonly used in computer science for a look-up table constructed from a database. Such tables represent an abstract or index of the information present in the database, and are often used for fast searches. Much of the new terminology in this section therefore is derived from computer science. A hash table is also called an associative array, where a specific name or 'key' is associated with each piece of data or 'value' stored in it. As applied in sequence searching, hashing is the process of breaking the sequence into small 'words' or 'ktuples' of a specified size, and creating a hash table with those words keyed to position numbers. The values associated with each key are stored in 'buckets'. The number of buckets obviously is the same as the number of keys specified, and this is dependent on the size of the ktuple chosen. Then when any of the word positions match part of an entry in the database, that match, the 'offset,' is saved. In general, hashing reduces the complexity of the search problem to the order of the length of all the sequences in the database. We will illustrate the use of this method in sequence searching by the following example.

Target 'Database': GTAAGGGTGCCCCTGTGAACCGGTAATTCCC

Query 'pattern': TGAA

If we chose the key size (or ktuple value) as 1, there will be four buckets, and the hash table is the following (Table 3.1).

Table 3.1

	Key	Position
Bucket 1	A	3,4,18,19,25,26
Bucket 2	T	2,8,14,16,24,27,28
Bucket 3	G	1,5,6,7,9,15,17,22,23
Bucket 4	C	10,11,12,13,20,21,29,30,31

The values in the tables are the positions of each key value in the sequence. Thus 'A' occurs at positions 3,4, etc. The number of entries in each bucket is clearly a function of the ktuple size – the smaller the size, the greater the number of entries. For instance, if we build the above hash table again, with a key size of 2, we have 16 buckets, but fewer entries in each bucket, as follows (Table 3.2).

Table 3.2

Key	Position
AA	3,18,25
AT	26
AG	4
AC	19
TA	2,24
TT	27
TG	8,14,16
TC	28
GA	17
GT	1,7,15,23
GG	5,6,22
GC	9
CA	--
CT	13
CG	21
CC	10,11,12,20,29,30

We will use this hash table, with ktuple = 2, to perform a search for the query pattern. The next step is the construct a similar hash table for the query sequences as well. This is the third column in the table below (Table 3.3). The fourth column in the table is the 'offset', obtained by subtracting each number in column 3 from each number in column 2. It represents the number of positions the query would have to be moved along the target in order for that particular key to find an identical match. Once these columns are calculated and filled, the next step, i.e. the actual search, is simple enough. We just look through the 'offset' column and identify that particular offset value that will yield the maximum number of matches. This is nothing but the offset value associated with the maximum number of keys. In our example this is 15. If we now move the query by that offset along the target, will find the best match for the two. In other words we have found one occurrence of the query pattern in the target, as follows. The search is ungapped if we require that the offset chosen be such that every key in the query line up against an identical key in the target. Sometimes this requirement may be relaxed, in which case gaps are introduced in the match. However it is much more difficult to introduce insertions/deletions in the match, and therefore, usually this procedure is used only for fast ungapped matches of small query patterns with large databases.

Table 3.3

Key	Position in target	Position in query	Offset
AA	3,18,25	3	0,15,23
AT	26	--	--
AG	4	--	--
AC	19	--	--
TA	2,24	--	--
TT	27	--	--
TG	8,14,16	1	7,13,15
TC	28	--	--
GA	17	2	15
GT	1,7,15,23	--	--
GG	5,6,22	--	--
GC	9	--	--
CA	--	--	--
CT	13	--	--
CG	21	--	--
CC	10,11,12,20,29,30	--	--

Target 'Database': GTAAGGGTGCCCCTG**TGAA**CCGGTAATTCCC
Query 'pattern': **TGAA**

In local sequence alignment and in pattern searching, using any method, there is always a trade-off between speed and specificity on the one hand, and sensitivity on the other. In the hash coding method, the relative importance of the two is adjusted using the value of ktuple. Specificity is the ability to pick up accurate and meaningful matches. Sensitivity is the ability to pick up approximate or distant matches that are yet biologically meaningful. If the ktuple is large, the speed is high, specificity is high and the sensitivity is low. Conversely, if the ktuple is small, the speed is low, the specificity is low, but the sensitivity is high.

The hash coding method is widely used for initial matches or for quick pattern matches. Note that the hash table for a database needs to be constructed only once. If the database is updated, it is a simple matter to simultaneously update the hash table. And since the query patterns are usually small, it is again very simple to construct a hash table for it, and to use this to search the table for the database. The algorithm is used in the two very widely used sequence matching and search programs BLAST and FASTA. These are described later in this chapter.

3.3 Dynamic Programming in Sequence Alignment

Dynamic programming is the name a family of techniques used in a variety of technological, scientific and commercial fields to find the set of parameters that would give the desired optimum solution. Consider the following example. A company wishes to establish a new factory. It has to evaluate the location in terms of the distance and ease of approach to the following: the supply of raw materials, the market where the product is sold, the availability of skilled labour, the availability of power and water,

a suitable climate, and several other such factors. Clearly many of the requirements would be contradictory. For example it may not be possible to locate the factory close to the source of raw materials as well as to the market. But good roads, or proximity to a port, may offset this disadvantage. Dynamic programming is one of the techniques used to take into consideration all the parameters and find the best location. The similarity of the problem of factory location to that of sequence alignment is clear when we consider that in the latter, too, we have to reconcile contradictory parameters and arrive at an optimal solution. When aligning two sequences we need to ensure that we get the maximum possible number of matches, the minimum number of mismatches and the minimum number of gaps. It is easy to maximize the matches, if we ignore the gaps and the mismatches. It is also easy to minimize the gaps, if we ignore the matches and mismatches. But to carry out all the procedures simultaneously calls for dynamic programming (DP) algorithms. There are several DP algorithms that have been developed for this purpose. We will deal here with two of them – one global alignment algorithm, namely the Needleman-Wunsch method, and one local alignment method, namely the Smith-Waterman method.

3.3.1 The Needleman-Wunsch algorithm

Historically this was the first application of dynamic programming to the alignment of sequences. Needleman and Wunsch published it in 1970 in an elegant paper in the Journal of Molecular Biology. Their exposition of the method using a simple example cannot be bettered and here we will use the same example. This is the one already introduced as a practice problem at the end of section 3.1.1.

Sequence 3: ABCNJRQCLCRPM
Sequence 4: AJCJNRCKCRBP

We will call Sequence 3 as S3(i), i = 1,m. (Here m = 13). For example, S3(7) is the residue Q. Similarly Sequence 4 is S4(j), j = 1,n (n = 12). The first step in the algorithm is to create a matrix that we shall call a similarity matrix F(i,j). One sequence is written along the top of the matrix, and each residue of this sequence labels one column. The other sequence is written vertically along the left of the matrix, and each residue of this sequence labels one row. Thus a pair of residues, one from each sequence, labels each element of the matrix (Figure 3.9a). For example the matrix element F(5,7) is labelled by S3(5) = J and S4(7) = C. The matrix will have m x n elements or cells.

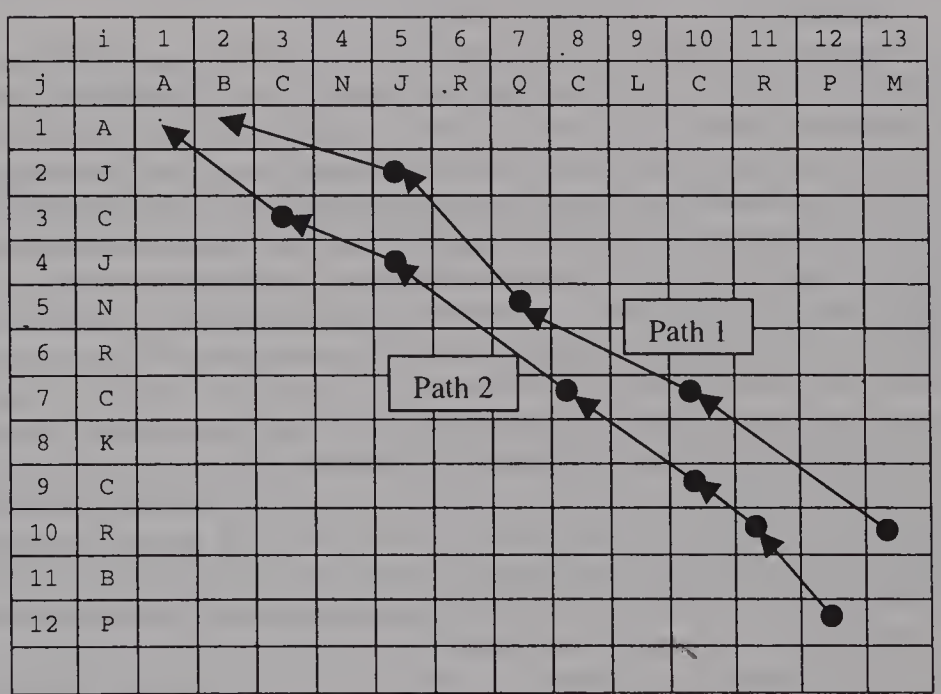
Before we proceed, let us consider some features of this matrix. Lines drawn connecting the cells are called paths through the matrix. A valid path is defined as follows. If a line connects the cell with indices (i,j) to the cell with indices (m,n), then either ($i < m$ and $j < n$) or ($i > m$ and $j > n$). In other words, the lines joining the cells are all diagonal, never vertical or horizontal. If we now consider a continuous line going from one corner of the matrix to the other, hopping from cell to cell, then every such continuous line is a valid path through the matrix. Every valid path through the matrix represents one valid alignment between the two sequences, and every cell along the path represents the alignment of one residue of the first sequence against one residue of the second sequence. (This is illustrated in Figure 3.9b.) The reason for the prohibition on vertical and horizontal lines now becomes clear. If such a line were on the path, it would imply the match of two residues from one sequence with one residue from the other. This of course is not allowed. The above rules also ensure that the order of the residues in the sequences is not changed. Thus the biological meaning of the sequences is preserved during the manipulations. The problem of finding the best alignment between the two sequences may now be stated as one of finding the 'best' path through the sequence. The meaning of the word 'best' may here be stated as 'largest score', but will become clearer as we go along.

To proceed in constructing the similarity matrix, we fill in the values of the matrix M according to the following rules.

$F(i,j) = 1$ for $S3(i) = S4(j)$
$F(i,j) = 0$ for $S3(i) \neq S4(j)$

	i	1	2	3	4	5	6	7	8	9	10	11	12	13
j		A	B	C	N	J	R	Q	C	L	C	R	P	M
1	A													
2	J													
3	C													
4	J													
5	N													
6	R													
7	C													
8	K													
9	C													
10	R													
11	B													
12	P													

Figure 3.9a The matrix F(i,j). Refer text for details.



Alignments: Path 1: Sequence 3: A B C N J R - Q C L C R P M
Sequence 4: A - - J C J N R - C K C R B P

Path 2: Sequence 3: A B C N J R Q C L C R - P M
Sequence 4: A J C - J N R C K C R B P

Figure 3.9b Paths through the matrix F(i,j), each corresponding to a valid alignment (with matches, mismatches and gaps) of the two sequences.

If a cell corresponds to two residues that are identical, the score 1 is written in the cell. If the two residues are not the same, the score is zero. The results of this operation are shown in Figure 3.9c.

	i	1	2	3	4	5	6	7	8	9	10	11	12	13
j		A	B	C	N	J	R	Q	C	L	C	R	P	M
1	A	*1*												
2	J					*1*								
3	C			*1*					*1*		*1*			
4	J					*1*								
5	N				*1*									
6	R						*1*					*1*		
7	C			*1*					*1*		*1*			
8	K													
9	C			*1*					*1*		*1*			
10	R						*1*					*1*		
11	B		*1*											
12	P												*1*	

Figure 3.9c The first step in the Needleman-Wunsch algorithm.

Residue comparison, especially when comparing amino acids, need not always be such a binary operation. There could be different degrees of similarity between any pair of residues, and the score need not be only 1 or 0. We have explained this briefly above (Section 3.2.1), and will do so in greater detail below. Here, in this example, we will use the simple binary scoring scheme given above.

The next step in the algorithm is to rewrite the matrix according to the following rules. We start with the cell at the lower right hand corner of the matrix, i.e. the cell F(m,n). We then proceed to the cells along the last row and then to those along the last column. When these are over, we go back the last cell in the next row and column. In this manner we gradually proceed up to the top left-hand corner of the matrix. Before we move from one cell to the next, we update the value in the current cell as follows. We check all the cells that are to the right of the current column and below the current row, and identify the cell that contains the largest value of the score. This largest value is then added to the value already present in the current cell, and the sum is the new, updated value of the cell. In general, the updated value F'(i,j) in each cell is given by

$$F'(i,j) = (\text{maximum of } \{F'(k,l),\ k = i+1,m,\ l = j+1,n\}) + F(i,j)$$

At the very first cell, i.e. F(m,n), as for all cells in the first row (F(m,j), j=1,n) and in the first column (F(i,n), i=1,m) that we consider, we take the term within brackets in the above equation to be 0, and set F'(i,j) = F(i,j). The updating of the matrix is shown in the series of figures 3.3d – 3.3i.

Again, before proceeding, let us consider what this operation actually achieves. The updated value of the matrix in each cell is an indication of the score attainable if that cell is added to the best path that has been already discovered up to that point. The first cells we consider, i.e. the one in column m and row n, have no previous path. Therefore if we add any one of those cells to the path, the score will be whatever value is in that cell. Note that we can add only one out all these cells to the path, since according to the definition, the path cannot move horizontally or vertically. Thus if we choose one cell, we cannot choose one more in the same column or row. When we go to a cell in the next row

	i	1	2	3	4	5	6	7	8	9	10	11	12	13
j		A	B	C	N	J	R	Q	C	L	C	R	P	M
1	A	*1*												0
2	J					*1*								0
3	C			*1*					*1*		*1*			0
4	J					*1*								0
5	N				*1*									0
6	R						*1*					*1*		0
7	C			*1*					*1*		*1*			0
8	K													0
9	C			*1*					*1*		*1*			0
10	R						*1*					*1*		0
11	B		*1*										**0**	0
12	P	0	0	0	0	0	0	0	0	0	0	0	1	0

Figure 3.9d

	i	1	2	3	4	5	6	7	8	9	10	11	12	13
j		A	B	C	N	J	R	Q	C	L	C	R	P	M
1	A	*1*											0	0
2	J					*1*							0	0
3	C			*1*					*1*		*1*		0	0
4	J					*1*							0	0
5	N				*1*								0	0
6	R						*1*					*1*	0	0
7	C			*1*					*1*		*1*		0	0
8	K											**1**	0	0
9	C			*1*					*1*		*1*	1	0	0
10	R						*1*					2	0	0
11	B	1	2	1	1	1	1	1	1	1	1	1	0	0
12	P	0	0	0	0	0	0	0	0	0	0	0	1	0

Figure 3.9e

	i	1	2	3	4	5	6	7	8	9	10	11	12	13
j		A	B	C	N	J	R	Q	C	L	C	R	P	M
1	A	*1*										1	0	0
2	J					*1*						1	0	0
3	C			*1*					*1*		**3**	1	0	0
4	J					*1*					2	1	0	0
5	N				*1*						2	1	0	0
6	R						*1*				2	2	0	0
7	C			*1*					*1*		3	1	0	0
8	K										2	1	0	0
9	C			*1*					*1*		3	1	0	0
10	R	2	1	1	1	1	1	1	1	1	1	2	0	0
11	B	1	2	1	1	1	1	1	1	1	1	1	0	0
12	P	0	0	0	0	0	0	0	0	0	0	0	1	0

Figure 3.9f

	i	1	2	3	4	5	6	7	8	9	10	11	12	13
j		A	B	C	N	J	R	Q	C	L	C	R	P	M
1	A	*1*							3	3	2	1	0	0
2	J					*1*			3	3	2	1	0	0
3	C			*1*					4	3	3	1	0	0
4	J					*1*			3	3	2	1	0	0
5	N				*1*				3	3	2	1	0	0
6	R						*1*		3	3	2	2	0	0
7	C			**4**	3	3	3	3	4	3	3	1	0	0
8	K	3	3	3	3	3	3	3	3	3	2	1	0	0
9	C	2	2	3	2	2	2	2	3	2	3	1	0	0
10	R	2	1	1	1	1	1	1	1	1	1	2	0	0
11	B	1	2	1	1	1	1	1	1	1	1	1	0	0
12	P	0	0	0	0	0	0	0	0	0	0	0	1	0

Figure 3.9g

	i	1	2	3	4	5	6	7	8	9	10	11	12	13
j		A	B	C	N	J	R	Q	C	L	C	R	P	M
1	A	*1*			6	5	4	4	3	3	2	1	0	0
2	J				6	6	4	4	3	3	2	1	0	0
3	C			**7**	6	5	4	4	4	3	3	1	0	0
4	J	6	6	6	6	6	4	4	3	3	2	1	0	0
5	N	5	5	5	6	5	4	4	3	3	2	1	0	0
6	R	4	4	4	4	4	5	4	3	3	2	2	0	0
7	C	3	3	4	3	3	3	3	4	3	3	1	0	0
8	K	3	3	3	3	3	3	3	3	3	2	1	0	0
9	C	2	2	3	2	2	2	2	3	2	3	1	0	0
10	R	2	1	1	1	1	1	1	1	1	1	2	0	0
11	B	1	2	1	1	1	1	1	1	1	1	1	0	0
12	P	0	0	0	0	0	0	0	0	0	0	0	1	0

Figure 3.9h

	i	1	2	3	4	5	6	7	8	9	10	11	12	13
j		A	B	C	N	J	R	Q	C	L	C	R	P	M
1	A	**8**	7	6	6	5	4	4	3	3	2	1	0	0
2	J	7	7	6	6	6	4	4	3	3	2	1	0	0
3	C	6	6	7	6	5	4	4	4	3	3	1	0	0
4	J	6	6	6	6	6	4	4	3	3	2	1	0	0
5	N	5	5	5	6	5	4	4	3	3	2	1	0	0
6	R	4	4	4	4	4	5	4	3	3	2	2	0	0
7	C	3	3	4	3	3	3	3	4	3	3	1	0	0
8	K	3	3	3	3	3	3	3	3	3	2	1	0	0
9	C	2	2	3	2	2	2	2	3	2	3	1	0	0
10	R	2	1	1	1	1	1	1	1	1	1	2	0	0
11	B	1	2	1	1	1	1	1	1	1	1	1	0	0
12	P	0	0	0	0	0	0	0	0	0	0	0	1	0

Figure 3.9i

Figure 3.9d – Figure 3.9i. Various steps in the updating of the matrix F(i,j) to F'(i,j). At each stage, the last cell updated is shown in **bold**. The numbers in *italics* are the values of F(i,j), zeroes not shown. The other numbers are the values of F'(i,j), zeroes are also shown.

or column, we update its value by taking the best possible score till that point, and adding to it the value of the current cell, thus giving the best score can be obtained if the current cell is added to the path. Proceeding in this way, we cover all possible paths, keeping track at the same time of the best possible path up to that point.

Now the final step of the algorithm is obvious. This is to construct a trace back, usually starting from a point at the left-hand top portion of the matrix, where a cell that gives the score corresponding to the best possible path will be present, i.e a cell with the highest score. From that cell, the next cell on the best possible path is easily found as the one with the next highest score, and then the next one, and so on till the final cell is added to the path. To then convert the path traced into the appropriate alignment is a trivial task. Note that at some points during the trace back, there may be more than one way to proceed to the next cell. At such branch points we consider both paths, and the corresponding alignments, as equally likely, since both lead to the identical score. The trace back procedure is illustrated in Figure 3.3j. This completes the alignment procedure.

	i	1	2	3	4	5	6	7	8	9	10	11	12	13
j		A	B	C	N	J	R	Q	C	L	C	R	P	M
1	A	8	7	6	6	5	4	4	3	3	2	1	0	0
2	J	7	7	6	6	6	4	4	3	3	2	1	0	0
3	C	6	6	7	6	5	4	4	4	3	3	1	0	0
4	J	6	6	6	6	6	4	4	3	3	2	1	0	0
5	N	5	5	5	6	5	4	4	3	3	2	1	0	0
6	R	4	4	4	4	4	5	4	3	3	2	2	0	0
7	C	3	3	4	3	3	3	3	4	3	3	1	0	0
8	K	3	3	3	3	3	3	3	3	3	2	1	0	0
9	C	2	2	3	2	2	2	2	3	2	3	1	0	0
10	R	2	1	1	1	1	1	1	1	1	1	2	0	0
11	B	1	2	1	1	1	1	1	1	1	1	1	0	0
12	P	0	0	0	0	0	0	0	0	0	0	0	1	0

Figure 3.9j

```
Best alignment      Sequence 3: A B C N J - R Q C L C R - P M
                                | X |   |   |   | X | |   |
                    Sequence 4: A J C - J N R - C K C R B P -

Alternate alignment Sequence 3: A B C - N J R Q C L C R - P M
                                | X |   |   |   | X | |   |
                    Sequence 4: A J C J N - R - C K C R B P -
```

Figure 3.9j The final step of the Needleman-Wunsch algorithm. A trace back procedure identifies the best path, and therefore the best alignment. There are two equally good paths in this example.

We have arrived reliably at the best possible alignment. We have also seen the largest possible number of matches. In this example there can be a maximum of 8 matches. Note that we have freely allowed mismatches and gaps to occur. Normally a penalty is associated with every gap, since this indicates that there has been the insertion of a residue or residues in one sequence, or equivalently the

deletion of a residue or residues in the other sequence. Gaps therefore are a feature of the alignment, and not of either sequence. To highlight this, gaps are also called indels (for insertion/deletion). The indel penalty varies with the nature of the sequences being aligned i.e., whether they are DNA or protein sequences, as well as with the biological relevance of the alignment. They are discussed briefly below along with other scoring schemes. (A fuller discussion is reserved for the next chapter). Here for illustration, if we assume a penalty of 0.5 points for every gap, as well as a penalty of 0.25 points for every mismatch we may now calculate the total score of the alignment shown in Figure 3.3j. This has 8 matches, 2 mismatches and 4 gaps, if we count the overhang also as a gap. The total score is therefore 8 x 1 (1 point for each match) + 2 x (-0.25) (since this is a penalty) + 4 x (-0.5) (also a penalty) = 5.5. Note that the scoring scheme we used while constructing the alignment optimised only the number of matches. It ignored the gaps and the mismatches. If we had chosen a scoring scheme with a strong gap penalty, then the emphasis would be on minimizing the number of gaps, and this would lead to a different alignment. For example, with a gap penalty of, say, 8 points, the best alignment would be one in which the two sequences line up with no gaps at all. There would only be 4 matching residue pairs in such an alignment. Instead of making an alignment with no penalties for mismatches or gaps, and considering only matches, we could incorporate such penalties in the algorithm. In that case the alignment would have been already optimised with respect to gaps and mismatches also. Such algorithms are considered below, after a brief discussion on scoring schemes.

3.3.2 Scoring schemes – briefly

A binary scoring scheme is inappropriate for analysing biological sequences, especially protein sequences. All replacements of amino acids do not change the structure and/or the function to the same extent. For example, if a cysteine residue in one sequence is replaced by a different residue in another sequence, it could lead to the absence of a disulphide bridge in the second protein, and this could have disruptive consequences for its structure and function. Such replacements are therefore unlikely to occur between related proteins, and the scoring scheme must reflect this improbability. Conversely, there are residue pairs that could replace one another with very little effect on the structure and the function. Again the scoring scheme must reflect this fact.

The variety of such scoring schemes available, the procedures for devising them, and their pros and cons are all discussed in Chapter 4. Here we will introduce, in Table 3.4 the PAM100 scoring scheme. The PAM series of matrices are 20 x 20 matrices also known as 'substitution' matrices. Each element of the matrix tells the score we have to use if, in an alignment, we find the residue pair labelling that element as matching residues. The unit of measure is 'bits', since the matrices are derived from information theoretical considerations. This matrix is called the substitution matrix s(x,y), where x and y represent amino acids.

Gap penalties are also part of the scoring scheme, and must be chosen along with the substitution scores. Again a detailed discussion of these penalties is left for a later chapter. Here we briefly remark that there are two aspects to gaps – the number of gaps in the alignment and their respective sizes. Normally, therefore, there is a 'gap opening' penalty, which is the basic penalty applied whenever a gap exists. In addition for each gap there is a 'gap extension' penalty, which depends on the size of the respective gap. We could use sophisticated functions to reflect various biological realities, but in the example below will use a very simple linear function, which is given as

$$G = k \times n$$

where k is a constant, set to -8 in the examples below, and n is the number of gaps.

3.3.3 The Needleman-Wunsch algorithm – part II

We will now describe a modified version of the Needleman and Wunsch algorithm. This version was first suggested in 1982, and is more efficient than the original one. It allows the use of any one of the

Table 3.4 The PAM100 substitution matrix

	A	R	N	D	C	Q	E	G	H	I	L	K	M	F	P	S	T	W	Y	V
A	4	-3	-1	-1	-3	-2	0	1	-3	-2	-3	-3	-2	-5	1	1	1	-7	-4	0
R	-3	7	-2	-4	-5	1	-3	-5	1	-3	-5	2	-1	-6	-1	-1	-3	1	-6	-4
N	-1	-2	5	3	-5	-1	1	-1	2	-3	-4	1	-4	-5	-2	1	0	-5	-2	-3
D	-1	-4	3	5	-7	0	4	-1	-1	-4	-6	-1	-5	-8	-3	-1	-2	-9	-6	-4
C	-3	-5	-5	-7	9	-8	-8	-5	-4	-3	-8	-8	-7	-7	-4	-1	-4	-9	-1	-3
Q	-2	1	-1	0	-8	6	2	-3	3	-4	-2	0	-2	-7	-1	-2	-2	-7	-6	-3
E	0	-3	1	4	-8	2	5	-1	-1	-3	-5	-1	-4	-8	-2	-1	-2	-9	-5	-3
G	1	-5	-1	-1	-5	-3	-1	5	-4	-5	-6	-3	-4	-6	-2	0	-2	-9	-7	-3
H	-3	1	2	-1	-4	3	-1	-4	7	-4	-3	-2	-4	-3	-1	-2	-3	-4	-1	-3
I	-2	-3	-3	-4	-3	-4	-3	-5	-4	6	1	-3	1	0	-4	-3	0	-7	-3	3
L	-3	-5	-4	-6	-8	-2	-5	-6	-3	1	6	-4	3	0	-4	-4	-3	-3	-3	0
K	-3	2	1	-1	-8	0	-1	-3	-2	-3	-4	5	0	-7	-3	-1	-1	-6	-6	-4
M	-2	-1	-4	-5	-7	-2	-4	-4	-4	1	3	0	9	-1	-4	-3	-1	-6	-5	1
F	-5	-6	-5	-8	-7	-7	-8	-6	-3	0	0	-7	-1	8	-6	-4	-5	-1	4	-3
P	1	-1	-2	-3	-4	-1	-2	-2	-1	-4	-4	-3	-4	-6	7	0	-1	-7	-7	-3
S	1	-1	1	-1	-1	-2	-1	0	-2	-3	-4	-1	-3	-4	0	4	2	-3	-4	-2
T	1	-3	0	-2	-4	-2	-2	-2	-3	0	-3	-1	-1	-5	-1	2	5	-7	-4	0
W	-7	1	-5	-9	-9	-7	-9	-9	-4	-7	-3	-6	-6	-1	-7	-3	-7	12	-2	-9
Y	-4	-6	-2	-6	-1	-6	-5	-7	-1	-3	-3	-6	-5	4	-7	-4	-4	-2	9	-4
V	0	-4	-3	-4	-3	-3	-3	-3	-3	3	0	-4	1	-3	-3	-2	0	-9	-4	5

various scoring schemes mentioned above, instead of the binary scoring scheme used in Section 3.2.1. The trace back is almost automatic and is performed as the matrix is updated, so that at the end of the procedure both the best possible score, as well as the particular alignment that leads to this score are both readily available. Finally, gap penalties are directly incorporated into the scoring scheme, and thus the alignment obtained is an optimal 'gapped ' alignment.

In order to describe this algorithm we will use the pair of sequences we have used in Section 3.2.1, but with modifications to ensure that the symbols correspond to the twenty proteinaceous amino acids. Thus we now have to find the best (gapped) alignment between the following pair of sequences

Sequence 7: ADCNGRQCLCRPM

Sequence 8: AGCGNRCKCRYP

Also we will use the PAM100 scoring scheme shown in Table 3.1, and a gap penalty of 8 per gap.

The algorithm consists of manipulating a matrix, named F(i,j). This matrix is first constructed in the usual way, by writing the two sequences, one on the top and the other as the first column on the left (Figure 3.10a). We first address the column i = 0 and the row j = 0. These are filled as follows.

$F(i,0) = -i \times d$, for i = 1, m

$F(0,j) = -j \times d$, for j = 1, n

where d, the gap penalty, is assumed to be 8 in the present case. m and n are the lengths of the two sequences respectively. The result of this operation is shown in Figure 3.10b. Next we address the other cells. We start from the top left hand corner of the matrix (i.e. cell (i = 1, j = 1)) and move towards the bottom right hand corner (cell (i = n, j = m), where n and m are the lengths of the two sequences respectively), filling up the squares recursively according to the following rules.

$$F(i,j) = \text{maximum} \begin{cases} F(i-1,j-1) + s(x_i,y_j) \\ F(i-1,j) - d \\ F(i,j-1) - d \end{cases}$$

	i	0	1	2	3	4	5	6	7	8	9	10	11	12	13
j			A	D	C	N	G	R	Q	C	L	C	R	P	M
0		0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80	-88	-96	-104
1	A	-8													
2	G	-16													
3	C	-24													
4	G	-32													
5	N	-40													
6	R	-48													
7	C	-56													
8	K	-64													
9	C	-72													
10	R	-80													
11	Y	-88													
12	P	-96													

Figure 3.10a The F(i,j) matrix used in the modified Needleman-Wunsch algorithm.

Here $s(x_i,y_j)$ are the values taken from the substitution matrix in Table 3.1, x_i and y_j are the pair of amino acids that label cell (i,j), and d is the gap penalty mentioned above. At each stage of the operation, the value that is placed in the current cell comes from one of three previous cells that have already been filled. An arrow is also stored to indicate which of the three previously filled cells contributes to the present one. It may happen that two or sometimes even all three previous cells yield the same final sum. In that case arrows are stored to both or all three, as the case may be. The arrows or pointers help to easily construct the trace back. Some steps of the procedure are illustrated in Figures 3.10b to 3.10d. Once the matrix is filled, it is a trivial matter to follow the arrows backwards to construct the best alignment, as shown in Figure 3.10e. The score of this alignment is immediately available. For this example, consisting of Sequence 7 and Sequence 8, the best global alignment is

Sequence 7: A D C N G R Q C L C R - P M
Sequence 8: A G C - G N R C K C R Y P -

and the score of this alignment is 28 bits. Note that in the above alignment, there is at least one possible exact match that has been ignored (between R residues). This is because the cost of opening a gap to accommodate this pairing is more than the advantage gained from it. Thus R in sequence 7 is paired against N in sequence 8, and R in sequence 8 is paired against Q in sequence 7. A different scoring scheme, or a different gap penalty would result in a different alignment. Note also that the score does not increase monotonically as we proceed along the alignment. For example, if the ignore the last two residues of both sequences, we would get an alignment with a greater score of 29 bits. However, 28 bits is the best possible score for a global alignment of the two sequences, taking all the residues into consideration.

	i	0	1	2	3	4	5	6	7	8	9	10	11	12	13
j			A	D	C	N	G	R	Q	C	L	C	R	P	M
0		0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80	-88	-96	-104
1	A	-8													
2	G	-16													
3	C	-24													
4	G	-32													
5	N	-40													
6	R	-48													
7	C	-56													
8	K	-64													
9	C	-72													
10	R	-80													
11	Y	-88													
12	P	-96													

Figure 3.10b The first step in the modified Needleman-Wunsch algorithm.

	i	0	1	2	3	4	5	6	7	8	9	10	11	12	13
j			A	D	C	N	G	R	Q	C	L	C	R	P	M
0		0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80	-88	-96	-104
1	A	-8	4	-4											
2	G	-16	**-4**												
3	C	-24													
4	G	-32													
5	N	-40													
6	R	-48													
7	C	-56													
8	K	-64													
9	C	-72													
10	R	-80													
11	Y	-88													
12	P	-96													

Figure 3.10c The first few cells are filled according to the algorithm. The value in **bold** indicates the last cell filled. Arrows show the previous cell that leads to the value in the current one.

	i	0	1	2	3	4	5	6	7	8	9	10	11	12	13
j			A	D	C	N	G	R	Q	C	L	C	R	P	M
0		0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80	-88	-96	-104
1	A	-8	4	-4	-12	-20	-28	-36	-44	-52	-60	-68	-76	-84	-92
2	G	-16	-4	3	-5	-13	-15	-23	-31	-39	-47	-55	-63	-71	-79
3	C	-24	-12	-5	12	4	-4	-12	-20	-22	-30	-38	-46	-54	-62
4	G	-32	-20	-13	4	11	9	1	-7	-15	-23	-31	-39	-47	-55
5	N	-40	-28	-17	-4	9	10	7	0	-8	-16	-24	-32	-40	-48
6	R	-48	-36	-25	-12	1	4	3	8	0	-8	-16	-17	-25	-33
7	C	-56	-44	-33	-16	-7	-4	-1	0	17	9	1	-7	-15	-23
8	K	-64	-52	-41	-24	-15	-10	-2	-1	9	13	5	3	-5	-13
9	C	-72	-60	-49	-32	-23	-18	-10	-9	8	5	22	14	6	-2
10	R	-80	-68	-57	-40	-31	-26	-18	-9	0	3	14	29	21	13
11	Y	-88	-76	-65	-48	-39	-34	-26	-17	-8	-3	6	21	22	16
12	P	-96	-84	-73	-56	-47	-41	-34	-35	-16	-11	-2	13	28	**20**

Figure 3.10d The completed matrix with all cells filled. (Note the value in **bold** at the bottom right corner, indicating this is the last value filled). For the sake of clarity, the arrows are not shown.

	i	0	1	2	3	4	5	6	7	8	9	10	11	12	13
j			A	D	C	N	G	R	Q	C	L	C	R	P	M
0		0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80	-88	-96	-104
1	A	-8	**4**	-4	-12	-20	-28	-36	-44	-52	-60	-68	-76	-84	-92
2	G	-16	-4	**3**	-5	-13	-15	-23	-31	-39	-47	-55	-63	-71	-79
3	C	-24	-12	-5	**12**	4	-4	-12	-20	-22	-30	-38	-46	-54	-62
4	G	-32	-20	-13	4	11	**9**	1	-7	-15	-23	-31	-39	-47	-55
5	N	-40	-28	-17	-4	9	10	**7**	0	-8	-16	-24	-32	-40	-48
6	R	-48	-36	-25	-12	1	4	3	**8**	0	-8	-16	-17	-25	-33
7	C	-56	-44	-33	-16	-7	-4	-1	0	**17**	9	1	-7	-15	-23
8	K	-64	-52	-41	-24	-15	-10	-2	-1	9	**13**	5	3	-5	-13
9	C	-72	-60	-49	-32	-23	-18	-10	-9	8	5	**22**	14	6	-2
10	R	-80	-68	-57	-40	-31	-26	-18	-9	0	3	14	**29**	21	13
11	Y	-88	-76	-65	-48	-39	-34	-26	-17	-8	-3	6	21	22	16
12	P	-96	-84	-73	-56	-47	-41	-34	-35	-16	-11	-2	13	**28**	20

Figure 3.10e The trace back. Only the relevant arrows are shown. **Bold** values correspond to paired residues in the alignment.

3.3.4 The Smith-Waterman algorithm

This is a local alignment method, in which portions of one sequence are aligned to similar portions of the second sequence. It is of course especially useful in comparing a relatively short sequence with a very long sequence, or a set of long sequences. It frequently happens that a portion or portions of the relatively short query sequence is similar to not just one, but many regions of the long target sequence. Also it happens often that portion 1 of the query sequence aligns with one part of the target, and portion 2 of the query aligns with another part, but these two portions of the target do not occur in the same order, or at the same distance as the portions of the query. The Smith-Waterman algorithm is designed to handle such cases. We shall again illustrate the method using an example. In order to make the nature of the method clear, we will use another pair of sequences.

Sequence 9: CAGCRMADCGNRQ

Sequence 10: AGCGNRCAKGCRM

We start again by constructing a matrix F(i,j), as shown in Figure 3.11a. Once again we begin at the top left corner and proceed to the bottom right. The matrix elements are now filled recursively according to the following rules.

$F(i,0) = 0$, for $i = 1, m$

$F(0,j) = 0$, for $j = 1, n$

$$F(i,j) = \text{maximum} \begin{cases} 0 \\ F(i-1,j-1) + s(x_i,y_j) \\ F(i-1,j) - d \\ F(i,j-1) - d \end{cases}$$

	i	0	1	2	3	4	5	6	7	8	9	10	11	12	13
j			C	A	G	C	R	M	A	D	C	G	N	R	Q
0		0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	A	0	0	4	**1**										
2	G	0	0												
3	C	0													
4	G	0													
5	N	0													
6	R	0													
7	C	0													
8	A	0													
9	K	0													
10	G	0													
11	C	0													
12	R	0													
13	M	0													

Figure 3.11a The first few cells are filled according to the Smith-Waterman algorithm. The value in **bold** indicates the last cell filled. Arrows show the previous cell that leads to the value in the current one.

	i	0	1	2	3	4	5	6	7	8	9	10	11	12	13
j			C	A	G	C	R	M	A	D	C	G	N	R	Q
0		0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	A	0	0	4	1	0	0	0	**4**	0	0	1	0	0	0
2	G	0	0	1	9	1	0	0	1	**3**	0	5	0	0	0
3	C	0	9	1	1	18	10	2	0	0	**12**	4	0	0	0
4	G	0	1	10	6	10	13	6	3	0	4	**17**	9	1	0
5	N	0	0	2	9	2	8	9	5	6	0	9	**22**	14	6
6	R	0	0	0	1	4	9	7	6	1	1	1	14	**29**	21
7	C	0	**9**	1	0	10	2	2	4	0	10	2	6	21	21
8	A	0	1	**13**	5	2	7	0	6	3	2	11	3	13	19
9	K	0	0	5	10	2	4	7	0	5	0	3	12	5	13
10	G	0	0	1	**10**	5	0	0	8	0	0	5	4	7	5
11	C	0	9	1	2	**19**	11	3	0	1	9	1	0	0	0
12	R	0	1	6	0	11	**26**	18	10	2	1	4	0	7	1
13	M	0	0	0	2	3	18	**35**	27	19	11	3	0	0	5

Figure 3.11b The completed matrix with all cells filled. Also shown are the traceback for two possible local alignments. Only relevant arrows are shown. **Bold** values correspond to paired residues in the alignment

Again, $s(x_i,y_j)$ are the values taken from the substitution matrix in Table 3.1, x_i and y_j are the pair of amino acids that label cell (i,j), and d is the gap penalty. The major difference in this method as compared to the previous one is the complete absence of negative numbers. Of the four terms considered in each cell, three are calculated from values already present in previously filled cells. The fourth number considered is zero, and if the other three numbers are negative, then zero is the maximum value and is filled into the current cell. Again, arrows may be stored at each step to help in the trace back. Now, since we are looking for local alignments, we may have multiple starts. Each cell with a high score is a possible start site for a good local alignment. From every such start site we construct the trace back and continue to follow it until we come to cell with the value zero. That signifies the end of the local alignment. Figures 3.11b shows the final filled matrix along with the trace back corresponding to two local alignments. The two alignments are

```
Sequence 9:    C A G C R M A D C G N R Q - - - - - -
                           | x | | | | x
Sequence 10:   - - - - - - A G C G N R C A K G C R M
```

and

```
Sequence 9:    - - - - - - C A - G C R M A D C G N R Q
                           | |   | | | |
Sequence 10:   A G C G N R C A K G C R M - - - - - - -
```

with scores 29 and 35 bits respectively. Note that the same matrix also points to other possible local alignments, but with lower scores. Thus whether a local alignment is considered or not will depend on the cut-off score used. For example, if we decide to consider only scores above 30 bits, then there is only one 'correct' local alignment. If we consider scores above 15, there is at least one more possible 'correct' local alignment. Not only that, the two alignments given above may be extended to include a few more residues, though the score now becomes less than previously, while staying above the cutoff.

There are, of course, now several alignment algorithms that are modifications of the above schemes to satisfy specific requirements, as for example to find repeat matches, or overlapping matches and so on. These however are beyond the scope of this book. We will end this section by remarking that the Smith-Waterman and the Needleman-Wunsch methods are important components of the two most popular sequence alignment programs in current use, namely FASTA and BLAST.

3.4 BLAST and FASTA

Of these two programs, BLAST has assumed almost iconic status, and has become representative not only of sequence matching and comparisons, but very nearly of all of bioinformatics. While both programs do approximately the same things, with approximately the same efficiency, there are some differences as well. Let us consider the similarities first. Both are sequence search and comparison algorithms. Both do fast searches through large databases for matches to the query sequence, and both then do more detailed alignments of the query sequences with the matches. Both use hash tables as well as dynamic programming. Both are heuristic algorithms[8]. Both return local alignments, and are not a single program, but rather a family of programs with implementations designed to compare a sequence to a database in nearly every possible way. They compare a DNA sequence against a DNA database, a translated (in all six frames[9]) version of a DNA sequence against a translated (six-frame) version of the DNA database, a translated (six-frame) version of a DNA sequence against a protein database, a protein sequence against a translated (six-frame) version of a DNA database, or a protein sequence against a protein database. Both give statistical indicators of the quality of each match, which may be used to decide on its significance. Both are freely available to all users over the Internet, either as a service, or as a program that may be downloaded and installed on a local computer system.

Let us now consider the differences. BLAST was developed at NCBI. It can find more than one region of gapped similarity. It has very fast heuristics and parallel implementations. It is restricted to precompiled, specially formatted databases. The FASTA family of programs was developed at the University of Virginia. It can find only one gapped region of similarity. It is relatively slow as compared to BLAST, and should often be run in the background. It does not require specially prepared, preformatted databases. We will now look at each algorithm in detail.

3.4.1 BLAST

The BLAST algorithm uses a word-based heuristic to execute an approximate version of the Smith-Waterman algorithm known as the 'maximal segment pairs' algorithm. A word list is prepared for the query sequence and is searched against the table for the database to identify exact matches. These are the maximal segment pairs or MSPs. MSPs do not allow gaps, and have the very valuable property that their statistics are well understood. Thus, we can readily compute a significance probability for a maximal segment pair alignment. The price for being able to do this is that the alignments cannot have

---

[8] 'Heuristics' are algorithms that use approximation techniques. The word is also defined as 'serving to guide, discover, or reveal, but unproved or incapable of proof'. In database similarity searching techniques the heuristic usually restricts the necessary search space by calculating a statistical quantity that allows the program to decide whether further scrutiny of a particular match should be carried out. However many correct possibilities may be missed, since the search may not be exhaustive.

[9] Each DNA sequence is translated in the three possible reading frames, the complementary sequences are then translated again in three reading frames, yielding six frames in all.

gaps. The word size W is chosen as 3 for proteins, and 11 for nucleic acid sequences. This is because, since there are only four nucleotides, the amount of background noise is too large with smaller word sizes. Once the look-up table of exact matches has been compiled, BLAST tries to find all double word hits along the same diagonal (i.e. no indels) within some specified distance. These word hits of size W do not have to be identical; rather, their scores have to be better than some threshold value T. Each double word hit that passes this step then triggers a process called un-gapped extension in both directions, such that each diagonal is extended as far as it can go, until the running score starts to drop below a pre-defined value within a certain range. The result of this pass is called a High-Scoring segment Pair or HSP. Those HSPs that pass this step with a score better than a minimum then begin a gapped extension step utilizing dynamic programming. Those gapped alignments with expectation values or E-values better than the user specified cutoff are reported. E-values are normalized versions of the probabilities described in section 3.1.4. They are now the 'expectation' that an alignment with a score better than the one reported may be obtained purely by chance.

BLAST is most frequently used over the Internet on the BLAST server (http://www.ncbi.nlm.nih.gov/BLAST/). Figure 3.12 shows the BLAST home page. There are several versions of BLAST and the home page lists all of them. 'blastn' is 'nucleotide-nucleotide' blast, or the matching of a nucleic acid sequence against the nucleic acid sequence database. Similarly 'blastp' is protein-protein blast. A different subset of programs performs translated searches, where either the

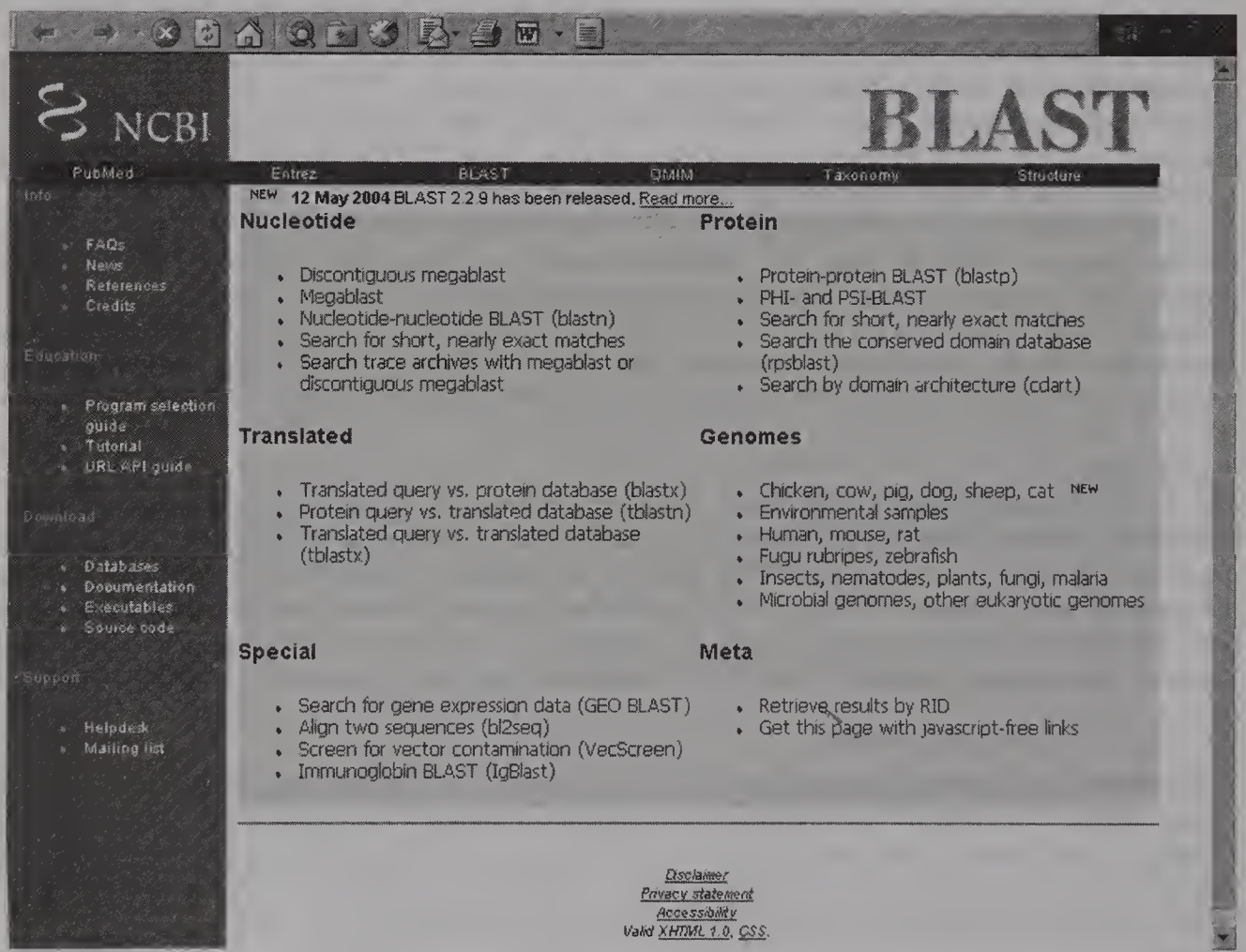


Figure 3.12 The BLAST home page

query ('blastx'), or the database ('tblastn'), or both ('tblastx') are nucleic acid sequences translated in six frames into protein sequences. There are other specialized programs such as, for example, 'megablast'. This program uses a different algorithm for nucleotide sequence alignment search. It is optimized for aligning sequences that differ slightly, perhaps as a result of sequencing errors. BLAST may also be run against specialized databases, such as particular genomes, etc. Figure 3.13 shows the input page for blastp. We have entered the sequence of a probable bacterial hemoglobin, as an

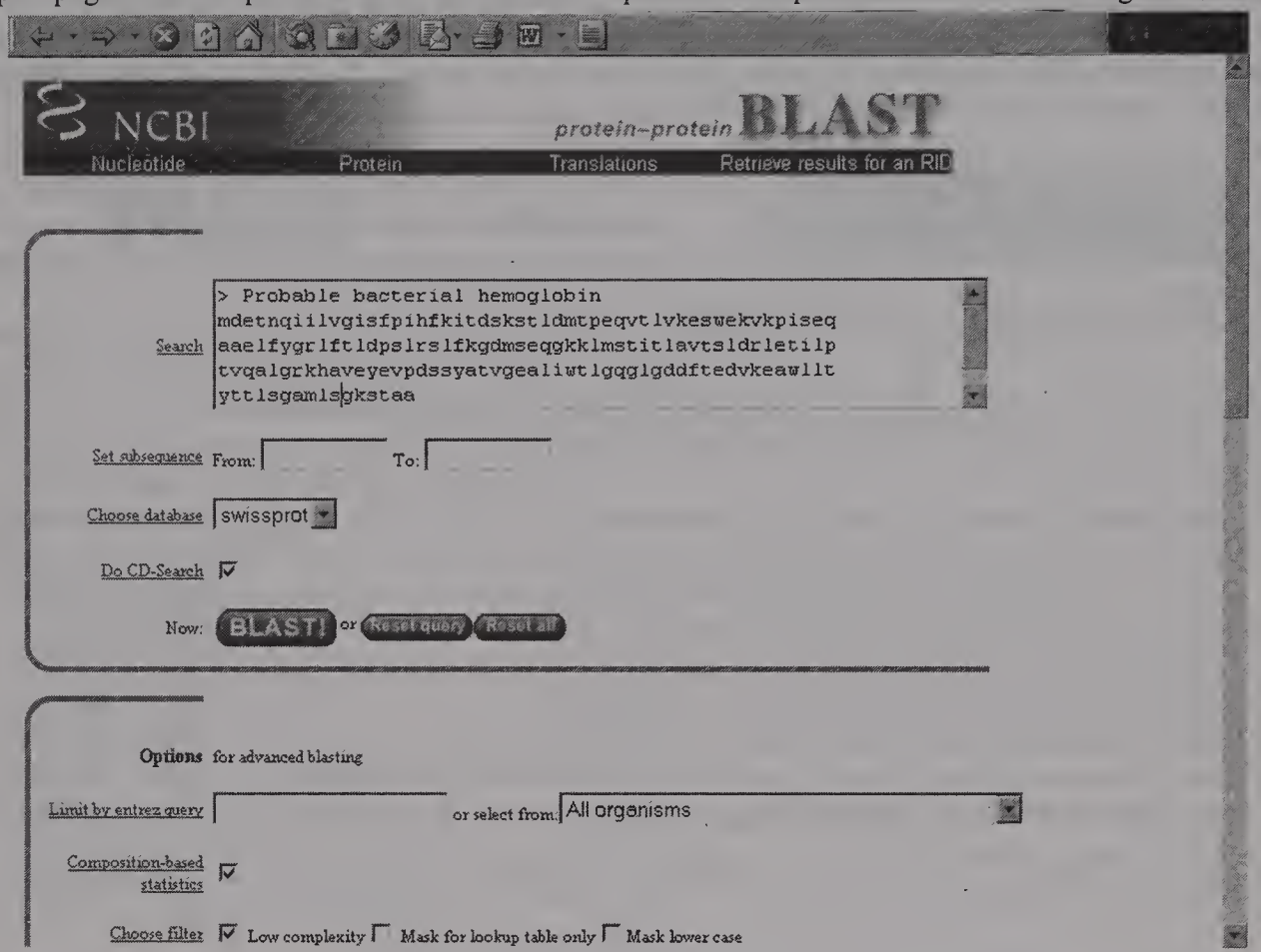


Figure 3.13 The BLAST query page

example. The sequence has been entered in the so-called FASTA format, in which the first line, always beginning with the symbol '>', contains a brief user-controlled description of the sequence or any other information the user requires. The sequence begins on the next line and proceeds without gaps to the end, with any number of line breaks. There are several user-definable options. Some of the more important of these are mentioned below.

- The database to be searched. (This entry is mandatory. Here we choose 'Swissprot')
- The expectation value cut-off for reporting matches (default value is 10).
- The word size of the MSPs (default: 3).
- The substitution matrix to be used (default: BLOSUM62).
- Gap penalties (default: gap opening penalty 11, extension penalty 1).
- Format of the output.
- The number of alignments to be displayed in the output, etc.

Figure 3.14 shows a section of the output page. We see first a list of all hits of the query sequence with sequences in the database that have an expectation value better (i.e. less than) than the number

specified. The Z score and the E value of each hit are given alongside. This list is followed by display of the alignment of each of these hits. We have chosen a rather unusual example, namely a protein in a bacterial species that appears to be similar to Hemoglobin. The first hit is with a neuroglobin from Zebra fish, proteins that are also hypothesized to be involved in oxygen transport. This match has a score of 69 bits and E value of 3×10^{-12}, indicating a strong biological significance for the match. Note that the strongest match with mammalian (guinea pig) hemoglobin is hit number 15 with a score of 48 and E value of 1×10^{-5}. The alignment these two sequences is shown below, along with other statistics, such as the number of identical residues in identical locations after the match, and the number of positives, or similar residues at the same position.

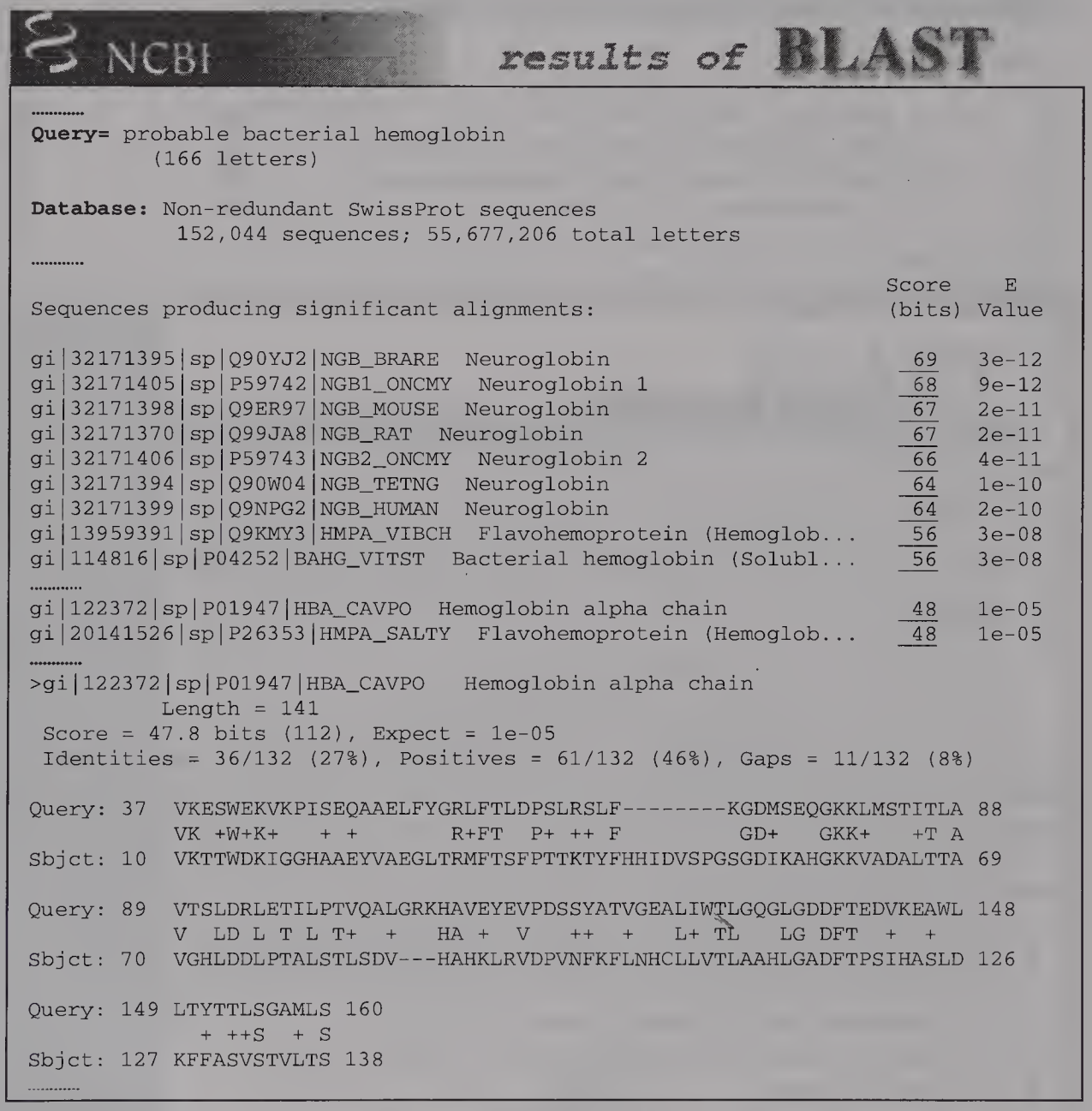


Figure 3.14 Portions of the BLAST output page. Dotted lines indicate portions omitted for clarity

This section would be incomplete without a description of two additional algorithms that have been added in 1998 to the BLAST family. The first is called PSI-BLAST, and stands for Position Specific Iterated BLAST. This algorithm returns more distantly related sequences from the database than BLAST. In other words, the sensitivity of the search is improved, without overly compromising on the specificity. PSI-BLAST brings this about by using a profile[10] to search the database instead of just the query sequence. The first step in PSI-BLAST is a search of the query sequence against the database using normal BLAST. In the second step, all hits with E values better than (i.e. less than) a threshold, usually 0.01, are selected and aligned using multiple sequence alignment. This alignment is then reduced to a position-specific profile. All sequences in the alignment are not weighted equally while evaluating the profile. In fact, if there is a set of sequences with a very high degree of homology to the query sequence, as well as to each other, then all sequences in this set are given low weights. Thus, the weight of the contribution of each sequence to the profile is calculated based on E-values, Z scores, and the number of related sequences that occur in the alignment. The profile so calculated is used to search the database for related sequences. Aligning a profile to a sequence is conceptually the same as aligning two sequences with each other, and algorithmically only slightly different, and only a slight modification of the BLAST algorithm is required to carry this out. The hits generated in this search are then iteratively used to refine the profile, another search is carried out with the new profile, and so on, until even distant relatives of the original query sequence have been recovered. The final test of whether the sequences are actually related is made by performing pairwise alignment of each sequence recovered with the original query sequence, and calculating the Z score and the E value. A variant of PSI-BLAST is called RPS-BLAST for Reverse PSI-BLAST. This algorithm searches a query sequence against a database of profiles, the reverse of the method in PSI-BLAST.

The second algorithm referred to above is called PHI-BLAST and stands for Pattern-Hit Initiated BLAST. This is a search program for which the input is not only a query DNA or protein sequence, but also a pattern. The pattern is written as a small sequence of residues or sets of residues, with wild cards[11] and spaces also allowed. This way of writing the pattern is called a 'regular expression' (see Chapter 4). PHI-BLAST helps to answer the following question: Given a query sequence that contains a particular recognized pattern, what other sequence in the database has the same pattern and is homologous to the query sequence in the neighbourhood of the pattern? In the cases where such patterns are known, PHI-BLAST is particularly useful is filtering out false positives of sequence similarity that may arise by chance.

3.4.2 FASTA

This program was originally called Fast, and the most popular version of it is called Fast A or FASTA. Since FASTA is the more commonly used name, we will use that henceforth. FASTA was the first widely used, powerful sequence database searching algorithm. It has been continually refined such that it remains a viable alternative to BLAST, especially if one is restricted to searching DNA against DNA without translation. It is also helpful in situations where BLAST finds no significant alignments. FASTA may be more sensitive than BLAST in these situations.

The first step of FASTA is again a hashing style algorithm. It builds words of a specific k-tuple size. By default this is two for peptides. It then identifies all exact word matches between the query

---

[10] A 'profile' is a probabilistic representation of a multiple sequence alignment. For proteins it is obtained by evaluating the probability of occurrence of each of the 20 residues at each position of the sequence. A profile that is L residues long is thus a matrix of L x 20 numbers. Profiles are described in greater detail in later chapters.

[11] A 'wild card' is sequence position in which any residue may occur. This is usually represented by a star symbol '*'.

sequence and the database members. Note that the word matches must be exact for FASTA. (In BLAST, on the other hand, after the creation of a lookup table, the matches between the two sets of words were considered hits if the similarity was above some threshold value.) From these exact word matches scores are assigned to each continuous, ungapped, diagonal by adding all of the exact match BLOSUM values. The ten highest scoring diagonals for each query-database pair are then scored again using BLOSUM similarities as well as identities. The ends are trimmed to maximize the score. Next the program looks around to see if nearby off-diagonal alignments can be combined by incorporating gaps. If so, a new score is calculated by summing up all the contributing scores, penalizing gaps with a penalty for each. The program then constructs an optimal local alignment for all pairs with scores better than some specified, threshold using a variation of dynamic programming. Here, only those portions of the matrix that lie in the neighbourhood of the diagonal line of hits are considered. The region for the dynamic programming alignment is bounded by the 'window size'. This variable specifies the neighbourhood of the diagonal. The window size limits the number of insertions or deletions one sequence can accumulate with respect to the other sequence in the alignment. Thus, the significant speedups in observed in a FASTA search relative to a full dynamic programming search is due to the prior restriction in alignment space. Next, FASTA calculates a normalized Z score for the sequence pair. It then compares the distribution of these Z scores to the actual extreme-value distribution of the search. Using this distribution, the program estimates the number of sequences that would be expected to have, purely by chance, a Z score greater than or equal to the Z score obtained in the search. This is reported as the E value or expectation value. Finally the program uses full Smith-Waterman local dynamic programming to produce the final alignments, before reporting them.

Some of the disadvantages in the FASTA approach can be illustrated by two extreme examples. In the first example there are two proteins that share 50% identity - but the proper alignment consists of alternating matches and mismatches. With a word size of two, there would be no word matches along the main diagonal of the dot plot for the sequences and the proper alignment is not found. The second case consists of two proteins that are almost identical, except the second protein has a 20-residue insertion into the middle of the sequence. If the window size is 15, then the dynamic programming alignment phase of FASTA will not have enough alignment space to jump the 20-residue insertion. Thus, the resulting alignment will be either the sequence prior to or after the insertion (whichever had the higher diagonal scores) and the fact that the proteins were basically identical (with only one long insertion) will be missed.

Like BLAST, FASTA is now a family of programs to perform different types of searches and sequence comparisons. FASTA compares a protein sequence to another protein sequence or to a protein database, or a DNA sequence to another DNA sequence or a DNA library. FASTX/FASTY compares a DNA sequence to a protein sequence database, translating the DNA sequence in three forward (or reverse) frames and allowing frame shifts. TFASTX/TFASTY compares a protein sequence to a DNA sequence or DNA sequence library. The DNA sequence is translated in three forward and three reverse frames, and the protein query sequence is compared to each of the six derived protein sequences. The DNA sequence is translated from one end to the other; no attempt is made to edit out intervening sequences. Termination codons are translated into unknown ('X') amino acids. FASTF/TFASTF compares an ordered peptide mixture, as would be obtained by Edman degradation of a protein, against a protein or DNA database. FASTS/TFASTS compares set of short peptide fragments, as would be obtained from mass-spectroscopic analysis of a protein, against a protein or DNA database. The WWW FASTA server is hosted by EBI, the European Bioinformatics Institute. (http://www.ebi.ac.uk/fasta33/).

3.5 Summary

- One of chief tasks of bioinformatics is to compare DNA and protein sequences and find similarities, or differences, and infer structural, functional or evolutionary relationships.
- Sequence comparison and alignment is not a trivial task.
- Any written language may be analysed by almost exactly the same methods used to analyse DNA and protein sequences.
- Many alignments are possible between any two sequences. To decide which of the alignments is the best, we need a function that helps us to find some figure of merit or score for each alignment.
- Brute force or trial and error approaches to sequence alignment lead to combinatorial explosion.
- We say two sequences are similar to each other when, after the best alignment, identical (or similar) residues occur at identical (or similar) positions.
- Similarity could arise by chance, or it could be a convergence towards a common sequence and structure and therefore function, through evolution, or, the similarity could arise from divergent evolution of the two sequences from a common ancestral sequence.
- The similarity that rises from this last mechanism alone is called homology.
- Homologs, heterologs, analogs, orthologs, paralogs, xenologs are words that describe the different ways in which sequence similarity could arise.
- Extreme value statistics helps us to calculate the statistical significance of an alignment.
- For each alignment, we calculate two numbers – the Z score and the expectation or E value.
- The Z score is the score of the alignment – the larger this number the better.
- The E value is a numerical estimate of the likelihood that the given alignment would have occurred by pure random chance, and is therefore devoid of any significance. The smaller the E value, the more significant is the alignment.
- A 'dot matrix' is a visually appealing, intuitive, but qualitative tool for the comparison of sequences.
- Portions of one sequence that are similar to portions of the second sequences are indicated by diagonal rows of dots.
- Dot matrices are used for various types of sequence comparisons
- Filtering techniques help to improve the signal-to-noise ratio in dot matrices.
- The hash coding method is particularly suited for fast, ungapped searches of a small sequence, sequence pattern or motif, through a large database.
- A hash table is an associative array, where the positions of specific subsequences (of small size) in the sequence are stored.
- Comparison of hash tables for the query sequence and target sequence database helps to quickly identify the positions of the query in the database.
- Dynamic programming is the name a family of techniques used in a variety of technological, scientific and commercial fields to find the set of parameters that would give the desired optimum solution.
- The Neeedleman-Wunsch method was the first application of dynamic programming to sequence comparison.
- The first step in the algorithm is to create a matrix F(i,j), where the index i represents the residues of the first sequence being compared, and the index j represents the residues of the second sequence.
- F(i,j) = 1 if residue i is the same as residue j, = 0 otherwise.

- The next step in the algorithm is to rewrite the matrix according to the following rule – F'(i,j) = (maximum of {F'(k,l), k = i+1,m, l = j+1,n}) + F(i,j)
- The final step of the algorithm is to find the element with the maximum score and trace back the path through the matrix the leads to this score and specifies the best alignment between the two sequences.
- In an improved version of the algorithm, the following updating rule is used – F(i,j) = maximum of [F(i-1,j-1) + $s(x_i,y_j)$; F(i-1,j) - d; F(i,j-1) - d]. Here $s(x_i,y_j)$ is the substitution matrix values between the two sequences, and d is the gap penalty.
- In the Smith-Waterman algorithm the updating rule is again different – F(i,j) = maximum of [0; F(i-1,j-1) + $s(x_i,y_j)$; F(i-1,j) - d; F(i,j-1) - d]
- The Neeedleman-Wunsch algorithm is a global alignment method, while the Smith-Waterman method is a local alignment method.
- BLAST and FASTA are computer programs that implement the above algorithms to do fast searches of databases.
- The BLAST algorithm uses a word-based heuristic to execute an approximate version of the Smith-Waterman algorithm known as the 'maximal segment pairs' algorithm.
- BLAST is most frequently used over the Internet on the BLAST server (http://www.ncbi.nlm.nih.gov/BLAST/).
- FASTA is again a heuristics based algorithm, very similar to BLAST.
- The differences in the algorithm lead to it being more sensitive, but also more time-consuming.
- The WWW FASTA server is hosted by EBI, the European Bioinformatics Institute. (http://www.ebi.ac.uk/fasta33/).

4

Multiple Alignment, Substitution Matrices, and Phylogenetic Trees

This chapter deals with three somewhat heterogeneous topics that are too small to be treated individually. The first topic is multiple sequence alignment. The previous chapter dealt in detail with the pair wise alignment of two protein or DNA sequences. Here we discuss methods to align several sequences together. Much important biological information may be gleaned from such alignments, and several different algorithms are available for this. Some are discussed here. The second topic deals with one of the uses of the multiple sequence alignments, namely constructing substitution matrices. Substitution matrices have been briefly introduced in Chapter 3. One way of constructing these matrices is to use existing knowledge about accepted mutations to estimate the similarity between amino acid residues. Such knowledge is obtained by aligning sets of sequences known to have the same structure and/or function. A few other methods of constructing substitution matrices are also discussed. The final topic, namely phylogenetic trees, describes methods of constructing trees of relationships between organisms on the basis of sequence similarities. Again, multiple sequence alignment is often the starting point of algorithms that perform this task.

The algorithms described in this chapter are often simple in concept, but quite tedious to actually work out. Computer programs are of course available to relieve us of the tedium. However the disadvantage in this, as far as this textbook is concerned, is that it is not possible to work out examples in detail, and the reader has to content herself with a description of the algorithm, or a few very elementary examples.

4.1 Multiple Sequence Alignment

4.1.1 Goals of multiple sequence alignment

A multiple sequence alignment, or MSA, may be formally defined as a two-dimensional table in which each row represents a protein or nucleic acid sequence, and the columns are the individual residue positions (Figure 4.1). The table is obtained by aligning all the sequences being considered simultaneously in order to obtain the best overall score. (Definitions of the score of a multiple sequence alignment are discussed below.) Such simultaneous alignment of several sequences has lead to many important results regarding common sequence patterns or motifs in proteins and nucleic acids.

```
Humlbpa : M---MGALARALPS-ILLALLLTSTPEALGA-NPGLVARITDKGLQYAAQEGLLALQSELLR :
Rablpb  : M---MGTWARALLGSTLLSLLLAAAPGALGT-NPGLITRITDKGLEYAAREGLLALQRKLLE :
Ratlbp  : M---MKSATGPLLP-TLLGLLLLSIPRTQGV-NPAMVVRITDKGLEYAAKEGLLSLQRELYK :
Humcetp : M---MLAATVLT---LALLGNAHACSKGTSH-EAGIVCRITKPALLVLNHETAKVIQTAFQR :
Maccetp : M---MLAATVLT---LALLGNVHACSKGTSH-KAGIVCRITKPALLVLNQETAKVIQSAFQR :
Rabcetp : ----------------------ACPKGASY-EAGIVCRITKPALLVLNQETAKVVQTAFQR :
Humbpi  : MRENMARGPCNAPRWVSLMVLVAIGTAVTAAVNPGVVVRISQKGLDYASQQGTAALQKELKR :
Bovbpi  : M---MARGPDTARRWATLVVLAALGTAVTTT-NPGIVARITQKGLDYACQQGVLTLQKELEK :
```

Figure 4.1 A small portion of a multiple sequence alignment of 8 lipase sequences. The code names of the sequences are given at the extreme left.

The alignment of promoter DNA sequences, which identified consensus regions, is a good example. One of the common goals of building multiple sequence alignments is to characterize protein and/or gene families, and identify shared regions of homology. This often happens when a user has run a BLAST or FASTA type search for sequences in a database that are similar to a given query sequence. Such a search may reveal strong similarities to several other sequences, which are then put together to perform the MSA. The alignment so obtained may then be used to determine consensus sequences, such as the –10 and the –35 consensus regions in promoter sequences. MSA is also used to reinforce a weak indication of a particular biological feature by increasing the 'signal-to-noise' ratio. For example, for a given set of proteins, common structure, function, or origin may be only weakly reflected in sequence, and multiple comparisons may strengthen a weak signal. MSA also helps to classify sequences into families. All the sequences in such a family may have been derived from some common ancestral sequence, indicating an evolutionary relationship. Or the similarity could have arisen by convergent evolution towards a common structure or function. In general, therefore, MSA helps to establish phylogenetic relationships between sequences, and by extension, between the parent organisms. The study of evolution at the molecular level is strongly assisted by establishing such phylogenetic networks, and MSA usually provides the initial information to build the networks. And lastly, MSA is usually the first step also in building three-dimensional models of protein structure. MSA helps to predict the secondary and tertiary structures for new sequences, and identify templates for threading and homology modeling, which are methods for 3-D structure prediction.

Like pair wise alignment, multiple sequence alignment also could be global or local. Global alignment is best suited to small sequences that are of approximately the same length, and that may have global relationships. If the sequences to be compared are large or have varied lengths, it is more meaningful to look for common sub sequences using local matching methods.

In theory, making an optimal alignment between two sequences is computationally straightforward (e.g. the Smith-Waterman algorithm), but aligning a large number of sequences using the same method is almost impossible. Consider a 'multi-dimensional dynamic programming' algorithm. Such an algorithm would require the construction of a multi-dimensional matrix, which is then filled according to a set of rules, as described in Chapter 3, but modified to take care of the increased number of dimensions. A traceback procedure may then yield the best MSA. Simple as this sounds, a very little thought and practice soon shows that this is no 'solution' at all to the problem of finding the best possible multiple sequence alignment. The chief objection is of course that while the algorithm may work for, say, 3 or 4 small sequences, and give results in a reasonable amount of time, it is, in the language of computer science, an 'NP-complete' algorithm. This means that as the number of sequences increases, the time required for the execution of the algorithm increases exponentially. Already when the number is 7 or eight, we reach the limits of what may be accomplished with reasonable computing resources. Any further increase in the size of the problem increases the computational cost beyond what is in the realm of possibility even in future.

Dynamic programming, therefore, is only of limited utility. Nevertheless, with certain modifications, this is still a useful and interesting algorithm, and in one of the next few sections we

will describe a commonly used DP method for MSA. If we give up on the idea of arriving at a single "correct" alignment, and look only for an alignment that is "optimal" according to some set of calculations, we have many other algorithms to choose from. In such methods, the onus is on the user to determine which alignment is best for a given set of sequences. A few of these procedures are also described below.

4.1.2 Representation of a multiple sequence alignment

Once the best multiple sequence alignment has been discovered, it is usually reported as shown in Figure 4.1, i.e. as a set of sequences, written one sequence to a line, with the residues in the columns showing the similarities. The columns may be shaded (Figure 4.2) or coloured according to the degree of similarity, and there are programs such as PRETTYBOX, BOXSHADE, SeqVu and GeneDOC to do this under minimal user control. These programs also have the functionalities of a sequence editor,



Figure 4.2 The same multiple sequence alignment of 8 lipase sequences as in the previous figure. Three levels of shading indicate residues with similar (i.e. hydrophobic/hydrophilic) properties.

and therefore are useful in annotating the alignment, or in modifying a computer alignment by hand, to highlight sequence similarities that the program has missed. A multiple sequence alignment may also be represented as a consensus sequence, as already described in Chapter 1. A consensus sequence derived from an MSA is simply a sequence obtained by putting together the most commonly occurring residue at each position (each column). For example, alignment of all bacterial promoter sequences has shown that the consensus sequence for the so-called TATA box is $T_{80}A_{95}T_{45}A_{60}A_{50}T_{96}$. The numbers in the subscripts indicate the percentage occurrence of each of the four bases at this position. The consensus sequence does not of course abstract all the information in the MSA. In the above example, we see at T in position 3 occurs 45 % of the time, but the consensus does not tell what the bases are in the other 55% of the promoters. A 'profile' of an MSA is a way of summarising a greater amount of the information than a consensus. Profiles may be written either as 'regular expressions' or as a position specific matrix of frequencies (or probabilities). Figure 4.3a shows a regular expression.

```
N P G L V A R I T
N P G L I T R I T
N P A M V V R I T
E A G I V C R I T
K A G I V C R I T
E A G I V C R I T
N P G V V V R I S
N P G I V A R I T
```

Regular expression `n(P/A)G x V X R I T`

Figure 4.3a A regular expression representing a multiple sequence alignment of 8 sequences. Capital letter indicates a strongly conserved residue, small letter a weakly conserved one, x and X indicate no well conserved residue at different strengths, parentheses indicate approximately equal possibility of the enclosed residues.

It uses the usual symbols for the residues, along with a set of special symbols, to summarise the occurrence of a residue at each position in the alignment. For example, if a residue is shown with a symbol in capital, this would indicate a strong conservation of that residue at that position in all the sequences. Other conventions are indicated in the figure.

A position specific matrix is again a two dimensional table, but here each row represents one of the possible residues (four for nucleic acids, and twenty for proteins), and the columns represent the relative frequency of occurrence of the residue at that position. If the longest sequence is L residues long, then for protein sequences, the matrix is of order L x 20 and for nucleic acids of order L x 4. If we are to use the matrix to classify new sequences as belonging to a given family or not, then the relative frequencies with the appropriate normalisation, may be considered as probabilities. For protein sequences, a profile may then be written in general as

$$P = p_1 p_2 p_3 p_4 p_5 \ldots p_L$$

where $p_1 = [p_{1,1}\, p_{1,2}\, p_{1,3} \ldots p_{1,20}]$

$p_2 = [p_{2,1}\, p_{2,2}\, p_{2,3} \ldots p_{2,20}]$

...

and so on for all p up to p_L. Here $p_{1,1}$ is the probability of amino acid number 1 at position number 1, $p_{1,2}$ is the probability of amino acid number 2 at position number 1... and so on for all the twenty amino acids and for all the L positions in the profile. The matrix of probabilities represented by P is also called a 'position specific scoring matrix' or PSSM. Figure 4.3b shows a PSSM. In different contexts these are also called weight matrices.

Amino acid residues	Relative frequency of occurrence at each site in above MSA								
	Site 1	Site 2	Site 3	Site 4	Site 5	Site 6	Site 7	Site 8	Site 9
A		0.375	0.125			0.250			
C						0.375			
D									
E	0.250								
F									
G			0.875						
H									
I				0.500	0.125			1.000	
K	0.125								
L				0.250					
M				0.125					
N	0.625								
P		0.625							
Q									
R							1.000		
S									0.125
T						0.125			0.875
V				0.125	0.875	0.250			
W									
Y									

Figure 4.3b A position specific scoring matrix for the same multiple sequence alignment as in previous figure. The blank cells in the table are zeros.

4.1.3 Scoring a MSA

The most common way of finding the score of any given MSA is the so-called 'sum-of-pairs' or SP score. We consider the representation of the MSA as a two-dimensional matrix. In the SP scoring scheme, the score of the alignment is the sum of the scores of each of the columns. The score of column i is given by the expression

$$S_i = \sum s(residue_i^k, residue_i^l)$$

In this expression, the indices k and l refer to the different sequences. $Residue_i^k$ is the residue in the kth sequence and i^{th} column, and likewise $residue_i^l$ is the residue in the lth sequence and i^{th} column. $s(residue_i^k, residue_i^l)$ is the substitution matrix score for the pair of residues indicated. The summation is made over every pair of residues in the column. In case one of the pair of residues is a gap, a gap score is defined as s(residue,gap) or s(gap,residue) and added to the sum.

The SP scoring scheme appears to be a natural extension of the scoring for pair wise alignments. However it is not foolproof. It can lead in some cases to counter-intuitive results. For example it may happen that, as the number of sequences in the MSA increases, the difference between the best score and the next best score actually decreases, though we would expect that as the data increases, the signal-to-noise ratio should become better.

SP scores may also be calculated in ways different from the above. For example, one of the sequences in the alignment could be considered the 'ancestor' sequence and the score of all other sequences with respect to this ancestor sequence could be summed, rather than all possible pairs. Another intuitively satisfactory method is to define a consensus sequence from the MSA and then calculate the SP score with the consensus as the reference. A completely different way of calculating the SP score is to define an 'entropy' term for each column, based on the probability of occurrence of each residue in a column, the probabilities being calculated from the MSA and the expected occurrence frequency of the residue. The entropy is then a measure of the information content in the column, and a summation over all the entropies of all the columns yields the total entropy of the MSA. The best MSA is then the one that minimises the total entropy (or maximizes the information content).

4.1.4 Dynamic programming for MSA

As explained in the previous chapter, when comparing two sequences, a dynamic programming algorithm, such as the Needleman-Wunsch method, finds an optimal path through a rectangular two-dimensional matrix representing a comparison between two sequences. Every path represents one way of aligning the two sequences, and the procedure examines all possible paths in this matrix to arrive at the optimal one, i.e. the one with the best score. To extend this to the comparison of n sequences involves an analogous search through all possible paths in an n-dimensional matrix. The updating rule specified for the Needleman-Wunsch algorithm is suitably modified to take into account the larger number of dimensions. This is a straightforward task, and in principle, the problem of MSA may be tackled by just increasing the number of dimensions in the dynamic programming algorithm. In practise, however, it is not possible to use this method for more than three or four sequences. This is because the computational cost increases exponentially with the number of dimensions, i.e. the number of sequences. If we consider n sequences of length 100 residues each, and if we can align 2 sequences in 40 milliseconds, then the following are the approximate computational times required to perform MSA using the above straightforward dynamic programming: 3 sequences require 8 seconds; 4 sequences 5 hours; 5 sequences 100 hours; and 6 sequences require 2 years. Clearly this method is not a practical one.

A method has been proposed that tries to overcome this problem by considering the MSA as a series of pair wise alignments. Each pair wise alignment is viewed as the projection of the optimal path in the n-dimensional matrix on to the respective two-dimensional matrix. Consider the following three sequences.

Sequence 1 : GTYS
Sequence 2 : HGTY
Sequence 3 : GTS

In this simple case, it is easy to see that the 'best' MSA would be as follows

Sequence 1 : -**GT***Y*S
Sequence 2 : H**GT***Y*
Sequence 3 : -**GT**-S

We have used bold letters, italics and underlining to indicate similar residues. The three-dimensional matrix representation of this MSA is shown in Figure 4.4a. Also shown in this figure is one of the three two-dimensional projections of the best path through the matrix. Consider now that we do not know the three-dimensional path, which is the situation before we obtain the MSA. At this stage therefore, in order to find the path, we need to perform dynamic programming on the entire three-dimensional matrix, which as we have pointed out, is not an efficient way of performing MSA. However it is much simpler to directly obtain the two-dimensional paths. When we have these paths, we can project these back into the three-dimensional matrix. This will help us define boundaries inside the n-dimensional the matrix, within which the optimal path is sure to lie (Figure 4.4b). . Now the task of searching the entire matrix is reduced to searching only within the boundaries. In most cases this represents an enormous saving of computation time. This particular algorithm is called the Carillo-Lipman algorithm, and has been implemented in a program called 'MSA'. Despite the large saving in time, however, the exponential increase in computation cost prevents the algorithm from being applied to a large number of sequences at a time. Given current computer speeds, about 10 sequences of length approximately 300 residues each may be aligned in a reasonable amount of time. For aligning a greater

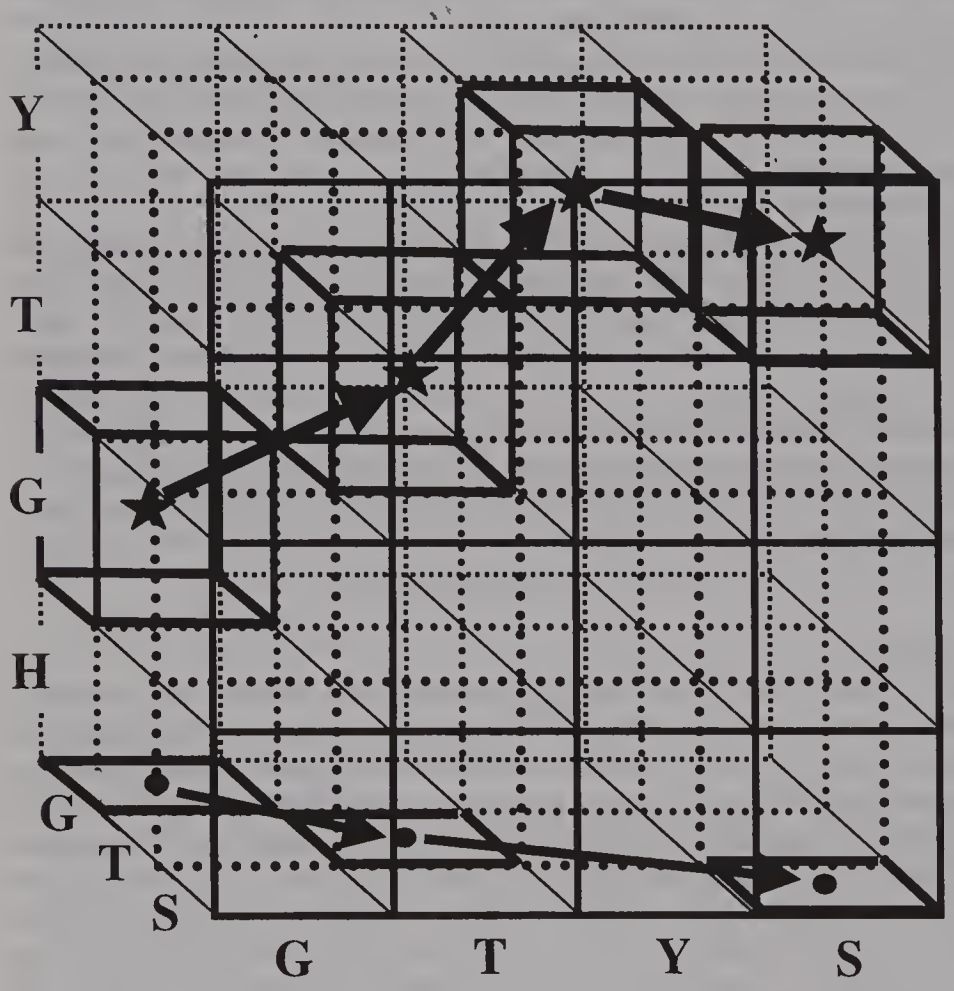


Figure 4.4a. A three-dimensional dynamic-programming matrix, showing the best multiple sequence alignment for the three sequences given in the text. Also shown is one of the three projections onto the respective plane. For clarity the other two projections have been omitted from this diagram, but may be easily imagined.

number of sequences, we cannot use dynamic programming, and we have to give up on the finding precisely the 'correct' alignment. Instead we settle for one that may be sub-optimal, but is nevertheless reasonably and intuitively correct, as well as achievable at low computational cost. A family of such algorithms go under the general title of progressive or hierarchical alignment techniques.

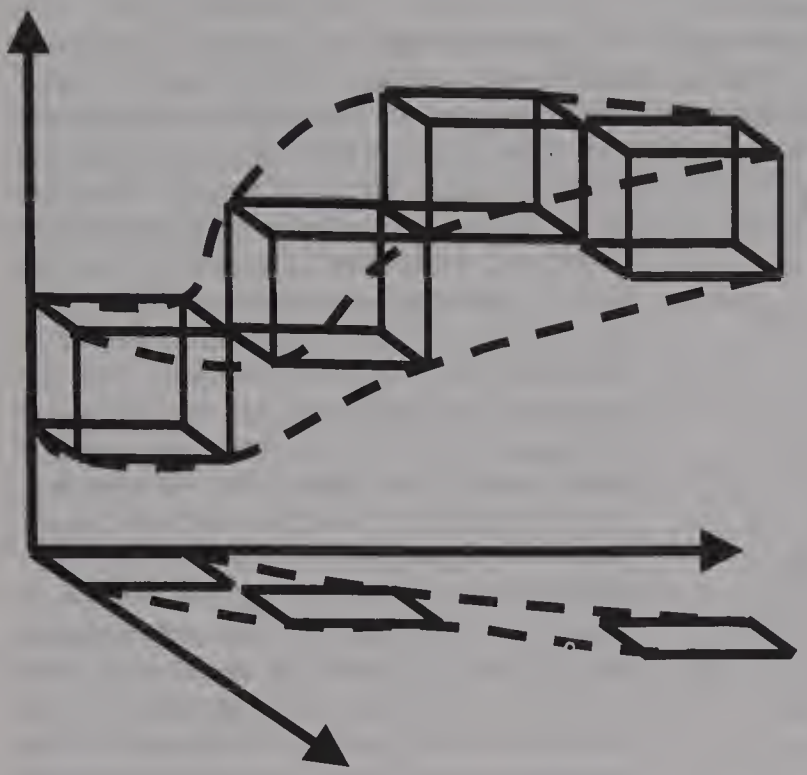


Figure 4.4b. The three 2-D dynamic programming solutions may be used to approximately reconstruct a limited volume (enclosed by the three dashed lines) in which to search for the correct solution.

4.1.5 Progressive or hierarchical alignment

In general terms, progressive alignment methods add one sequence at a time to the MSA. Thus, usually an initial sequence, or alignment of two or three sequences acts as the seed of the alignment. Then based on some measure of similarity, each of the remaining sequences is added progressively to the MSA. One way of picking the seed alignment is to choose the sequence that is most similar to each of the other sequences. This is the 'centre' of the alignment. The other sequences, forming the spokes of 'star' in terms of similarity to the centre, are then aligned one at a time to the centre. Such a technique is called the 'centre-star' alignment. There are other ways of choosing the centre, for example by choosing a consensus sequence. Of course this would imply an iterative procedure since the consensus itself is obtained usually by MSA. A more general iterative technique is summarised as follows: Align a pair of sequences chosen according to some criterion. Next pick a sequence that is most similar to this alignment. Align this to form a MSA consisting of three sequences. Repeat the procedure until all the sequences are part of the MSA. There are many different variants to this technique. Here, we will consider the Feng-Dolittle algorithm, and then discuss how this has been modified and implemented in one of the most popular programs used for MSA, namely CLUSTAL.

Before we take up those algorithms we will discuss briefly the concept of a 'distance' between two sequences. Any pair wise sequence alignment has a score associated with it, for example the 'z score'. This is a measure of similarity between the two sequences, and the larger the score, the greater the similarity. The concept of 'distance' between two sequences is reciprocal to that of the similarity score – the greater the distance, the less the similarity between the sequence pair. There are many ways of calculating the distance between two sequences after a pair wise alignment. One simple technique is to take the reciprocal of the 'Z score' or the appropriate similarity score, normalised in some way. A Hamming distance may also be calculated. This is a count of the number of 'mutations' that have to be made in one sequence in order for it to match the other sequence identically.

The Feng-Dolittle algorithm consists of the following steps. Consider that we have n sequences to be aligned. First construct a half-matrix of n(n-1) distances between all pairs of n sequences by

standard pair wise alignment. Feng and Doolittle calculated the distance between two sequences a and b by the following expression.

$$D_{ab} = -\log[(S_{ab} - S_{rand})/(S_{max} - S_{rand})]$$

where S_{ab} is the best similarity score between a and b, S_{rand} is the random score obtained by aligning two sequences with the same length and residue composition, S_{max} is the maximum possible score, obtained by aligning each of the two sequences to itself and taking the average of the two maximal scores. Only a half matrix is required since the distance between two sequences is a commutative value, the distance between a and b being the same as between b and a. The next step in the algorithm uses the Fitch-Margoliash clustering technique to cluster the sequences together into different groups based on distance. This information is used to build a guide phylogenetic tree, which gives a representation of the relationships between the sequences. In this tree, sequences that are similar are arranged close to each other, and sequences that are dissimilar are arranged further away. One may use this information to draw a diagram of the tree such as the one shown in Figure 4.5. Note that the Fitch-Margoliash algorithm is only one method to convert a set of distances into a phylogenetic tree. We will be discussing phylogenetic trees in greater detail in a later section. Here we further note that the tree obtained above is a crude one and cannot to be used to infer biologically relevant phylogenetic relationships between the sequences, or between their parent organisms. The tree is only used as a guide in constructing the MSA. The next few steps in the Feng-Doolittle algorithm are straightforward. The nearest two sequences, as indicated by the guide tree are aligned using a pair wise alignment method. The next nearest sequence is then added to this alignment, and so on, until all sequences are part of the MSA. During this procedure, we may align one sequence to another, a sequence to an already aligned set of sequences (i.e. an alignment), or one alignment to another. Standard dynamic programming is used to align a sequence to another. To align a sequence to an alignment, the sequence is aligned pair wise to every sequence in the alignment, and the best pair wise alignment decides how the new sequence is added. To align one alignment to another, all possible pair wise alignments between the sequences in the two groups are carried out, and once again the best of these decides how the two alignments are aligned.

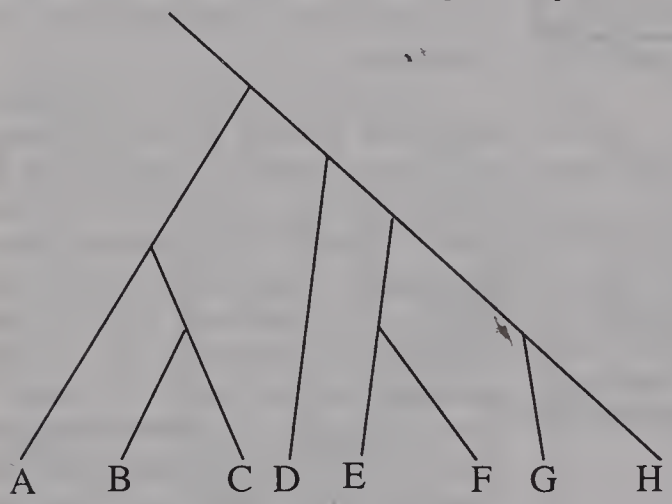


Figure 4.5. An example of a 'guide' phylogenetic tree used in the Feng-Dolittle algorithm. A, B.... are sequences, and the lines joining them represent relationships between them. For example, B and C are more similar to each other (i.e. 'closer' to each other) than each of them is to A.

CLUSTAL is a popular program for MSA that uses an extensively modified version of the Feng-Doolittle algorithm. The chief problem with the algorithm is that it uses only pair wise alignments to build up the MSA. This can also be regarded as its main strength, since it leads to the massive speed-up in the calculations. However, information about alignments already made is lost every time a new alignment is carried out. As described earlier, it is possible to summarize and save the information in an alignment by building a profile, either as a regular expression, or a probability matrix. The CLUSTAL algorithm builds up the MSA by using such profiles wherever appropriate. The steps in the procedure are as follows. As in the previous algorithm, the first step is to construct a half-matrix of n(n-1) distances between all pairs of n sequences by standard pair wise alignment. The conversion of

similarity scores to distances is performed on the basis of the Kimura model of the evolution, the so-called 'neutral drift' theory. There are several theories and models of evolution and explanations as to how diverse features arise in the organisms. While all models conform to the basic Darwinian principles of mutation and selection, there are conceptual differences in their details. While this is discussed in somewhat greater detail in a later section, here we will content ourselves by noting that the differences impact upon the methods used to convert similarity scores to evolutionary distances. In the CLUSTAL algorithm, as already stated, the Kimura model of evolution is used. In the next step of the algorithm, a guide tree is constructed by using the neighbour-joining algorithm for clustering. Details of this algorithm are explained later. The final step in CLUSTAL consists of using the guide tree to progressively add sequences to the MSA, starting from that pair of sequences that are the closest to one another. Every time an alignment is made, a profile is generated, and in the subsequent steps of the MSA construction, the profile is used, instead of the individual sequences. Thus we have sequence-sequence comparisons, sequence-profile comparisons and profile-profile comparisons. There are several other features in the program that contribute to its accuracy. Some of these are: weighting functions to correct for biases in the sequences (e.g. too many sequences from the same family); choice of appropriate substitution matrix to calculate the score; choice of appropriate gap penalties; and adjustment of the guide tree even during the progressive alignment (third step) if necessary.

The whimsically named program T-COFFEE is an improved progressive alignment technique introduced recently. The first step in this alignment is to generate a library of information about the alignments between the sequences being aligned. The information in the library is generated by a variety of different types of comparisons between each pair of sequences, including local and global sequence alignment. The information in the library is not required to be consistent, and even two or more alignments of the same pair of sequences may be included. The information is represented as sets of alignments between the pairs of sequences or portions of the sequences. A weighting scheme consisting chiefly of evaluating the percentage of identical residues aligned is used to calculate a weight for each alignment in the library. In the next step the all the information in the library is combined appropriately to obtain a unique weight for a match between each pair of residues. A heuristic library extension algorithm is used to carry this out. At the end of this operation every pair of sequences would have gathered information from all other possible alignment pairs. When the library generation and extension is over, it is now possible to calculate the score of every pair-wise alignment, and use these scores in a progressive sequence alignment, using a modified version of the Feng-Dolittle method. A recent version of T-COFFEE, called 3D-COFFEE, incorporates information from three-dimensional structure alignments also.

In another multiple alignment algorithm, the sequences to be aligned are placed in arbitrary order and each sequence is divided into n segments of m residues each, where m ranges from 10 to 40. First, all possible alignments of the segments from the first two sequences are evaluated by scoring them according to some matrix. The best 1000 such matches are saved in an array called the heap T1. Next all alignments between the segments of the third sequence and the segments in the heap T1 are evaluated, again according to the same scoring matrix. The best 1000 results of this comparison are placed in the second heap T2. Now the segments of the fourth sequence are compared with the segments in T2 and once again the best 1000 results are saved into T1, overwriting the previous set. This procedure is repeated for each sequence in turn, alternately storing the best results in T1 and T2. Finally after the last sequence, either T1 or T2 will contain the best set of alignments. The number of results stored, i.e., 1000 is chosen to minimize the number of omitted alignments that may be weak in the first comparisons but become stronger later. This method has been applied to several sets of sequences such as the DNA binding proteins with good results when the number of sequences to be compared is 5 to 10.

4.2 Substitution Matrices

4.2.1 What are substitution matrices?

If two residues are not exactly the same but are closely related such that the replacement of one residue by the other does not affect the biological functions of the sequence, these two residues may be given a very high similarity score. On the other hand if the replacement will seriously affect the function, the similarity score will be small or even negative. The use of such scores makes the analyses of sequence similarity quantitative and therefore more precise. A matrix of values that is used to score residue replacements or substitutions is called a substitution matrix. There are a variety of such scoring schemes available, constructed on the basis of different principles. The first and the simplest is the binary scheme that we mentioned above. This can be recast as a scoring matrix called the Unitary matrix. We write the 20 amino acids along the topmost row as well as along the leftmost column of the matrix. Every element of this matrix then represents the score when the residue corresponding to the row index of the element is replaced by the residue corresponding to the column index, or vice versa[12]. The Unitary substitution matrix has a score of one along the diagonal and zero everywhere else. This reflects the fact that according to this scoring scheme, a score of one is given wherever the matching residues in the alignment are identical, and zero otherwise.

It is clear that a binary scoring scheme, such as the one we have considered above, is inappropriate for analysing protein sequences. Howver, the unitary scoring matrix is quite effective for DNA sequences. The following is another example of an effective matrix for DNA.

	A	T	G	C
A	1	-1	-0.5	-1
T	-1	1	-1	-0.5
G	-0.5	-1	1	-1
C	-1	-0.5	-1	1

This matrix is constructed on the basis that if a purine replaces a pyrimidine, and vice versa, this is more harmful than a purine replacing a purine, or a pyrimidine replacing a pyrimidine. Note that, as expected the matrix is symmetric about the main diagonal, and the maximum score, when there is no substitution, has been set to 1.

For protein sequences some substitutions are clearly more likely to occur than others, due to similar chemical properties of the amino acids involved. All amino acids replacements do not change the structure and/or the function to the same extent. For example, if a cysteine residue in one sequence is replaced by a different residue in another sequence, it could lead to the absence of a disulphide bridge in the second protein, and this could have disruptive consequences for its structure and function. Such replacements are therefore unlikely to occur between related proteins, and the scoring scheme must reflect this improbability. Conversely, there are residue pairs that could replace one another with very little effect on the structure and the function, e.g. isoleucine for valine, serine for threonine, etc. These are the so-called conservative substitutions. Again we get considerably better alignments if the scoring scheme reflects this fact.

---

[12] The replacement of residue A by residue B is assumed to have the same score as the replacement of B by A. This is because, when we compare sequences and build phylogenetic trees, we have no *a priori* knowledge of which sequence came first, or even more particularly, which mutation came first.

Biologically sensible scoring matrices for proteins may be specified as follows. Identical amino acids in the two sequences should be given greater score than any substitution. Conservative substitutions should be given greater score than non-conservative ones. Finally, different sets of values may be required for scoring different types of alignments. Very similar pairs of sequences (e.g. homologues in mouse and rat) may require one set of substitution matrices, while highly divergent sequences (e.g. homologues in mouse and yeast) require another. We want our scoring matrices to take into account the evolutionary distance between the sequences involved.

One simple matrix for amino acids built on biological principles corresponds to the 'Genetic code' scoring scheme. It is based on the fact that changes in the sequence of a protein arise basically from mutations in the corresponding gene. A single base change in the DNA sequence could change the codon from representing one amino acid to representing another. For example the codon for Lysine is AAA. Changing the central adenine to guanine yields AGA, the codon for Arginine. On the other hand, changes between other pairs of amino acids may require more than one change, sometime two, sometime three. For example, a change from Lysine (AAA) to Serine (TTA) requires changes in two nucleotide bases, while to change from Lysine to Cysteine (TGC) requires all three nucleotides to change. Such analysis helps to calculate a distance between the amino acids, which can then be used to construct the scoring scheme. The degeneracy of the genetic code adds a level of complexity to the calculations, as does the differing codon preferences in different organisms. However these parameters can be approximated, or dealt with in other ways to build the genetic code substitution matrix. For example, in one variation of the genetic code matrix, called the 'minimum mutation distance' matrix, the scores are based on the minimum number of bases that must be changed to convert a codon for one amino acid into a codon for a second amino acid.

Other substitution matrices may be constructed that are based on biochemical and biophysical properties of the amino acids, such as size, shape, local concentrations of electric charge, the conformation of the van der Waals surface, and the ability to form hydrophobic bonds, salt bridges and hydrogen bonds. However, the most commonly used matrices are ones that are based on an analysis of similarities between sequences of proteins that have the same known function and structure. Broadly the following method is used to build such matrices. A set of homologous sequences is considered. A multiple sequence alignment then yields statistics regarding amino acid substitutions. These numbers are converted to substitution scores. The two most popular of such matrices are the PAM matrices and the BLOSUM matrices.

4.2.2 Evolutionary models

The PAM and BLOSUM matrices, as well as other methods of calculating distances between residues or sequences, are based upon a particular model of natural selection and evolution. There are many evolutionary models, and the score calculated varies depending on which model is chosen. In this subsection we will briefly describe some of the models.

Most models assume that the residues at each position in the sequence evolve (i.e. suffer mutations) independently of residues at other positions. This is not entirely realistic, since the effect of a particular residue at a particular position, especially in proteins, is strongly dependent on its neighbours. Natural selection acts at the level of structure and function, which rarely depends on single residues. Also, most models assume that the rate of accepted mutation at all the sites in the sequence are the same. Again this is a questionable assumption. For example, mutations of residues in the active site of a protein will certainly be more likely to harm the parent organism, and therefore not be accepted, than mutations at other sites. Some models do allow different sites to evolve at different rates. But such models are very complicated. The more complicated the model we use, the more complicated it is to compute the substitution scores. Since simple models give reasonable results,

complicated models are used only when studying evolutionary theories, or for other such specialist applications.

A simple model for the evolution of DNA sequences is as follows. Each sequence is considered as a chain of independent sites. Each of the sites can be in one of the four states specified by the bases {A,T,G,C}. Each sites has a probability of being in any one of these four states, called the stationary distribution. It also has a probability of changing, or mutating, from one state to another, called the transition probability. Such a model is known, in general, as a Markov model. In the calculation of the scoring matrices, we are interested only in the transition probabilities. To complete this model of DNA we need to calculate the 16 transition probabilities below.

$$\begin{matrix} P_{AA} & P_{AT} & P_{AG} & P_{AC} \\ P_{TA} & P_{TT} & P_{TG} & P_{TC} \\ P_{GA} & P_{GT} & P_{GG} & P_{GC} \\ P_{CA} & P_{CT} & P_{CG} & P_{CC} \end{matrix}$$

Here P_{AA} is the probability of a transversion (or mutation) from A to A, P_{AT} is the probability of transversion from A to T, and so on. Jukes and Cantor suggested a further simplification of this model, reducing the number of parameters to be calculated to one single number, called α. The probabilities are then assigned as follows.

$$\begin{matrix} P_{AA} & P_{AT} & P_{AG} & P_{AC} \\ P_{TA} & P_{TT} & P_{TG} & P_{TC} \\ P_{GA} & P_{GT} & P_{GG} & P_{GC} \\ P_{CA} & P_{CT} & P_{CG} & P_{CC} \end{matrix} = \begin{matrix} 1-3\alpha & \alpha & \alpha & \alpha \\ \alpha & 1-3\alpha & \alpha & \alpha \\ \alpha & \alpha & 1-3\alpha & \alpha \\ \alpha & \alpha & \alpha & 1-3\alpha \end{matrix}$$

Here all the transversion probabilities (i.e. the probability of change from one residue to another) are the same, α, while the residual probability is assigned to the situation when the residues in the aligned sites of the two sequences are identical. α is the probability of transversion in one generation. Its value depends on the time scale, measured in terms of generations. If one unit time corresponds to 100 generations, α would have a smaller value (i.e. the transversion probability in unit time is smaller) than if unit time corresponds to 200 generations. The upper bound for α in any case is 1/3.

To overcome some of the oversimplification inherent in this model, Kimura suggested the following scheme of assigning the probabilities.

$$\begin{matrix} P_{AA} & P_{AT} & P_{AG} & P_{AC} \\ P_{TA} & P_{TT} & P_{TG} & P_{TC} \\ P_{GA} & P_{GT} & P_{GG} & P_{GC} \\ P_{CA} & P_{CT} & P_{CG} & P_{CC} \end{matrix} = \begin{matrix} 1-\alpha-2\beta & \beta & \alpha & \beta \\ \beta & 1-\alpha-2\beta & \beta & \alpha \\ \alpha & \beta & 1-\alpha-2\beta & \beta \\ \beta & \alpha & \beta & 1-\alpha-2\beta \end{matrix}$$

Here there are two parameters. α is called the 'transition' probability, corresponding to changes from a purine to a purine (either the same one or another), or a pyrimidine to a pyrimidine. β is called the 'transversion' parameter again, but here it is the probability of a change from a purine to a pyrimidine, or vice versa. In both the Jukes-Cantor and the Kimura models, the probabilities assigned above are for one unit time, defined in terms of number of generations. These numbers act as the basis to obtain the probabilities after a given number of evolutionary time units. Conversely, if the actual substitutions are calculated for any particular alignment, it is possible to use already calculated probabilities in the above models to determine the evolutionary distance between the two sequences.

For protein sequences, similar models may be constructed. However, the complications of accounting for transitions and transversions within a set of twenty different amino acids mean that simple one or two parameter models such as one above are not useful. However we may use substitution matrices constructed from statistical analyses of similarities and differences in sequence data as an expression of the underlying model.

4.2.3 PAM substitution matrices

PAM stands for Percent Accepted Mutation. In an alignment between two protein sequences, if an amino acid in the first is substituted by another in the second, it indicates a point mutation in the sequence. (Note that though statistics are compiled for the amino acid sequences, the mutation in fact occurs in the corresponding gene.) Since the alignment is performed on experimentally determined sequences, this means that the mutation has had no deleterious effect on the parent organism. In the other words it has been 'accepted'. These are the two chief features considered in compiling the PAM matrices, viz. there must a replacement of one amino acid by another; and this substitution must be accepted by natural selection.

PAM matrices are based on a Markovian model of evolutionary change in the sequences. Each site, i.e. residue, in the sequence is considered to evolve independently of the other sites. In the course of evolutionary time, at each site, the residue will make a transition to another residue with some transition probability. One can measure evolutionary time in years, or millions of years. But for the purpose of sequence comparisons it is more convenient to measure this in terms of the time required to make a certain standard number of mutations, say 1 in 100, or 1%. In other words, the amount of time required to produce one accepted transition in one hundred residues is counted as one unit of evolutionary time and is called 1 PAM. If used in the interpretation of sequence alignments, if two sequences have 1% difference in the residues between them, the evolutionary distance between them is 1 PAM. This method of measuring time overcomes the following problem that arises when the measurement is made in years. Some proteins are crucial to the functioning of all life forms and the corresponding sequences change only very slowly. Other proteins are less crucial and they incorporate accepted mutations into their sequences at a much faster rate. Thus the number of mutations in a sequence relative to another may not be a very good indicator of the evolutionary distance in years between them. If the measurement is made in PAMs, however, the values are normalised with respect to the mutation rate specific to that set of proteins. Once can therefore use PAMs to compare evolutionary distances using different sets of proteins. Nevertheless, it is possible to estimate that overall most proteins evolve at the same rate. Thus, it has been estimated that 1 PAM corresponds approximately to 10 million years of evolutionary distance between two sequences. The fact that this represents a very approximate number is clear when we note that new species have arisen and diverged from the parent in far shorter spans of time than this (i.e. as short as 1 million years).

The transition probabilities within the set of amino acids were obtained from statistical analyses of the sequence data. In 1978 Margaret Dayhoff and colleagues used ungapped multiple alignments of certain well-conserved regions from closely related proteins for this purpose. They selected 71 groups of proteins. Each group contained the sequences of proteins with the same or closely related functions from different organisms. The protein groups included the cytochrome C family, the ferrodoxins, the flavodoxins, and other such well characterized proteins. In any block, any two sequences did not did not differ more than 15%, i.e. the sequences in any particular multiple sequence alignment were 85% identical. (The idea was to keep the number of sites that have encountered several changes low.) These aligned regions then were used to infer the underlying evolutionary tree. Figure 4.6 shows an example of such a tree for the following sequence data.

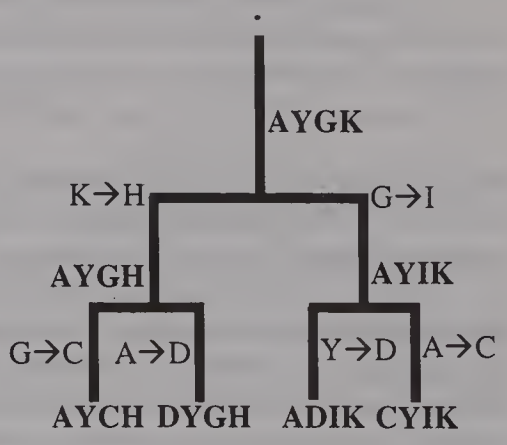


Figure 4.6. An example of a 'guide' phylogenetic tree used to build PAM substitution matrices

Sequence 1: A Y C H
Sequence 2: D Y G H
Sequence 3: A D I K
Sequence 4: C Y I K

The algorithm used to construct the phylogenetic trees from the alignments is called the Maximum Parsimony method and will be described in detail in section 3 of this chapter. Here we state that the most parsimonious tree is one that explains the given sequence alignment by postulating the least number of mutations. These trees are then used as guides in counting the number of mutations that have occurred and have been accepted. The reason we first construct the trees is, firstly, to avoid over counting of mutations, since we count only the changes between one sequence and its neighbour in the tree. Secondly, we ensure that the mutations have occurred in closely related sequences, not too far apart in evolutionary time, thereby avoiding un-realistic mutations that may not actually appear in Nature. A matrix of mutations may then be constructed from such trees, called the MDM or Mutation Data Matrix. This simply counts the number of times a particular residue has mutated to another along the branches of the tree. The MDM for the tree in Figure 4.6 is the following.

	A	Y	C	D	G	H	I	K
A			1	1				
Y			1	1				
C	1	1						
D	1	1						
G							1	
H								1
I					1			
K						1		

In general, from all the trees, for the twenty amino acids, we may write a matrix A whose elements A_{ij} are the counts of the number of times the residue i has changed to residue j. To convert the counts into probabilities, we must normalize the numbers with respect to the frequency of occurrence of all mutations. We write these transition probabilities as follows.

$$P_{ij} = c \times a_{ij}$$

where $i \neq j$. Here c is a scaling constant, and a_{ij} are defined as

$$a_{ij} = A_{ij} / \Sigma_l A_{il}$$

where the summation is over all twenty residues, l = 1,20. In the case where i = j,

$$P_{ii} = 1 - \Sigma_{i \neq k} (c \times a_{ik})$$

The scaling factor is required to normalize the probabilities so that all of them refer to the transition probabilities in 1 PAM of evolutionary time, or such time that 1 % of the residues undergoes mutation. With an appropriate choice for the scaling constant c, the matrix P is a transition probability matrix for 1 PAM. This matrix can now be converted into a scoring matrix as follows. If two sequences are aligned over a length of n residues (or n sites) we can calculate the probability of the alignment being a random one by estimating the probability of the matches and mismatches occurring by chance using the product of the random match/mismatch probabilities at all the sites, i.e.,

$$P_{random} = \Pi_{i=1,n}[q(a_i) \times q(b_i)]$$

where $q(a_i)$ is the relative frequency of the occurrence of the residue a at the site i in the first sequence, and $q(b_i)$ is the same for residue b in the second sequence[13]. We may also estimate the probability for

[13] The symbol Π stands for 'product of' just as the symbol Σ stands for 'sum of'.

the alignment indicating an evolutionary relationship between the two sequences.

$$P_{related} = \Pi_{i=1,n}[q(a_i) \times P(a_i,b_i)]$$

Here $P(a_i,b_i)$ are the same as the transition matrix elements P_{ij} we have calculated earlier. The different notation is used to indicate that we are now using these to calculate the alignment score. In principle, we want our score to reflect the chance (or the likelihood, or the 'odds') that we have aligned evolutionarily related sequences, i.e. we want a high score if the odds are high that we have aligned related sequences, and a low score id the odds are high that we have aligned two unrelated sequences. A natural choice for the score is then a comparison of the probabilities $P_{related}$ and P_{random}. The likelihood ratio, or the odds ratio, is given by

$$\begin{aligned} \text{Score} &= P_{related} / P_{random} \\ &= \Pi_{i=1,n}[q(a_i) \times P(a_i,b_i)] / \Pi_{i=1,n}[q(a_i) \times q(b_i)] \\ &= \Pi_{i=1,n}\{[q(a_i) \times P(a_i,b_i)] / [q(a_i) \times q(b_i)]\} \\ &= \Pi_{i=1,n}[P(a_i,b_i) / q(b_i)] \end{aligned}$$

Since these numbers are very small, it is more convenient to take the logarithm of the likelihood ratio (or the logarithm of the odds ratio). This is also better justified from an information theoretic viewpoint.

$$\begin{aligned} \text{Score} &= \log_2\{\Pi_{i=1,n}[P(a_i,b_i) / q(b_i)]\} \\ &= \Sigma_{i=1,n}\{\log_2[P(a_i,b_i) / q(b_i)]\} \end{aligned}$$

The entries in the substitution matrix are thus

$$S(a,b) = \log_2[P(a,b) / q(b)]$$

Since the logarithm is taken to the base 2, the score is in bits. The matrix, of course helps to determine the score at each site. And since we have used logarithm of the odds to construct it, the score of the alignment is just the sum of the scores at each site.

$$\text{Score} = \Sigma_{i=1,n} S(a_i,b_i)$$

We have chosen the scaling constant such that the matrix represents the transition probabilities in 1 PAM. To find the probabilities, and hence the substitution matrix for more evolutionarily distant sequences, we build matrices at different PAM values. A 'n PAM' matrix is obtained from a 1 PAM matrix by taking the nth power of the transition probability matrix before taking the logarithm.

$$\begin{aligned} &\text{If for 1 PAM, } S(a,b) = \log_2[P(a,b) / q(b)] \\ &\text{for n PAM, } S(a,b) = \log_2[P^n(a,b) / q(b)] \end{aligned}$$

Thus the 100 PAM (or PAM100) substitution matrix is obtained as

$$S(a,b) = \log_2[P^{100}(a,b) / q(b)]$$

We multiply P(a,b), the transition matrix for 1 PAM, into itself 100 times, and then find the log odds values. Note that the evolutionary distance specified by 100 PAM does not imply 100% change in the sequence. It means a 1% change applied 100 times. Since the same residue may change many times, we cannot accurately say what will be the percentage residue differences between two sequences that have diverged over 100 PAM. Figure 4.7 gives the correspondence between the evolutionary distance and the observed percentage difference in the sequences originally analysed by Dayhoff and co-workers. Scoring matrices at various distances calculated from the 1 PAM matrix have been constructed and are available for reference. In Table 3.4 we have given the commonly used 100 PAM matrix. For detecting more distant evolutionary relationship the PAM 250 matrix is also a popular choice, corresponding very approximately to about 2 billion years of divergent evolution. This is given in Table 4.1. PAM matrices are among the most commonly used substitution matrices, and one of the main reasons for their popularity is their solid foundation of theory and analyses.

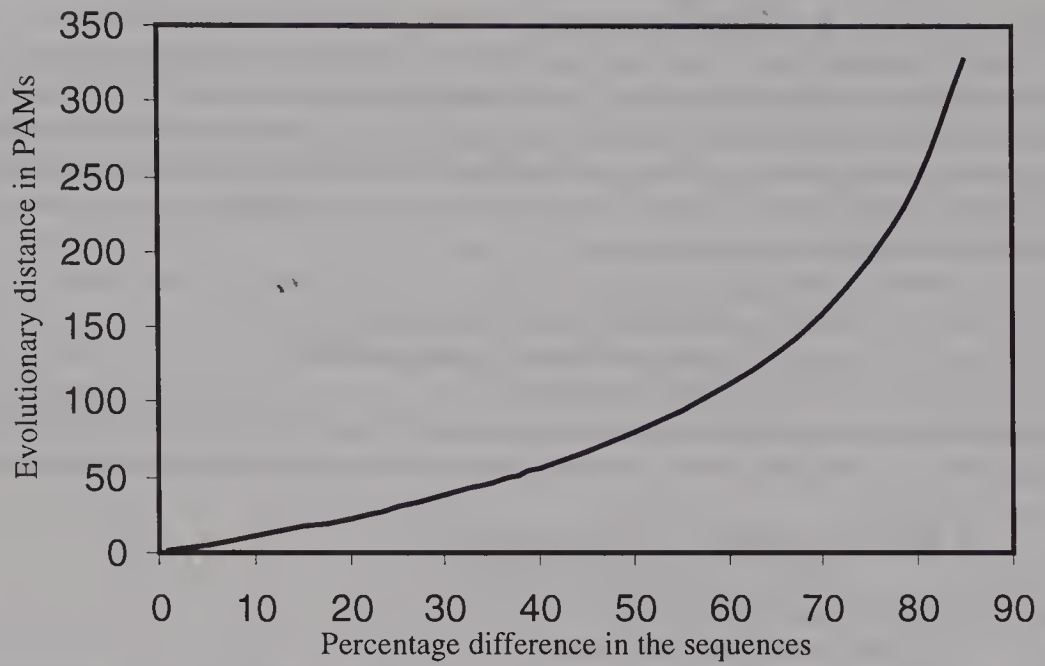


Figure 4.7 Relationship between percentage difference in two sequences and their evolutionary distance.

Table 4.1 The 250 PAM (or PAM250) substitution matrix

	A	C	D	E	F	G	H	I	K	L	M	N	P	Q	R	S	T	V	W	Y
A	2	-2	0	0	-4	1	-1	-1	-1	-2	-1	0	1	0	-2	1	1	0	-6	-3
C	-2	12	-5	-5	-4	-3	-3	-2	-5	-6	-5	-4	-3	-5	-4	0	-2	-2	-8	0
D	0	-5	4	3	-6	1	1	-2	0	-4	-3	2	-1	2	-1	0	0	-2	-7	-4
E	0	-5	3	4	-5	0	1	-2	0	-3	-2	1	-1	2	-1	0	0	-2	-7	-4
F	-4	-4	-6	-5	9	-5	-2	1	-5	2	0	-4	-5	-5	-4	-3	-3	-1	0	7
G	1	-3	1	0	-5	5	-2	-3	-2	-4	-3	0	-1	-1	-3	1	0	-1	-7	-5
H	-1	-3	1	1	-2	-2	6	-2	0	-2	-2	2	0	3	2	-1	-1	-2	-3	0
I	-1	-2	-2	-2	1	-3	-2	5	-2	2	2	-2	-2	-2	-2	-1	0	4	-5	-1
K	-1	-5	0	0	-5	-2	0	-2	5	-3	0	1	-1	1	3	0	0	-2	-3	-4
L	-2	-6	-4	-3	2	-4	-2	2	-3	6	4	-3	-3	-2	-3	-3	-2	2	-2	-1
M	-1	-5	-3	-2	0	-3	-2	2	0	4	6	-2	-2	-1	0	-2	-1	2	-4	-2
N	0	-4	2	1	-4	0	2	-2	1	-3	-2	2	-1	1	0	1	0	-2	-4	-2
P	1	-3	-1	-1	-5	-1	0	-2	-1	-3	-2	-1	6	0	0	1	0	-1	-6	-5
Q	0	-5	2	2	-5	-1	3	-2	1	-2	-1	1	0	4	1	-1	-1	-2	-5	-4
R	-2	-4	-1	-1	-4	-3	2	-2	3	-3	0	0	0	1	6	0	-1	-2	2	-4
S	1	0	0	0	-3	1	-1	-1	0	-3	-2	1	1	-1	0	2	1	-1	-2	-3
T	1	-2	0	0	-3	0	-1	0	0	-2	-1	0	0	-1	-1	1	3	0	-5	-3
V	0	-2	-2	-2	-1	-1	-2	4	-2	2	2	-2	-1	-2	-2	-1	0	4	-6	-2
W	-6	-8	-7	-7	0	-7	-3	-5	-3	-2	-4	-4	-6	-5	2	-2	-5	-6	17	0
Y	-3	0	-4	-4	7	-5	0	-1	-4	-1	-2	-2	-5	-4	-4	-3	-3	-2	0	10

4.2.4 BLOSUM substitution matrices

BLOSUM stands for BLOcks SUbstitution Matrices. In 1992, Henikoff and Henikoff devised the BLOSUM family of substitution matrices. Just as in the case of the PAM matrices, the scores are obtained as the logarithms of likelihood ratios. However, they are not based on any specific evolutionary model. Therefore sequence evolution is not modeled as a Markov chain, and no phylogenetic trees need to be constructed. Instead the following procedure was adopted.

Groups of related protein sequences were obtained from the PROSITE database, version 8.0. As explained in Chapter 2, this database contains all known protein sequences determined and verified by biochemical analysis. The relationships between the proteins within a group have therefore been clearly established. Each group of sequences was then aligned into blocks using local ungapped alignment techniques. Each block consists of a set of sequences, of around 15 residues each. In other words, there are up to about 15 columns in each block and as many rows as the number of sequences in the chosen group. The blocks represent highly conserved local regions in the proteins, and were created using a dynamic programming algorithm for local alignment, using a program called PROTOMAT. Table 4.2 gives four sample blocks from the BLOCKS database.

Table 4.2: Sample blocks

Block 1	Block 2	Block 3	Block 4
WWYIR	CASILRKIYIYGPV	GVSRLRTAYGGRK	NRG
WFYVR	CASILRHLYHRSPA	GVGSITKIYGGRK	RNG
WYYVR	AAAVARHIYLRKTV	GVGRLRKVHGSTK	NRG
WYFIR	AASICRHLYIRSPA	GIGSFEKIYGGRR	RRG
WYYTR	AASIARKIYLRQGI	GVGGFQKIYGGRQ	RNG
WFYKR	AASVARHIYMRKQV	GVGKLNKLYGGAK	SRG
WFYKR	AASVARHIYMRKQV	GVGKLNKLYGGAK	SRG
WYYVR	TASIARRLYVRSPT	GVDALRLVYGGSK	RRG
WYYVR	TASVARRLYIRSPT	GVGALRRVYGGNK	RRG
WFYTR	AASTARHLYLRGGA	GVGSMTKIYGGRQ	RNG
WFYTR	AASTARHLYLRGGA	GVGSMTKIYGGRQ	RNG
WWYVR	AAALLRRVYIDGPV	GVNSLRTHYGGKK	DRG

The first set of blocks was created by analysis of several hundred groups of proteins. This analysis yielded more than 2000 blocks. The elements of the transition probability matrix were obtained by the analysis of these blocks of aligned sequences. The values of $P_{related}$ and P_{random} for each pair of residues were computed from the frequencies of replacement of one by the other in the above blocks. The rest of the computation of the substitution matrix is the same as for the PAM matrices – the logarithm of the odds ratio for each pair is the score for that pair.

The advantage of this process, over the PAM process is that it does away with the need to build phylogenetic trees, by choosing related groups of sequences for building the blocks. The disadvantage is that sequences very closely related to each other may contribute too heavily to the probabilities, again leading to degradation of information regarding distant relationships. To overcome this, a clustering procedure is adopted. Within a block, if a pair of sequences is identical at, say, 80% of the sites, these two sequences are merged into one seqùence, and the residue pair frequencies with the rest of the sequences in the block are counted as the average of the frequencies of each one of the merged pair. For example, if the percentage is set at 80%, and sequence segment A is identical to sequence segment B at 80% of their aligned positions, then A and B are clustered and their contributions averaged in calculating pair frequencies. If a third sequence C is identical to either A or B, again at

80% of aligned positions, it is also clustered with them and the contributions of A, B and C averaged, even though C might not be identical to both A and B at 80% of aligned positions. A scoring matrix constructed by this clustering procedure, with the percentage set to N%, is called the BLOSUM'N' matrix. Thus if N is 80%, we obtain the BLOSUM80 matrix. Obviously, the BLOSUM100 matrix is almost the same as the matrix constructed without any clustering, since very few sequence pairs would be identical at all positions. However as the percentage is decreased, the number of sequences in each block gets reduced. Indeed some of the blocks disappear, since all the sequences get merged into a single sequence, which, of course, cannot be included in further calculations.

There are two major differences between the BLOSUM matrices and the PAM matrices. The first is the kind of data used: the PAM matrices were constructed from 62 families of sequences; the BLOSUM matrices made use of data from hundreds of related families of sequences. The second difference is in the concept: the first used data from sequences with close evolutionary relationships and extrapolated to more distant relationships; the second directly used all protein sequences in a related family without regard to evolutionary relationships. A comparison of the two sets of matrices has been made using a measure of average information per residue pair. This comparison shows that the PAM250 matrix is comparable to the BLOSUM45 matrix, while PAM120 is comparable to BLOSUM80. BLOSUM62 gives roughly the same results as PAM160.

4.2.5 Gap penalties

A gap is a consecutive run of spaces in a single sequence of an alignment. It corresponds to an insertion or deletion of a subsequence. Gaps are caused in many ways. A single mutation can create a gap – this is perhaps the most common cause. Sometimes unequal crossover during meiosis can lead to insertion or deletion of strings of bases in the DNA sequences. DNA slippage during replication could result in the repetition of a set of consecutive bases, and this could lead to a gap when the corresponding protein sequence is aligned to another that has not resulted from such slippage. A further cause of gaps in the alignment is the insertion of small subsequences by a retrovirus in one of the two sequences aligned. Gaps may occur at three possible locations in an alignment: before the first character of one of the sequences, inside one of the sequences or after the last character of one of the sequences. Gap penalties are also part of the scoring scheme, and must be chosen along with the substitution scores. There are two reasons for applying a penalty whenever a gap is introduced, one practical, and the other biological. The practical reason is that, if there were no penalties for gaps, and any number of gaps of any size was to be allowed, even two random sequences may be aligned with high scores. The biological reason for allowing gaps is that during the course of evolution related sequences diverge by acquiring insertions and deletions (or 'indels'). If these indels are accepted by natural selection, then clearly they do not interfere drastically with the function of the protein. However, in general, if a mutation has to occur, biology would prefer a transversion rather than an indel. Gap penalties would then be required to ensure that the former are preferred over the latter in any sequence alignment. The choice of a gap penalty thus requires a lot of finesse. A low penalty leads to a heightened sensitivity, but a lack of specificity, thus giving rise to high scores, high background 'noise', and therefore reduced biological significance. A high penalty works in just the opposite way, increasing the specificity, but decreasing the sensitivity.

There is no general theory that can guide the selection of gap penalties, especially, for local sequence alignment. (For global alignments, low gap penalties would in general be preferred). Using the sequence alignment data generated in the construction of the PAM matrices, we may consider an indel to be the 'twenty first residue', and determine a transition probability for each one of the twenty amino acids to occur opposite an indel. To calculate the odds ratio, however, we also need to know what are the odds of such a transition occurring purely by chance. While a Markovian model, based on the total frequency of occurrence of each amino acid, exits for the other twenty residues, there is no

such theory for indels, and P_{random} cannot be estimated for the transitions (amino acid, indel). In general we write the total gap penalty for an alignment as

$$\gamma = \Sigma_{i=1,\,n}[f_i(g_i)]$$

where $f_i(g_i)$ is a function that depends on the size of the gap (i.e. the number of contiguous sites in it), and n is the number of gaps A linear gap cost or gap penalty is written as

$$f_i(g_i) = - k \times g_i$$

where k is a constant, and g_i is the size of the gap number i. Such a linear gap cost, however is unrealistic, especially for DNA sequences. A single insertion or deletion event can easily affect large chunks of DNA, creating big gaps, and a linear function would penalize this event far more strongly than another single mutation that created only a small indel. Consider also the difference between a coding sequence and a non-coding one. In the former, an indel of size one, or two, nucleotides could affect large portions of the corresponding protein sequence by introducing a shift in the gene reading frame. Such mutations are called frame-shift mutations. Indels of size 3, or multiples of three, do not shift the reading frame, and are consequently much less harmful to the protein sequence. In this case the gap penalty is best specified as a periodic function, with a period of 3. Even in the case of protein sequences, a linear gap cost function is unrealistic. A better function is the so-called 'affine' gap cost, written as

$$f_i(g_i) = -d-\{(g_i-1) \times e\}$$

where d is the 'gap opening' penalty, e is the 'gap extension' penalty, and once again g_i is the size of the gap i. Typically d is much greater than e.

A recent study has shown that gap penalties vary in their effectiveness depending on the evolutionary distance at which relationships between the sequences are sought. Thus in applying the affine gap penalty to a particular set of sequence alignments, the most effective value of d depends on the substitution matrix used. The optimal gap opening penalty for a given matrix (e.g. BLOSUM50, PAM200) has been determined to be d = 25 - 0.1 x (target PAM distance). However the value of the gap extension penalty e is not as sensitive, and a constant value of 5 may be used. For example if we use the PAM120 substitution matrix (equivalent to BLOSUM80), d = 25 – 0.1 x 120 = 13, and e = 5.

4.3 Phylogenetic Trees

4.3.1 Introduction

Phylogeny refers to the evolutionary relationships among species. Speciation is the process through which one species becomes divided into two or more new species. The pattern of evolutionary relationships among species is called their phylogeny. It is convenient to represent phylogeny as a tree in which lines represent species. At some places a line splits into two to represent points where ancestral species evolved through speciation into two new species. However, one could discuss phylogeny and phylogenetic trees without reference to evolution and the formation of new species, and simply as the relationships between different individuals being compared. In this sense, a phylogenetic tree is considered as a graph with a set of nodes, and lines joining them. The tree has internal nodes, indicating branch points, and external nodes, usually indicating the data points used in constructing the tree. In the context of biological evolution, these data are referred to as Operational Taxonomic Units, or OTUs for short. Taxonomic units are, in general, the characteristics or features or data used to compare organisms and infer relationships between them. They could be morphological features, such the size and shapes of, say, the beaks of birds. Or they could behavioural features, such as, say, whether the mother gives birth to live young or lays eggs. In this book all examples and discussion will be under the assumption that the OTUs being considered are protein or DNA (or RNA) sequences. Taxonomic units could refer to hypothesized features that may have been present in an ancestral, now

extinct, organism. To differentiate such cases, we define OTUs in particular as those taxonomic units that have been actually measured in the laboratory or in the field, and are now being used to build the tree. The correct tree, or the best tree is one that explains all the relationships between all the OTUs to the most satisfactory extent.

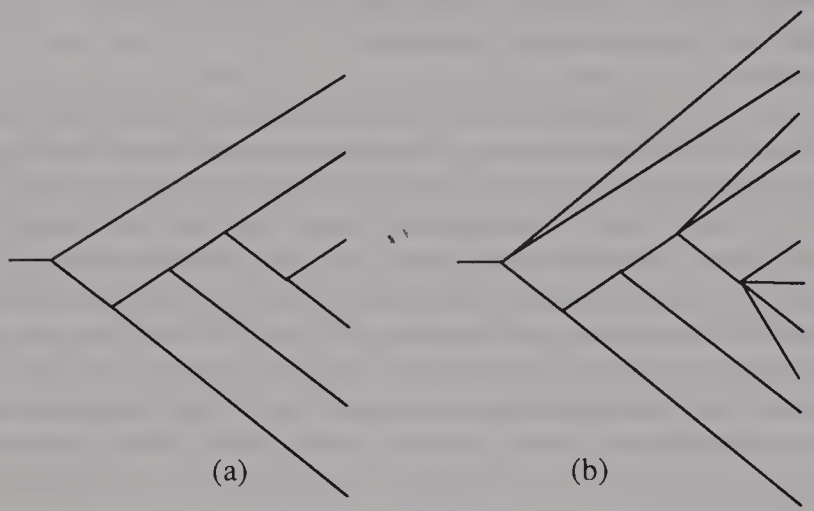


Figure 4.8 Bifurcating (a) and multifurcating (b) phylogenetic trees.

In building a phylogenetic tree that represents evolutionary relationships, we choose a graph that does not have any closed rings or loops. Thus only one branching point will connect two adjacent points at the end of the tree. A tree may be bifurcating (Figure 4.8a) in which at every node there are only two branches. Else the tree may be multifurcating (Figure 4.8b) in which each node may branch into two or more stems. Phylogenetic trees representing evolutionary relationships are always considered bifurcating. An evolutionary tree must identify which groups of the creatures being compared had more recent common ancestors, and which groups had more ancient common ancestors. Such trees are called 'rooted trees', as shown in Figure 4.9a. On the other hand the same set of relationships between the organisms may also be represented by the graph in Figure 4.9b, called an un-rooted tree. Most mathematical algorithms to build, i.e. to draw, phylogenetic trees from data regarding the characteristics of the organisms yield only the best un-rooted tree, i.e. the un-rooted tree that explains most satisfactorily all the data used. To convert such an un-rooted tree into a rooted one, we require additional information, not part of the data already used in building the tree. This information should identify which one of the organisms diverged from the main group at the earliest point during the course of evolution. The

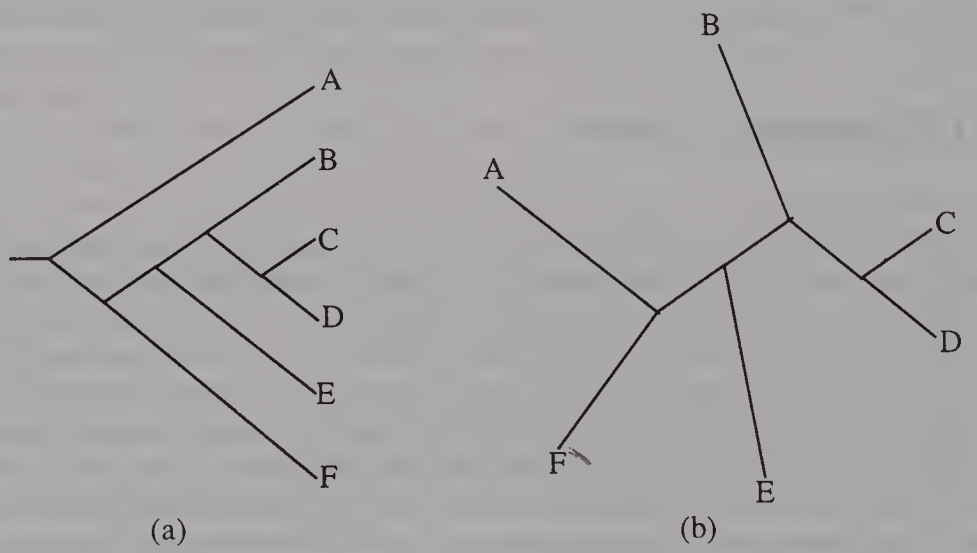


Figure 4.9 Rooted (a) and unrooted (b) phylogenetic trees for the six OTUs A, B, C, D, E and F. The scale of distances between the OTUs is not the same in the two trees.

earliest diverging organism is called the 'outgroup'. With this proviso, we now do not consider a phylogenetic tree as directly indicating the course of evolution. Instead we consider it to be a graph that best illustrates all the relationships indicated in the data at hand.

To appreciate the mathematical problem of constructing phylogenetic trees consider a set of OTUs A,B,C,D... etc. We will assume that we have some way of calculating if one pair, say A and B are more similar to each other than another pair, say A and C. The problem is to represent these relationships, for all the OTUs, by means of a tree. If we have only two OTUs, say A and B, there is only one tree that we could construct (Figure 4.10a). If we consider 3 OTUs, there is one way of building an unrooted tree, but 3 ways of building a rooted tree, depending on which of the three OTUs is considered the outgroup (Figure 4.10b). With 4 OTUs the number of possible un-rooted trees increases to 3, and the number of possible rooted phylogenetic trees is 15 (Figure 4.10c).

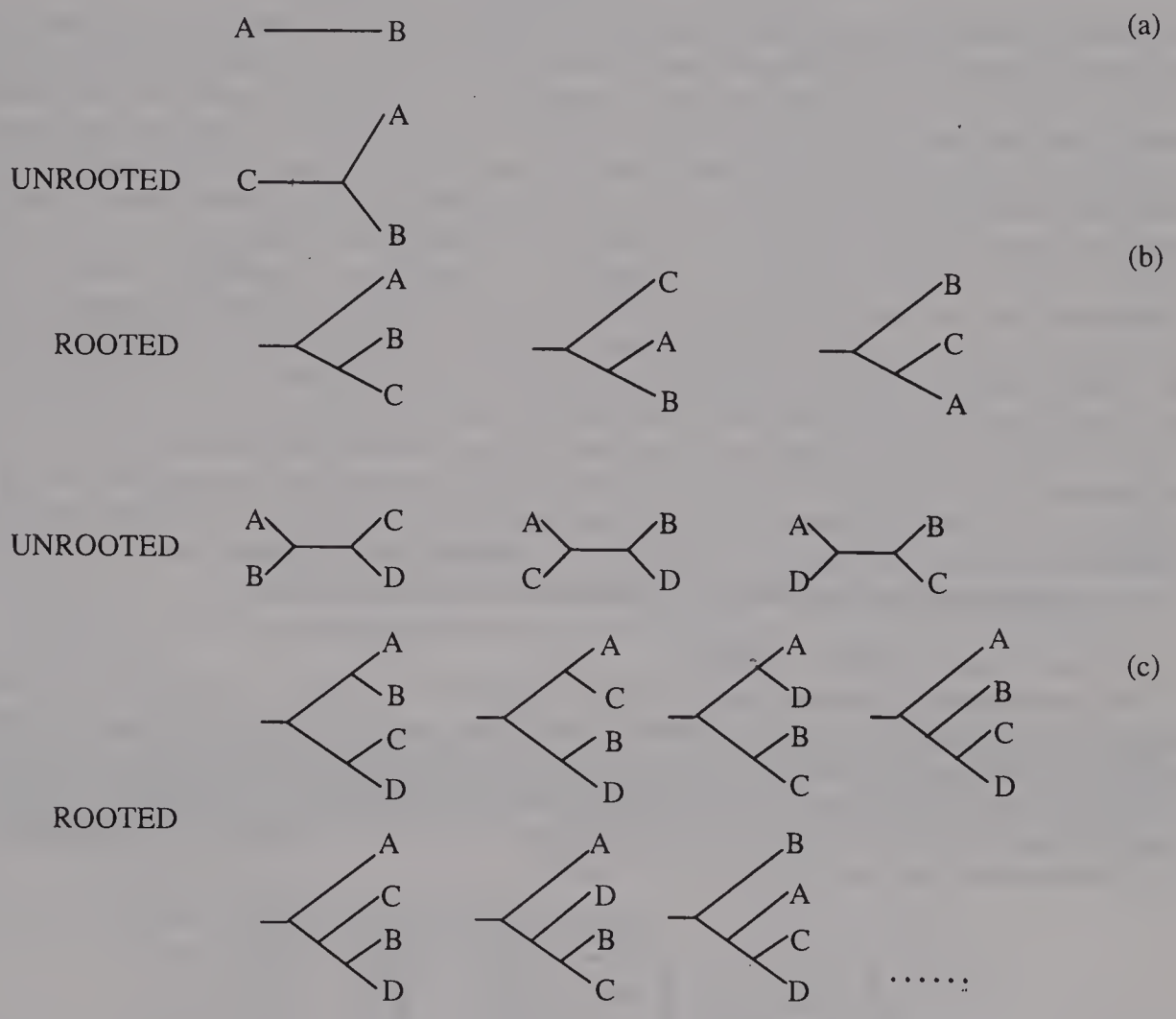


Figure 4.10 Possible rooted and unrooted phylogenetic trees for the two (a), three (b) and four (c) OTUs. Of the 15 possible rooted trees for four OTUs, only 7 are shown. The rest may be easily derived.

Table 4.3

Number of OTUs	Possible number of unrooted trees	Possible number of rooted trees.
2	1	1
3	1	3
4	3	15
5	15	105
8	10,395	135,135
10	2 million	35 million

The number increases even more rapidly after this, as shown in Table 4.3. Building a phylogenetic tree for a given number of OTUs means we need to identify the best tree among these, i.e the one that best explains all the data. A simplistic, brute-force approach is to consider all possible trees one after the other, scoring each one by some system that indicates how closely the tree explains the data about the relationships between the OTUs, and then choose the one that does it best. Such an approach, even if performed on the fastest of computers, soon takes impossible large computation times for anything more than about 12 or 15 OTUs. In the parlance of computational complexity, such an approach belongs to the set NP, or the set of algorithms for which the computational times increases exponentially with the size of the problem, and not as a polynomial of the size (see Chapter 7, section 3.2). There are, however, other algorithms that terminate in a relatively small amount of computation time, and also give a very reasonable tree, though it cannot be proven that this is in fact the very best one. Such algorithms, in common with other similar ones in computer science, are referred to as heuristics. There are many, many such algorithms. In fact, since this problem may be cast as one in optimization of an objective function, most optimization methods may be applied to solving this problem as well. We will however consider three methods that are commonly used and have proved immensely successful. The first is in fact a class of methods, called 'distance matrix' methods. The other two are the maximum parsimony and the maximum likelihood methods.

4.3.2 Distance matrix methods – UPGMA

As the name suggests, distance matrix methods depend on the ability to calculate an evolutionary distance between any pair of OTUs. Earlier in this chapter, we have already seen how this may be carried out when the OTUs are DNA or protein sequences. We will not repeat the discussion here, but will simply start the discussion of the UPGMA method, and the other distance matrix methods, by assuming that a matrix of distances between every pairs of OTUs is available, such as, for example, the one given below for the five OTUs A, B, C, D and E.

	A	B	C	D	E
A	0	d_{AB}	d_{AC}	d_{AD}	d_{AE}
B	d_{AB}	0	d_{BC}	d_{BD}	d_{BE}
C	d_{AC}	d_{BC}	0	d_{CD}	d_{CE}
D	d_{AD}	d_{BD}	d_{CD}	0	d_{DE}
E	d_{AE}	d_{BE}	d_{CE}	d_{DE}	0

In this table d_{AB} is the distance between the OTUs A and B, etc.

UPGMA stands for Unweighted Pair Group Method with Arithmetic mean. This method of constructing a phylogenetic tree uses the distance matrix to identify the first branch of the tree as consisting of the two OTUs that have the shortest between them. Let us assume that d_{AB} is the least of all the distances. The OTUs A and B thus form the nodes, or 'leaves' of the first branch of the tree (Figure 4.11a). These two OTUs are merged together and considered as a single composite OTU,

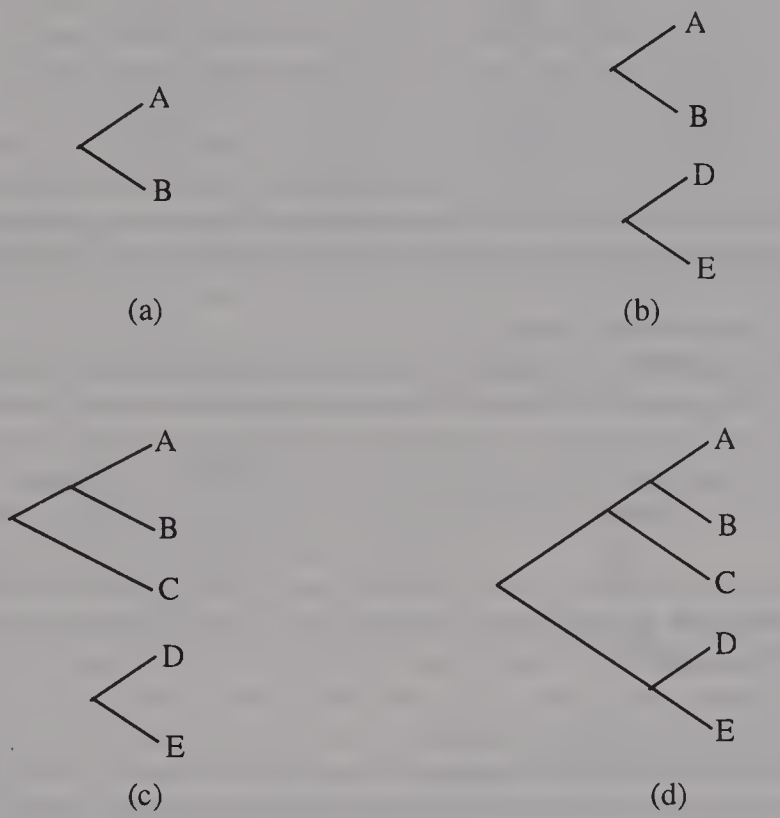


Figure 4.11. The various stages of the hierarchical clustering method of constructing phylogenetic trees. (It is emphasized that these are unrooted trees, regardless of the style of drawing them)

called 'AB'. The distances between this new OTU and all the others is calculated by taking the unweighted arithmetic mean of all the pair wise distances of the two OTUs A and B in the new OTU AB with all the other OTUs in the group, viz. C, D and E. For example, the distance between AB and C is given by the expression

$$d_{(AB)C} = (d_{AC} + d_{BC})/2$$

The table of distances is therefore recalculated as follows.

	AB	C	D	E
AB	0	$d_{(AB)C}$	$d_{(AB)D}$	$d_{(AB)E}$
C	$d_{(AB)C}$	0	d_{CD}	d_{CE}
D	$d_{(AB)D}$	d_{CD}	0	d_{DE}
E	$d_{(AB)E}$	d_{CE}	d_{DE}	0

Note that only the first column and the first row need to be recalculated. The other distances, for instance the one between the OTUs C and D, remain unchanged. Next, step 1 is repeated, by identifying the shortest of all distances in the table. Let as assume that this is d_{DE}. The next node of the tree is then identified as the one connecting the OTUs D and E, and these are then the two new leaves of the tree (Figure 4.11b). The new OTU is DE, and the distance matrix is again recalculated by taking unweighted arithmetic means.

	AB	C	DE
AB	0	$d_{(AB)C}$	$d_{(AB)(DE)}$
C	$d_{(AB)C}$	0	$d_{C(DE)}$
DE	$d_{(AB)(DE)}$	$d_{C(DE)}$	0

In the UPGMA method, the distance between two composite OTUs, such as AB and DE, is computed as the unweighted arithmetic mean of the pairwise distances between the constituent OTUs. Thus in the matrix above,

$$d_{(AB)(DE)} = (d_{AD} + d_{AE} + d_{BD} + d_{BE})/4$$

Once again this matrix is examined to identify the shortest distance (say $d_{(AB)C}$). Once again, a new node is added to the tree (Figure 4.11c), and the distance matrix recalculated.

	(AB)C	DE
(AB)C	0	$d_{((AB)C)(DE)}$
DE	$d_{(AB)(DE)}$	0

This process is repeated until no new nodes remain to be added. Figure 4.11d indicates the final phylogenetic tree for this example.

4.3.3 Distance matrix methods – Neighbour joining

This method is best explained through an example. Consider N OTUs 1, 2, 3, 4,...N. We assume, as above, that a matrix of evolutionary distances between every pair of OTUs is available to us. We begin the process of building the phylogenetic tree, by assuming a 'tree' without any clustering, in which every OTU is placed in an equal relationship with every other OTU (Figure 4.12a). To identify the first pair of OTUs to cluster together, so that they diverge from the same node in the tree, we proceed as follows. We assume that OTUs 1 and 2 form this first cluster. We then use the following formula to calculate the total 'length' of a tree that would have 1 and 2 as diverging from one node, and the other N-2 OTUs from another node (Figure 4.12b).

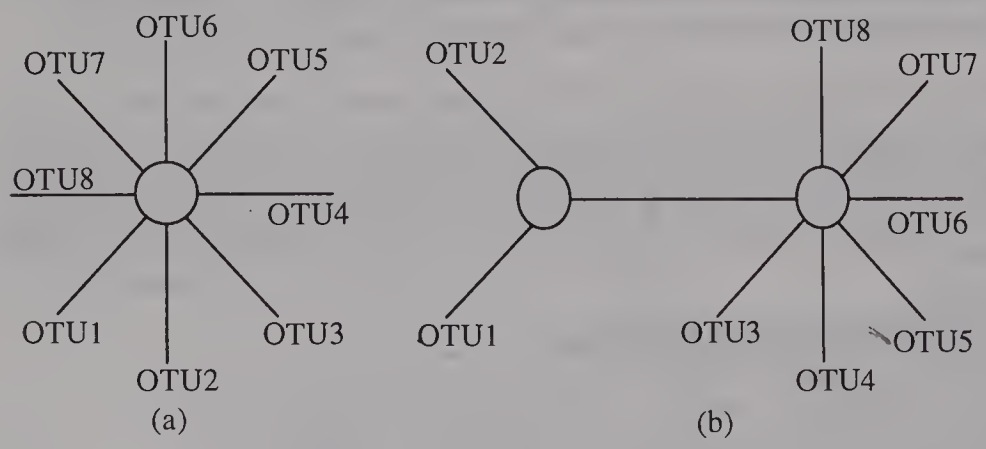


Figure 4.12. An illustration of the neighbour joining method for 8 OTUs (N = 8).

$$S_{12} = X + Y + Z$$

where $X = 1/\{2(N-2)\} \times \sum_{j=3,N} (d_{1j} + d_{2j})$

$Y = 1/2 \times d_{12}$

$Z = 1/(N-2) \times \sum_{3 \le i \le j \le N} d_{ij}$

In these equations d_{ij} is the distance between OTU i and OTU j as given in the distance matrix. The first term X is the mean distance from OTU 1 and OTU 2 to the rest of the OTUs 3 to N. The second term Y is just half the distance from OTU 1 to OTU 2. The third term Z is the average of all the pairwise distances between the OTUs 3 to N. Thus S_{12} is the total tree length if we assume that OTU 1 and OTU 2 form the first cluster. Similarly S_{13} is calculated as the total tree length if OTU 1 and OTU 3 form the first cluster. Likewise the tree lengths $S_{14}....S_{1N}$ are calculated. The lowest of these then indicates the first pair of OTUs to be clustered, i.e. the first pair of neighbours to be joined. Next, just as in the UPGMA method, the first pair of neighbouring OTUs are merged into one composite OTU, and the distance matrix is recalculated as the unweighted arithmetic mean of the pairwise distances between the constituent OTUs (of the composite OTU) and all the other OTUs in the group. The newly calculated distance matrix is then used to repeat the procedure to identify the next pair to be clustered. The entire process of calculating the total tree lengths for all possible ways of pairwise clustering, identifying the cluster that will yield the tree with the smallest total length, merging the two OTUs identified for the cluster into one single composite OTU, and then recalculating the distance matrix, all these steps are repeated until no more OTUs remain to be clustered, and the complete tree has been formed. Clearly the method is too tedious to perform manually for anything over say 4 or 5 OTUs, but the computation time actually increases only as the square of N, the number of OTUs. Thus in terms of computational complexity, the algorithm is in the set P.

Besides UPGMA and neighbour joining, there are several other distance matrix methods, such as the transformed distance method and the neighbours relation method. We will however not consider these further, chiefly because these are not as commonly used as UPGMA and neighbour joining. The trees constructed using the distance matrix methods may be made more quantitative by calculating the lengths of the branches of the tree. There are two commonly used methods for this purpose, namely the Fitch-Margoliash algorithm, and the least squares technique. We will illustrate the first of these with a simple example, a tree consisting of three OTUs A, B and C. There is, of course, only one unrooted tree connecting these (see Figure 4.10b). Let us assume that the branch lengths from the single internal node connecting the three OTUs are x, y and z for the three branches, respectively. We may now write three simultaneous equations in these three unknowns, as follows.

$$d_{AB} = x + y$$

$$d_{AC} = x + z$$

$$d_{BC} = y + z$$

Here d_{AB}, etc., and obtained, as usual, from the distance matrix. These three equations may be solved to obtain x, y and z.

$$x = (d_{AB} + d_{AC} - d_{BC})/2$$

$$y = (d_{AB} + d_{BC} - d_{AC})/2$$

$$x = (d_{AC} + d_{BC} - d_{AB})/2$$

For a larger number of OTUs, a similar procedure is adopted by first calculating, wherever necessary, a single branch length for two or more OTUs that have been merged into a composite OTU. In such cases we choose the appropriate, recalculated distance matrix. The branch lengths of the OTUs composing the single merged OTU are calculated in turn by the same procedure.

4.3.4 Maximum parsimony

This set of methods is not based on a numerical distance between the OTUs. As implemented here, it is applicable when the OTUs are protein or DNA sequences, since it is based on character comparisons.

The principle underlying the method is to choose the tree that will explain all the differences in the sequence data by postulating the least number of mutations among them. Thus we are parsimonious (or 'stingy') in allowing mutational changes between the sequences. This follows from a general principle in science, called 'Occam's Razor', that when we have to decide between two or more explanations for a given set of data, we choose the one that makes the least number of assumptions. The application of this principle in building phylogenetic trees using molecular sequences is illustrated using a simple example. Consider the following four OTUs

```
OTU1:  A A G A G T G C A
OTU2:  A G C C G T G C G
OTU3:  A G A T A T C C A
OTU4:  A G A G A T C C G
```

This multiple sequence alignment of the four DNA sequences is the starting point to build the correct maximum parsimony tree. The general procedure is to check every possible tree and choose the one that requires the least number of substitutions or mutations. We do not however carry this all at once for the entire sequence, but examine each column of characters, or 'site', to determine a maximum parsimony tree for each. Once the examination has been carried out for all the sites, the results are averaged and the best tree for all the sites together is identified. In the above example, the four OTUs have the three possible unrooted trees given in Figure 4.13a. (Note that just as in the case of the distance matrix methods, or indeed all the methods discussed here, we can only identify the best unrooted tree. Converting the unrooted tree to a rooted tree requires the identification of one of the OTUs as an outgroup.) Let us now consider each site in the alignment. The three trees

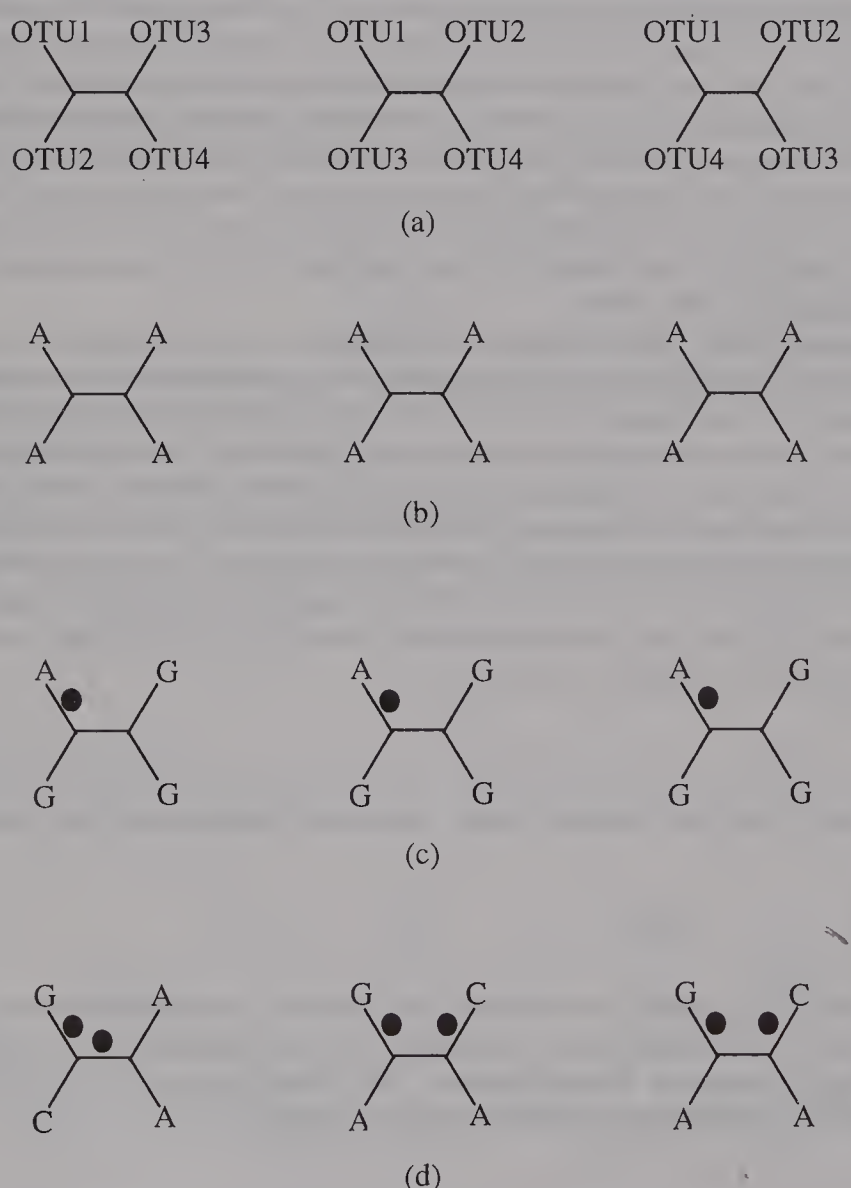


Figure 4.13

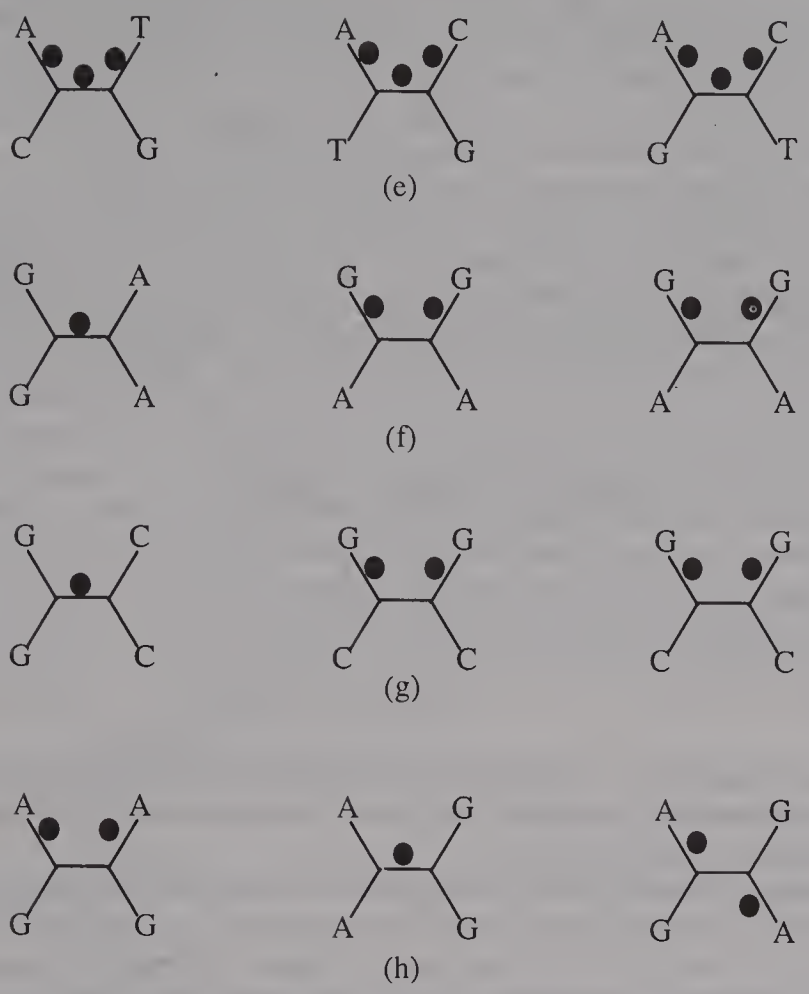


Figure 4.13. The minimal unrooted trees at the seven sites (of the nine) described in the text. The dark circles stand for mutations (or substitutions). Note that, since we do not know the nucleotides at the internal nodes, shown here is only one possible set of mutations. However these are chosen to yield the minimal substitutions in each case. Figures 4.13 (b), (c), (d), (e), (f), (g) and (h) correspond to Sites 1, 2, 3, 4, 5, 7, and 9 respectively. In each figure, from left to right, Tree I, Tree II and Tree III.

for the first site are shown in Figure 4.13b. Clearly, since all four characters (nucleotides) at this site are the same, no substitutions need be postulated for any of the trees, and all three are equally likely. Such sites, which give no useful information about the best tree, are called uninformative sites. We can discard these from consideration. However we need to identify which are the un-informative sites and which are the informative ones, and this is not always straightforward. For DNA or RNA sequences, a simple rule is that if, at a site, there are at least two different nucleotides, each occurring at least two times, then that is an informative site. In the present example, we will not apply this rule, but instead we will check each site and only discard it if we show it to be un-informative. Continuing in this manner, the three trees for site no. 2 are given in Figure 4.13c. We see that no matter which of the three trees we choose, we require to assume that one mutation has taken place. Again, this result does not help us to decide which one of the trees is the correct one, and site no. 2 is also uninformative. Consider site no. 3. The three possible trees for this site are given in Figure 4.13d. Tree I has 2 substitutions. For Tree II we can postulate either 2 substitutions, or, assuming different nucleotide changes, 3 substitutions. To choose between these two possibilities for Tree II, we again apply the principle of maximum parsimony, and choose to explain the tree with just 2 substitutions. In other words, at every stage, we choose that options that requires the least number of assumptions. Therefore, Tree II is explained by 2 substitutions, and, as shown in the figure, so is Tree III. Thus, site no. 3 is also an uninformative site. Consider site no. 4. The three

minimal substitution trees are shown in Figure 4.13e. Again all have the same number of substitutions, and this site also in uninformative. Site no. 5 gives use the first informative site in the series (Figure 4.13f). Tree I is explained by assuming just one substitution, while trees II and III require 2 substitutions each. If we were to decide on the best tree based on this site alone, we would chose Tree I as the maximum parsimony tree. However we have to consider the remaining 4 sites also, and therefore, we will record the results for this site, and move to the next one, site no. 6. Since this has the same nucleotide in all four positions, this is an uninformative site. Site no. 7 is an informative site, as shown in Figure 4.13g, with 1 substitution for Tree I and 2 substitutions each for Trees II and III. Site no. 8 is again an uninformative site, while at site 9 (Figure 4.13h) 2 substitutions are required to explain Trees I and III, and one substitution to explain Tree II. That completes the examination of all 9 sites and the results are summarized in the following table, which gives the minimum number of substitutions required to explain each of three trees at each site. Note we have included only the informative sites in the table.

	Tree I	Tree II	Tree III
Site 5	1	2	2
Site 7	1	2	2
Site 9	2	1	2
Total (of all informative sites)	4	5	6

Examination of the table shows that overall, Tree I requires the least number of assumptions to be made regarding possible mutations between the sequences. Thus, Tree I best explains the available data and is considered the correct 'maximum parsimony' tree.

The maximum parsimony method is an algorithm in the set NP, i.e. the computational time increases exponentially (or even faster!) with the number of OTUs considered. This is because, for every site, all possible trees have to be evaluated, and we have seen in section 4.3.1 that this number increase faster than exponentially as a function of the number of OTUs. For greater than about 11 or 12 OTUs, therefore, the maximum parsimony method becomes impossible to use. However, the algorithm may be modified by introducing the so-called 'branch and bound' technique to reduce the number of potentially optimal trees to be considered. One may even use heuristic techniques such as 'stepwise addition' or 'branch swapping' to reduce the computation time to manageable limits, though these latter techniques do not guarantee a rigorously optimal solution. Detailed discussion of these techniques is beyond the scope of this book.

4.3.5 Maximum likelihood

Like maximum parsimony, this algorithm also is based on a comparison of character states. In the maximum parsimony method the best tree was chosen by minimizing the number of mutations required to explain the tree. In the maximum likelihood method, a likelihood function is defined for every possible tree, and the tree with the maximum value of this function is chosen as the best tree. The likelihood function is calculated from the probability of a residue changing to another residue or remaining the same over a specified period of time. The probabilities are calculated on the basis of an evolutionary model. The likelihood function is calculated for each residue site for each possible tree, and finally, when all the sites and trees have been calculated, the best tree is chosen by considering the entire sequence.

We will illustrate the method taking the example of tree based on DNA sequences. There are four nucleotide residues, and we have to consider an evolutionary model that allows us the calculate the

probability of a residue i at a given site in a given sequence changing to the residue j at the same site in another sequence over a period of time t. The Jukes and Cantor one-parameter model is one such. In section 4.2.2 we had given the transition or mutation probabilities as a matrix. Equally, we may write it as follows.

$$P_{ii} = ¼ + ¾[\exp(-4\lambda t/3)]$$
$$P_{ij} = ¼ - ¼[\exp(-4\lambda t/3)]$$

Here P_{ii} is the probability of residue i remaining as such with no change, and P_{ij} is the probability of residue i changing to residue j. Since we are dealing with DNA, residues i and j correspond to the four nucleotides A, T, G and C. λ is the rate of substitution per unit time, and t is the time period. In order to build the likelihood function, consider a phylogenetic tree for four OTUs. At a particular site, consider that these four OTUs, OTU1, OTU2, OTU3 and OTU4, have the residues i, j, k and l respectively, corresponding to one of the four nucleotides. We also label the nucleotide at the node connecting OTU1 and OTU2 as z, the one at the node connecting z and OTU3 as y, and the one at the node connecting y and OTU4 as x. We further call as t_1 the time period required for the nucleotide x to mutate to y and l; t_2 is the time period of change from y to z and k; and t_3 is the time period of change from z to i and j. Note that we have no knowledge of which of the four nucleotides actually correspond to x, y and z. The likelihood function for a site with the nucleotides i, j, k and l in the four OTUs is computed as follows. Consider the first branch. The nucleotide x has changed into the pair y and l. Since l is the nucleotide that is present now, the total time taken for x to change to l is $t_1 + t_2 + t_3$. The probability of this happening is given by

$$P_{xl}(t_1 + t_2 + t_3)$$

The value maybe calculated from the Jukes Cantor model equations give above. Similarly the residue x changes to y in a time period t_1, and the probability of this happening is given by

$$P_{xy}(t_1)$$

Consider now the next node. y changes to z and k. Arguing as above, the probability expressions for these events are

$$P_{yk}(t_2 + t_3)$$
$$P_{yz}(t_2)$$

Likewise at the third node, the probabilities are

$$P_{zj}(t_3)$$
$$P_{zi}(t_3)$$

Since these are all independent events, the total probability of having the nucleotides i, j, k and l in the four OTUs is given by the product of all these probabilities, i.e.

$$P_{xl}(t_1 + t_2 + t_3)P_{xy}(t_1)P_{yk}(t_2 + t_3)P_{yz}(t_2)P_{zj}(t_3)P_{zi}(t_3)$$

As already noted, we do not know the particular ancestral nucleotides that we represent as x, y and z. We assume that they could be any one of the four nucleotides, each occurring with a probability g_x, which we may compute from the present nucleotide frequencies. The likelihood function, which we now call H, is written to accommodate this fact, and to sum over all the possibilities for x, y and z, as follows.

$$H(i,j,k,l) = \sum_x g_x P_{xl}(t_1 + t_2 + t_3) \sum_y P_{xy}(t_1) P_{yk}(t_2 + t_3) \sum_z P_{yz}(t_2) P_{zj}(t_3) P_{zi}(t_3)$$

This is the likelihood of observing the nucleotides i, j, k and l in the given order, in the given tree. To indicate this we write the likelihood as

$$H(i,j,k,l,T)$$

This is the likelihood for a single site. We may similarly calculate the likelihood for a given tree for all the sites in the sequence alignment data. The product of the likelihood at all the sites is the total likelihood for a given tree. In general, we write a likelihood function

$$L(\theta_1,\theta_2 \ldots \theta_n | x_1, x_2 \ldots x_N, T)$$

This is a function of the unknown parameters $\theta_1,\theta_2...\theta_n$ that correspond to features such as branch lengths (or branching dates), nucleotide substitution rates and so on. $x_1,x_2...x_N$ are the observed data, i.e. the sequences and the nucleotide substitutions in them, while T is the tree being considered. The task is to maximize this likelihood function by varying the unknown parameters and considering different trees. This is, in general not a straightforward optimisation problem, and is usually solved using a heuristic algorithm such as the branch and bound method. The best maximum likelihood tree identified by this procedure is a quantitative tree that gives precise branch lengths as well as the phylogenetic relationships.

There are several other methods of phylogenetic reconstruction that have not been considered above. This is a somewhat controversial field, since no one method works well in all circumstances, and the choice of a particular method depends also on individual choice, at least to some extent. To evaluate the relative effectiveness of the different methods, we must consider the basic assumptions, such as, for example, the assumption that the rate of residue substitution is constant for all the branches of the tree. We must also consider the computational time required, the statistical consistency, and the relative performance of the method under all conditions. Tests using computer simulations have shown that, in general, if the assumption of non-uniform rates of substitution is made, then the maximum likelihood method performs the best, followed by neighbour joining, with maximum parsimony being the least effective. In choosing a particular program or method to construct a phylogenetic tree, however, we need to consider other aspects as well, such as computational costs.

4.4 Summary

- The three topics, multiple sequence alignment, substitution matrices and phylogenetic trees are heterogeneous, but are related in that the results of one are used in the other, e.g. multiple sequence alignment is used to construct substitution matrices, as well as phylogenetic trees.
- A multiple sequence alignment may be defined as a two-dimensional table in which each row represents a protein or nucleic acid sequence, and the columns are the individual residue positions.
- One of the common goals of building multiple sequence alignments is to characterize protein and/or gene families, and identify shared regions of homology.
- Multiple sequence alignments may be represented by consensus sequences or profiles, or the entire alignment may be shown with colours or shades to highlight different features of the alignment.
- The most common way of finding the score of any given multiple sequence alignment is the so-called 'sum-of-pairs' score.
- The use of dynamic programming algorithms for multiple sequence alignment is, in practise, not possible for more than three or four sequences, because the computational cost increases exponentially with the number of dimensions, i.e. the number of sequences.
- The Carillo-Lipman algorithm tries to overcome this problem by limiting the area of search.
- Progressive or hierarchical alignment methods add one sequence at a time to the multiple sequence alignment.
- The Feng-Dolittle algorithm is a progressive alignment technique that uses a distance matrix to construct a crude phylogenetic tree of relationships between the sequences to guide the alignment. Based on the tree, sequences are progressively added to the multiple sequence alignment, until all sequences are a part of it.
- CLUSTAL is a popular program for multiple sequence alignment that uses an extensively modified version of the Feng-Doolittle algorithm.

- A matrix of values that is used to score residue replacements or substitutions is called a substitution matrix.
- The Unitary substitution matrix has a score of one along the diagonal and zero everywhere else. This reflects the fact that according to this scoring scheme, a score of one is given wherever the matching residues in the alignment are identical, and zero otherwise.
- Biologically sensible scoring matrices for proteins may be specified on the basis of the similarity, or otherwise, of the biochemical properties of every pair of residues.
- The most commonly used matrices are based on an analysis of similarities between sequences of proteins that have the same known function and structure. Usually, a multiple sequence alignment of a set of homologous sequences yields statistics regarding amino acid substitutions, which are converted to substitution scores
- There are many evolutionary models, and the score (and the substitution matrix) calculated varies depending on which model is chosen.
- Most models assume that the residues are each position in the sequence evolve (i.e. suffer mutations) independently of residues at other positions.
- The Kimura and the Jukes-Cantor models of evolution for nucleotide sequences help to calculate the probabilities of nucleotide transversions. Similar models are available for amino acid residues.
- The two most popular statistically derived matrices are the PAM matrices and the BLOSUM matrices.
- PAM (Percentage Accepted Mutation) matrices are based on a Markovian model of evolutionary change in the sequences. Each site, i.e. residue, in the sequence is considered to evolve independently of the other sites.
- The amount of time required to produce one accepted transition in one hundred residues is counted as one unit of evolutionary time and is called 1 PAM
- Margaret Dayhoff used ungapped multiple alignments of certain well-conserved regions from 71 groups of closely related proteins to obtain the transition probabilities within the set of amino acids.
- The matrix of transition probabilities is converted into a scoring matrix by calculating the logarithm of the odds (or likelihood) ratio for each pairwise alignment of the residues. This gives the substitution scores for a divergence distance of 1 PAM (i.e. PAM1).
- At a divergence distance of n PAM, the substitution matrix is obtained as $(PAM1)^n$. A commonly used substitution matrix in sequence alignment is the PAM100 matrix.
- BLOSUM stands for BLOcks SUbstitution Matrices.
- Just as in the case of the PAM matrices, the scores are obtained as the logarithms of likelihood ratios. However, now they are not based on any specific evolutionary model.
- Groups of related protein sequences were obtained from the PROSITE database. Each group was then aligned into blocks using local ungapped alignment techniques.
- The frequencies of replacement of one residue by the other in the blocks were used to compute the logarithm of the odds ratio for each pair as the substitution score for that pair.
- The PAM120 substitution matrix is comparable to the BLOSUM80 matrix.
- A gap is a consecutive run of spaces in a single sequence of an alignment. It corresponds to an insertion or deletion of a subsequence.
- Gap penalties are also part of the scoring scheme, and must be chosen along with the substitution scores.
- Phylogeny refers to the evolutionary relationships among species. It is convenient to represent phylogenetic relationships as a tree in which lines represent species.

- Three classes of methods are commonly used to build phylogenetic trees from a set of sequence data. They are distance matrix methods, maximum parsimony methods and maximum likelihood methods.
- Distance matrix methods depend on the ability to calculate an evolutionary distance between any pair of sequences (or, in general, between the data used to build the trees).
- UPGMA stands for Unweighted Pair Group Method with Arithmetic mean. This method of constructing a phylogenetic tree uses the distance matrix to identify the first branch of the tree as consisting of the two sequences that have the shortest distance between them. These two are now merged into one composite unit.
- The distance to all other sequences is calculated as the unweighted arithmetic mean of the pairwise distances from each constituent sequence of the composite sequence.
- Neighbour joining is another distance matrix method, in which the best tree is the one for which the total tree length is the minimum.
- Maximum parsimony is based on character comparisons, and is therefore best applicable to sequence comparisons. The principle underlying the method is to choose the tree that will explain all the differences in the sequence data by postulating the least number of mutations among them.
- In the maximum likelihood method, a likelihood function is defined for every possible tree, and the tree with the maximum value of this function is chosen as the best tree. The likelihood function is calculated from the probability of a residue changing to another residue or remaining the same over a specified period of time.
- To evaluate the relative effectiveness of the different methods, we must consider the basic assumptions, such as, for example, the assumption that the rate of residue substitution is constant for all the branches of the tree. We must also consider the computational time required, the statistical consistency, and the relative performance of the method under all conditions.

5

Protein and DNA Sequence Analyses

This chapter deals with the techniques used in analysing protein and DNA sequences. The search for common patterns in sequences that point to similar structures, functions and, perhaps, evolutionary origins, is one of the principal tasks of such sequence analyses. There are several pattern recognition techniques adapted from computer science that have been successfully applied to this task. Databases of sequences with common patterns and motifs have been built, and these are of immense use particularly in the area of functional genomics and genome annotation. Many of the databases of patterns have been constructed by multiple sequence alignment.

5.1 Pattern Representations and Characterisation

In chapter 4, we have seen some methods to represent multiple sequence alignments. With a slight shift of focus, we may consider some of them to be ways of representing patterns in protein sequences. In this section we will discuss in greater detail deterministic patterns, such as regular expressions, as well as probabilistic patterns, such as profiles.

5.1.1 Deterministic patterns – regular expressions

The PROSITE 'pattern' is a good example of a regular expression. The method of identifying such patterns is discussed later. Here we describe the way in which the pattern is represented once it has been identified. Since the PROSITE patterns are all motifs discovered in protein sequences, the rules for the regular expression refer to amino acids. However, with appropriate changes, such rules may also be developed for nucleic acid sequences. The PROSITE rules are as follows.

1. The standard IUPAC one-letter codes for the amino acids are used.
2. A pattern or a regular expression is made of elements. Each element is separated from its neighbour by a hyphen '-', and describes the situation at each position in the pattern.
3. The symbol 'x' is used for a position where any amino acid is accepted.
4. When more than one amino acid is possible at a position, all the acceptable amino acids are listed between square brackets '[ ]'. For example: [RKH] stands for arginine or lysine or histidine.

5. To specify that some amino acids are not acceptable at a particular position, these amino acids are listed between a pair of curly brackets '{ }' For example: {RK} stands for any amino acid except Arginine and Lysine.
6. To signify the repetition of an element of the pattern, it is followed with a numerical value or a numerical range between parentheses. For example x(3) corresponds to x-x-x; x(2,4) corresponds to x-x or x-x-x or x-x-x-x.
7. When a pattern occurs at the N-terminal of a sequence, it starts with the symbol '<'
8. When a pattern occurs at the C-terminal of a sequence, it ends with the symbol '<'
9. A full stop ends the pattern.

The example that follows represents the sequence pattern of a 'serine active site' in the protein cutinase. It occurs in about 12 different protein sequences in the Swiss-Prot database.

P-x-[STA]-x-[LIV]-[IVT]-x-[GS]-G-Y-S-[QL]-G.

The pattern starts with a conserved proline, and has any amino acid at the next position. The third position is occupied by one of serine, threonine or alanine, and the fourth position again has no constraint on the amino acid. The fifth and sixth positions maybe be chosen from [leucine, isoleucine or valine] and [isoleucine, valine or threonine] respectively. The seventh residue may be any amino acid, and the eighth residue may be either glycine or serine. The ninth, tenth and eleventh residues are conserved to be glycine, tyrosine and serine, respectively. At the twelfth position the choice is only between glutamine and leucine. The pattern ends with a conserved glycine.

The above set of rules for building a regular expression is a typical one, but there are variations of these that apply in other situations. Essentially, it is a deterministic set of rules, and this makes it a relatively easy task to decide whether a given sequence may be classified as belonging to the pattern or not – if all the rules are obeyed, the sequence belongs to the pattern; if even one rule is disobeyed, then it does not. However, it is precisely this ease of applying the rule that also makes it incapable of detecting small, but perhaps unimportant, variations in the pattern. In the above example, we have total of 5 residues that must be exactly conserved, if the new sequence is to be classified as belonging to the 'serine active site' sequence of a cutinase. If the pattern has been built after study of hundreds of sequences, all of which exactly follow the pattern of conservation, then it is reasonable to expect that the new sequence should also follow that pattern, if it to be admitted into that classification. However if the pattern has been built only after study of a few tens of sequences, then it is possible that even though the new sequence violates the rule of exact conservation at one or two positions, nevertheless it still belongs to that classification. The situation is even more uncertain if we consider the elements of the pattern that are not exactly conserved. Thus, it would be useful to have ways of representation that indicate how much a given sequence may deviate from the pattern and still be classified in it. Such methods of representation are given by the probabilistic methods, for example, sequence logos.

5.1.2 Probabilistic patterns – sequence logos

As mentioned earlier in chapter 4, consensus sequences are a useful way of representing patterns, but they are even more deterministic than regular expressions. They are a succinct way of representing the information present in a multiple sequence alignment, but they abstract only the most prominent of such information and discard all the rest of it. Sequence logos, though requiring specialized software and hardware, are a way of writing consensus sequences using probabilistic information. As shown in Figure 5.1, sequence logos show all the following information in a single figure – the consensus sequence, the order of predominance of each residue at each position, the probability of each residue at that position, the amount of information present at each position, measured in bits, and, if appropriate, the start and stop points. To build a logo, we start with an aligned set of sequences. The residues that occur most frequently at each position are identified, and they form the consensus sequence, which is displayed most prominently in the logo. The probability of occurrence of every possible residue at each

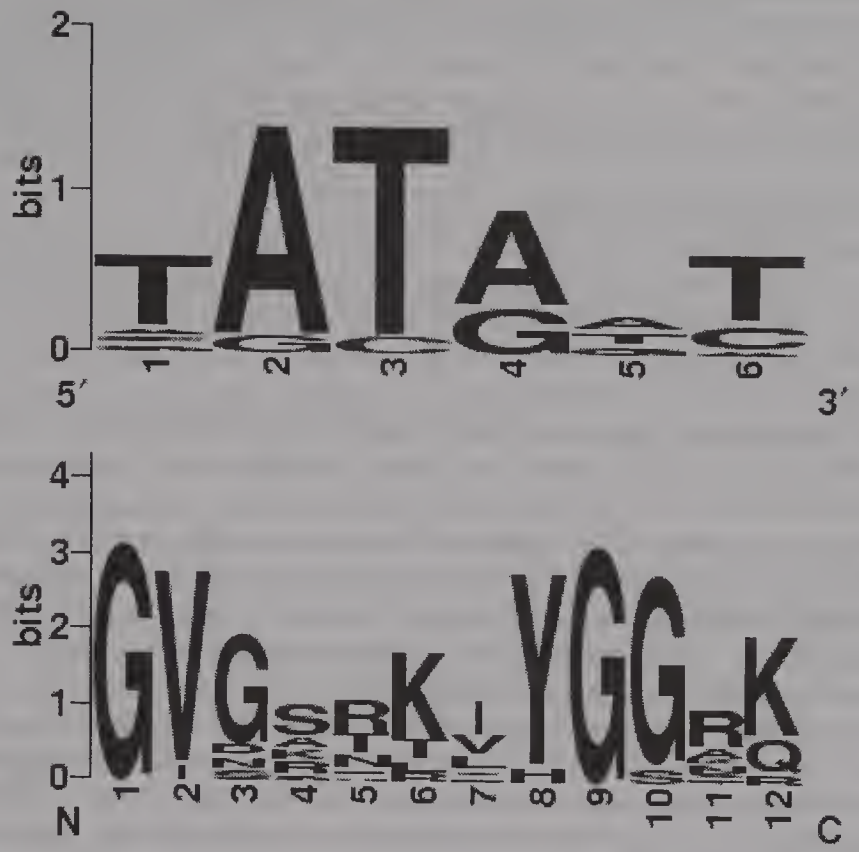


Figure 5.1 (a) Sequence logo for the –10 consensus site of promoters (This figure and Figure 5.1b were created from a small set of aligned sequences chosen arbitrarily, using the online program WEBLOGO at weblogo.berkeley.edu. It does not represent a true –10 consensus sequence). (b) Sequence logo for an arbitrary set of aligned protein sequences.

position may be calculated from the frequency table derived from the multiple sequence alignment. The probabilities are used to calculate the amount of information present at each position in the sequence alignment. This is done through the use of the Shannon formula taken from Information Theory. According to this, we first define a measure of uncertainty H(i) at a particular position i as

$$H(i) = \sum_{j=1.N} [f(R_j,i) \log_2 f(R_j,i)]$$

where $f(R_j,i)$ is the frequency of the residue j at the position i, and the summation is over all the N residues possible at every position. This measure is converted to the amount of information at each position by subtracting it from the maximum possible uncertainty at a point, which is given by $\log_2 N$.

$$I(i) = \log_2 N - H(i)$$

The logo is then constructed by placing the symbols of the residues on top of each at each position. The size of each residue symbol at a position i is given by

$$\text{Height}(R_j, i) = f(R_j,i) \times I(i)$$

The sum of all the heights at a position i is I(i) and is the information content at a position, i.e. it indicates the importance of that position. All measurements are in bits. Figure 5.1 shows two examples of sequence logos, one for DNA sequences, and the other for protein sequences. The maximum uncertainty at any position in a DNA sequence is $\log_2 4 = 2$ bits, since there are four possible residues at that site. For protein sequences this number is $\log_2 20 = 4.32$ bits.

Other possible probabilistic representations of patterns include profiles (or weight matrices, or position specific scoring matrices), which have been explained in Chapter 4. In the context of using

these for pattern discovery and pattern matching, we add the following information. We have described examples of matrices constructed using probabilities of occurrence of each residue at each position. When the number of residues is large, and the probability of occurrence becomes small, it is more convenient to use log odds values to build the matrices, than just the probabilities. Similarly in some situation it becomes more convenient to use the frequencies of occurrence of the residues.

5.1.3 Pattern characterisation and classification

The quality of a pattern is assessed in slightly different ways depending on its biological use. Patterns or motifs are used in two major ways. The first is to use the sequence pattern to identify structural, and consequently functional features that are common to a set of proteins. This entails strict conservation of residues with specific chemical properties, or even the residue itself, at least at a few locations in the pattern. A good example of this is the 'catalytic triad' of residues in proteases, lipases and catalases. The second major use of protein sequence motifs is to classify a new sequence into a known family of proteins. In this case the definition of the pattern may allow a slight departure from strict conservation.

To address the first issue, all we would normally require is that the putative pattern match the requirement of specific residues at specific locations. However, variations on this are possible. Also it is quite possible, especially for new patterns of unknown structure and function, that the conservation is a result of chance and has no biological significance. A common technique used to resolve this issue is to estimate the probability of this occurrence. It is possible to calculate or compute the probability of a random occurrence of a pattern in a set of sequences as large as the one used to discover the pattern. If this probability is sufficiently low, then we may safely assume that the detected pattern is indeed biologically significant. As to what a 'sufficiently low' probability means, we need to consider the size of the database used to discover the pattern. For example, if the analysis of one hundred sequences of average length 100 residues has lead to the identification of 10 residues long containing, say, two exactly conserved residues, the probability of this happening purely by chance may be calculated to be approximately 10^{-5}. (An exact calculation may be made using Bayesian statistics, but we will not consider that here). Such a probability is good enough to assure us that the discovered pattern is a true one. Real data however are likely to be more noisy than this, and probabilities at the level of 10^{-3} are good enough to initiate further investigations into the probable biological relevance of the discovered pattern.

In the case of deciding how well a particular pattern is diagnostic of membership in a specific sequence family the approach is different. We already have a pattern extracted from a group of sequences. At this stage we may or may not already know what common structural and/or functional similarity the sequences in this group share. We wish to decide whether a new sequence belongs to this family. We do this by looking for the family motif in the new sequence. In such a case we require the pattern to be defined with just sufficient precision to include all the sequences that actually belong to the family, without rejecting any genuine members. In other words the sensitivity must be high. At the same time the pattern must be precise enough to reject sequences that do not actually belong to the family, i.e. the specificity also must be high. To put it in other words, the pattern definition must be good enough to include all true positives and reject all true negatives. At the end of the classification there should be no false positives (sequences that do not actually belong to the family, but matching the pattern), neither should there be any false negatives (sequences that belong to the family, but not matching the pattern). The sensitivity and specificity may be calculated by a commonly used 'jack-knife' procedure. A set of sequences is generated by random rearrangements of the residues. This set bears statistical (i.e. in terms of residue composition, etc.) resemblance to the set of sequences used to identify the pattern, as well to a 'typical' new sequence, but clearly do not belong to the family identified by the pattern. This artificial set allows us to determine the number of true negatives (T_N) as well as the number of false positives (F_P). The original set of sequences helps us to calculate the other

two numbers required – the number of true positives (T_P) and the number of false negatives (F_N). The sensitivity and specificity are determined from these numbers as follows.

$$\text{Sensitivity} = T_P/(T_P + F_N)$$

$$\text{Specificity} = T_N/(T_N + F_P)$$

A good pattern definition will have both these numbers as close to 1.0 as possible, though this is a somewhat contradictory requirement – an increase in sensitivity usually leads to a decrease in specificity, and *vice versa*. Another useful number is the so-called 'positive predictive value', or PPV, that tells what proportion of sequence that match the pattern are actually in the family.

$$\text{PPV} = T_P/(T_P + F_P)$$

Again this number should be close to 1.0 for a well-defined pattern.

5.2 Pattern Discovery and Sequence Classification in Proteins

5.2.1 General methods

Multiple sequence alignment is one the most commonly used techniques to discover new patterns in sequences. It is a powerful and conceptually simple method, but has two major disadvantages. Firstly, it requires the pre-knowledge that a certain set of sequences possibly have a common pattern. Secondly, it requires the analysis of a large number of sequences to correctly identify a pattern, especially a subtle one. A further disadvantage, related to the first one above, is that it requires extensive manual intervention, especially in the initial stages, and therefore cannot be used for completely automatic pattern discovery. Once the possible existence of a motif has been identified in a set of sequences, however, one of the multiple sequence alignment techniques described in Chapter 4 may be used to find and refine the exact pattern and then present it as a regular expression or profile.

To consider how pattern discovery may be completely automated, we analyse the process into three separate tasks, each with different considerations. The first task is to understand and decide on the type of patterns that the process will result in. For example, we may be interested, say, in only repeating patterns, in which identical, or similar residues repeat at regular fixed intervals along the sequence. Or in the more commonly understood meaning of the word 'motif', we may interested in sets of similar residues that occur at similar locations in a set of sequences. These two ways of defining patterns correspond to two different 'solution spaces' respectively. The second task in pattern discovery is to define a measure of 'fitness'. This measure will indicate how well a sequence fits into a family. We may also invert this requirement and use the fitness function to measure how well a newly discovered pattern fits the input set of sequences, i.e. how well a family is defined. The third task is to develop an algorithm to automatically perform the pattern discovery. The algorithm will analyse the input set of sequences, and output one or more patterns, along with a measure of fitness for each.

Algorithms may be classified into pattern-based methods, and sequence-based methods. Pattern-based methods start with all possible patterns and check which of them are supported by the sequence data. Clearly, this requires a very precise definition of the solution space, for else, not only will the number of possible patterns be astronomically large, but also it may be impossible to enumerate all of them. An example of such an approach is to start with patterns of the type $A_1 - n_1X - A_2 - n_2X - A_3$, where A_1, A_2, and A_3 are three specific amino acids, X represents any amino acid, and n_1 and n_2 are constant integers. A search through the sequence data for this pattern using all possible values for the amino acids and the integers, establishes if the data support a given set of values. A fitness function such as the one described in the paragraph above is used to characterize the pattern and decide its acceptability.

Sequence-based discovery techniques start with the sequence data, searching for regularities that lead to the identification of possible patterns, that are then refined. Normally pairs of sequences are

compared to establish possible similarities. Any such seed patterns discovered are then built up by further pairwise comparisons. Many of these methods are very similar to sequence alignment techniques, in particular multiple sequence alignment techniques, such as CLUSTAL.

5.2.2 General methods for proteins

Among the most sophisticated methods used in pattern discovery are the probabilistic techniques, such as artificial neural networks (ANN) and hidden Markov models (HMM). In Chapter 7 we describe the use of ANN in protein secondary structure prediction. A general introduction to neural networks is given there. Here, we describe its application to sequence pattern discovery. The ultimate reason for looking for sequence motifs is to identify structural, and more importantly, functional motifs. Very often, sequences that appear dissimilar to many of the pattern discovery methods may yet correspond to a specific structural or functional unit in the folded protein. Identification of such motifs from the sequence is then a task requiring analyses of underlying cooperative effects among the amino acids. ANNs are well suited to carry out this task of identifying hidden patterns. As an example of their use for protein sequence analyses, consider the identification of signal peptides. A signal peptide is a small region of a protein sequence that occurs at the N-terminal end. They help secretory proteins to enter the secretory pathway and be targeted to its appropriate final destination. Signal peptides are cleaved off while the protein is being translocated through the membrane. Signal peptides from different proteins do not share a strict consensus sequence, and in fact the sequence similarity between them is low. The identification problem may be considered to have two parts – identification of a peptide sequence as belonging to a signal peptide region, and identifying the specific residue at which the cleavage of the signal peptide occurs. The ANN-based program 'SignalP' uses two types of neural networks to analyse the input sequence. It outputs two scores called the 'S' and 'C' scores for every residue in the sequence. If a signal peptide is present, a plot with these two scores on the y-axis against the sequence represented on the x-axis clearly indicates regions of the sequence with high values of the S score as signal peptides. A single sharp peak for the C score shows the cleavage site. There are two different types of networks to calculate the different scores respectively. The reason for this arose from the fact that the S scores required the sequence window in the input layer to be symmetric about the reference residue, while the calculation of the C score was more accurate if the window was unsymmetrical, considering only 2 to 4 residues to the right, and about 12 residue to the left of the reference residue.

Another example of ANN-based identification of motifs in protein sequences is the work on the identification of glycosylation sites. One of the important post-translational modifications that occur on protein sequences is the addition of a sugar moiety called N-acetylglucosamine via a 'beta anomeric' linkage to a serine or threonine residue in the sequence. The acceptor site on the sequence, i.e. the site of attachment of the sugar, does not display any definite consensus sequence. The only requirement appeared to be the presence of proline or valine residues in the vicinity of the glycosylation site along the sequence, the presence of a small stretch of serine residues downstream of the site, and the absence of leucine and glutamine residues around the site. The exact positions of these residues could not be defined, as they did not invariably occur at precise locations in all the known glycosylated sequences. The pattern is therefore a fuzzy one, and it was difficult to use it to annotate glycosylation sites in new sequences. A neural network was then constructed; using a set of sequences with experimentally determined *in vivo* glycosylation sites to train the network. The sequences were first analysed for the information content around the glycosylation site. Sequence windows centred on the site were aligned and the information content per site was calculated and visualized using sequence logos. Secondary structure predictions were made around the site, and this information was also taken into account. Six neural networks were designed using this data, and then trained on the known sequences. About 10% of the sequences were used to cross-validate the network. A jury or consensus of all the six networks made the final prediction. The prediction was modified by the results from surface prediction and

signal peptide prediction algorithms. A particularly interesting result of this work was the accurate prediction and subsequent experimental verification of glycosylated sites in a critical surface protein of the malaria-causing parasite *Plasmodium falciparum*.

5.2.3 Hidden Markov models

Like ANN, hidden Markov models are probabilistic mathematical constructs used in a wide variety of different scientific and engineering applications. Their most common use is in areas that require recognition of complex patterns, such as speech recognition or signature verification. Again like ANN, are built by analysis of an existing set of data that has already been classified as belonging or not belonging to a particular class. The results of this probabilistic analysis are embodied in the form of a hidden Markov model (HMM). The model then may then be used to classify new sequences. Strictly speaking, therefore, HMM are used not so much in discovering new patterns in sequences, as discovering the presence of already known patterns in new sequences. Nevertheless, it also possible to analyse a group of sequences with no known patterns, build a HMM, and then use the probabilities that characterize that HMM to discover new patterns. In this section, we give a very brief explanation of HMM before going on to describe some applications in protein sequence analysis.

A Markov chain is formally defined as a linear collection of symbols, chosen from a finite set, each symbol occurring at every position with a specified probability. If we know these probabilities for a given sequence of symbols, then we have constructed a Markov model corresponding to that sequence. We may think of this in another way. Consider a machine that emits a symbol at regular intervals. The emitted symbol is always one chosen from a finite set with a specific probability, and represents the state of the machine. The set of emitted symbols are arranged linearly in the order in which they are emitted. This string of symbols is then a Markov chain, and if we have the set of associated probabilities, then we have a Markov model for this chain. In the context of bioinformatics, DNA, protein and RNA sequences are considered Markov chains, with sets of symbols of size 4, 20 and 4 respectively. The emission probabilities are written as e_s, where s = 1 to N_S, the number of symbols. We have defined here a first order Markov model, in which the probabilities depend only on the symbol itself. In a second order Markov chain the probability of emission depends not only on the symbol at the current position, but also on the previously emitted symbol. We may therefore term the emission probabilities in this case as transition probabilities. Figure 5.2 shows a 'cellular automata' representation of this model of a biological sequence, in this case a DNA sequence. Each symbol is considered a 'state' of the system, and at any given point of time (or equivalently, at any given position in the sequence) the system exists in one of states, with a finite probability of either making a transition to another state or remaining in the same state at the next point of time. Again, knowledge of the transition probabilities implies that we have a Markov model of the system.

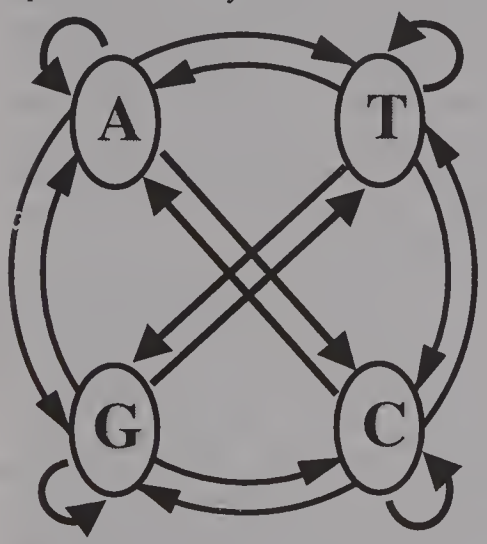


Figure 5.2 Cellular automata representation of the Markov model of a DNA sequence. The arrows represent the transition probabilities.

The meaning of the term 'transition probabilities' changes when we consider hidden Markov models. Let us consider a first order (non-hidden) Markov chain. This would have a set of emission probabilities, corresponding to the symbol states of the system, as described above. However it is possible that the entire chain exists in one of a set of hidden states at any given time, and that it has the

probability of making a transition to another one of these hidden states. To clarify this with an example consider a DNA sequence, say 1000 nucleotides long. We further consider that only the second half of this sequence codes for a gene. The first half, therefore, is a non-coding sequence. Thus, as we proceed along the sequence, it starts from the state 'non-coding', and halfway through it makes a transition to the state 'coding'. These two states of the system, viz. coding and non-coding, are hidden states not easily apparent from cursory inspection of the sequence. The set of symbols are the same in both hidden states – what differs are their respective emission probabilities. Thus we will have two sets of probabilities associated with the set of symbols, one set corresponding to the state 'non-coding' and the other corresponding to the state 'coding'. There is also a transition probability associated with the changeover from coding to non-coding and vice versa. Since the transition may occur at any position along the sequence, we need to know the set of emission probabilities for any symbol in one state, not only to go to any symbol in the same hidden state, but also to go to any symbol in the other hidden state. To build this hidden Markov model, therefore, we have to calculate all the transition probabilities, both from one state to another within a hidden state, as well as from a state in one hidden state to a state in the other.

In the simple example above, we have considered a two-state architecture for the HMM with the states fully interconnected. Transitions are allowed between any pair of symbols, both either in the same state, or in different states. However in real biological applications there will typically be more states, as well as a lesser number of possible transitions. It is common, however, in the analysis of protein and DNA sequences to simplify the situation as far as possible, and to consider only a two state architecture, such as coding/non-coding, CpG island/non-CpG island, signal peptide/no signal peptide etc. A so-called 'left-right' architecture is chosen, ensuring that the states always change from left to right in the sequence, and never in the other direction – i.e. no reversals of any transitions are allowed. We also add a 'start' state and an 'end' state to signify the first and final transitions (or first and last symbols) respectively. The selection of the HMM architecture is the first step in building the model. This process includes the incorporation of any prior information available, such as, for example, specific required residues (or transitions) at particular points in the sequence.

The next step in HMM construction is parameter estimation, or in other words, the assignment of values for the emission and transition probabilities. To do this we require sets of sequences, each set clearly identified by experimental or other theoretical techniques (such as sequence comparison) as belonging to one of the states specified in the model architecture. These 'training' sets of sequences are then analysed using one of several algorithms to estimate the parameters. If the sequences are clearly labelled as belonging to one of the states, then simple counting will yield the parameters. If the sequences are not labelled, but it is known that some portions belong to one state and the other portions to the other state (for a two-state model) one may use, for instance the Baum-Welch algorithm to obtain the probabilities. Obviously in this case, there would be a degree of uncertainty associated with the set of parameters, and it is necessary to iteratively optimise the parameters and obtain a set that best explains the observed data. The model is complete once the parameters have been estimated. To use this model to annotate and classify a new sequence, or portions of it, as belonging to one or the other of the underlying, hidden states, there are again several techniques. The one most commonly used is a dynamic programming method called the Viterbi algorithm.

HMMs are used in a wide variety of contexts in DNA and protein sequence analyses, including sequence comparisons, secondary structure prediction, and so on. Their most widespread and successful use is in pattern discovery. Some of these applications are described below.

5.2.4 HMM – Application to analyses of protein sequences

- As an example of the use of HMMs for pattern recognition and classification, we will consider the case of G-protein coupled receptors (GPCRs), which are important membrane bound proteins,

involved in all the most important signal transduction functions in the cell. GPCR sequences extracted from the PROSITE database were used to build a HMM. Based on this, a score is calculated for every new sequence presented. If the score is above a threshold limit, the new sequence is classified as a GPCR sequence. The ability of the model to discriminate between GPCR and non-GPCR sequences was tested by calculating scores for all the sequences in Swiss-Prot of a specific length, as well as random sequences generated using the compositional characteristics of the known GPCR sequences. If the scores for all these sequences are calculated on the basis of the model, and plotted as a function of sequence length, all the non-GPCR sequences lie on a straight line, while the GPCR sequences fall on a different distinctly different line, parallel to the first one, but with a lesser intercept on the y axis. Comparison of the predictions by the HMM with other computational techniques such a hydropathy plots, also indicated the superior performance of the model. It was also possible to fine-tune the model to identify different classes of GPCRs, such as adenosine receptors, acetylcholine receptors, etc.

- HMMs have also been built for signal peptides. As already described in section 5.2.2 above, there are two aspects to recognition of signal peptides. First, we identify a portion of the protein sequence as the signal peptide, and next we identify the precise cleavage site within it. Complications arise due to wide variation in the lengths of the peptide. A further complication is the occurrence in some proteins of a signal peptide-like sequence at the N-terminal end (rather than the C-terminal end). This also helps in translocating the protein across the membrane, but since it is not cleaved, it is called a signal anchor, and has to be distinguished from the signal peptide. A HMM has been designed to perform all these tasks – viz. differentiate not only between signal peptide and non-signal peptide, but also identify sequences that are in fact signal anchors, as well as identify the cleavage site. This model includes an explicit representation of different possible lengths for the signal peptides, as well one for signal anchors. The model was trained using all types protein sequences – with and without signal peptides and anchors, with clearly identified cleavage sites, where appropriate. For signal peptides and cleavage sites, the performance of this model is better than the ANN in some cases, and worse in others. For signal anchors, however, the HMM is clearly superior to the ANN.
- Profile HMMs are the most extensive use of HMMs in bioinformatics. A profile HMMs abstracts the statistical information present in a multiple sequence alignment of a family of sequences. It thus describes in a compact way the essential features required in a sequence to belong to that family, and may be used to decide if a new sequence belongs to that family. Construction of a profile HMM starts with a multiple sequence alignment of known members of a family. The information present in the alignment is first summarised as a PSSM (see section 4.1.2). The PSSM is used to design the HMM architecture with the addition of possible gaps as 'insert states' Parameterisation of the HMM is then carried out using specially designed versions of the algorithm mentioned above. The profile HMM is used to identify new members of the family, again by modified versions of algorithms mentioned above. Pfam is a web-based database that comprises protein sequence families and their motifs identified by use of profile HMMs. Some of the well-known sequence families present in this database include those that give rise to specific structural motifs, such as 'leucine zippers' and trans-membrane helices.

5.2.5 Other analyses of protein sequences

Apart from pattern recognition and sequence classification, various other analyses may be performed on protein sequences. One of these, viz. protein secondary and tertiary structure prediction is dealt with in the next chapter. In this subsection we briefly describe some other types of useful analyses, which are chiefly based on the chemical and physical properties of the 20 amino acids in protein sequences.

- It has been noticed that amino acid composition varies according to protein family, and it is possible to classify proteins, based not on the sequence, but on the amino acid composition. Programs (e.g. AACompIdent and AACompSim) are available to carry out database searches on this basis. The user is required to input the amino acid composition of the new protein. This information may be obtained by biochemical analysis that stops short of sequence determination, and therefore more easily obtained. Together with the composition, the user also inputs the pI (the isoelectric point – the pH at which there is no net charge on the protein) and molecular weight, where this information is available. The program then conducts a search of the Swiss-Prot database, which contains not only all protein sequences, but also valuable annotation regarding biochemical properties. The output consists of a list of proteins with composition and other values similar to that input. Such searches may be conducted also when the sequence is known. The composition is then calculated from the sequence and used in the search. This search is particularly useful when the input sequence bears less than about 25% similarity with any other sequence in the database. The extra information obtained from the composition analysis may then be used to better identify the class to which the protein belongs.
- A program called PROPSEARCH performs a similar task. The search in this case, however, is considered far more robust, since, instead of the amino acid composition of the protein, a total of 144 different parameters calculated from the protein sequence, are used to carry out the search. For example one of the parameters is the percentage of bulky amino acids in the sequence. Others are molecular weight, average hydrophobicity and average charge. A 'vector' of 144 dimensions is constructed for each protein in the database and compared with the vector for the input sequence by calculating a Euclidean[14] distance between the vectors. A distance score of less than 10 indicates a good chance that the two proteins belong to the same family and have the same function.
- A program called MOWSE uses a set of molecular weights determined for a protein to search for a match in the database. The molecular weights, not just of the complete protein, but also of various fragments and digests may be determined by biophysical techniques such as mass spectrometry. The comparison is made with sets of molecular weights computed for the protein in the database. For each protein, the molecular weight of the entire protein, as well as of the possible digests is calculated and stored. For each query set of molecular weights, comparisons that yield a high number of matches with the set of values calculated for a single protein sequence in the database are scored as hits. The list of hits may then be scanned to identify the protein family to which the query protein most likely belongs. Note that here too, as in some of the algorithms noted earlier, the sequence of the query protein is not required to detect familial relationships.
- Some programs are available to calculate the physical and chemical properties of a protein given its sequence. For example the program 'Compute pI/MW' calculates the isoelectric point and the molecular weight of a sequence. The pI is calculated from experimentally determined pK values for the amino acids. It also possible to use a similar technique to calculate the extinction coefficient of the protein. Individual amino acids are treated as if they were free in solution, and the extinction coefficient of the protein is obtained by summing the coefficients of the individual residues. Other programs output cleavage sites in a sequence if it were to be treated with various enzymes or chemical reagents such as trypsin, chymotrypsin, cyanogen bromide, etc. These calculations are based on known signals in the sequence, as well on information on specific proteins available in the annotated database. The output is in the form of a list of peptides, and their molecular weights obtained by the treatment.

---

[14] If the vector **A** has the components $(x_i, i=1, N)$, and vector **B** has the components $(y_i, i=1, N)$, the Euclidean distance between **A** and **B** is given by $\sqrt{[\sum_i (x_i - y_i)^2]}$

5.3 Pattern Discovery and Sequence Classification in Nucleic Acids

The wealth of DNA sequences (10^{11} bases in the public databases, at the last count!) cries out for automatic annotation algorithms, in order that the data is converted to biological meaning. One of the most important tasks to be undertaken in this regard is to identify genes, i.e. protein coding sequences, in the genomic sequences. Shotgun sequencing methods have made it possible to quickly sequence entire genomes of even the most complex organisms. It is estimated that in the higher organisms, such as mammals, about 10% of the genome corresponds to regions that code for proteins. The problem of identifying which portions of the genome correspond to protein coding sequences is a difficult one, and like many other tasks in bioinformatics, has not been completely solved. This is illustrated by the estimates of the number of proteins coded in the human genome that vary from about 30,000 to about 50,000, reflecting the fact that different algorithms give different estimates. Below we review some of these methods. Analyses of DNA sequences are not restricted to gene discovery. The search for sequence motifs, for example, promoters, enhancers, repressor binding sites, and other such regulatory regions, is an important task that, as for protein sequences, requires a battery of tools. Some of these are also discussed below. A further type of analyses that may be performed on DNA sequences is with respect to their physical and chemical properties. Some of these aspects are dealt with in Chapter 8. Others are described here.

5.3.1 Gene discovery

Gene discovery requires attention to a variety of complementary signals in the DNA sequences, including promoters, transcription and translation start and stop signals, reading frames, polyadenylation signals, introns and exons, nucleosome positioning signals, etc. Some of these are weak and some are strong. Some are particular to eukaryotes, others to prokaryotes. And some are specific to the organism – a clear example being codon choice, which may give rise to different compositional gradients between coding and non-coding regions.

A simple method of checking if a DNA sequence codes for a protein is to determine if it contains an open reading frame. An open reading frame, or ORF, is a stretch of DNA that can be translated into a protein sequence of a reasonable length, without coming across any STOP codons. 'Reasonable' length would imply that the DNA sequence codes for a protein of at least about 50 amino acids, though this is in fact the size of the smallest proteins, and on an average, the size is expected to be greater than 100 residues. (Note that these numbers are true mainly for prokaryotes. Eukaryotic genes are interrupted by introns, and the exon size, or the ORF, could be smaller than expected from the figures above.) In random sequence DNA, we would expect that a stop codon occurs 3 times every 64 codons, or approximately once every 21 codons. This is often the first analysis performed on a DNA sequence. The translation, which is performed by looking up a genetic code, is carried out in all six reading frames possible. The six frames arise due to the fact that three consecutive nucleotide bases correspond to an amino acid, and that the codon for the next amino acid immediately follows the previous one. Thus we may start the translation from the first base of the sequence, and obtain one sequence of amino acids. If we start from the second base, or the third, we obtain two different sequences. This accounts for three reading frames. The other three arise from the fact that, since DNA is a double (complementary) stranded molecule, we do not know whether the DNA sequence is the one that contains the coding region, or its complement. We therefore now take the sequence complementary to the one already translated. This sequence now has a further three reading frames, totalling six in all. Figure 5.3 shows this graphically. Finding an ORF in the DNA sequence is only the first step. We still need to confirm that in fact the sequence is actually expressed as a protein *in vivo*. Again, without going to the wet-lab, we may perform a database search with the protein sequence to see if a similar protein has already been identified in a different organism. This method thus relies on 'extrinsic'

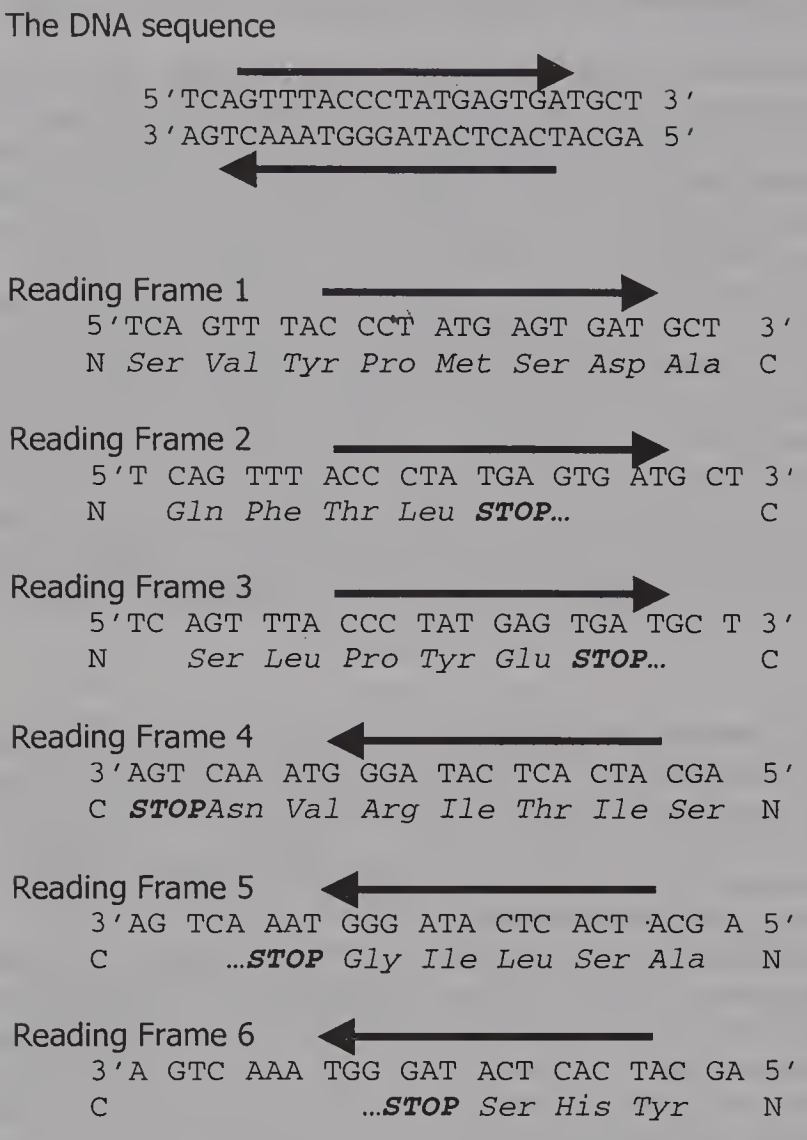


Figure 5.3 The six possible reading frames in a DNA sequence. Note that only one of the frames yields the 'complete protein' sequence.

signals, viz. sequence similarity. With validated protein sequence databases growing larger all the time, this is an attractive choice of procedure. If this fails, and no sequence with greater than about 20% identity turns up in the database search, we have to identify the presence or absence of other signals such as promoters, ribosome binding sites, polyadenylation sites, termination sequences, etc. This method therefore relies on 'intrinsic' signals. In the absence of a sufficient length of sequence, or of sufficient strength in the signals, we have to rely on looking for probabilistic base sequence patterns that differentiate between coding and non-coding regions. Methods that do this are known as *ab initio* methods.

A major set of algorithms use probabilistic techniques to integrate various strands of information into a single gene finding tool. These methods use ANNs or HMMs to rapidly scan large DNA sequences and identify possible coding regions. Table 5.1 gives a list of only those programs that incorporate one of these two techniques, but even this number is large. Some of these programs identify only the start site, or else the intron/exon splice junctions, or other such signals. However, used in combination with an ORF finding program, they can as well be used for gene discovery. As an example, we will discuss briefly one program under each of the two techniques mentioned. It must be noted that both HMMs and ANNs are crucially dependent on the training set of data. Thus if only sequences from a particular set of organisms (or, as in the case of the human genome sequence, from one single organism) are used to evaluate the various parameters required to construct the HMM or ANN, then the program may said to be safely validated only for that organism or set of organisms. Application of the algorithm to sequences from other organisms has to be carried out with caution, or requires further training of the models and networks. Thus many of the above programs, as well as others not mentioned here, are specific, and may not be applied generally. In particular, detection or prediction of genes in eukaryotes relies on signals and statistics that are completely different from those in bacteria or archebacteria.

Table 5.1

HMM based gene discovery programs

1. Ecoparse [*ecoparse@cse.ucsc.edu*]
2. GeneHacker [*www-btls.jst.go.jp/GeneHacker/*]
3. GeneMark [*exon.biology.gatech.edu/~james/genemark.html*]
4. Genie (uses both HMM and ANN) [*www.fruitfly.org/seq_tools/genie.html*]
5. Glimmer [*www.tigr.org/software/glimmer/*]
6. VEIL [*www.cs.jhu.edu/labs/compbio/veil.html*]
7. GENSCAN [*gnomic.stanford.edu/~chris/GENSCANW.html*]

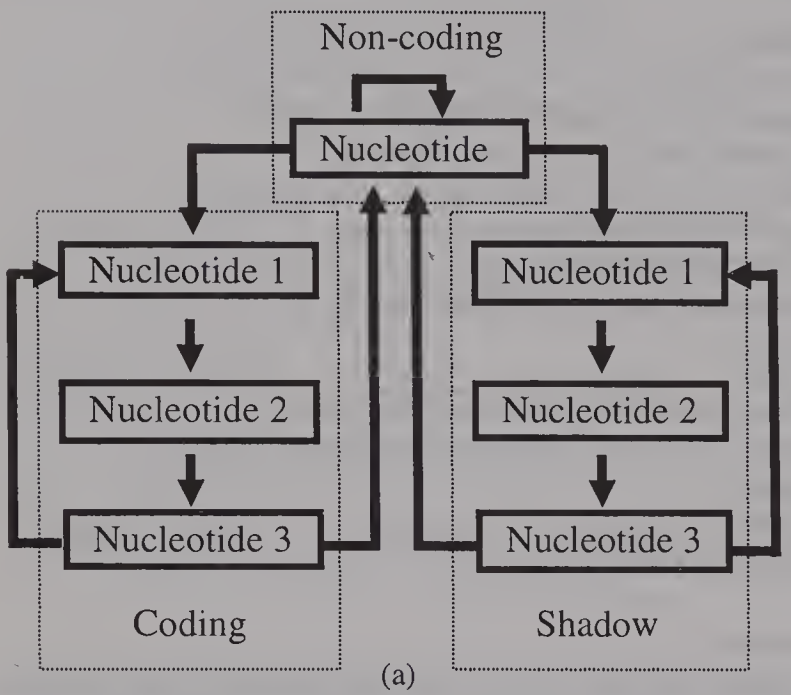
ANN based gene discovery programs

1. Geneparser [*beagle.colorado.edu/~eesnyder/GeneParser.html*]
2. Genie (uses both HMM and ANN) [*www.fruitfly.org/seq_tools/genie.html*]
3. Grail [*avalon.epm.ornl.gov*]
4. Netgene [*www.netgene.co.jp*]
5. Netstart [*www.cbs.dtu.dk/services/NetStart/*]

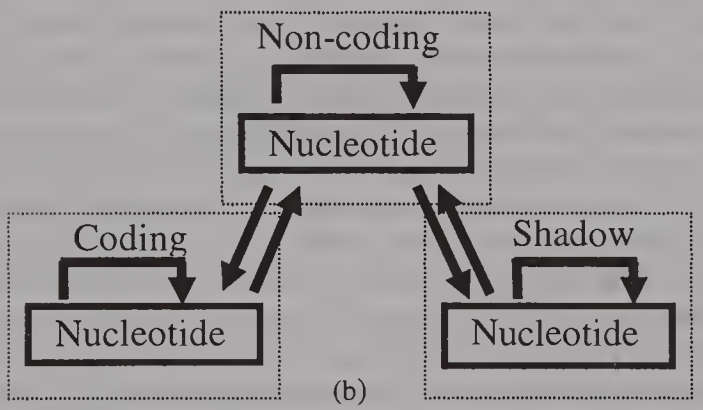
5.3.2 Gene discovery using HMM - GeneMark

The program GeneMark uses a simple Markov model to calculate the probability of any new sequence being a coding region based on the parameters generated from known coding regions. Later extensions of the algorithm used an interpolated Markov model (as in the program Glimmer), or hidden Markov models (as in the program GeneMark.hmm). The first model mentioned above is also known as an inhomogeneous Markov model and classifies the DNA sequence as belonging to one of three states – coding regions, coding regions on the complementary strand or 'shadow' regions, and non-coding regions. Thus, by incorporating a gene shadow model in the Bayesian statistical formalism, the algorithm identifies genes on both strands using the DNA sequence of one strand only. GeneMark also incorporates reading frame dependent oligonucleotide frequencies, and is therefore called an inhomogeneous model. In other words, the frequencies of occurrence of mononucleotides, dinucleotides, trinucleotides, as well as higher oligonucleotides at each position on the DNA sequence are not expected to be homogenous, but vary according to whether the position is at the beginning, middle or end of the codon, i.e., it depends on the phase, or reading frame. This feature is used in calculating the probabilities of occurrence of oligonucleotides of various lengths (usually 1, 2 and 3, i.e. mono-, di- and trinucleotides) in known coding, shadow and non-coding sequence. These numbers are then used to estimate the *a posteriori* probability for each portion of a new sequence to belong to a coding, shadow or non-coding region. The new sequence is scanned by calculating these three numbers in a sliding window. The window can slide over a sequence of any length, including entire genomes. The size of the window is set to a default value of 96 nucleotides, but could be longer, or shorter. Inspection of the 'profile' of probabilities calculated for the sequence then differentiates the coding and shadow regions, where the *a posteriori* probabilities for coding or shadow would be high, from the non-coding regions. The hidden Markov model extension to this algorithm assumes a hidden 'state' sequence, i.e. a sequence of hidden states in which each nucleotide exists. Examples and explanations have been given in section 5.2.3 above. Figure 5.4 shows the HMM architecture used in the program GeneMark.hmm. Figure 5.4a shows the three-phase, three-state model. The three states are, of course, coding, shadow and non-coding. Within the coding and shadow states, note that the transitions are designated as being possible from the first position in the reading frame (or first position in the codon) to the second to the third. Transitions are also possible between coding and non-coding, and between

shadow and non-coding, but not between coding and shadow. The model could be generalised to the one shown in Figure 5.4b, which is called a 'HMM with duration'. Now there are three states, coding, shadow and non-coding, and the sequence may stay in a given state for 'duration' measured in nucleotides. Known sequences were used to derive all the required parameters (the transition and emission probabilities) by a process of unsupervised learning. When assessed in terms of the specificity (percentage of predicted genes that correspond to real genes) and sensitivity (percentage of real genes that were identified by the algorithm), the quality of performance of the algorithm was very good, scoring well above 90 percentage points in both measures, over all the data used in the assessment.



(a)



(b)

5.3.3 Gene discovery using ANN - GRAIL

The program GRAIL uses a neural network to identify protein coding regions in DNA. The basic concept was derived by analogy with robotic systems that perceive the surroundings through the integration of information from many different sensors, such as cameras for 'visual' information, sonar transducers for 'aural' information, laser range finders to detect distances and shapes, and tactile sensors for 'touch and feel'. A neural network is used to integrate the output of the sensors to form a combined best estimate of the environment being perceived. Exposing the sensors to well-defined environments and adjusting the weights of the network nodes to obtain the required results accomplishes the training of the network. This approach has been applied to the recognition of gene sequences. The 'sensors' in this case are seven different statistical measures of the sequence, calculated at every position along it. The sensors, along with the neural network are referred to as the 'coding recognition module'. The advantages of this approach are the following. First, it is a convenient way of representing and combining all (or most of) the statistical information present in the sequence. Second, machine-learning algorithms carry out the integration and the assignment of weights in the network in an unbiased and mathematically rigorous manner. Third, the system is robust, i.e. it is tolerant of input errors or noise in the sequence, because it does not overly depend on any one sensor. And finally, the modular nature of the design allows the addition (or subtraction) of any further sensors. The seven sensors are the following.

1. A *frame bias matrix* that is based on the non-random frequency bias in the occurrence of each of the four nucleotides in each of three reading frames in a DNA coding sequence. The differences between these set of frequencies, and the virtually random frequencies in non-coding regions helps to identify the former.
2. The implementation of an algorithm that considers several different properties of coding sequences such as the overall base composition, etc.
3. The frequency of occurrence of the sixteen possible dinucleotides is different in coding sequences from that in non-coding sequences, and is used as a recognition signal.
4. We may count the frequencies of occurrence of DNA 'words' of different lengths, and use these to distinguish the character of the sequences. This sensor uses words of length 6 nucleotides.
5. Again, just like mononucleotides, sextuple word frequencies vary according to whether we are looking at a reading frame or not, and this is used as a signal.
6. Some DNA words occur very commonly in coding sequences, while others are less common. The 'commonality' index of a sequence may be calculated in terms of the number of common words it contains, and may be used as a signal.
7. Finally, the number and frequencies of repetition of words, in particular sextuple words, may be calculated, compared with known non-coding repeating sequences, and used as a signal.

The neural network was set up with the seven sensors comprising the input layer. There are two hidden layers consisting of 14 nodes and five nodes respectively, the first hidden layer receiving inputs from the input layer, and the second hidden layer getting them from the first hidden layer. The output layer is of course just one node that displays the final decision, i.e. coding or non-coding. The neural network is of the 'back-propagation' type. It was trained by providing it with the correct output value for each position of known sequences. The network learnt by comparing the output value at each position in the sequence with the expected correct output and adjusting its internal parameters to satisfy the expectation as closely as possible. For each position in the sequence, the seven sensors calculated the appropriate numbers by considering a window of 99 nucleotides, centred at the current position, i.e. the window consisted of 48 residues on each side of the position. The network was trained on a set of sequences known to be coding, or non-coding respectively, obtained from GenBank. The program performed well on a set of test sequences, locating more than 90% of the coding sequences of length greater than 100 bases, with less than 80% false positives. GRAIL is an implementation of the above algorithm and stands for Gene Recognition and Analysis Internet Link. It incorporates a web-based gene prediction service, and is particularly well tuned to recognise genes in the human genome. A more recent implementation is GrailEXP, or the Grail Experimental Gene Discovery Suite. This is a software package that predicts exons, genes, promoters, poly-adenylation sites, CpG islands and repetitive elements within experimentally determined DNA sequences. GrailEXP has been used particularly for annotating the human genome. Another version of the algorithm more suited for microbial genomes is call Generation.

5.3.4 Gene discovery using Fourier analysis - GeneScan

GeneScan should not be confused with Genscan, which is also a successful gene prediction program. The latter uses a maximal dependence decomposition (MDD) method for representing intron/exon splice sites that takes into account nonadjacent correlations in the splice site positions. As such it outputs a list of possible exons, that are then confirmed to be genes or not, based on other signals and experimental data such as ESTs (or expressed sequence tags[15]). It will not be dealt with further here.

[15] Expressed Sequence Tags are DNA sequences obtained by sequencing mRNA obtained from the organism, and therefore represent portions of the DNA that have been expressed, presumably to be translated into proteins.

The former, GeneScan[16], is based on Fourier analysis of DNA sequences and is discussed in greater detail below.

Consider a DNA sequence length n,

$\alpha_1\ \alpha_2\ \alpha_3 \ldots\ldots\ \alpha_n$

where α_i is the base (A, T, G or C) at position i. We define an 'indicator function' or 'projection operator' U_α which when applied to a DNA sequences yields a binary string, with the value 1 if the nucleotide symbol at that position is α, and 0 otherwise. For example, consider the following DNA sequence.

ATGCTTCCGGTTAAGCTGCCCCAAGT

If we apply the projection operator U_A we obtain the following binary string.

U_A 10000000000011000000001100

Similarly, the projection operators U_T, U_G and U_C yield the following three strings, respectively.

U_T 01001100001100001000000001

U_C 00100000110000100100000010

U_G 00010011000000010011110000

Thus any DNA sequence may be reduced to four binary strings. We search for periodicities in each of these binary strings by performing a Fourier transform of each one. A Fourier transform for each string is defined as

$$S_\alpha(f) = |\Sigma j\{U\alpha(j)\exp(2\pi ifj)\}|^2$$

where the summation is over j = 1, n, i.e. over all the positions in the string (i is the imaginary number, sqrt{-1}). The frequency f is given as k/n, where k = 1, 2, 3...n/2. The spectrum S is computed for different frequencies such as 1/n, 2/n, etc., and may be plotted as S(f) v/s f. A peak at frequency f = m/n in the spectrum indicates that the string has m-position periodicity. It should be pointed out that while a m-position peak does not necessarily imply the presence of a repeat unit of length m, the converse is true: the existence of repeats of length m will give rise to a peak at frequency f = 1/m. The Fourier transform of the DNA sequence is obtained as the sum of the individual spectra, i.e.

$$S(f) = \Sigma_\alpha 1/n^2(S_\alpha(f))$$

One may also calculate the average total spectrum symbolised by <S>, as the average of the spectra for the sequence at each frequency, and use this to estimate the signal-to-noise ratio at every peak noticed in the spectrum. Calculation of the Fourier transform for various sequences known to be coding or non-coding revealed a distinct peak at a f value of 0.3333... (i.e. f = 1/3) in the coding sequences, while the non-coding sequences had no such peak. The peak at f = 1/3 was sharp and had, in general, a signal to noise ratio of greater than 4. The discrimination between coding and non-coding sequences was possible even when the input sequence was a fairly short sequence, of the order of a few hundred bases.

The GeneScan gene discovery program uses the above information to distinguish between coding and non-coding regions. The procedure is straightforward. The input DNA sequence is first converted to four binary strings. A window of about 400 bases is defined. (In tests a window size of 380 bases was shown to have the fewest false positives). The Fourier spectrum of the sequence within the window is calculated as described above. A window that shows a discrimination factor greater than 4 is considered to belong to a coding region of the DNA.

The performance of the algorithm was very good, but not significantly better than other methods used in gene discovery. However there are several advantages in using this method. The chief of these is that it does away with the requirement for a 'training set' of sequences. Methods that use training sets are highly biased by these sets, and if the set is skewed towards one particular pattern, the

---

[16] We must also note that a commercial company uses the name Genescan for their gene sequencing software. This software is not in any way related to the one discussed here.

algorithm does not recognize genes that occur without this pattern. On the contrary, GeneScan uses a perfectly general Fourier transformation technique, which does not require the algorithm to be trained beforehand. Also it is perfectly reasonable to assume that the 3-base periodicity used in discriminating between coding and non-coding sequences is a general feature of genes due to the triplet nature of the codon. A second advantage of the present method is that it may be applied to very short DNA sequences or to large genome sequences. The method is robust, and is tolerant even to sequencing errors that may result in a shift of the reading frame. The method is also insensitive to base composition, and may be applied to genomes with unusual G+C content.

5.3.5 Other analyses of DNA sequences

Gene discovery is the chief task carried out in analysing DNA sequences. However there are several other signals and patterns in DNA sequences. Some of these, such as promoters, exon/intron junctions and splice sites, etc. are related to gene identification. Others such as base composition analyses, etc., yield other types of information. In this section we will briefly describe a few of these analyses.

- Base composition analyses has revealed a non-random distribution of the four bases A, T, G and C. The 'null hypothesis' is that all four nucleotides occur with approximately the same frequency in a DNA sequence. This means that if we consider the duplex sequence, consisting of two complementary strands, the proportion of G+C should be approximately the same as that of A+T. It has however been found that G+C content (and therefore by implication the A+T content) varies from as low as about 30% to as high as about 70%. The variation is seen not only between one genome and another, but also between one part of a genome and another part of it. Even in the about 130 bacterial genomes that have been completely sequenced, the G+C content shows such wide variations. The study of base composition is interesting from many points of view in biology. One study showed that the phylogenetic tree for bacteria constructed using the G+C content as the OTU follows the one constructed from functional or morphological characteristics almost exactly, indicating a possible role for the base composition as a phylogenetic marker in DNA sequence analyses.
- Dinucleotide composition analyses revealed that genomic DNA has a paucity of the dinucleotides CpG compared to the number expected on the basis of the mononucleotide composition[17]. The biochemical reason for this is that cytosine in CpG is often methylated at the 5 position of the base. An enzyme known as deaminase frequently replaces the amine atoms at the 4 position of cytosine by a keto oxygen atom, thus converting the methylated cytosine base to thymine. During the next cycle of replication, the CG base pair is now converted to an AT base pair. In other words a point mutation is introduced into the sequence, leading to potentially disastrous consequences for the organism. One way of avoiding this is to suppress the use of CpG dinucleotides. The analyses of new DNA sequences with respect to the CpG suppression, or possible elevation, is therefore capable of yielding important information regarding functional as well as phylogenetic aspects of the DNA sequence. Dinucleotide analyses may be performed not just for CpG, but for all the 16 possible dinucleotides. Their frequencies of occurrence show non-random behaviour that is indicative of underlying functional or other causes. The 'null hypothesis', or the normal expected behaviour, as in the case of mononucleotides, is that the distribution is random, but of course based on mononucleotide composition. This could be expressed by calculating a probability of occurrence of each dinucleotide, given the mononucleotide composition. If P_A, P_T, P_G and P_C are the relative frequencies of occurrence of A, T, G and C respectively, the probability of occurrence of the dinucleotide ApT, say, is

[17] CpG represents the base C followed by the base G in the DNA sequence. The 'p' is the intervening phosphate group, and is given to emphasize that it is a CG dinucleotide and not a CG base pair

$$P_{ApT} = P_A P_T$$

We assume here that the DNA chain is a first order Markov model, in which the probability of occurrence of a nucleotide at a given position depends only on the nucleotide itself. We may then compare the expected frequencies with the ones that actually occur in the DNA sequences. Such comparisons have revealed that the distribution of dinucleotides is not random, and DNA sequences appear to have distinct biases in the dinucleotide composition, perhaps related to function.

- Studies of codon bias in genes distinguish between bias in the codon usage, and bias in the codon preference. Codon usage refers to the observed frequencies of the different codons in all the genes in an organism. Codon usage is thus an average property of the organism as a whole. Codon preference on the other hand differs from gene to gene within an organism, and correlates with the gene expression levels. The genetic code is degenerate, and for many amino acids more than one codon code for the same amino acid. For example, Arginine has six codons. Transfer RNAs that accept the same amino acid but having different codons are called 'isoaccepting tRNAs'. Genes that are highly expressed use codons that correspond to the most abundant of the isoaccepting tRNAs. This bias is called codon preference. Codon usage is again highly related to phylogeny, and similar codon usage often implies a strong phylogenetic relationship between the organisms concerned. Codon usage tables are available for various groups or organisms, and used as an indicator of coding regions. ORFs that in addition obey the expected codon usage statistics are more likely to be coding regions. Also, a plot of codon usage within a sliding window, with respect to the sequence position, gives the possibility of locating any frame-shift mutations, since in such case there would a sudden change in the codon usage from normal to abnormal values. Calculation of the usage in the other two reading frames would then help to identify the new reading frame.
- Gene sequences have been analysed both with respect to the DNA sequence, as well as with respect to the corresponding protein sequence, to understand the precise roles of mutation at the level of the DNA and evolutionary selection at the level of the protein. Such studies have, for example, revealed that codon choice is correlated with the G+C composition bias. Thus organisms that have genomes with high G+C content choose codons that also have a high G+C content. The base composition bias is also reflected in the amino acid composition, and in such genomes in general, amino acids whose codons have more G and C are preferred.
- Pattern recognition techniques such as HMMs and ANNs, have been developed not just to recognize entire genes, but also for each individual pattern within a DNA sequence. Some these are promoters, operators, intron/exon junctions, termination signals, polyadenylation signals, ribosome binding sites in the mRNA, rRNA and tRNA genes, attenuation signals, enhancers, transcription factors, restriction enzyme recognition sites, restriction digest maps, nucleosome binding sites, DNA bending regions, unusual secondary structure regions in DNA and repeats. Methods have also been developed to compare entire genomes, and study the origin, evolution and 'death' of genes. For example, while most genes probably arise by descent from the ancestral organism, some important ones are acquired by the horizontal transfer of genes from one organism to an unrelated one. It is important to have techniques to recognise which genes have followed the former route, and which the latter.

5.4 Summary

- This chapter deals with the techniques used in analysing protein and DNA sequences, especially pattern discovery and sequence classification
- Patterns in sequences may be deterministic patterns, such as regular expressions, as well as probabilistic patterns, such as profiles.
- The PROSITE 'pattern' is a good example of a regular expression.
- The following represents the sequence pattern of a 'serine active site'.
  P-x-[STA]-x-[LIV]-[IVT]-x-[GS]-G-Y-S-[QL]-G.
- Sequence logos are a way of writing consensus sequences using probabilistic information.
- Sequence logos show all the following information in a single figure – the consensus sequence, the order of predominance of each residue at each position, the probability of each residue at that position, the amount of information present at each position, measured in bits, and, if appropriate, the start and stop points.
- The quality of a pattern is assessed in slightly different ways depending on its biological use.
- It is possible to calculate the probability of a random occurrence of a pattern in a set of sequences as large as the one used to discover the pattern. If this probability is sufficiently low, then we may safely assume that the detected pattern is biologically significant.
- The pattern definition must be good enough to include all true positives and reject all true negatives. There should be no false positives, nor should there be any false negatives.
- Multiple sequence alignment is one the most commonly used techniques to discover new patterns in sequences.
- Other motif-discovery algorithms may be classified into pattern-based methods, and sequence-based methods.
- Pattern-based methods start with all possible patterns and check which of them are supported by the sequence data.
- Sequence-based discovery techniques start with the sequence data, searching for regularities that lead to the identification of possible patterns.
- Among the most sophisticated methods used in pattern discovery are the probabilistic techniques, such as artificial neural networks (ANN) and hidden Markov models (HMM).
- ANNs are well suited to carry out this task of identifying hidden patterns. Examples of their use for protein sequence analyses include the identification of signal peptides, and the identification of glycosylation sites.
- Hidden Markov models are probabilistic mathematical constructs used in a wide variety of different scientific and engineering applications.
- A Markov chain is defined as a linear collection of symbols, chosen from a finite set, each symbol occurring at every position with a specified probability.
- The Markov chain may exist in one of a set of hidden states at any given time, with the probability of making a transition to another one of these hidden states. It then requires a hidden Markov model.
- To build a hidden Markov model we have to calculate all the transition probabilities between the hidden states from a set of 'training' sequences that have clearly identified states.
- An example of the use of HMMs for pattern recognition and classification is the case of G-protein coupled receptors (GPCRs). GPCR sequences extracted from the PROSITE database were used to build a HMM. Based on this, a score is calculated for every new sequence presented. If the score is above a threshold limit, the new sequence is classified as a GPCR sequence.

- A profile HMM abstracts the statistical information present in a multiple sequence alignment of a family of sequences. It thus describes in a compact way the essential features required in a sequence to belong to that family, and may be used to decide if a new sequence belongs to that family.
- Apart from pattern recognition and sequence classification, various other analyses may be performed on protein sequences, such as amino acid composition analyses, calculation of physical properties, hydropathicity plots, pI calculations, etc.
- One of the most important tasks to be undertaken in the analyses of DNA sequences is to identify genes, i.e. protein coding sequences, in the genomic sequences.
- Gene discovery requires attention to a variety of complementary signals in the DNA sequences.
- A simple first step in checking if a DNA sequence codes for a protein is to determine if it contains an open reading frame.
- A major set of algorithms use probabilistic techniques to integrate various strands of information into a single gene finding tool.
- The program GeneMark uses a simple Markov model to calculate the probability of any new sequence being a coding region based on the parameters generated from known coding regions.
- The program GRAIL uses a neural network to identify protein coding regions in DNA.
- The GeneScan gene discovery program is based on Fourier analysis of DNA sequences to distinguish between coding and non-coding regions.
- Other analyses performed on DNA sequences include base composition analyses, dinucleotides frequency calculations, codon bias, and patterns and motifs such as RNA sequences, ribosome binding sites, complex repeating patterns, etc.

6

Determination and Analysis of Molecular Structures

This chapter deals with methods of structural analyses. It begins with descriptions of the experimental techniques used to obtain the three-dimensional structures of molecules. The three-dimensional structure of a molecule is considered known when the precise location of each and every atom in it is known; in other words, when we have a list of the three-dimensional coordinates of all the atoms. There are only two well-established experimental methods to obtain the three-dimensional structure of the molecule as defined above, namely X-ray crystallography and NMR. The former determines the structure as present in the solid or crystalline state. This is because the experimental technique requires that crystals of the molecule be first obtained. This requirement may seem to be limitation, especially for biological molecules where most of the activities of the molecules occur in solution, especially in aqueous solution. However it is not so, since biological molecules in crystals are always surrounded by water and other small molecules, closely simulating their natural environment. Thus gigantic leaps in the understanding of biological activity has been achieved on the basis of structures determined by X-ray crystallography, details of the function of the ribosomal particle being among the most recent of these. Nuclear magnetic resonance or NMR is a complementary technique of molecular structure determination. It does not require crystals to be grown, and it deals with the molecules in solution. This makes it possible to tune the experiment either to determine the average structure of the molecule or to follow the dynamics of movement of parts of the molecule. However, due to experimental and theoretical limitations, NMR yields the structures only of relatively small molecules, i.e. those with molecular weights less than about 50,000 daltons. Apart from X-ray crystallography and NMR, there is a third technique which is becoming more and more useful, but which has not still reached the effectiveness of the other two. This is electron diffraction. This technique is particularly useful for large membrane-bound proteins that are hard to crystallise, and are too big for NMR. However such molecules often form thin regularly arranged layers, or two-dimensional crystals, and are therefore eminently suitable for electron diffraction. The first part of this chapter describes these techniques in detail.

Once the structure of a molecule has been obtained, it may be visualised at various levels, from an overall appreciation of its shape, to a detailed understanding of the disposition, for example, of various atoms in the active site. This kind of visualisation is especially convenient with the aid of computer graphics, which allows us to 'zoom in' on particular aspects of the molecule or 'zoom out' to obtain a gross picture. Computer graphics also help to rotate the molecule and view it from different

angles, and to instantly choose different models of representing the molecule, such as 'ball and stick' or 'space-filling'. The second part of this chapter deals with the techniques used to represent complex three-dimensional molecules on the two-dimensional computer screen.

But is not sufficient to simply look at the molecules. To reach a deeper understanding of their function it is necessary to perform various geometrical calculations, such as bond lengths and angles, torsion angles, plane calculations, etc. It is also necessary to calculate the Ramachandran map, which is an important tool to analyse protein structures. Further, we need to recognize and specify the secondary structures in the molecule. These methods are detailed in the next part of this chapter.

Finally, there are several methods used to compare two or more three-dimensional structures, to determine structural similarities, if any. This is an important task in bioinformatics, and, as seen in Chapter 2, there are several databases that employ structural comparison techniques to make hierarchical classifications of protein structure. The last part of this chapter describes these techniques.

6.1 Experimental Structure Determination Techniques

In this section we describe the three experimental techniques used in the structure determination of molecules. In general, these techniques, especially X-ray crystallography, are applicable to all sizes of molecules, from NaCl to ribosomes, and even to 'infinite' polymers such as nylon and cellulose. However, some of the details are different for different types of molecules. We will base our discussions on the use of these techniques in structural biology.

6.1.1 X-ray crystallography

From the year 1913, when Laurence Bragg determined the structure of NaCl in the crystalline form using X-ray analysis, to the year 2000, when four groups of scientists in USA, UK and Germany, determined the structure of the ribosomal particle, there has been continuous development of theory, experimental practice and technology in the science and art of crystallography, until it has become one of the most powerful tools of a biologist. The developments continue apace, and the focus now is on automating all steps of the procedure, such that 'high-throughput' crystallography, in which crystal structures are mass-produced as in a factory, has now become a near possibility.

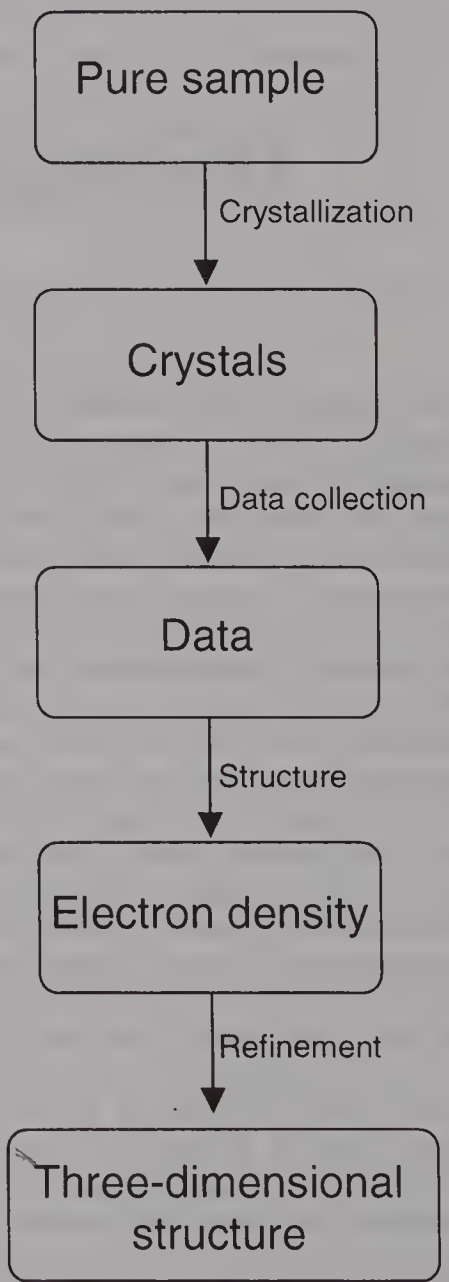


Figure 6.1 The method of X-ray crystal structure determination

Crystallography is the study of crystals. Since biological materials are only rarely crystalline in the natural state, the reason why this technique is used is that, as mentioned above, except for NMR and electron diffraction, this is the only technique that results in the precise structure of the molecule. The method of X-ray crystallography (Figure 6.1) can be described

very briefly as follows: Crystallise the molecule, subject it to irradiation by a beam of X-rays and make a record of the 3-D diffraction pattern. Analysis of this data through computer calculations results in a model of the molecule, which can then be refined against the observed X-ray data. Finally one obtains the three-dimensional coordinates, which describe the position in space of each atom in the molecule.

A crystal is a regular, repeating three-dimensional array of atoms or molecules or groups of molecules. The chief difference between crystals of biological substances and those of other important materials such as silicon is that in the former the repeating units are invariably molecules or groups of molecules, while in the latter the repeating units are atoms or groups of atoms. This implies that the forces of bonding between one repeating unit and another are much weaker in the former, being van der Waals forces or hydrogen bonds. In the latter, on the other hand, the crystals are built up from strong covalent or ionic bonds. Crystals of biological materials are thus usually soft and brittle as compared to inorganic substances. Another difference is the size. Biological and organic molecules usually form very small crystals (approximately 1 mm^3) as compared to inorganic materials (about 1 cm^3). Bio-crystals are usually grown from solution and may have a large solvent content. This is especially true of crystals of macromolecules such as proteins and nucleic acids where the solvent can take up as much as 80% of the crystal. The growth of bio-crystals is a multi-parameter process that could be affected by several different factors such as the concentrations of the different components in the crystallisation mixtures, the temperature, the method of crystallisation, and so on. Growing crystals, especially of biological macromolecules, is a delicate art and usually represents the foremost rate-limiting step in crystallographic technique. Once the crystals are grown, they are placed in an X-ray beam and the three-dimensional diffraction pattern is measured.

Diffraction is a well-known phenomenon that occurs when two or more waves are combined. If light falls on a regularly spaced series of slits, called a diffraction grating, the light that comes out forms a complex pattern of light and dark lines, called a diffraction pattern. The pattern is not simply a projection of the grating but occurs due to the different combinations of the waves of light as they come out of the grating. This is the effect of a one-dimensional grating, but it is possible also to have two and three-dimensional diffraction gratings. A crystal is nothing but a three-dimensional grating. However, it cannot diffract visible light. This is because for the diffraction effect to be noticeable, the spacing between the slits in the grating should be of the same order of magnitude as the wavelength of the incident radiation. In a crystal the distance between the lattice points corresponds to the dimensions of atoms and molecules i.e., a few Angstroms. X-rays are radiation with wavelengths of this size, and hence diffraction from crystals is observed with X-rays. The diffraction pattern from a three-dimensional grating, such as a crystal, is a set of dark spots in three dimensional space spread all around the crystal and representing the angles at which the waves coming out of the crystal combine constructively to add to the total intensity. Thus, the next step, after growing the crystal, is to place it in a beam of X-rays and measure the angles and intensities of the diffracted spots. The experimental set up (Figure 6.2) consists of an X-ray source, a goniometer on which to mount the crystal, and a detector to measure the intensity of the reflected X-ray beam. This detector may be a film, or a radiation counter, but modern devices such as imaging plates or charge

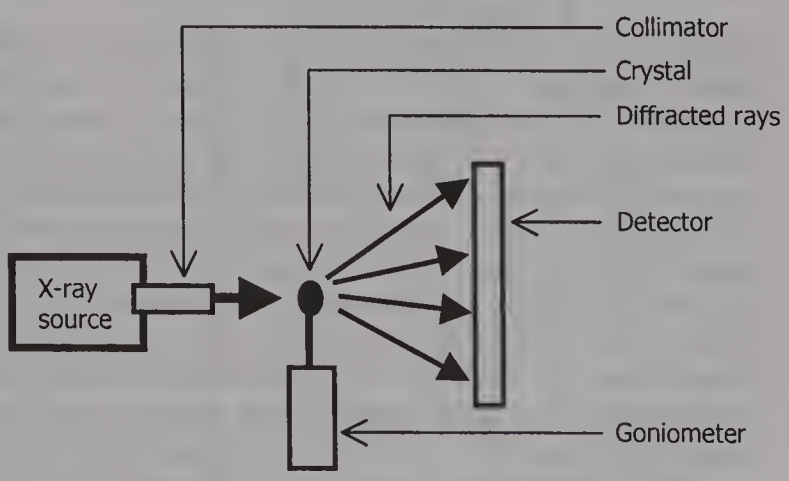


Figure 6.2 The experimental setup for X-ray diffraction

coupled devices (CCDs) offer the advantages of both films and of counters and are now almost exclusively used in the collection of data from crystals of biological macromolecules.

The outcome of the data collection process is a list of angles, indicating the orientation of the crystal and the detector, and the intensity of the reflection from the crystal plane at this position. Instead of the angles, it is usual to identify the set of planes that give rise to each diffracted spot by the so-called Miller indices (h,k,l) and associate with each set of (h,k,l) the appropriate value of the intensity. From the list of intensities, the amplitudes of each reflected wave is extracted by the simple process of taking the square root, since the measured intensity of a wave is the square of its amplitude. These amplitudes, represented as F_{hkl}, are now used in the analysis of the structure of the molecule that makes up the crystal. The structure of the molecule is obtained by a Fourier transform of the observed amplitudes F_{hkl}. However, in order to carry out this mathematical operation, it is not only necessary to know the amplitude (and wavelength) of the waves that correspond to each spot in the diffraction pattern, but also to know the relative positions, or phases of the waves in order to reconstruct the original periodic function. The problem of X-ray crystallography is now one of obtaining the phase information. This is the next major rate-limiting step. It is called the phase problem in crystallography since, in the X-ray diffraction experiments, the phases cannot be measured directly and have to be determined by other indirect means. Among these methods are the heavy atom or multiple isomorphous replacement (MIR) method that obtains the phases from knowing the position of a heavy atom like platinum, which has been diffused into the crystal; the so-called direct methods, which uses a statistical analysis of the X-ray data to obtain the phases; the molecular replacement technique, which is used when the unknown structure can be assumed to be similar to an already known structure; and the anomalous scattering technique, in which the difference in the intensities of diffraction from opposite sides of the crystal is exploited to obtain the phases.

Once the approximate values of the phases are determined the next step is to refine them. Instead of directly refining the phases, the refinement of the structure is usually carried out by performing the Fourier transform and refining the approximate positions of the atoms obtained by including other known data such as stereochemistry. One of the most commonly used techniques is the least squares method. This is a well-known mathematical technique used to fit a model to the observed data by changing the model iteratively in small steps so as to minimise the difference between it and the data. The function to be minimised is called the Residual or the R factor and is usually defined as

$$R = (\Sigma||F_o|-|F_c||)/ (\Sigma|F_o|)$$

where F_c is the calculated structure amplitude for a particular reflection hkl, F_o is the observed structure amplitude for the same reflection and the summation is over all the reflections. The R factor is one of the most important of the figures of merit used to estimate the quality of a structure determination. It is sometimes expressed as a percentage and R factors of less than 20% are considered acceptable in structural biology. In order to work, the least squares technique requires a large number of observations. In the case of biological macromolecules, this condition is seldom satisfied. Owing to various factors, including high solvent content of the crystals, the molecules are not very well ordered and reflection planes with spacing of less than about 2.0 Å are usually not sufficiently regularly spaced to give measurable diffracted intensities. In other words the limit of resolution of the data in such cases is usually not beyond 2.0 Å. This means that the observation-to-parameter ratio is not good enough to allow refinement using the standard least squares techniques, and stereochemical constraints have to be introduced to improve the observation-to-parameter ratio. This in turn means that the experiment is not completely unbiased but carries some of our preconceived notions about the structure. The division of the data on the basis of Bayesian statistical theory into a working set and a test set, using the former for refinement and the latter for checking has lead to the definition of a figure of merit called R_{free} which helps to avoid the model bias to some extent. R_{free} values of less than about 30% are considered acceptable. The advent of high-resolution computer graphics has made possible the development of

several new techniques of refinement and of utilising known chemical and biological information in the refinement process. The model is superposed on an electron density map on the computer graphics terminal, and wherever the fit of the model to the electron density is not good enough, adjustments are made to improve the fit. Thus the degree of correspondence between the map and the model is another check on the quality of the structure. At the end of the refinement process we obtain a set of coordinates for each of the atoms of the molecule and another number (or set of numbers) representing the thermal vibrations of the atom.

A note on the resolution of an x-ray structure: The number of X-ray reflections that may be obtained from a crystal depends on the degree of order present in it. A greater degree of order implies that lattice planes with smaller inter-planar spacing are regularly arranged in the crystal and that X-ray reflections from these planes can also be measured. This implies that in the final structure, features at smaller distances can be resolved (i.e. seen as separate entities). The terminology of the resolution can be somewhat confusing. Since a higher resolution means smaller separation of the observable features, a resolution of 6 Å, is lower than a resolution of, say, 2 Å. The larger the number, the lower the resolution, and vice versa. For biological molecules, a resolution of about 2.0 Å is considered good and resolutions down to 3.5 Å are acceptable. Ultimately the test of a good structure is the amount of confirmed biological information that can be obtained from it. The greater this amount, the better is the structure.

6.1.2 Nuclear magnetic resonance

Nuclear Magnetic Resonance spectroscopy is one of the most important experimental techniques currently in use in biophysics. Until the early 1970's NMR was of interest mainly to physicists and chemists. With the development of techniques such as Fourier transform NMR, and two-dimensional and three-dimensional NMR, biochemists and biophysicists use the method to determine the structures of large molecules such as proteins and also to study the dynamic behaviour of such molecules. Magnetic Resonance Imaging is becoming increasingly popular in medicine as a means to obtain accurate three-dimensional images of the interior organs of the human body in an essentially safe and non-invasive manner.

In common with other spectroscopic techniques, the NMR experiment consists of the measurement of the intensity of absorption (or emission) of electromagnetic radiation at each frequency over a specific range. In NMR this range is the radio-frequency range, i.e. from a few MHz to a few hundred MHz. At this frequency, the energy is absorbed or emitted when the nuclei of the atoms go from one quantized spin state to another. Spin is a property possessed by only certain atomic nuclei such as the hydrogen nucleus H or the phosphorous isotope ^{31}P. A classical analogy to this property of spin is the rotation of a body about an axis that passes through its centre. Since the nucleus has a charge, the spinning motion will give rise to a magnetic field, and the spinning nucleus can be thought of as a small bar magnet. The axis of the bar magnet

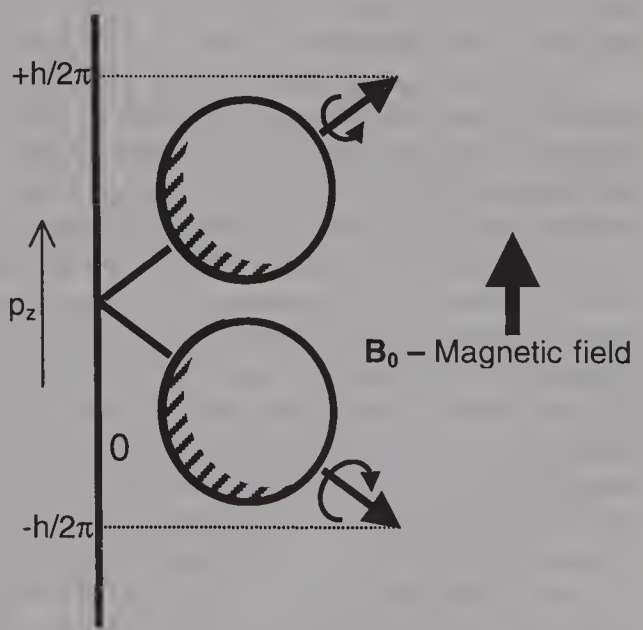


Figure 6.3 A nucleus with spin may align in two possible ways when placed in a magnetic field

will be parallel to the axis of spin. When this nucleus is placed in a strong external magnetic field, then, by the laws of quantum mechanics and due to the interaction of the two magnetic fields, the axis of spin will orient itself in one of two possible orientations (Figure 6.3). Transitions between these two states can occur when another external oscillating magnetic field supplies sufficient energy for the spin system, i.e. the nucleus to go from the state or orientation of lower energy to the state of higher energy. The difference in energy of the two states corresponds to the energy of a radio-frequency field and hence a nuclear spin, placed in a static magnetic field, must be excited by radio-frequency electromagnetic radiation in order that the transition takes place. The energy absorbed from the radio-frequency field can be detected and this is the fundamental parameter measured in an NMR experiment. The frequency at which the transition occurs is called the resonance frequency. The resonance frequency of a particular type of nucleus, e.g. ^{1}H, will be different when it is placed in different chemical environments. For example the ^{1}H in a methylene group will have a distinctly different resonance frequency from one in a methyl group. This change in frequency arising out of the different chemical environments is called the chemical shift. By changing the frequency of the exciting radio-frequency field and measuring the intensity of the absorption at each frequency, it is possible to detect the nuclei and in each case identify their respective environments. This feature is used to identify various chemical groups in molecules. If the magnetic field in which the spins are placed is very high, subtler effects become noticeable, i.e. the resolution of the spectrum increases. In this way it is possible to detect even conformational changes in the molecule.

Continuously changing the frequency of the radio field and measuring resonance is a tedious and time-consuming task. So much so, in the case of large molecules, where the changes in intensity are very small, formidably long data collection time would be required. Fourier transform NMR and multi-dimensional NMR have reduced the experimental time required immensely and have made it possible to use NMR to study large biological molecules and complexes. In FTNMR the radio-frequency stimulus is applied as a pulse, which may last typically for about 200μs. The pulse is sufficiently strong and is constituted of a sufficiently large number of frequencies to excite all the resonances in the sample. The response to such an excitation would be an NMR signal from each of the spins in the sample. The composite would be another complicated wave, which may be Fourier transformed to extract the NMR spectrum. The expression free induction decay or FID is used to describe the response on the application of the radio-frequency pulse.

The nuclei in the sample will be present in several different environments, and the resulting FID will consist not only of several different frequencies, but also the intensity peak corresponding to each different frequency will decay at a different rate. The Fourier transform of the FID therefore contains information not only about the resonance frequency of each nucleus but also about the decay of the resonance and hence the rate and mode of energy transfer from the nucleus to the environment.

The nuclear Overhauser effect or NOE illustrates the so-called 'double resonance' technique in which a second radio frequency field is applied to the sample in addition to the observing radio frequency field. The two fields may be denoted as H_2 and H_1 respectively. The second, strong, H_2 field is usually used to saturate particular resonances and is used in spin decoupling, saturation transfer or NOE experiments. H_2 is maintained at the resonance frequency of one resonance, to saturate it while another resonance is monitored by the non-saturating H_1 field. The monitored resonance may change in intensity if affected by the saturated resonance due to a change in the populations of the nuclear energy levels. Under normal conditions the relative populations of the spin states (or nuclear energy levels) is given by the Boltzmann distribution. If this distribution is altered by a saturation of some of the nearby resonances, and the consequent transfer of magnetisation by dipole-dipole coupling, it can lead to an enhancement of the observed signal. The magnitude of dipolar coupling is strongly distance dependent and is proportional to $1/r^6$ where r is the distance between the nuclei. NOE is a 'through-space' effect, and it does not need the interacting nuclei to be covalently linked to each other. Thus NOE is a

powerful aid to the conformational analysis of molecules. A typical NOE experiment would consist of identifying all the resonances in the molecule and then saturating them one by one, each time measuring the enhancement of the intensity of the other resonances. In this way, it is possible for example to identify those portions of a polypeptide chain which may fold back to be close together in space though they are far apart in sequence.

Detailed conformational studies of macromolecules are now possible by applying a combination of some of the above techniques, like NOE with multi-dimensional NMR. A one-dimensional FTNMR experiment involves the application of a RF pulse after the sample has been allowed to achieve equilibrium during the 'preparation' period. Immediately after this, comes the detection phase where the FID (i.e. the signal S) is collected as a function of time (represented by the variable t_2). A Fourier transform of the signal into the frequency domain results in the NMR spectrum.

$$S(t_2) \rightarrow S(\omega_2)$$

In the case of two dimensional NMR spectroscopy an additional time period (denoted t_1) and called the evolution period is introduced, after which the FID is collected. During the evolution period another RF pulse may be applied and a further period of time, called the mixing period, may be allowed to elapse before the FID is collected. In each experiment, the time t_1 may be given a different value, ranging from zero to some maximum value depending on the relaxation times. Each time the FID is collected and Fourier transformed to yield $S(t_1,\omega_2)$. The several spectra so obtained are arranged in a data matrix which can be subjected to a second Fourier transform, column wise

$$S(t_1, \omega_2) \rightarrow S(\omega_1, \omega_2)$$

The spectrum thus will have two dimensions. ω_1 is along one of the dimensions and ω_2 along the other. The ω_2 axis contains information about coupling interactions with other nuclei during the evolution period t_1, and the ω_2 axis contains information about the chemical shifts. The precise pulse sequence applied to the samples, gives rise to different types of coupling information being displayed along the ω_1 axis. There are currently more than 50 different pulse sequences that have been published. The two most commonly used sequences in biological research are abbreviated as COSY and NOESY.

COSY is an abbreviation for correlated spectroscopy. COSY spectra are of great use in assigning the peaks to the nuclei that cause them. Solving the chemical structure of the molecule can be accomplished through a COSY spectrum. The assignment of peaks in the spectrum of a large biomolecule such as a protein is relatively easily accomplished.

NOESY is an abbreviation for nuclear Overhauser enhancement spectroscopy. The nuclear Overhauser effect is a 'through-space' effect and can be used to determine the spatial separation between nuclear spins. The strength of the effect varies as $1/r^6$ where r is the distance between the spins, and measurement of the strength of the off diagonal NOE peaks in a 2D NOSEY spectrum can be used to estimate distances between spins, and thereby to determine the three-dimensional structure of the molecule.

The experiment to determine the three dimensional structure of a macromolecule usually consists of two steps. The first step is to assign all the resonance peaks that occur in the spectrum. A 2-D spectrum such as COSY is used to trace the connectivity in the molecule and assign the peaks. The second step in the structure determination is to identify secondary and tertiary structures from the J-connectivity patterns and from NOE data. A NOESY spectrum will give information about the 'through-space' couplings, and therefore distances, between the spins in the molecule. The distance data so obtained may be used to attempt a reconstruction of the three-dimensional folding of the biopolymer chain. This is mathematically a complicated problem and no general algorithm exists which can lead from the distances to the geometry in a reliable way. There are however several approaches to solving this problem, each being successful to a lesser or greater degree. Distance geometry algorithms therefore use all chemical information available and may be performed in torsional-angle space where the degrees of freedom are the dihedral angles of the main chain and the

side chains. Additional constraints in the form of non-bonded van der Waals contacts, hydrophobic interactions and so on, may also be used in arriving at the structure. A commonly used technique is restrained molecular dynamics. Here the observed experimental data, in this case the NOE distances, are written as a pseudo potential, along with other semi-empirical potential terms such as bond length energy, bond angle energy etc. This method has been successfully used in many instances in arriving at precise three-dimensional structures of small proteins (Mw ~20,000). Structures of larger proteins (Mw > 50,000) are still somewhat inaccessible to current techniques.

6.1.3 Electron microscopy, image reconstruction and electron diffraction

Electrons are negatively charged subatomic particles. Their use in imaging can be explained in terms of their quantum mechanical wave-like property, which allows a beam of electrons to be considered in exactly the same way as a beam of electromagnetic radiation such as light. Electrons possess two additional advantages for use in microscopy. The first of these is that according to quantum physics, the wavelength of the waves associated with an electron varies inversely as its energy. Thus by accelerating the electron between charged plates with a very high applied voltage, the wavelength of the 'radiation' used in the microscope could be made very small. This implies a concomitant increase in the resolution and hence in the possible magnification, to the extent that we obtain good images of the large macromolecular complexes or even DNA molecules. The second advantage, which actually allows the use of electrons for microscopy, is that, unlike electromagnetic radiation of short wavelengths such as X-rays, electrons can be focussed by means of magnetic lenses. This is dependent on the well-known principle in physics that a charged particle in a magnetic field is deflected in a direction perpendicular to its direction of motion. This principle has been used in the construction of electron microscopes. Modern electron microscopes use accelerating voltages in the range 1000 volts to 1000 kilovolts. The most popular microscopes use about 100 kilovolts. According to quantum physics, this voltage will correspond to a wavelength of about 0.03 Å. This should, in principle, allow the imaging of details on an atomic scale. However several factors come in the way of reaching this high resolution. Vibration, damage to the specimen and contamination are some of the problems encountered in microscope design. The most important factor limiting the resolution is the inability of the magnetic lenses to focus the beam accurately. The final quality of the image depends not only on the resolution but also on the contrast. Contrast is the difference in intensities between the object and the background as a fraction of the background intensity. For visibility, this ratio should be larger than 0.2. The contrast can be enhanced if the object is dense as compared to the background. Usually, the support for the specimen is a carbon film and biological materials, especially, have about the same density. The use of electron microscopy in biology is thus especially problem prone. But one can use staining methods to increase the contrast. DNA strands are normally imaged by using a heavy atom such as platinum to stain them. Thus, because of the above problems and also because of others not fully discussed here, the actual practical resolution and magnification in biological electron microscopy is below what in principle is possible. Nevertheless, objects such as cells, viruses, DNA strands, ribosomes, etc. can be imaged clearly at resolutions of a few tens of Angstroms, at magnification of 100000X or more. The transmission electron microscope (TEM) is one of the most commonly used instruments. The scanning electron microscope (SEM) is very useful in obtaining images of the surface of thick specimens. In the scanning-transmission mode (STEM), the SEM can also be used to study thin specimens and in this respect has some advantages over the conventional TEM.

Very often the electron micrographs of biological samples such as viruses or ribosomes or other large molecular assemblies consist of several repeated indistinct images of the object. The image can be made more distinct by reconstruction procedures. There are chiefly two such procedures. The first involves simply superposing all the images, such that the 'true' parts of the images, or the signal, gets

reinforced, while the background 'noise' will be cancelled. The selection of the identical parts of the image for superposition can be accomplished visually. A more rigorous method is to calculate a superposition function in two dimensions. The superposition function or the cross correlation function will have a maximum value when one image is rotated and translated to superpose on another image optimally. Such optimisation can be accomplished on the computer if the images are digitised and fed to it. A more accurate way to accomplishing a clear image from several indistinct images of the same object is possible if the images are arranged in a regular repeating two-dimensional pattern. If the image is used as a kind of imperfect diffraction grating and is placed in the path of a source of light, a diffraction pattern will be formed. The diffraction pattern can then be filtered to allow through only those parts, which correspond to the periodic lattice. The filtered diffraction pattern is in turn used as the grating and placed in the path of a light beam. The image formed will correspond to a much clearer version of the original object. Mathematically, the procedure is equivalent to performing a Fourier transform, cleaning up the coefficients by filtering, and then performing a reverse Fourier transform. The method of image reconstruction has been widely used to study crystalline materials. Virus structures such as the stem of the T phage, membranes and two-dimensional arrays of ribosomes have all been studied by this method. The method can be extended to three dimensions by taking several two-dimensional images obtained by tilting the specimen at different angles. In fact a single micrograph of a suspension of particles is bound to contain images of the particle in various three-dimensional orientations. Image reconstruction techniques can be fruitfully used to put together these images to form a 3-D picture of the particle.

The electron microscope can be also be used with a few modifications, to obtain electron diffraction patterns from two-dimensional crystals. Instead of analysing these patterns through geometrical optics, it is possible to use Fourier theory to mathematically transform the diffraction pattern into a three-dimensional image of the particle. Since the electron wavelength is much smaller (~ 0.03 Å) than that of X-rays, it may be thought that a much higher resolution will be obtained. However several obstacles come in the way of attaining such high resolution. Since electrons are easily absorbed, only very thin samples can be used. Electron diffraction is ideal for use in those cases where three-dimensional crystals do not grow but two-dimensional arrays form easily, such as to study membrane proteins. From such a thin 'crystal', almost the entire diffraction pattern will be in a single plane and most of the three-dimensional pattern required to perform a complete Fourier transform is not available. Another problem, also present in the case of X-ray diffraction, is the loss of phase information and the consequent inability to perform the Fourier analysis. One way in which this problem is got over is by using the electron micrograph to obtain some information about the approximate phases and using this information in combination with the measured electron diffraction intensities. In spite of these difficulties, electron diffraction is a popular technique to obtain low-resolution images of large particles that do not easily form crystals suitable for X-ray diffraction. The images are only pictures, not coordinates, and therefore suitable only for obtaining an overall structural picture of the biological system under study. Unlike as for X-ray crystallography and NMR, the results of these experiments do not find a place in the PDB or other such structural databases.

6.2 Visualisation and Representation of Molecular Structure – Computer Graphics

6.2.1 Methods of representing a three-dimensional object on a two-dimensional surface

Artists in the fifteenth and sixteenth centuries first grappled seriously with the problem of realistically representing three-dimensional objects on a two-dimensional surface such as canvas or paper. Many of

the techniques developed then are used even today, not only in art, but also at one of the interfaces of art and science, namely molecular display and modelling. In addition, the extensive use of computer graphics for this purpose has given rise to the adoption of additional specialised methods for creating three-dimensional images.

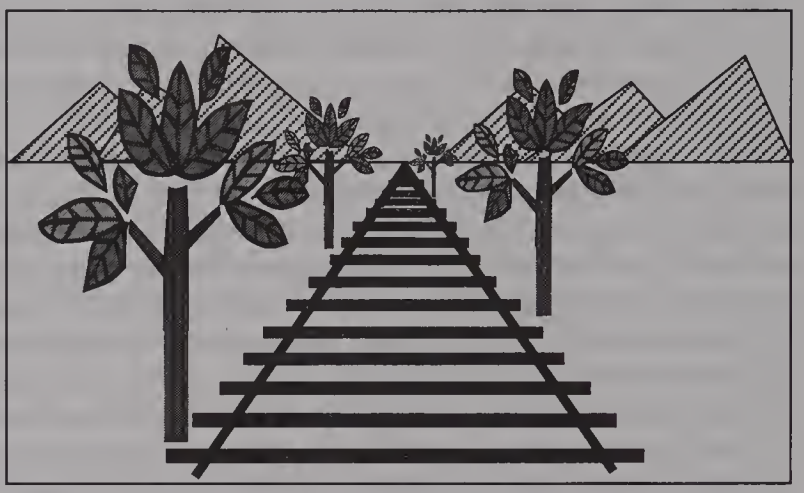
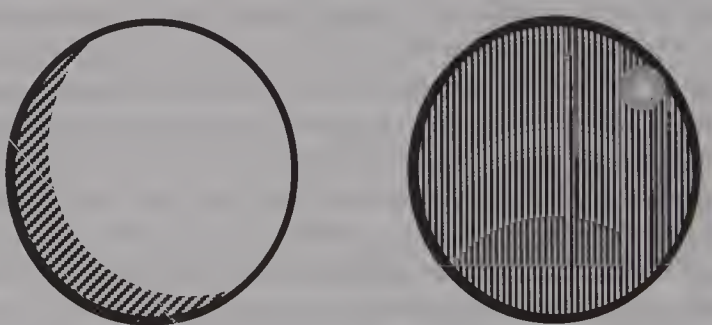


Figure 6.4. Perspective

Certain attributes of an object or a group of objects arranged to form a scene arise specifically from its three-dimensional nature. One of these is *hidden line* or *hidden surface removal*. This means that if two opaque objects are placed one behind the other, the one at the rear will have at least a part of its surface hidden by the object in front. For a single object the front and side surfaces will be visible, but not the rear. The lines or surfaces that are obscured are removed from the picture, and in this manner an indication of the relative depths of the various objects is made. Another clue to the third dimension or depth in pictures comes from *perspective*. This refers to the fact that if there are two objects of the same size at different depths, the one that is more distant appears smaller than the one that is closer to the viewer. Figure 6.4 shows the well-known example of a railway track vanishing into the horizon. For this feature to be effective the viewer must have a clear idea of the relative sizes of the objects. While representing molecules, perspective is useful in indicating depth since the actual atomic sizes are constant, regardless of their position in the molecule. Surface curvature and indentations in an object are represented by differences in the brightness of the surface. This process is called *shading* (Figure 6.5a). A related process, specially useful to represent a metallic or smooth plastic surface is *highlighting* where the highlights or glints which occur on the highest point of the object due to reflection of the incident light are shown as bright, nearly white, spots (Figure 6.5b). A computationally expensive algorithm known as 'ray tracing' is often used to carry out both shading and highlighting. In this technique an imaginary ray is drawn from the (potential) viewer's eye, through a point on the computer screen and on to the surface of the object. The intersection point determines the visible surface at this point. To determine shadows, another imaginary line is drawn towards the source of the light. If this line is intercepted before it reaches the light source, a shadow is indicated. If not, the angle made by this line with the surface of the object indicates the degree of shading required. If the object is not completely opaque, two more lines, one in the direction of reflection and one in the direction of transmission, help to calculate the distortion in the reflecting and refracting surfaces. These calculations are repeated for every point in the scene to be displayed, and for



(a) (b)

Figure 6.5 a Shading. b Highlighting

every point on the computer screen, and thus require advanced computation resources. Ray tracing also helps in *shadowing*, i.e. displaying an object along with its shadow so as to enhance the 3-D effect.

The easy availability of computer-generated graphics has led to several new techniques to represent three-dimensional objects. *Rocking* is a method wherein the object is subject to small oscillations (about 10°) about the vertical or horizontal axis. By simultaneously applying rocking, hidden line/surface removal and perspective, a clear idea of the three-dimensional nature of the object is obtained. The disadvantage is, of course, that the display is always in motion and this prevents a more careful look at any details of interest. A feature that is an integral part of almost every graphics software program used to display molecules is user-controlled rotation about all three axes. In this, the mouse, or a special set of dials, or the arrow keys on the keyboard, are used to control the direction and speed of the rotation. Again rotations about the vertical and horizontal axes will give an idea about the third dimension.

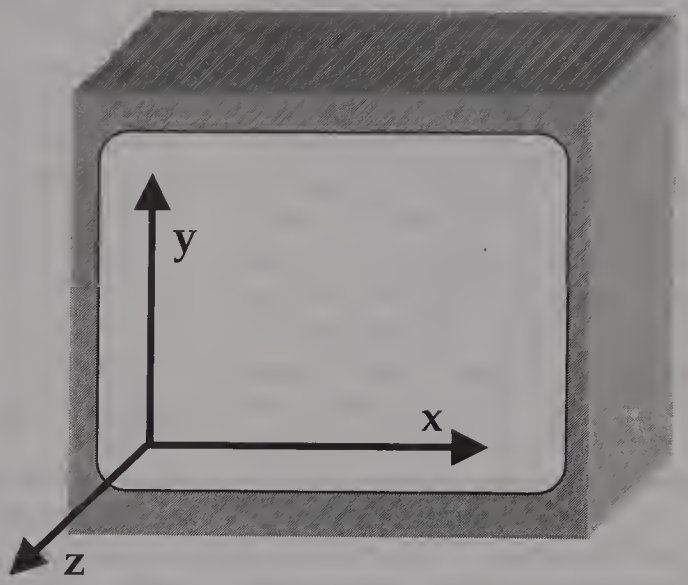


Figure 6.6 The axis system on the computer screen.

There are two different axial systems used simultaneously in computer graphics to display molecular coordinates. The atomic positions are defined in the Cartesian coordinate system, as found in the PDB. Rotations and translations of the molecule as displayed on the screen are performed with respect to a different axial system, which is fixed with respect to the computer screen. In this system the x axis runs horizontally from left to right along the bottom of the screen, the y axis is vertical from bottom to top at the left hand edge of the screen and the z axis is perpendicular to the screen towards the viewer from the left lower corner (Figure 6.6). Thus the rotations are applied not only to the atomic coordinates, but usually also to the Cartesian coordinate system. This however is true only when the entire molecule, or all the molecules displayed on the screen are rotated. When a part of the scene displayed or a part of a molecule is rotated or translated, the Cartesian coordinate system is kept fixed. Another system of axes that comes into consideration when displaying X-ray crystallographic data is the crystal system[18]. To incorporate the crystal symmetry information, it is more convenient to specify the coordinates with respect to the unit cell in the crystal, which, unlike the Cartesian system, is not always orthogonal. In such cases the software usually allows the user to switch views from just displaying a single molecule in the Cartesian system, to displaying a molecule and its symmetry-related neighbours in the crystal system.

The definition of the z-axis as being perpendicular to the screen facilitates another method of indicating the depth using the brightness of the object, called *depth cueing*. In this method, the object is made dim or bright according to its position on the z-axis with respect to the viewer. Thus the z coordinate of each part of the object is directly converted to a brightness value, which is then displayed. Computers with advanced graphics display options usually implement this feature in what is called a 'hardware z buffer'. This means that a section of the computer memory is set apart solely to represent the z value of each point to be display. As the scene changes, the z values and hence the

[18] See Figure 6.13

brightness values are constantly updated and displayed, so that the front portions of the objects always are brighter than the rear portions. The objects can be moved (translated) along the z-axis so that they become dimmer and dimmer and vanish altogether. They are of course brought back into view by having them move in the other direction.

Perhaps the most dramatic three-dimensional effects arise from an application of stereo projections. This method is particularly suited for displaying line diagrams, such as most molecular pictures, and less suitable for pictures of solid surfaces, though here too the effects can be equally good. The method is based on how we actually perceive the third dimension in everyday life. As is well known, when we see the external world through two eyes, the image on each eye is rotated slightly with respect to the other. Therefore the two images are not exactly the same. The brain interprets these differences to perceive the third dimension. It is possible to repeat this effect for a diagram of a three-dimensional object by producing two copies of it, each rotated with respect to the other by about 6°. The two images are placed side by side, separated by a convenient distance of about 6 cms. Now the eyes are made to view the two images separately, with the left eye focussing on the image on the left, and the right eye on the one on the right. This is equivalent to obtaining slightly different views of the three dimensional scene, and the brain interprets the combined view as a three-dimensional image of the object. Stereo viewing is usually carried out with special glasses that help each eye focus on one image. With practise, however, such devices are unnecessary and a pair of stereo pictures (Figure 6.7) can be viewed with naked eyes and be seen as a single three-dimensional image.

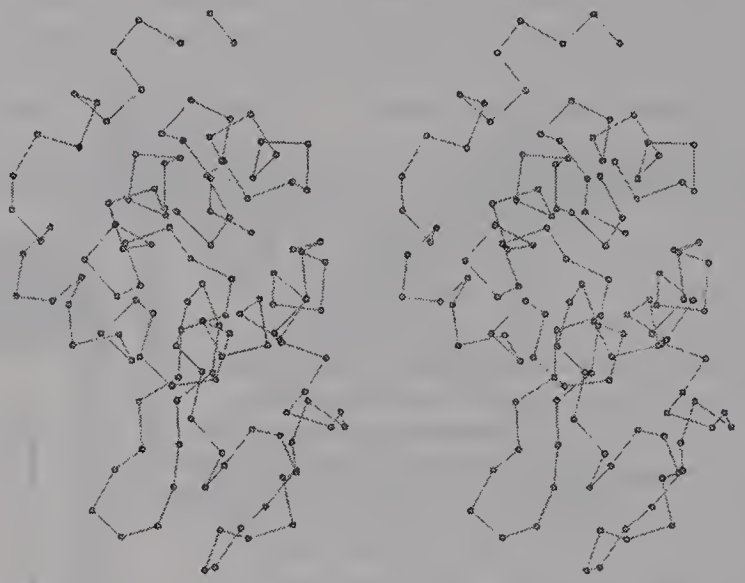


Figure 6.7 A stereo picture of the C^{α} trace of lysozyme

6.2.2 Methods of representing biological molecules

The two experimental methods of detailed structure determination described in the first section above, namely X-ray crystallography and NMR, yield a list of the coordinates of the atoms in the molecule. To appreciate the biological information contained in the structures, they have to be presented visually as models or graphic images. Before the advent of computer graphics, it was common to build large, detailed three-dimensional models of the molecules out of wood, plastic, or metal. These were cumbersome, inaccurate and difficult both to construct and to maintain. Nevertheless many important early discoveries, including perhaps the most important one of all – the structure of DNA, were made through the use of such models. Even though they remain aesthetically satisfying, they are seldom used today, and have been replaced by computer graphics. Computer graphics are accurate, portable, flexible, detailed and relatively inexpensive. They have one additional major advantage especially in relation to biological molecules – they allow the three-dimensional display of different types of information, such as secondary structure, electron or charge density, thermal motion and hydropathicity.

Molecules may be represented as three-dimensional chemical diagrams, with a single point indicating an atomic position and the lines joining the points corresponding to the bonds (Figure 6.8). Such pictures are called line diagrams. In the *ball and stick* pictures bonds between the atoms are drawn as thick sticks, and the atoms are indicated by balls whose radii are often made proportional to their respective van der Waals sizes (Figure 6.9). Colour is an important parameter used to convey information, and the colours of the balls are coded to represent the atomic elements, with black for carbon, red for oxygen, blue for nitrogen, white for hydrogen and yellow for phosphorous and sulphur. X-ray crystallography also gives information on the thermal vibration of each atom about its mean position. This information may be included in the diagram by replacing the balls with ellipsoids, the whose axial lengths are proportional to the value of the corresponding component of vibration. A computer program that specialises in such *thermal ellipsoid plots* is called ORTEP. The surface of the molecule may be studied using the so-called *CPK* or *Corey-Pauling-Koltun* models (Figure 6.10). These are also called *space-filling* diagrams of the molecule.

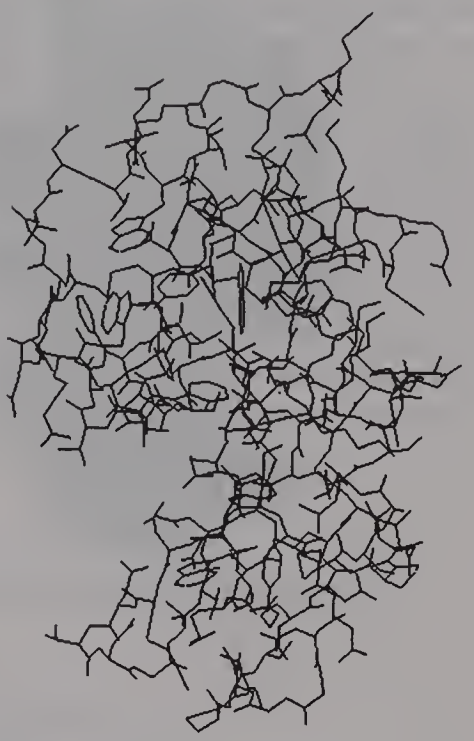


Figure 6.8 Line diagram of Lysozyme

Protein structures are too complex for the viewer to appreciate all the information present from a single diagram. Various models are used to convey various levels of structure. Line diagrams may be drawn using all the atoms, or the atoms of the polypeptide backbone alone, or as a trace joining the C^{α} atoms alone. Secondary structures are conveyed in the form of *ribbon* diagrams (Figure 6.11), where a trace of the polypeptide backbone is shown as a ribbon. Such ribbon diagrams are also useful in following the path of the ribose-phosphate backbone of nucleic acids. *Cartoon* diagrams may also be used to convey protein secondary structure information (Figure 6.12), in which the alpha helices are shown are cylinders and the beta strands as thick arrows. Macromolecular surfaces are calculated in a different way as compared to small molecules. In the latter case, the intersection of all the van der Waals surfaces of the atoms, which envelops the entire molecule, is considered its surface, and this may be easily calculated. The CPK surface mentioned above is just such a surface. Proteins and nucleic acids are also often shown as CPK space-filling models. But biologically important surface of a macromolecule is calculated as its *accessible surface area*. A water molecule, which is modelled as a sphere of radius 2.5 Å, is made to approach the macromolecule from every side. Each point where the two make contact, without any

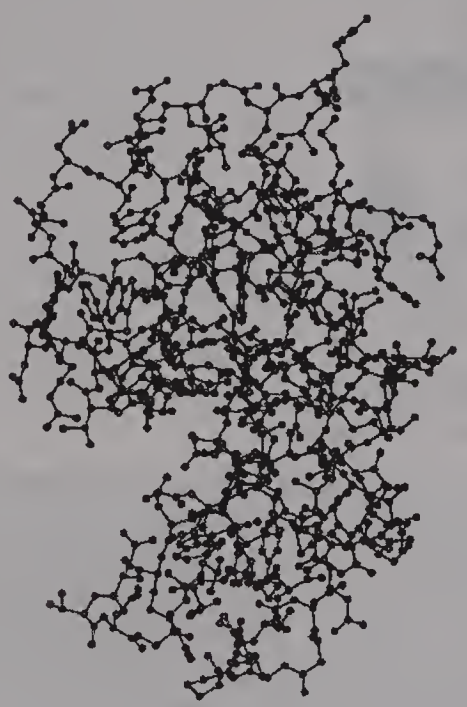


Figure 6.9 Ball and stick diagram of Lysozyme

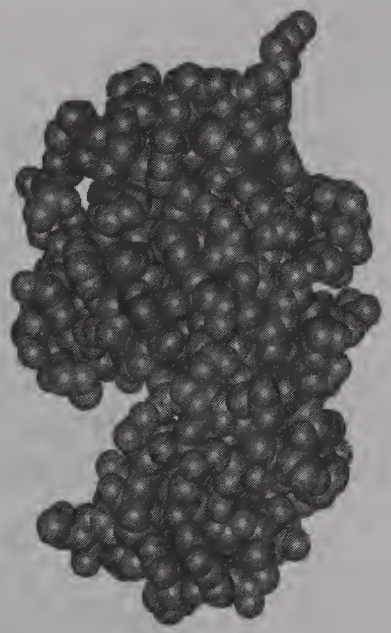


Figure 6.10 CPK diagram of Lysozyme

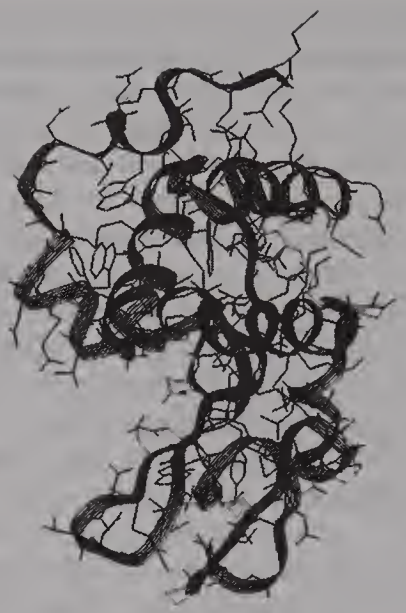


Figure 6.11 Ribbon diagram of Lysozyme

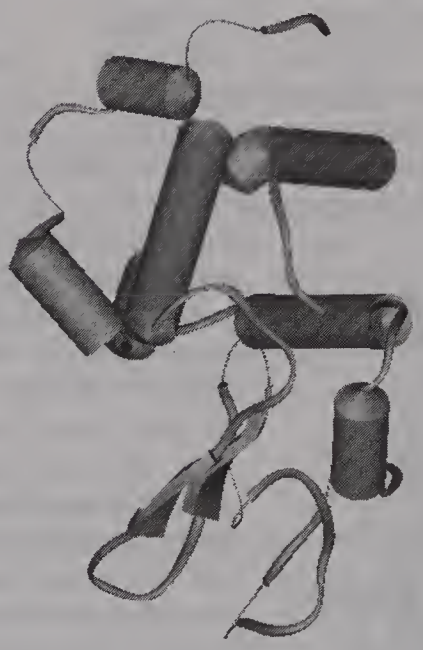


Figure 6.12 Cartoon model of Lysozyme

interpenetration of atoms, is considered to a point on the macromolecular surface. Thus indentations in the surface, that are too narrow to allow the water molecule to enter, are smoothed over, and a model of the surface as relevant to its biological function is obtained. (See also section 6.3.9). *Connolly surfaces* are another way of calculating the outer envelope of a molecule. Surfaces are represented using many different styles. The two most common ways are as evenly spaced dots placed sufficiently thickly on the envelope, and as a smooth surface, calculated from the points by interpolation. The surface is coloured to represent different properties of the surface such as charge or hydrophobicity, making it all the easier to understand the molecular structure and function.

Several programs and program packages are available in the public domain and commercially to carry out these tasks. The commercial packages, especially, come with graphical user interfaces (GUIs) that make it easy to use just a few clicks on the mouse to generate complex molecular pictures. The Insight II package is one such widely used commercial package. The TRIPOS suite of programs is another well-known commercial package for molecular graphics. Among the common programs available in the public domain are Swiss-PDB viewer, RASMOL, Molscript and Bobscript, SETOR and Grasp. The last is a specialised program that calculates and displays the surface electrostatic properties of a molecule, given its atom coordinates. Some of these programs allow the user to manipulate and change the molecule to build new models. Others are simply display programs, and have a wide range of display styles and options. Thus using SETOR it is possible to create graphics of molecular interactions that include portions of the molecule displayed as ribbons, other portions as ball and stick (e.g. the active site of an enzyme), and yet other portions as surfaces, all simultaneously in the same picture. Such pictures are an extremely powerful way of conveying complex information without loss of scientific precision.

6.3 Geometrical Analyses of Structures

6.3.1 Coordinate transformations

As mentioned earlier, structures determined by X-ray crystallography are conveniently listed in the axial system of the crystallographic unit cell. Thus the atomic positions are expressed as fractions of the three unit cell axes. Calculations of the molecular geometry are however usually carried out in the Cartesian coordinate system. Figure 6.13 indicates the relationship between the two systems. The conversion from the crystallographic system to the Cartesian system is given by

$$\mathbf{x'} = \mathbf{A}\mathbf{x}$$

where $\mathbf{x'}$ and $\mathbf{x}$ are the coordinates of an atom in the Cartesian system and crystallographic system respectively and $\mathbf{A}$ is the conversion matrix given by

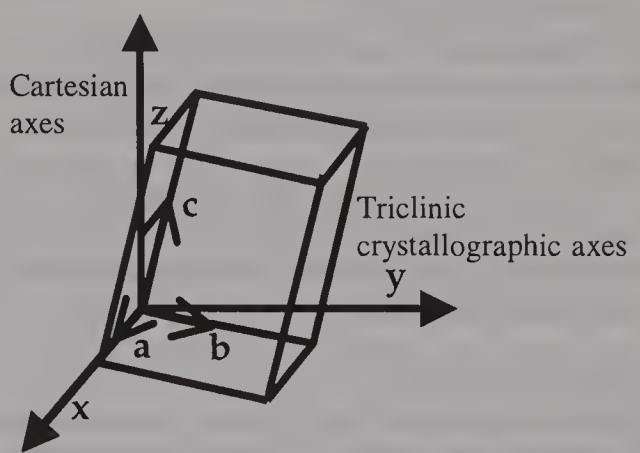


Figure 6.13 The relationship between a triclinic crystallographic unit cell and the Cartesian coordinate system.

$$\mathbf{A} = \begin{vmatrix} a & b\cos(\gamma) & c\cos(\beta) \\ 0 & b\sin(\gamma) & c\{\cos(\alpha) - \cos(\beta)\cos(\gamma)\}/\sin(\gamma) \\ 0 & 0 & P \end{vmatrix}$$

where

$$P = c[1-\cos^2(\alpha)+\cos^2(\beta)-2\cos(\alpha)\cos(\beta)\cos(\gamma)]^{1/2} / \sin(\gamma)$$

Here a, b and c are the unit cell sides, and α, β and γ are the angles between b and c, c and a, and a and b respectively.

Another convenient axial system that is encountered often in biomolecular analyses is the cylindrical coordinate system (Figure 6.14). This system is particularly useful in representing helical structures such as the DNA double helix or alpha helices in proteins. The three coordinates used in this system are the radius r, the angle θ and the height z. The conversion to the Cartesian coordinates x', y'

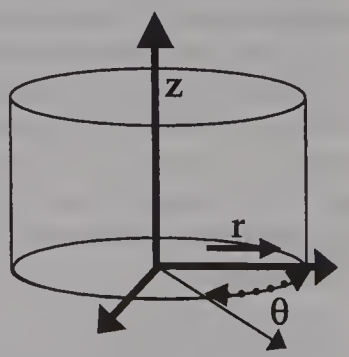


Figure 6.14 The cylindrical coordinate system. (The Cartesian system is also shown for reference).

and z' is given below.

$$x' = r\cos(\theta)$$
$$z' = r\sin(\theta)$$
$$z' = z$$

6.3.2 Bond lengths

Once the Cartesian coordinates of the atoms in the molecule are known it becomes easy to calculate the distance between any two atoms. If $\mathbf{x}_1$ and $\mathbf{x}_2$ are the vector coordinates of the two atoms and d the distance between them, then

$$d = \text{sqrt}\,[(x_1-x_2)^2 + (y_1-y_2)^2 + (z_1-z_2)^2]$$

in which 'sqrt' represents the square root of the term within brackets. This distance is used to decide whether a particular pair of atoms is bonded. In very accurate structures, the bond distance may be used to estimate the chemical bond order, i.e. whether it is a single, double, partial double or triple bond. Protein and nucleic acid structures however are very rarely so accurate, and bond distances are usually kept fixed at chemically reasonable or average values during the refinement of the structure. Thus in the analyses of such molecules, calculations of inter-atomic distances are carried out more to evaluate tertiary interactions such as hydrogen bonds or van der Waals interactions.

6.3.3 Bond angles

The angle between a set of three bonded atoms is known as the bond angle. If $\mathbf{x}_1$, $\mathbf{x}_2$ and $\mathbf{x}_3$ are the three atoms, then the angle made at $\mathbf{x}_2$ by the line joining $\mathbf{x}_1$ to $\mathbf{x}_2$ with that joining $\mathbf{x}_2$ to $\mathbf{x}_3$ is given by

$$\theta = \cos^{-1}[(\mathbf{A}\bullet\mathbf{B})/\,|\mathbf{A}||\mathbf{B}|]$$

where

$$\mathbf{A} = (\mathbf{x}_1 - \mathbf{x}_2) \quad \text{and} \quad \mathbf{B} = (\mathbf{x}_3 - \mathbf{x}_2)$$

Again in macromolecular structures, the angles between neighbouring bonds are not determined experimentally but evaluated from known chemistry. Thus the angle calculations are carried out mostly for tertiary interactions.

6.3.4 Hydrogen bonds

Hydrogen bonds and van der Waals interactions are extremely important in biological systems. Almost all molecular recognition processes occur through a combination of these two interactions. Hydrogen bonds occur between two electronegative atoms such as chlorine, oxygen, nitrogen, and occasionally, carbon. One of the pair must have a hydrogen nucleus, i.e. a proton, which it 'donates' and thus is called a donor. The other atom in the pair 'accepts' the proton and is called the acceptor. When a candidate pair of atoms had been identified, the presence of a hydrogen bond is confirmed by calculation the distance between the donor and the acceptor, and the one between the hydrogen atom and the acceptor. These distances should be less than the sum of the radii of the two atoms involved. The angle defined by the three atoms donor-hydrogen-acceptor is also required for confirmation. This should be close to 180°, and is usually 140-160°. If the chemical groups required for a hydrogen bond are not present between two proximal atoms, a possible van der Waals interaction may be confirmed by calculating the distance between the two atoms to see if this is less than the sum of their van der Waals radii.

6.3.5 Torsion angles

A torsion angle is defined by four atoms connected serially in a chain. It is the dihedral angle between the two planes defined by the first three and the last three atoms respectively. If A, B, C and D are the four connected atoms, the angle made by the bond BA with the bond CD, when viewed directly down the BC bond is called the torsion angle (Figure 6.15). The angle is zero when the bond BA eclipses the

bond CD, and is counted positive when the far bond is rotated clockwise with respect to the near bond. Note that the value and the sign of the angle is the same no matter in which direction it is viewed, i.e. whether down BC or down CB. In vector notation, the torsion angle is given by

$$\chi = \cos^{-1}[(\mathbf{E}\bullet\mathbf{F})/|\mathbf{E}||\mathbf{F}|]$$

where

$\mathbf{E} = (BA)\mathbf{x}(BC)$ and

$\mathbf{F} = (CB)\mathbf{x}(CD)$

BA being the vector from point B to point A, etc. There is a matter of the sign of the angle to be taken into consideration, since there are two scales of angles that may be used. The angle may be specified in a range from 0 to 360° or between –180° to 180°. The transformation of the angle from one range to another is carried out simply by checking if the angle is greater than 180°, and if it is, subtracting 360° from it.

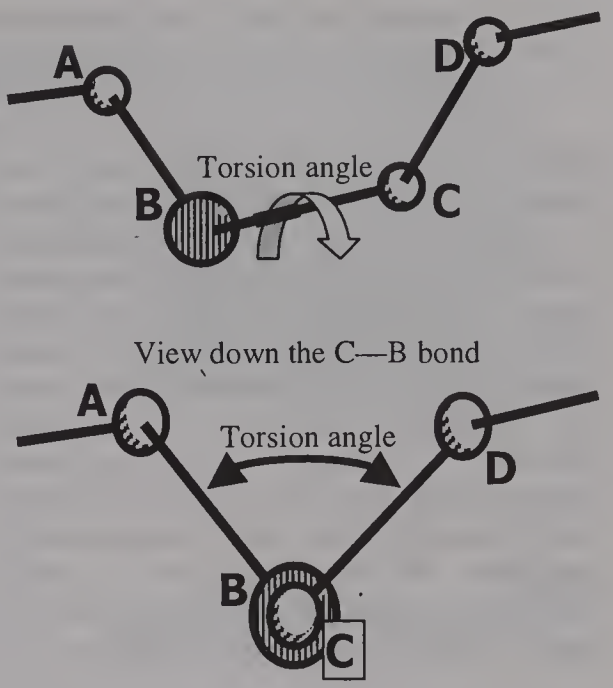


Figure 6.15 Schematic of a torsion angle

6.3.6 Calculations of planes

Groups of atoms that lie in a plane often do not do so perfectly. Recourse is therefore made to iterative techniques to obtain the equation of the plane and to estimate the deviations of the relevant atoms from it. The least squares technique is most commonly used for this. In general the equation of a plane is given as the equation of the normal to it, i.e.

$$lx + my + nz = P$$

where l, m and n are the direction cosines of the normal, x, y and z are the coordinates of a point on the plane, and P is the perpendicular distance from the origin to the plane. If we define the plane by just three points, then the cross product of any two vectors connecting the three points results in the normal to the plane. If more than three points are used to define the plane, the least squares technique is used to find the best plane.

Planes calculations are particularly useful in studying the conformation of the pyrrolidine ring in proteins and the ribose ring in nucleic acids. These five-member rings are not planar, but are puckered such that one or two atoms of the ring are out of the plane defined by the rest of the five. In order to determine the exact conformation of such rings, least squares planes are carried out for various sets of ring atoms and the best one estimated as the one that has the least average deviation of the atoms from it. The puckering is then determined by calculating which atoms deviate from this plane, by how much, and in which direction.

6.3.7 Pseudorotation parameters

The concept of pseudorotation was first introduced to describe the complex conformations of large heterocyclic rings. In macromolecules it is applicable to pyrrolidine and ribose rings. The deviations of the atoms from the mean plane of the other atoms of the ring may be described in the form of a wave moving around the ring, lifting or lowering one atom and then the other. In other words the puckering is a result of the pseudorotation of the 'virtual' wave. There are two parameters that specify the conformation of the five-member rings in terms of pseudorotation. These are the phase angle P, which specifies the position of the 'wave' with respect to some origin; and the amplitude of pucker τ_m, which specifies the height of the 'wave'. These parameters are calculated from the five endocyclic torsion

angles as indicated in Figure 6.16. Note that the endocyclic torsions and the pseudorotation parameters are two equivalent descriptions of the ring conformation. The equation connecting these two is

$$\text{Tan } P = [(\theta_4 - \theta_1) - (\theta_3 - \theta_0)] / 2\theta_2(\text{Sin}36 + \text{Sin}72)$$

$$\tau_m = \theta_2 / \text{Cos } P$$

where θ_i are the endocyclic torsion angles. It is mostly the ribose ring in nucleic acids that is described using the pseudorotation parameters, though it is also sometimes applied to the pyrrolidine ring of the amino acid proline in proteins. The origin of the phase angle is set by convention. For ribose this convention is indicated in Figure 6.17. This is a picture of the so-called pseudorotation pathway, and it shows the relationship between the pseudorotation parameters and the usual '*endo*' – '*exo*' description in terms of which atom is out of the least squares plane of the other atoms.

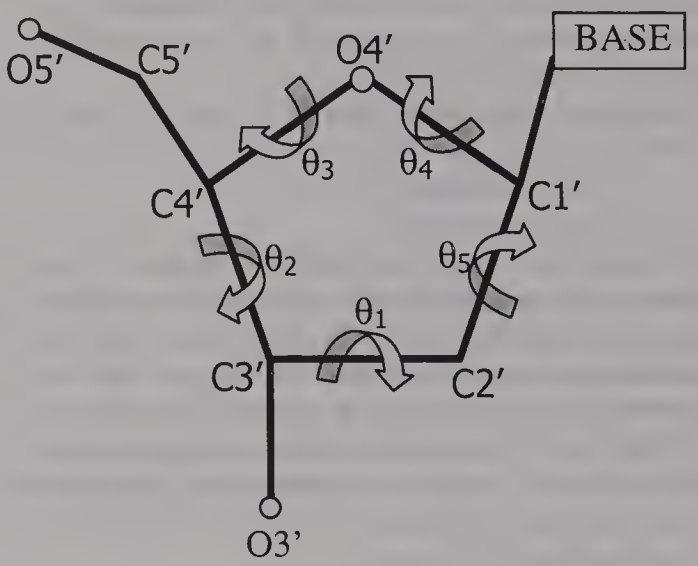


Figure 6.16. The five endocyclic torsion angle in the furanose ring

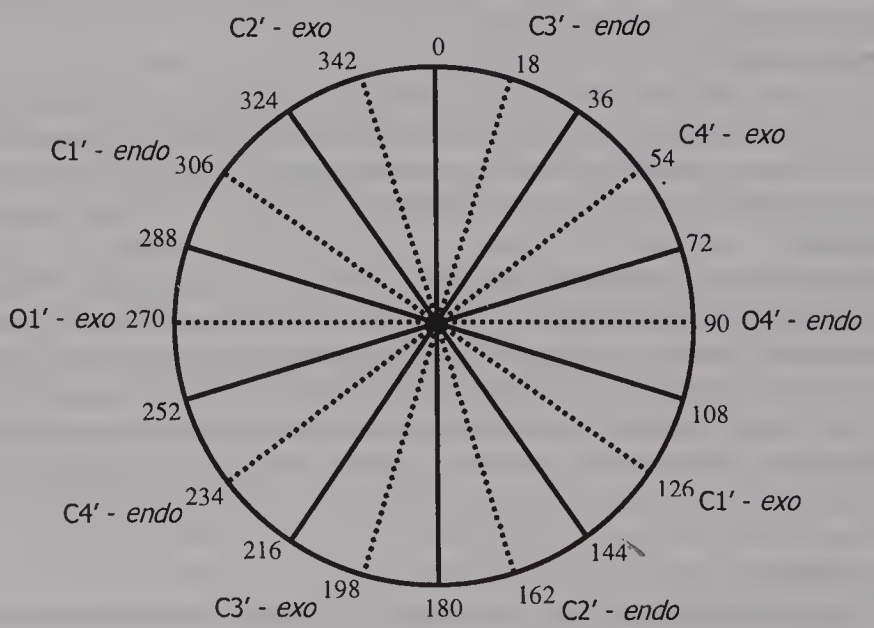


Figure 6.17 The pseudorotation circle or pathway

6.3.8 The Ramachandran map

The sequence in which the amino acids occur along the polypeptide chain closely controls how the chain will fold into the three-dimensional structure. In other words the primary structure of the protein determines its secondary, tertiary and quaternary structures and ultimately the function of the protein. The polypeptide chain is built up when the amino acids join to each other by means of a peptide bond. The peptide (C-N) bond has a partial double bond character, which means that the molecule cannot rotate about it. The six atoms that form the peptide group are constrained to lie on a plane thereby forming a planar peptide group (Figure 6.18). Ramachandran and his colleagues at the University of Madras realised that this property simplified the geometrical analysis of the polypeptide chain. If the rotational possibilities of the longer side chains are ignored, and the backbone of the protein chain alone is considered, then each residue is described by two torsion angles ϕ and ψ (Figure 6.19). The freedom of rotation about even these bonds is not absolute, but is restricted by possible steric hindrances. For example, if the $C'_{i-1} - N_i$ bond is *cis* to the $C^{\alpha}_i - C'_i$ bond when the $C_i - N_{i+1}$ bond is *cis*

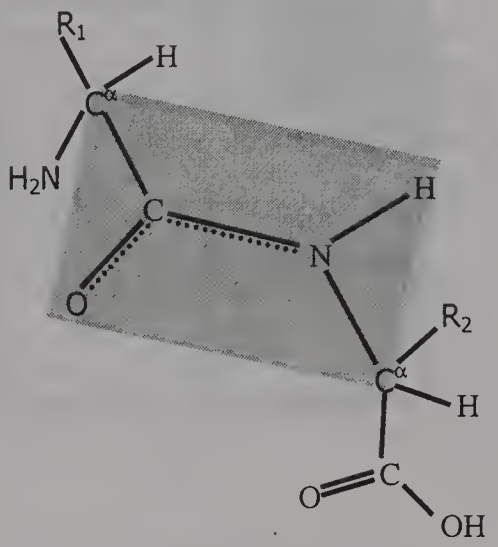


Figure 6.18 The planar peptide unit

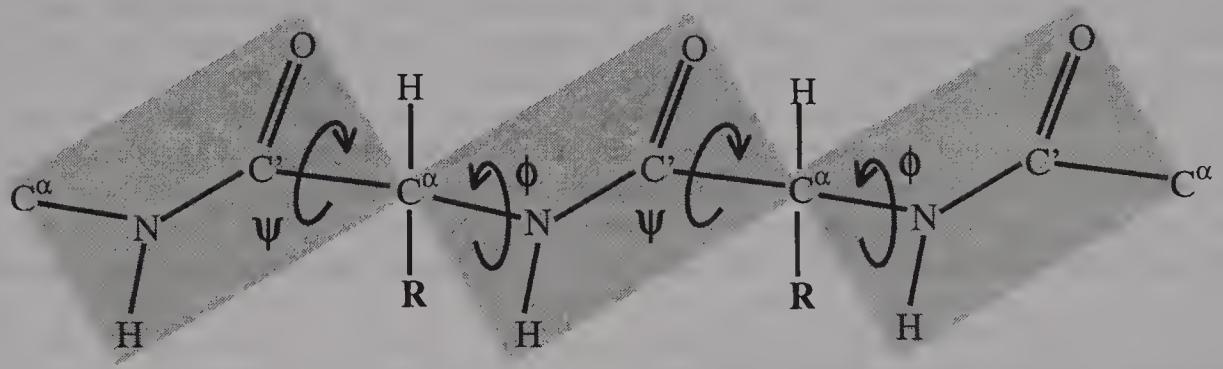


Figure 6.19 The Ramachandran torsion angles.

to the $N_i - C^{\alpha}_i$ bond, i.e. when both ϕ and ψ are 0°, then a severe clash between H_{i+1} and O_{i-1} will occur, making this particular pair of values disallowed. A systematic search of all pairs of values of ϕ and ψ reveals that only about 18% of the values are allowed. This is represented as the Ramachandran map (Figure 6.20). This map indicates the situation when the atoms are modelled as hard spheres allowing no interpenetration. The radii of the spheres were derived from the inter-atomic distances seen in the crystal structures of small molecules. Apart from these normal values, somewhat smaller values for the contact distances were also considered. These were called the extreme limits, and when they were used, the allowed area on the Ramachandran plot expanded to 22%. The map is correct for eighteen of the twenty side chains. The exceptions are glycine and proline. Glycine has a much larger allowed region owing to the absence of the C^{β} atom. The side chain of proline binds to the peptide nitrogen atom and this limits the value of ϕ to -60 ± 20°. The Ramachandran map has been fully validated by the protein structures determined since it was proposed. Over 98% of the experimentally determined values of ϕ and ψ fall within the allowed regions of the Ramachandran plot.

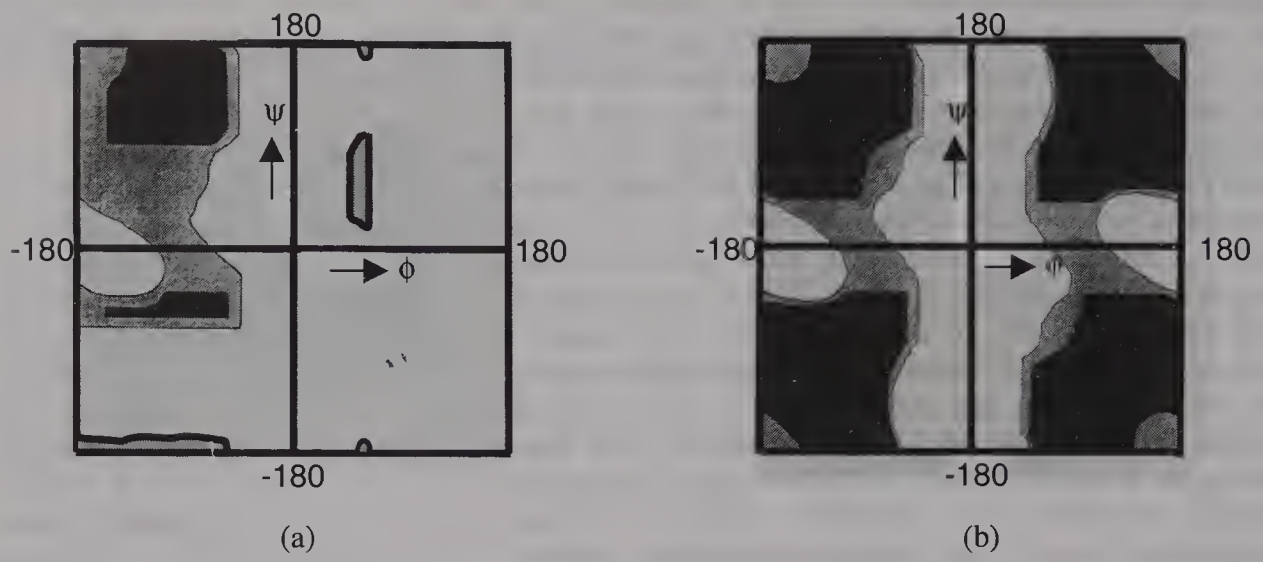


Figure 6.20 The Ramachandran map. (a) For all residues except glycine and proline. (b) For glycine alone. The allowed regions are shown in black or grey.

The greatest utility of the Ramachandran plot is as an analytical tool. Since the experimental techniques of protein structure determination are not precise, newly obtained structures are always plotted on the map to check for conformational correctness. If some of the angles fall outside the allowed regions of the map, the experimental data may have to be reinterpreted and the structure corrected. When building models of the proteins, too, the map helps to decide on possible conformations of parts of the polypeptide chain. And finally, if some of the pairs of values of ϕ and ψ which fall within the allowed region are repeated for many residues along the polypeptide chain, it leads to regular recognisable secondary structures, such as α-helices and β-sheets. Hence, the Ramachandran plot for a new structure helps to identify the secondary structural features it contains.

6.3.9 Protein surface and solvent accessibility

The surface of protein is calculated chiefly in one of two ways. The method of Connolly has been briefly mentioned in section 6.2.2 above. In this method, the surface is defined as a convex hull or 'skin' that is located as follows. First, the atom furthest from the molecular centre is determined. Then a procedure called Delaunay triangulation is used to locate two of its neighbours. These three atoms are used to define a triangular plane. A test is made to see if all the atoms of the protein lie on one side of this plane and none on the other. If this condition is not satisfied, the procedure is used again to find two other neighbours. The triplet of atoms that pass the test are considered a small triangular part of the surface. Next, using one

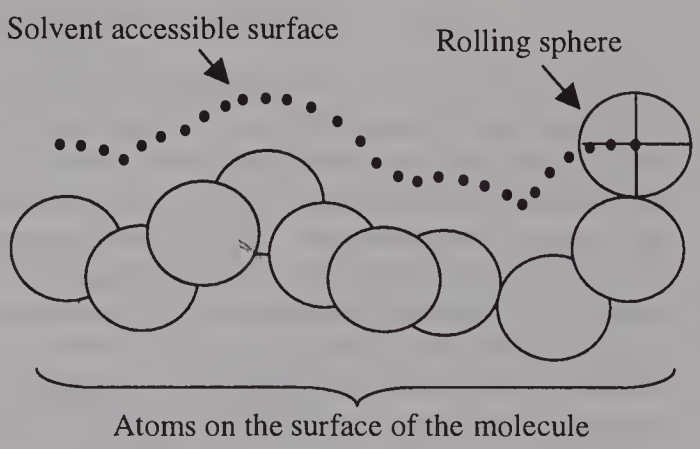


Figure 6.21 Solvent accessible surface defined by a 'rolling sphere'

of these three atoms, the procedure is repeated to obtain the next portion of the surface, and so on, until the entire protein surface has been determined.

The other method of calculating the surface involves 'rolling' an imaginary solvent molecule, considered spherical in shape, and called the probe sphere, on the surface of the protein. A map of the locus of a point on the sphere is then the surface of the protein (Figure 6.21). Depending on the size of the sphere and on which point is used, one obtains a number of different types of surfaces. When the radius is 0, and the centre of the sphere (or any point) is used, the surface is the van der Waals surface, corresponding to the intersection surface of the van der Waals spheres representing the atoms of the protein. With a probe radius of 1.4 (or 1.5) Å, and the centre of the sphere chosen for mapping, we obtain the solvent accessible surface of the protein. The value 1.4 Å corresponds to the radius of a spherically-averaged water molecule, and the surface calculated in this way represents the area on the protein surface that can be reached by the water molecule. A larger radius, say 10 Å, for the probe, yields the surface accessible to larger ligands. Instead of choosing the centre of the probe for mapping the surface, if the tangent at the point of contact between the probe atom and the protein atoms is chosen, this gives rise to the molecular surface of the protein.

6.3.10 *Nucleic acid structural parameters*

DNA forms a double helical structure. The double helix is characterised by the certain global parameters, that apply to the entire helix, and certain local parameters that are different at each base pair or each base step. Global parameters that vary from one type of helix to another are twist and rise. Twist refers to the average angle made by one base pair with respect to the next one about the helix axis. Rise refers to the average translation between two consecutive base pairs as measured along the helix axis. The values for these two parameters as averaged over thousands of base pairs are very different for different types of helix. Twist may also be expressed as the number of base pairs per turn of the helix. Similarly, rise may also be expressed as length of one turn of the helix, or pitch.

Detailed experimental studies have shown that both twist and rise vary from base step to base step, even within a single helix type, and what remains approximately constant are only their average values. This finding has lead to the concept of structural microheterogeneity in the DNA helix that is characterised by a set of local helical parameters. (Figure 8.26 illustrates some of these parameters.) They may be classified as base step parameters and as base pair parameters, or as translational parameters and rotational parameters. The translational parameters for a base step are the x and y displacements, stagger, stretch and shear. The rotational parameters for a base pair are tip, inclination, opening, propeller twist and buckle. The translational parameters for a base step are rise, slide and shift. The rotational parameters for a base step are twist, roll and tilt. Not all these parameters are important. Though the relationships among the parameters is not clear, some of the movements or displacements of the bases are chiefly responsible for the helix taking up different forms. Specifically these are x displacement, inclination, propeller twist and buckle for the base pair geometry, and rise, slide, twist, roll and tilt for base step geometry. Equations to calculate these parameters depend very much on their precise definition as well as the axial system chosen, and therefore will not be given here. Recent analyses has however shown that as long the calculations are performed in a self consistent manner, with correct choice of axes and their origin, the values obtained are the same. Software that addresses these issues and gives the correct values for these geometrical parameters is now freely available.

The parameters discussed above are in addition to the ones that have been in use even before the advent of detailed experimental structures of DNA fragments, such as the backbone torsion angles, the glycosidic torsion angle specifying the orientation of the base with respect to the ribose, and the conformation of the ribose ring itself (see section 6.2.8). Again the relationships between the backbone

parameters and the base-centred parameters are not clear. In practise all the parameters are calculated for every structure, but discussion is usually limited to only those that show some special features.

6.3.11 Software

Programs like the Insight II suite mentioned above also include utilities for geometrical analyses and for generating the Ramachandran plot for any given structure. In addition there are specialised programs designed solely for this purpose. Most geometrical calculations including bond lengths and angles, planes, possible hydrogen bonds, etc. may be calculated using the program PARST. This is an open software package that is widely used by crystallographers, chemists and biologists for geometrical calculations on the structures of small, accurately-determined molecules. PROCHECK is the similar program used especially for protein structures. An additional feature in PROCHECK is the calculation of the Ramachandran plot. For geometrical analyses of nucleic acid structures, there are several computer programs, including FREEHELIX, CURVES, NUPARM, and 3DNA. Most of them provide similar results, except for variations in respect of choice of axis and so on. Any of them may be used consistently for comparative analyses of nucleic acid structure.

6.4 Structure Comparisons

Just as sequence comparison tells us about similarities in structure and function, structure comparison yields information on functional relationships. In addition structure comparisons form the core of many classification schemes for proteins into hierarchical groups and families. In geometrical and algorithmic terms, several parameters are involved in comparing the shapes of two molecules. First of all recognizing common substructures between two highly convoluted structures, such as proteins, is in general a very complex combinatorial problem The human eye and brain are very good at recognizing shapes, but this manual technique becomes rapidly impractical when the number of molecules to be compared increase beyond just a few. Computers can perform some of these tasks very fast, but need proper software. Algorithms for structural comparisons require (i) a suitable representation of the objects of study, (ii) an objective function to be optimised, (iii) a comparison algorithm, and (iv) appropriate decision rules concerning the significance of the result. A suitable method of representing molecules is as a list of atomic positions referred to an appropriate system of axes. The objective function usually is the root mean square deviation in the positions of equivalent pairs of atoms in the two molecules, after one of them is placed correctly over the other. As for the algorithm, among the most important techniques used for comparisons of structures are the superposition methods and dynamic programming methods.

6.4.1 Superposition techniques

The most obvious way of comparing the structures of two molecules is to superpose them on top of each other and check which portions match and which do not. This however is easier said than done. In the first place the two molecules may not be chemically identical, with identical numbers of atoms connected identically. Such a situation would then entail the identification of a subset of atoms that would be present in both molecules, which could be used for superposition. In proteins, the C^{α} atoms or the polypeptide backbone atoms would suit this role ideally. If such a subset cannot be recognized, then the structural comparisons cannot be carried out at the atomic level. The primary task then would be to design a description of the structure that could be used for comparisons on the computer. This situation will not be further discussed here.

Secondly, even if a set of atoms that is present in both molecules is identified, the relative orientation and translation between the two in Cartesian space would be unknown. In general, the relationship between the atomic positions of the two identical molecules may be written as

$$\mathbf{x}' = [\mathbf{M}]\mathbf{x} + \mathbf{T}$$

where $\mathbf{x}'$ and $\mathbf{x}$ are the coordinates of the first and second molecules respectively, $[\mathbf{M}]$ is the rotation matrix, and $\mathbf{T}$ is the translation vector. $\mathbf{T}$ is easily determined. It is simply the vector connecting the centres of mass of the two molecules. Or

$$\mathbf{T} = (\Sigma\mathbf{x}' - \Sigma\mathbf{x})/N$$

where the summation is over the N atoms in each molecule. However the determination of the orientation matrix poses a major problem. Consider for example two copies of the same molecule, but with one copy rotated by 180° with respect to the other about an arbitrary axis. After the translation is calculated and performed on one of the molecules, the centres of mass of the two would coincide, but not the atomic positions. If the two structures were now compared, they would look completely different from each other, even though, in fact, they are identical. In order to determine the rotation matrix, it is necessary to have an idea of what we expect when the two are correctly oriented. This is obtained by calculating the root mean square deviations, or the rmsd, in the positions of the equivalent atoms,

$$\text{rmsd} = \text{sqrt}\{\Sigma(\mathbf{x}' - \mathbf{x})^2/N\}$$

In other words this is the square root of the average of the squares of the distances between equivalent pairs of atoms. This number should be zero when two identical molecules are correctly superimposed. If the structures themselves are different, the rmsd is different from zero, even if their relative orientation is correct. However, in this case, if the orientation is correct, the rmsd would be the least. Inverting the logic, the task may now be formulated as: find the rotation matrix that, when applied to one of the molecules after translating it, minimizes the rmsd.

There are several techniques that may be used to solve this problem, at least in an iterative fashion. We will discuss two of them here. The first one is a least squares technique that starts by defining the rotations in terms of the Eulerian angles. Eulerian angles are used to avoid the degeneracy that arises when the Cartesian axes are used to define rotations. In the latter system, a rotation of say 90° about the x-axis, followed by another of the same magnitude about the z-axis, leads to exactly the same orientation for the object as a rotation of 90° about the z-axis followed by one of 90° about the y-axis. Indeed there are several other combinations that would lead to the same result. This is true not only of right angle rotations but of other rotations as well. This degeneracy may be removed by specifying the rotations with respect to an axial system that is fixed in the object and moves along with it at each rotation, thus 'keeping track' of the rotation space already sampled. In the Eulerian system (Figure 6.22), the first rotation is defined with respect to the x-axis, the second with respect to the rotated y-axis, i.e. the y'-axis and the third rotation is with respect to the newly rotated x"-axis. Note that the first and the third rotations are performed around the 'same' axis, except that it has been rotated in between.

Casting the problem in this form then involves obtaining the rotation matrix in the Eulerian axis system that would minimize the rmsd between equivalent atoms when applied to one of the two molecules whose structures are to be compared after translating it appropriately. The least squares techniques may be used to refine the matrix, once a reasonable approximate guess is made. This guess normally derives from a visual inspection of the two molecules. Alternatively, the matrix required to superpose three atoms of one molecule on the three equivalent atoms of the other is taken as the first approximation. This initial matrix is refined in an iterative fashion by the least squares technique, using the rmsd in atomic position as the quantity to be minimized. At the end of this procedure, the best rotation matrix is obtained. When the translation, as defined above, and this rotation are applied to one of the molecules of the pair, it is superposed on the other. The superposed molecules may be

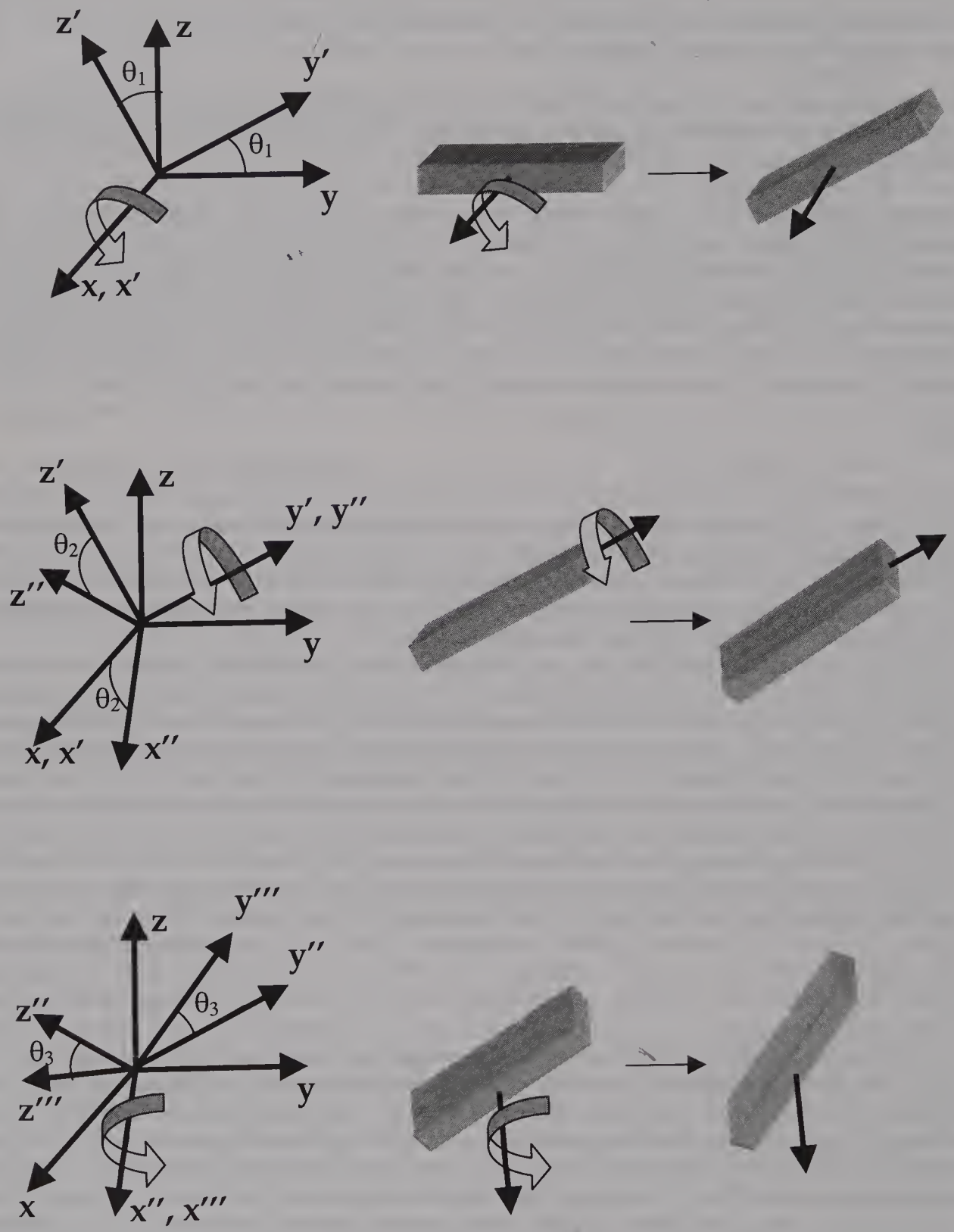


Figure 6.22 Eulerian axes of rotation

viewed to estimate visually their similarity (or otherwise). The rmsd in the atomic positions may be calculated to obtain a quantitative estimate of this similarity.

The second method to obtain the rmsd does not define the rotations with respect to the Eulerian axis. Instead, it uses a quaternion transformation. It has been shown that the rotations about the three Eulerian (or Cartesian) axes are mathematically equivalent to a single rotation about another axis that remains to be determined. The direction cosines specifying this axis constitute three unknowns, and the rotation angle about it is the fourth. These four unknowns (called a quaternion) replace the three unknown angles in the other axial systems. While this may appear to be making the problem more complicated, in fact it is not so. It is easier to determine the axis and the angle than the three angles, since the equations for the later are much simpler. Once again, a least squares technique is used to iteratively minimize the rmsd by varying the axis and the angle of rotation. When the procedure has converged the parameters obtained may be applied to one of the molecules, after translation, to superpose it on the other. Again the rmsd is a measure of the similarity or otherwise of the two molecules. Though the equations in this method are simpler to set up and solve, its effectiveness does not extend to large molecules with all their complexities. This is chiefly because it does not allow any user intervention to set the initial values, and therefore the rmsd may be an absolute minimum, but may not have much biological relevance. To elucidate this further, consider the case of two large proteins that have a small portion of their structures the same. In the first method, the initial guess at the rotation matrix may be made to emphasize this partial similarity. The second method would only reflect the fact that large portions of the two molecules are dissimilar and thus miss out what may well be important biological information.

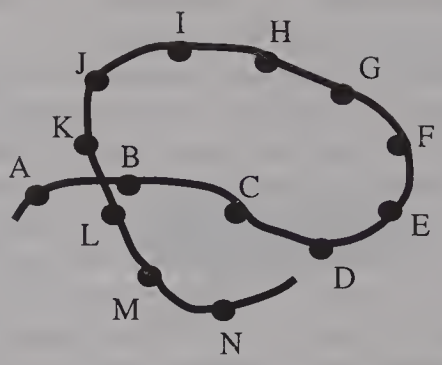
6.4.2 Distance matrices

Distance matrices[19] are an interesting way of converting the three-dimensional information present in the molecular structure into a two-dimensional map, while at the same time enabling analyses of the relationships among the structural elements. The distance map may also be used for structural comparisons, but this will be explained in the next section.

A distance matrix is drawn similar to a dot matrix comparison of a sequence with itself. Conventionally, the sequence of the protein (or RNA) is written from left to right across the bottom of the matrix, as well as from bottom to top on the left side (Figure 6.23). Each element of the matrix so defined is labelled by two residues, on from the top and one from the left. Each element of the matrix is the distance, in Angstroms, between the pair of residues labelling it. The position of the C^{α} atom of each residue may be taken as defining the position of the residue. Sometimes the position of the C^{β} atom is taken instead. If the structure is known at a high resolution with knowledge of the positions of all the atoms, the centre of mass of each residue may be calculated and used to define the residue position. The advantage of using the C^{α} positions lies in the fact that even at low resolutions, the protein backbone trace is defined at the very least by these coordinates. Since distance matrices are used also in structure comparisons, employing C^{α} positions to define the residue position enables the comparison of two structures at different resolutions. A distance map is symmetrical about its diagonal, since the same protein appears on both the axes. Thus it is normal to represent only one triangular half of it, usually the lower one.

The interpretation of a distance matrix is aided by contour lines that are drawn through points representing the same inter-residue distance. Thus one may draw a contour through all points corresponding to an inter-residue distance of 5 Å, another through points corresponding to 10 Å, a third at 15 Å, etc. Such a diagram would clearly indicate spatial relationships between the various parts

[19] We have used the term 'distance matrix' earlier, in Chapter 4, to indicate the degree of similarity between sequences. Here we refer to a matrix of distances in 'real' three-dimensional space.



N	8.0	7.5	4.0	6.0	7.0	9.0	7.5	***10.0***	***12.5***	***11.0***	8.5	6.0	3.0	0
M	6.0	6.0	5.0	7.0	7.5	9,5	8.5	***10.5***	***10.5***	***9.0***	6.0	3.0	0	3.0
L	**3.0**	**3.0**	6.0	9.0	8.5	10.0	9.0	8.0	8.5	6.0	3.0	0	3.0	6.0
K	**3.0**	**3.0**	7.0	9.5	9.5	10.0	9.0	6.5	5.5	3.0	0	3.0	6.0	8.5
J	5.0	6.0	7.5	10.5	10.0	11.0	8.5	5.5	3.0	0	3.0	6.0	***9.0***	***11.0***
I	7.5	7.0	7.5	9.0	9.0	9.0	6.0	3.0	0	3.0	5.5	8.5	***10.5***	***12.5***
H	8.0	7.5	7.0	7.5	7.0	6.0	3.0	0	3.0	5.5	6.5	8.0	***10.5***	***10.0***
G	10.0	8.5	7.5	6.0	5.5	3.0	0	3.0	6.0	8.5	9.0	9.0	8.5	7.5
F	10.5	9.0	7.0	5.5	3.0	0	3.0	6.0	9.0	11.0	10.0	10.0	9.5	9.0
E	10.5	8.5	5.5	3.0	0	3.0	5.5	7.0	9.0	10.0	9.5	8.5	7.5	7.0
D	8.5	6.0	3.0	0	3.0	5.5	6.0	7.5	9.0	10.5	9.5	9.0	7.0	6.0
C	6.0	3.0	0	3.0	5.5	7.0	7.5	7.0	7.5	7.5	7.0	6.0	5.0	4.0
B	3.0	0	3.0	6.0	8.5	9.0	8.5	7.5	7.0	6.0	**3.0**	**3.0**	6.0	7.5
A	0	3.0	6.0	8.5	10.5	10.5	10.0	8.0	7.5	5.0	**3.0**	**3.0**	6.0	8.0
	A	B	C	D	E	F	G	H	I	J	K	L	M	N

Figure 6.23 A schematic example of a distance matrix. Two regions are marked out. The dotted line marks out residues that are far in sequence but close in structure. The solid lines mark out residues that are far in structure.

of the molecule. In many cases the distance matrix (or distance map) is contoured at just two or three levels using different coloured dots. Thus Figure 6.24 is a distance map of the native structure of myoglobin with the light regions representing small inter-residue distances, and black representing distances greater than 25 A. Myoglobin consists chiefly of alpha helices, which are represented by bands of light regions close to the diagonal. A pair of helices that fold parallel to each other shows up as bands parallel to the diagonal, while a pair that folds antiparallel to each other shows up as a band perpendicular to the diagonal. Note that the internal organisation of the diagram is such that close contacts that arise from the secondary structures occur close to the diagonal, while tertiary and quaternary structures are more distant.

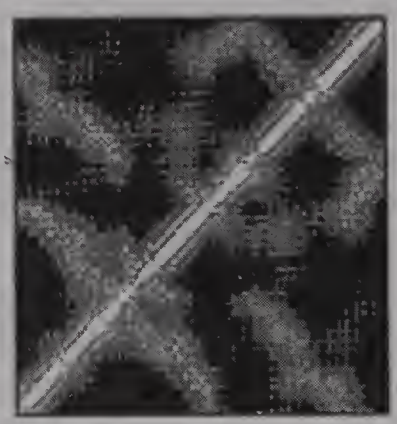


Figure 6.24 The distance map for Myoglobin

The diagram may be used to identify structural domains in the molecule. A structural domain may be considered as a group of residues that are closer to each other in three-dimensional space than they are to all other residues in the molecule. Domains usually have a functional unity, with each domain corresponding to one function or one aspect of the function. Figure 6.25 is the distance map for calmodulin, contoured as in Figure 6.24. Calmodulin has two functionally and structurally separate domains, connected by a long alpha helix. This arrangement shows up clearly on the map.

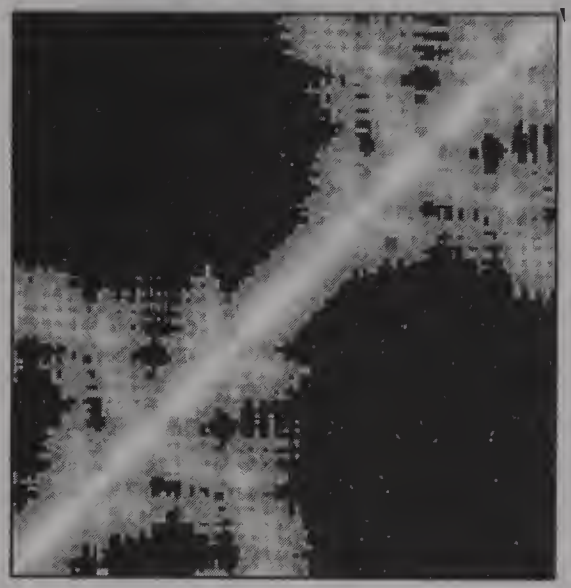


Figure 6.25 Distance map of Calmodulin

Distance matrices are also useful to represent the near globular structures of transfer and ribosomal RNA. Here the positions of the phosphorous atoms, or those of the carbon atom of the glycosidic bond are used to represent the position of the residue.

A useful variant of the distance map is the difference distance map, which can be used to compare the structures of two proteins. Here two maps are drawn, one each for structure. The two maps are superimposed and the difference of the distance value in each cell is represented in the difference map. In other words, the difference map represents the value D_{ij} for each cell (i,j), where

$$D_{ij} = r_{ij}^{A} - r_{ij}^{B}$$

Here r_{ij}^{A} and r_{ij}^{B} are the distances between the ith and the jth residue for two proteins A and B respectively. Obviously if the two structures are very different in sequence and structure to begin with, the difference map would be a case of 'comparing apples and oranges', i.e. unrelated numbers. Thus, an analysis using difference maps is really useful only when the structure of the same protein has been, say, determined by more than one technique, and these two structures are compared.

Two distance matrices may be shown in a single diagram by using the inherent symmetry of the matrix. The triangular top half is used for one of them and the lower half for the other. Such a diagram may be used to compare two proteins that have different sequences, but have some structural domains in common. Figure 6.26 shows an example for two proteins in which the so-called 'zinc-finger' domains are structurally equivalent. The top half is the distance map for a molecule called the tramtrack protein (PDB entry 2drpA), the bottom half corresponds to the human enhancer binding protein (PDB entry 1bbo). Though the similarity is already visible in Figure 6.26a, it is very clear in Figure 6.26b, in which the distance matrices have been brought into register by keeping only rows or columns corresponding to structurally equivalent residues.

An extension of this idea leads to an algorithm for quantitative comparison of protein structures, which has been implemented in a program called DALI. The principle underlying the algorithm is that similar three-dimensional structures have similar inter-residue distances. The problem is that often only a small portion of the structures of two proteins is similar, and such similarity may be hard to find if the distance maps of the complete structures are compared. To overcome this problem, the distance matrix for each protein is decomposed into overlapping sub-matrices (Figure 6.27). Each of the two structures to be compared is now represented by a set of distance sub-matrices. The size of the sub-matrix is chosen to represent comparisons between the structure of a hexa-peptide from one protein with a hexa-peptide from the other. The comparison proceeds by calculating a difference distance map for all pairs of sub-matrices, one from each protein. Whenever a particular pair has a sufficiently small value for the difference, it is considered a possible candidate to be included as equivalent

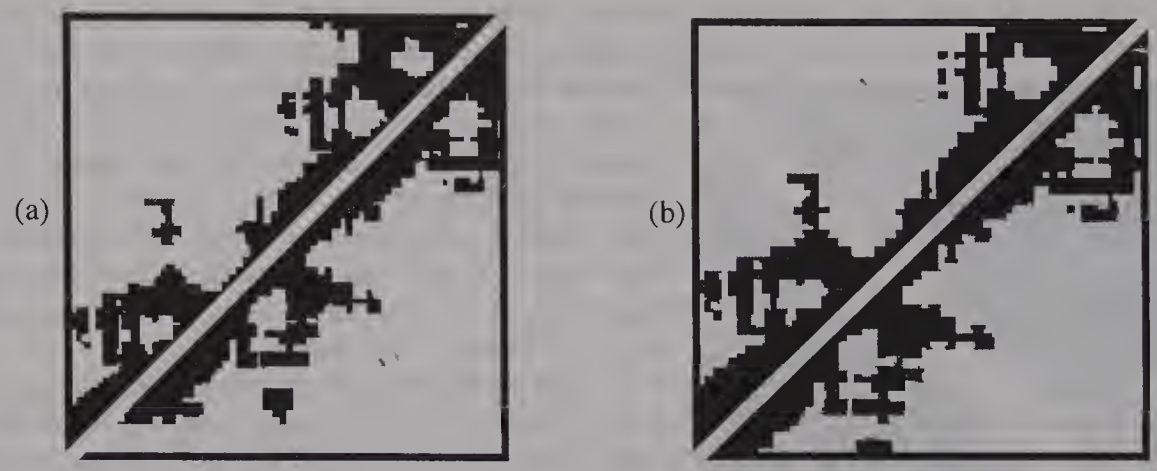


Figure 6.26 Comparison of two structurally similar proteins, namely tramtrack protein and human enhancer binding protein (a) Both complete sequences are shown – upper left, tramtrack protein; lower right, human enhancer binding protein. (b) The sequences have been aligned to show the similarity in their respective structures.

sub-structures in the final structural alignment. A list of all such candidates is prepared. The next step in the comparison is to start from a pair of fragments that have been identified as equivalent, and to build up a chain of connected fragments that are structurally equivalent. This takes place by an iterative method that uses random walks and the optimisation of many possibilities in parallel. Finally the best possible alignment of the largest number of substructures gives the best alignment of the two proteins, and, at the same time, brings out any possible similarities in the structure, in whatever way these similarities may be permuted in the actual structures. Thus if two well-defined substructures A and B occur in the order AB in one protein and BA in another, DALI will detect this occurrence, and show the way in which the first protein is related to the second. Quantitatively, DALI evaluates a similarity score as

$$S = \Sigma_i \Sigma_j \varphi(i,j)$$

where the summation is over i = 1,L, j = 1,L. i and j label pairs of matched residues considered structurally equivalent. Thus i stands for the pair of residues (iA, iB), where the two proteins being compared are A and B. $\varphi(i,j)$ is a measure of similarity between the two proteins and is in general a function of the distances $d_{i,j}^{A}$ between residues i and j in protein A, and distances d_{ij}^{B} in protein B. (To elucidate this point fully, consider that a particular sub-structure in protein A is matched with another sub-structure in protein B. The length of the two matched sub-structures is the same, say L. Then i and j correspond to the first and last residues respectively of the substructures both in protein A as well as in protein B. Note that the two substructures may occur anywhere along the sequence in either protein, being, for example, at the beginning of the sequence in one, and at the end in the other.) DALI optimises the structural comparison so that the largest value of S corresponds to the best set of matches of the residues in A with those in B. There are two principal ways of defining $\varphi(i,j)$. In one of them

$$\varphi(i,j) = \theta^{R} - |d_{i,j}^{A} - d_{i,j}^{B}|$$

Here θ^{R} is set as 1.5 Å and is considered the zero level of similarity. $d_{i,j}^{A}$ and $d_{i,j}^{B}$ have been defined above. This is called a 'rigid' similarity score, and is especially useful to search for the occurrences of a predefined structural pattern. The other expression used is

$$\varphi(i,j) = \{\theta^{E} - [|d_{i,j}^{A} - d_{i,j}^{B}|]/ d_{i,j}^{*}\} w(d_{i,j}^{*}) \text{ for } i \neq j, \text{ and}$$

$$\varphi(i,j) = \theta^{E} \text{ for } i = j$$

This expression uses relative deviations rather than absolute deviations, and is therefore more tolerant of small changes in the angles and geometry that could lead to large changes in the distances over even

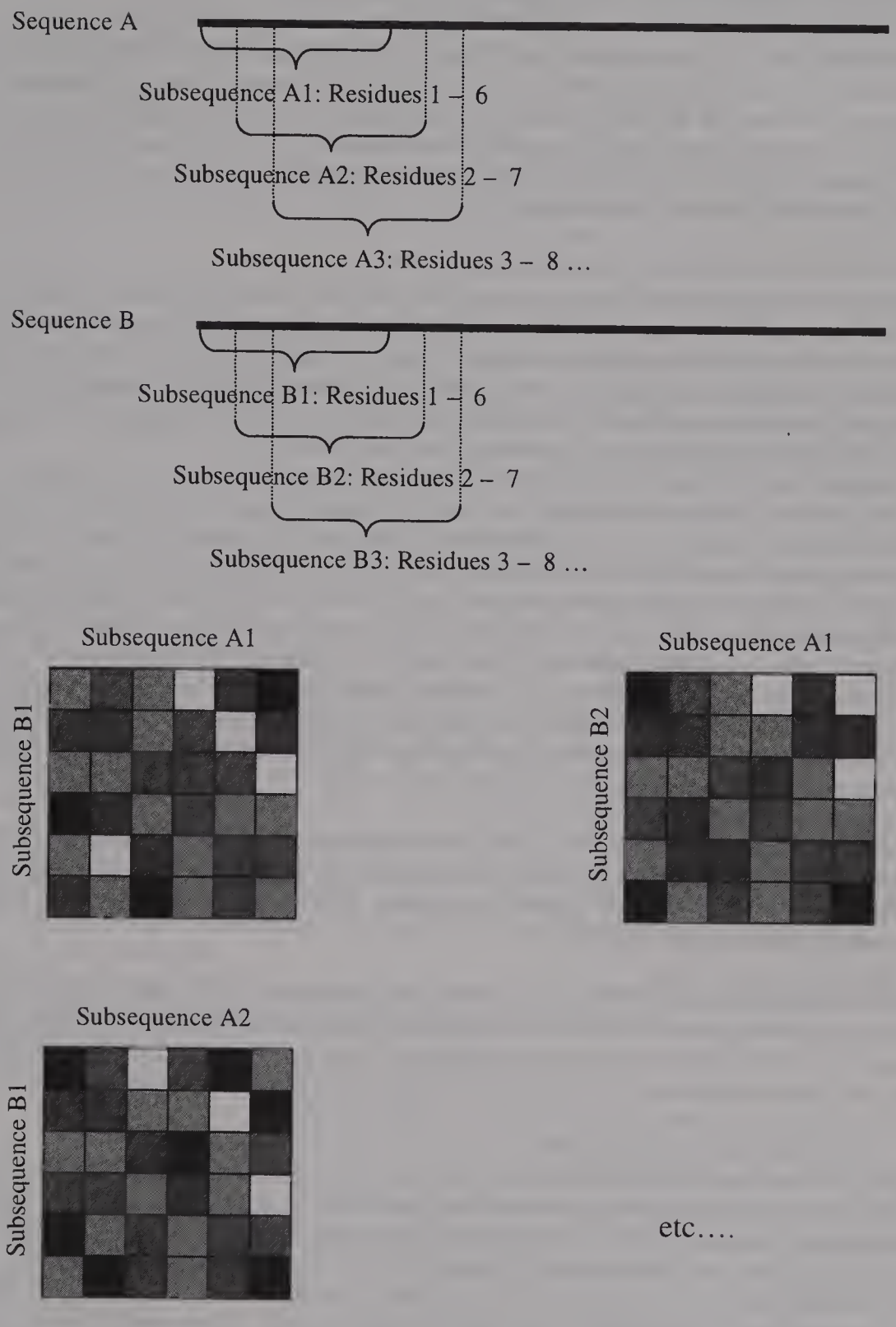


Figure 6.27 Decomposing the distance matrix into overlapping submatrices

a small length of the protein chain. $d_{i,j}^{*}$ is the average of $d_{i,j}^{A}$ and $d_{i,j}^{B}$, θ^{E} is a similarity threshold set equal to 0.20, i.e. a 20% deviation. w is a weighting function set as

$$w(r) = \exp(-r^2/\alpha^2)$$

where α = 20 Å, which is the radius of an average protein structural domain. The idea behind the weighting function is that when the distances d_{ij} are large, there are many more such distances but they are not unique. Without the weighting they would contribute a large sum to the similarity score even though the structures may not actually be similar. The similarity score S is maximised by means of a so-called 'greedy' algorithm that checks for all possible pairs of substructures, and builds up the structural alignment from a well-chosen starting pair.

6.4.3 Dynamic programming using distance matrices

The method used in DALI to find the optimum alignment of the two structures under comparison is a general combinatorial optimisation method, and not a dynamic programming method. In order to avoid a 'combinatorial explosion', or, in other words, in order to avoid having to sift through impossibly large numbers of possible combinations of similar sub-structures, certain approximations have to be made. This implies that the search is not complete, but may only be the best possible one subject to the various constraints, especially the one of optimising computational costs.

In contrast, the program SSAP uses a dynamic programming algorithm that ensures a search through all possible valid combinations of matching substructures in a reasonable amount of computation time. In Chapter 3 we encountered the dynamic programming method as applied to sequence comparison in the Needleman-Wunsch and the Smith-Waterman technique. The essential point about that particular application was that the objects to be compared were both one-dimensional strings of symbols. One could therefore simply compare one symbol against another, to build up a matrix of similarities and dissimilarities. This matrix was then quickly searched for the optimal alignment. To apply the same method to compare protein structures, therefore, it is first of all necessary to design a way of representing the three-dimensional structure in a single dimension as a string of symbols. One way of doing this is to associate with each residue in the protein sequence the set of distances to the other residues in the protein. Or

$$s_i = \{d_{ij}\},\ j = i\text{-}n,\ i+n$$

where n may be chosen to cover all the residues in the protein, or else only a subset of it. To use this representation of the protein chain for structure comparison, we define a measure of similarity between two residues, namely i from protein A and k from protein B, A and B being the two protein structures to be compared. This measure is written as

$$S_{ik} = \sum |^{A}d_{i,i+m} - {}^{B}d_{k,k+m}|$$

where the summation is over m = -n to +n. Generally the dynamic programming algorithm may be applied to obtain an alignment either with a minimum of the selected function or a maximum. In the present case we are looking for the greatest similarity between the structures, and the score should be a maximum for the best alignment. It is therefore necessary to recast the above measure of similarity to reflect this criterion as follows.

$$S_{ik} = \sum a / (|^{A}d_{i,i+m} - {}^{B}d_{k,k+m}| + b),$$

where a is a constant chosen to limit the maximum possible score, and b is constant chosen to prevent division by zero. (The values a = 50 and b = 2 to 5 were found suitable). Once again the summation is over m = -n to +n, implying a total of 2n distance comparisons, with each comparison being centred on one residue in protein A and one residue in protein B. S_{ik} is thus a measure used to compare position i in protein A with position k in protein B. Now a matrix of comparisons may be set up, and then the dynamic programming method may be used to trace the best path, i.e. the best alignment through it. This matrix is constructed by writing the sequence of the first protein on the top from left to right, and the sequence of the second protein on the left, from the top to the bottom. Each element in this matrix

is labelled by one residue from each protein sequence. The matrix element is S_{ik} as defined above. Now the dynamic programming method is carried out to obtain the best alignment, by first of all updating all the matrix elements. This is done by starting from the element at the lower right hand corner of the matrix and resetting the value of each element according to the following recurrence formula.

$$S_{ij} = S_{ij} + \text{maximum of } \{ S_{i+1,j+1} ; \text{maximum of } ([S_{k,j+1} - g], k = i+2, N_A) ; \text{maximum of } ([S_{i+1,l} - g], l = j+2, N_B) \} \quad (6.1)$$

Here g is a gap penalty applied to compensate for missing residues in any one of the pair of proteins, N_A and N_B are the numbers of atoms in proteins A and B respectively. Note that this is just a straightforward application of the Needleman-Wunsch algorithm detailed in Chapter 3. Normally this is a global alignment method, but it can be very easily transformed into a local alignment technique, such as for example, the Smith-Waterman algorithm. In that case, small portions of one protein that are similar to small portions of another may also be detected.

However, a serious drawback of the method that has not been addressed in the above is the following. The measure of similarity between a position in protein A and a position in protein B is given by S_{ik}, which is calculated by summing the difference of the distances between the reference residue and the 2n residues on each side of it, in each protein. The problem here is that if the range is large, i.e. more than about 15 residues, and one of the proteins has an insertion or deletion of one or more residues in this range, then S_{ik} will not reflect true equivalences between the two positions, if any. Consequently, the method might not be successful in comparing the structures of entire proteins, or of even two domains. A solution to this problem lies in applying the dynamic programming method at a lower level of distance comparison, to select the best possible value of S_{ik} for the pair of residues being compared. The range of summation in this case is the entire length of the respective polypeptide chain. In this case, first a matrix is constructed for each possible pair of residues, i.e. a total of (N_A x N_B) matrices, containing the differences of distances between each residue of one pair and the others of the respective chain. For example consider the structures of two 'proteins' HSERRVF and GQVMAC. To derive a score for the structural similarity between residue H and residue G, a matrix is constructed in which the first element represents the difference of the two spatial distances, d_{GG} (G to G) in one structure and d_{HH} (H to H) in the other. This would obviously be zero. The next element would have the value of the difference between the distances d_{HH} (H to H) in the first structure and d_{GQ} (G to Q) in the second. The next element is the difference between the distances H to H in the first structure and G to V in the second. In this manner the entire matrix is calculated. Note that the diagonal elements are not all of them zero. Thus the second diagonal element is the difference in the distance H to S in one structure and G to Q in the other.

Now the dynamic programming algorithm is applied to this matrix to obtain the best structural equivalence score S_{HG} for the residue pair (H,G). This entails, first of all, an application of a recursive formula similar to equation (6.1).

$$S_{ik} = d_{ik} + \text{maximum of } \{ [a / (|^{A}d_{ij} - {}^{B}d_{kl}| + b)] ; \text{maximum of } ([S_{k,j+1} - g], k = i+2, N_A) ; \text{maximum of } ([S_{i+1,l} - g], l = j+2, N_B) \}$$

A trace-back procedure, as explained earlier, yields the best score for the structural equivalence between this pair of residues. This score is then accumulated in another matrix to which a second level dynamic programming procedure will be applied. Similar scores are obtained for each possible residue pair selected from the two sequences, by applying the identical procedure to each of the (N_A x N_B) matrices, and each time the score so obtained is placed at the appropriate element of the second level matrix. Once this matrix has been constructed, the dynamic programming procedure will show the best structural alignment between the two proteins including any possible gaps in the sequence. The score

for this alignment is used as a measure of structural similarity between the two proteins, and may be used in classifying protein structures.

The dynamic programming method of comparing protein structures is fast, stable and gives consistent results. It makes allowances for gaps and for domain movements. The major drawback of this method is that the comparisons are sequential. If the sub-structures are permuted, the underlying similarity will be missed out. For example if one protein has sub-structure A followed by sub-structure B, and another has B followed by A, the method will miss out their similarity, even if A, B and their juxtaposition are identical in the two structures.

6.4.4 Generalised dynamic programming using a set of residue properties

COMPARER is a program for more than just comparison of structures. It incorporates a host of other chemical, physical and structural properties associated with each residue. The comparison between two proteins is then performed between the entire set of these properties, using a dynamic programming algorithm. A protein molecule is defined as a hierarchy of structures, viz. primary, secondary, super-secondary, motif, domain or the entire protein. At each level the protein is modelled as a string of elements arranged sequentially. For example at the primary level, the protein is a string of amino acids, and at the secondary level it is a string of secondary structures such as helices, strands and coils, again arranged sequentially. At every level, each element is associated not only with a set of chemical, physical or structural properties, but also participates in interactions and relationships with other elements at the same level, such as hydrogen bonding, geometrical interactions, etc. This complex and comprehensive set of parameters is reduced to a set of values associated with each residue. A residue-by-residue comparison is then carried out by comparing these values.

The program defines a matrix W, each element W_{ij} of which represents the summation of the differences in the properties at all levels between residue i and residue j. The dynamic programming algorithm is applied to this matrix. The properties used in the program are as follows.

1) Residue identity: The score is set to one if residue i is the same as residue j, zero otherwise.
2) Residue property: A set of five physical and chemical properties (i.e. hydrophobicity, turn preference, residue bulk, residue refractivity and anti-parallel strand preference), that have been estimated to be specially characteristic of the residue, are chosen. A single number is calculated for the entire set by appropriate weighting, normalisation and summation. The score contributed by this property is the difference between residue i and residue j in this number.
3) Residue local fold: This number is based on the distance between the C^{α} atom of the ith residue and those in a small length on either side of it. A similar number is calculated for the jth residue in the other protein. The score is the difference in the two numbers.
4) Residue distance from molecular centre: The distance of the C^{α} atom of the ith residue from the centre of gravity of the molecule is an indication not only of its relative position in the protein, but also of the compactness of the protein. The score is the difference between this number and a similar number calculated for residue j of the other protein.
5) Orientation of the side chain relative to the molecular centre of gravity: This number is defined by the angle between two vectors. The first is from the centre of gravity to the C^{α} atom of residue i. The second vector is from the molecular centre of gravity to the centre of gravity of the side chain. A similar angle is calculated for residue j of the other protein, and the score is the difference between the two angles.
6) Orientation of the side chain relative to the main chain: This is the again an angle between two vectors. One vector is from C^{α} of residue i to the side chain centre of gravity. The other is from C^{α} of residue i-1 to C^{α} of residue i+1. Once again the score is the difference of this angle with the one for residue j of the second protein.

7) Orientation of the main chain relative to the molecular centre of gravity: This score is the difference between the angles made by the vectors from the molecular centre of gravity to the C^{α} with the main chain direction at residues i and j in the two proteins respectively.
8) Side chain solvent accessibility: This is the difference in the side chain surfaces accessible to the solvent of residues i and j in the two proteins respectively.
9) Main chain solvent accessibility: A similar number is calculated for the difference in the main chain surface accessible to solvent.
10) Main chain torsion angle φ: This is one of the two angles defined by Ramachandran as being characteristic of the backbone fold of a polypeptide. The score finds the difference in the value of this angle at residue i and j respectively. The difference is always normalised to lie between 0 and 180°.
11) Main chain torsion angle ψ: This the second of the pair of Ramachandran angles. Again the normalised difference is taken to be the score.
12) Absolute position of the residues in space: This is the difference in the positions of the C^{α} atoms of residue i of protein A and residue j of protein B, after an initial least squares superposition of the two proteins. A and B are the two proteins being compared.
13) Main chain directions: The score is the difference in the direction of the vectors spanning C^{α} atoms within two segments that are centred on the ith and j^{th} residues of protein A and B respectively, after they have been superposed.
14) Various properties of the secondary structure element to which the residues belong, including type of structure, average accessibility of the main chain and the side chains of all residues in the segment, distance and orientation of the secondary structure element from the molecular centre of gravity, etc. All these values are associated with every residue in the segment. The score is thus the difference in each of these values for residue i and residue j in the two proteins respectively.
15) The final set of values associated with the residues are the relationships between them, such as hydrogen bonds, donor/acceptor relationships, van der Waals interactions, electrostatic interactions, etc. These are calculated for residues or segments, as appropriate, by simulated annealing along with a generalised dynamic programming algorithm.

After defining the residues and secondary structure elements, their properties and relationships are defined. The relationship terms are refined and obtained by the simulated annealing procedure. Property terms are calculated directly for the residues. These four set of quantities are used to define the matrix W. Application of the dynamic programming procedure to this matrix yields the best alignment between the two proteins, in terms of structure as well as in terms of the biophysical chemistry of the residues. The alignment may be carried out for pairs of structures, or with slight modifications, for a set of structures.

6.4.5 Other structure comparison methods

- Clustering: One of the many other methods that are available for comparison of structures is a clustering algorithm that starts by identifying similar segment in the two proteins being compared. The proteins are first divided into overlapping segments of about five or six residues each. Each segment from one protein is compared, by least squares superposition, with each segment from the second and a similarity score is calculated for each comparison. The segment pair with the best score is selected. This is then united with another pair. Thus, if (S_{Ai},S_{Bj}) is the best pair, the union of this with another pair (S_{Ak},S_{Bl}) would lead to $(S_{Ai}+ S_{Ak},S_{Bj}+S_{Bl})$. Here A and B refer to the two proteins being compared. Note that it is the structures that are united and not the sequences. Each segment is represented by a set of coordinates, and the union leads to a larger set of coordinates corresponding to the two segments being united. Due to this way of defining the union, the amino

acid sequence is not preserved, and the new unified segment would not have amino acids contiguously in sequence. The superposition is now refined and a new similarity score is calculated. This is repeated for all possible pairings of (S_{Ai},S_{Bj}) with the other remaining pairs. The union with the best score is chosen as the correct combination of segments at this step. The next step is a repetition of this. Again the best pair is united with each of the remaining pairs and once again the union with the highest similarity score is chosen as the best one at this step. The cycle is repeated until the similarity score falls below a threshold or until no more pairs remain that can be added to the structural comparison. Finally, the entire superposition may be refined and a similarity score calculated. This method is good at identifying a small fragment or domain in one protein that is structurally alike to a similar sized portion of another protein.

- Computer vision: A major problem with many of the methods described above is that they are best when comparing structures of contiguous amino acid sequences. To overcome this problem, a geometric hashing algorithm has been developed that uses techniques derived from computer vision to compare protein structures. It consists of three steps. In the first step a small subset of similar substructures is found in the two proteins. This will form the initial match. In the next step a superposition of the two similar substructures is carried out. In the third and final step the initial match is extended by adding new similar portions to the match and by re-evaluating the superposition after each addition. The first step is carried out using a technique of computer vision called geometric hashing. There are two parts that make up this step, namely pre-processing and recognition. If the problem is defined as one of searching a target structure for similarities with a query structure, in the pre-processing stage, the target structure is reformed as a geometric hash table. In Chapter 3, we have already introduced a hash table as used in sequence comparisons. In general a hash table is a list of occurrences of specific small patterns in the entire object. In the present case, the object is a structure and the patterns are sets of distances between the atoms. The keys or the column labels in the hash table are chosen to be a specific triangle of distances, and all atom triplets that satisfy this triangle are entered under that key into that column. If a database of structures has to be searched, the entire database may be represented in the hash table. The hash table has to be calculated only once and may be repeatedly used for searches, thus saving time. When a query structure has to be compared with the target, the hash table is used instead. The query structure broken down into a hash table of its own. The query hash table is then compared with the target hash table. If there are greater than about seven triangles of distances that are similar in the two tables, an initial match is made. There could be many such matches and some of them may not correspond to a match of the actual three-dimensional structures. However the best of these matches is chosen as the initial match and a three-dimensional superposition of the two structures is made. The structural matches may then be extended.
- Graph Theory: In this method the protein structure is defined purely in terms of its secondary structure, or more specifically, in terms of the helices and sheets contained in it. The secondary structural elements are first assigned automatically to the protein. Next an axis is fitted to each secondary structural element. The axes have the same length and orientation as the secondary structures, and the complete set of axes may be used to represent the entire structure. Once the axes through all the secondary structures in a protein have been found, a matrix of interactions between the elements is calculated for each protein. The interactions between a pair of elements are measured in terms of the angle between the pair of axes, the distance between their midpoints and the distance of closest approach. Such a matrix may be considered, in terms of graph theory, as a 'labelled graph', with the nodes of the graph corresponding to the linear representations of the helices and strands, and the edges of the graph to the inter-line distances and angles. Many algorithms are available to identify whether a certain sub-graph is contained within a graph. If the query structure is also represented in the form of a graph, one of these so-called sub-graph

isomorphism algorithms may be used to see if any part of the query structure is contained within the target structure. If there is a need to search through a database of structures for a particular motif or substructure, the database can be reduced to a set of graphs, which can be then be used as the target in the search algorithms. The program package POSSUM incorporates these methods. It uses the Ullman. sub-graph isomorphism algorithm for searching. This is a depth-first, tree-searching algorithm that compares successively larger sub-graphs from the query structure. In other words, it looks for a small match, and then keeps building on this initial match to make it larger and larger until it can be extended no longer. Only then it moves on to another initial match to repeat the procedure of build up. The final best match is then chosen from all these possibilities.

- Another method that uses geometrical relationships between secondary structure elements (SSEs) to define protein structures and substructures and use them in structure comparison is implemented in the database SARF2. For each pair $\{S_i,S_j\}$ of the SSEs from a protein five parameters are calculated. These are: the angle G_{ij} between them, the shortest distance between their axes, the corresponding closest points on the axes (C_{ij} on the axis of the S_i and C_{ji} on the axis of the S_j), and the minimum (D_{min}) and maximum (D_{max}) distances from each SSE to their medium line, which is defined by the middle point of the shortest segment between axes of the SSEs and the vector $(S_i/|S_i|+d_{ij}*S_j/|S_j|)$, $d_{ij} = \pm 1$. The sign of dij is chosen to place projections of the vectors S_i and S_j to the medium line on one side from the projection of the closest points. A pair of SSEs $\{SA_i, SA_j\}$ from protein A is considered compatible with a pair $\{SB_k, SB_l\}$ from protein B, if (i) $|GA_{ij} - GB_{kl}| < G_0$ (usually $G_0 = 50$ degrees); (ii) $D_{i,min} - D_{k,max} < e$ ($e = 1.5$Å); (iii)$D_{k,min} - D_{i,max} < e$; (iv) $D_{j,min} - D_{l,max} < e$; and (v) $D_{l,min} - D_{j,max} < e$. These distance and angle constrains filter out differently arranged pairs of SSEs, leaving only superimposable SSEs. Structure comparison then consists of superimposing the compatible SSEs and calculating a similarity score.
- The VAST algorithm, for "Vector Alignment Search Tool" also uses SSEs and their geometrical relationships to define structure. It focuses on similarities that are surprising in the statistical sense, and does not waste time examining many similarities of small sub-structures that occur by chance in protein structure comparison. In comparing two protein domains the most surprising sub-structure similarity is the one where the sum of superposition scores across these SSEs is greatest. The likelihood that this similarity would be seen by chance is then calculated as a 'p-value' using simple statistics. Alternative alignments are examined using a Gibbs sampling algorithm, beginning from the "seed" SSE-pair alignment. The optimal alignment is defined as the one that is most surprising relative to the background distribution of alpha-carbon superposition residuals obtained by chance drawing of structural fragments at random. The VAST p-value is a measure of the significance of the comparison, expressed as a probability. For example, if the p-value is 0.001 then the odds are 1000 to 1 against seeing a match of this quality by pure chance.
- Protein structure alignment may also be performed by a technique called combinatorial extension. If A and B are the two proteins to be compared, a distance matrix is first constructed for the two using a suitable function of the differences in the inter-residue distances in the two proteins. This matrix is used to discover suitable aligned fragment pairs, or AFPs. As the name suggests, AFPs are pairs of fragments, one from protein A and the other from protein B, which have similar structures, and which are about eight residues long. The best alignment between the two proteins is that combination of the AFPs that leads to the best overall similarity score. The best alignment may also be equivalently considered as the longest possible continuous path of AFPs in the similarity matrix mentioned above. The method used to combine the AFPs into a single continuous alignment is called combinatorial extension, as opposed to dynamic programming. The first aligned fragment pair can be chosen anywhere in the similarity matrix. Consecutive AFPs are added to this path using several criteria. The first among them is a set of conditions that ensure that the physical requirements of the structural comparison are satisfied. For example, these assure

that the same fragment of one protein does not form a pair with more than one fragment of the other protein, or that the sequential requirements of the polypeptide chain are not violated. The other conditions help to choose from among the several possibilities for extending the alignment path. Only the best AFP is added on to the path at each step. The best one is chosen by using a function based on the difference in the distance matrices, not only of the AFPs, but also of the entire newly extended path. Possible gaps are also given a score and are included if they lead to an overall better alignment. Finally a z score, used to quantify the reliability of the alignment, is calculated on the basis of a normal distribution, with the empirical parameters in the equation obtained from a random comparison of structures.

6.5 Summary

- X-ray crystallography and NMR are the two methods of obtaining the detailed three-dimensional structure of a molecule.
- X-ray crystallography consists of four main steps: crystallisation, data collection, structure solution and structure refinement.
- The chief rate-limiting step in X-ray crystallography is the need to obtain crystals of sample.
- NMR is used to obtain molecular structure in solution.
- The sample is placed between the poles of a powerful magnet and excited by a radio-frequency pulse. The resonance frequencies of the nuclei are measured as a spectrum.
- Different pulse schemes lead to information about through-bond and through-space connectivity in the molecule. This information is converted into the three-dimensional structure of the molecule using distance-geometry algorithms.
- Membrane proteins, especially, may form thin two-dimensional crystals that can be studied by electron diffraction.
- Precise images of biological molecules are generated based on the atomic coordinates through the use of computer graphics.
- Three-dimensional objects may be represented on a two-dimensional surface by several techniques including hidden surface removal, perspective, shading, shadowing, highlighting and stereo projection.
- The use of computer graphics allows additional methods such as rocking, rotating and depth cueing.
- There are different levels of protein structures, and each level has a different representation. Stick and ball and stick are used to show all the atoms, cartoon diagrams and backbone traces for secondary structures, and surface diagrams to indicate the overall shape of the protein.
- The structure is geometrically characterised by calculating bond lengths and angles, torsion angles, planes, accessible surfaces and the Ramachandran map.
- DNA double helices are characterised by base-centred parameters that indicate sequence specific microheterogeneity in the structure.
- One of the most commonly used methods for comparison of two molecular structures is by superposing one on the other and calculating the root mean square deviation in the atomic positions.
- A distance map for a protein is a two-dimensional matrix of inter-residue distances.
- The comparison of protein structures may be carried out by comparing their respective distance matrices.

- To make all possible comparisons of substructures in the proteins, dynamic programming algorithms are used on the distance matrices.
- A generalised dynamic programming algorithm compares not only the inter-residue distances in the two proteins, but also other physical, chemical and structural properties of the residues.
- Other structure comparison methods include clustering, graph theory and geometric hashing, vector searching and combinatorial extension.

7

Protein Structure Prediction and Protein Folding

It has been emphasized already in this book that in order to scientifically understand the function of any system it is important to know its structure. This is of course true also in the case of biological molecules. This chapter deals with the determination of the structures of proteins, using computational methods. The three-dimensional structure of a molecule is considered known when we have a list of the three-dimensional coordinates of every one of its atoms. In Chapter 6 we have described the chief experimental techniques used to determine molecular structure, namely X-ray crystallography, NMR and electron microscopy. In this chapter we describe computational or *in silico* methods to determine protein structure.

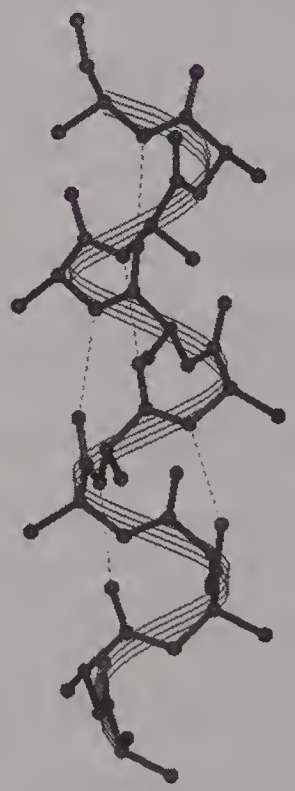


Figure 7.1 The α helix

The structure of a protein may be described at four major levels. The amino acid sequence of the polypeptide chain is called its primary structure. This is also the chemical structure of the protein since the sequence may be considered a compact way of writing down the types and numbers of atoms present in the molecule, and their interconnections. A description of the protein at this level lacks information about the three dimensional arrangement of the atoms. This information is partly supplied by the secondary structure.

The next level of the arrangement of atoms in the protein is called its secondary structure. More than 50 years ago, from X-ray diffraction experiments Astbury and co-workers observed that fibres of the protein Keratin existed in two structural forms. They called these α and β forms. The α structure was later identified as a helix, called the α helix (Figure 7.1). The polypeptide backbone chain progresses by 3.6 residues for every turn of the helical axis. The projective distance along the helical axis is 1.5 A per residue. This gives a pitch (length of

the helix per turn) of 5.4 A. The α helix has since been found to be a common structural motif even in globular proteins, in which small segments of the polypeptide chain assume this structure. It is one of the two major secondary structural elements. The other is the structure exhibited by the β form of Keratin and is called the extended or β strand structure (Figure 7.2). As the name suggests, the chain is maximally extended, consistent with the stereochemistry, and the atoms of the polypeptide backbone trace a zigzag path in a plane. This secondary structure element, too, is observed in short stretches in globular proteins. Other recognisable secondary structural motifs include the β bend or turn and several different types of helices, such as the 3_{10} helix, the π helix, the γ helix, and so on, though these occur somewhat less frequently. Thus, the secondary structure of a protein is described by the occurrence of relatively short, regular, well-identified arrangements of the atoms of the polypeptide chain. Those portions of the chain that are not associated with any particular secondary structural motif are assigned to the 'random coil' class.

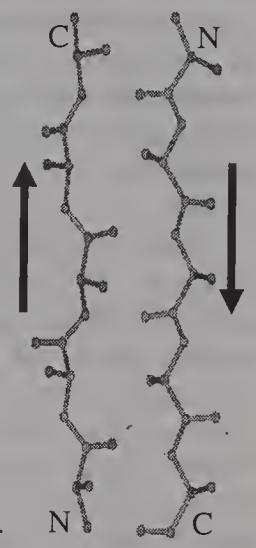


Figure 7.2 β strands. Two or more such strands may come together to form β sheets in parallel or antiparallel (shown here) orientation.

A description of the secondary structure of a protein is still essentially linear, since the three-dimensional arrangements of the atoms are local. The next level of structural description is the tertiary structure (Figure 7.3). Here we specify the relative arrangements of the secondary structural elements in three-dimensional space. We also need to specify the arrangements of the side chain atoms. In other words, a complete description of the precise relative position of every atom in the protein molecule constitutes the tertiary structure. In the case of proteins whose functional form consists of only one polypeptide chain, the tertiary structure is the fully folded, final, functional structure of the protein.

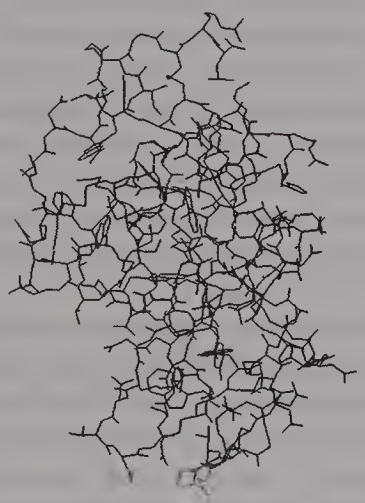


Figure 7.3 The tertiary structure of lysozyme, shown as a line diagram.

In the case of proteins made of more than one polypeptide chain, i.e. dimers, trimers, or in general multimeric proteins, there exists yet another level of structural description, namely the quaternary structure. This specifies the manner in which the different folded polypeptide chains are arranged in three-dimensional space to form the functional protein. The different polypeptides in the multimer may either have the same sequence and structure, in which case they are homodimers or homotrimers, etc., or they may have different sequences and structure, in which case they are heterodimers or heterotrimers, etc.

Protein structure prediction operates primarily at the level of the secondary and tertiary structure. The fundamental principle underlying all the methods is that the sequence of the protein, viz. its primary structure, completely specifies the final three-dimensional form of the functional protein. The experiments of C. B. Anfinsen were largely responsible for establishing this as a scientific fact. He worked primarily on the enzyme RNAase and showed that this protein, when subjected to conditions that denatured their native structure, such as high urea concentration, lost this structure and hence its activity. The structure and the activity were fully recovered when the original conditions were restored. Since there was nothing else in the solution to influence the proper three-dimensional folding of the protein, it was established that the three-dimensional structure was entirely coded by the amino acid

sequence. Almost all similar experiments since then have further confirmed this view of the relationship between sequence and structure. (There are some important exceptions. One of these is that for many proteins, other proteins or protein complexes, called chaperones, are important, if not essential, for their true and proper folding. In the absence of chaperones these proteins misfold and become dysfunctional. In such cases therefore, it is not correct to say that all the information required to fold the protein resides in its primary structure. These cases are not considered further in this chapter.)

While the fact that the primary structure specifies the secondary and the tertiary structure of a protein is a well-established one, the principles that govern this relationship are not yet fully known. It is necessary here to make the distinction between the problem of protein structure prediction and the problem of protein folding. The former refers to concepts and methods that address the use of computational methods to arrive at the three dimensional structure of a protein from knowledge of its sequence alone. This is technologically an important issue, especially since the experimental methods of obtaining the structure (described in Chapter 6) are so expensive, time-consuming and uncertain. However there is no requirement that a successful structure prediction method should be a realistic physical model of the actual folding process. The study of the way in which 'real' proteins fold in 'real' time, in other words protein folding, is an important branch of experimental, theoretical and computational biophysics, but that is not the problem addressed in structure prediction. It may happen, and frequently does happen, that insights from protein folding are used in predicting protein structure, and vice versa. However, the aim, focus and the possible techniques, in particular possible computational techniques, used in the two areas of study are quite distinct. Thus a computationally feasible method of arriving at the final folded form of the protein may be physically unrealistic, while the actual mechanisms of protein folding cannot, at this point of time, be fully simulated on computers. The main focus of this chapter is on protein structure prediction, but there is also some discussion of protein folding. Besides, there are areas where the methods and concepts overlap, and these will be pointed out as appropriate

The problem of protein structure prediction is usually tackled in two main steps. First comes prediction of the secondary structure, and then prediction of tertiary structure. Methods for the former task are better developed and show greater success rates that those for the latter. Many methods of tertiary structure prediction however do not have specific steps for secondary structure prediction, and instead simply compute the final three-dimensional structure, which of course contains both secondary and tertiary structure. In the next section of this chapter we will describe methods of secondary structure prediction. The section after that describes tertiary structure prediction methods. The final, brief section of this chapter is devoted to computational approaches to understand the physical principles of protein folding.

7.1 Protein Secondary Structure Prediction

For the purposes of prediction, every residue in a protein chain is always considered to exist in one of three (or four) secondary structural states. These are: helix, usually represented as H; beta strand, represented as B or E (for 'extended'); and random coil, signified as C. Lower case letters are used when the prediction is not very certain. Often an additional state is also predicted, namely turn, signified as T. The output of most secondary structure prediction algorithms and programs is the sequence of the protein along with one of the above symbols for each residue. Also given, if available, is an estimate of the strength of the prediction. Thus secondary structure prediction is a residue-by-residue process and not, for example, an estimate of the overall secondary structure content of the protein. Protein secondary structure prediction methods may be broadly classified according to the

techniques used as those based on single residue statistics, those based on explicit rules and those based on neural networks. The methods may be further classified historically as belonging to the first, second or third generation. First generation statistical methods were developed in the early 1970's and made use of the experimentally determined protein structures that became available. In the second generation, statistics concerning neighbouring residues were also used. In the third generation, information obtained from sequences with similar structures was incorporated into the prediction algorithm. Some of the methods predict all the three or four structural states. Others restrict themselves to predicting one or two states. Thus some of the methods can predict the occurrence of helices, or even more specifically trans-membrane helices often found in membrane protein structures. There are now several programs and web servers available for predicting protein secondary structure. In this section we will describe a few of the earliest formulated algorithms, and some of the most popular among the recent methods.

7.1.1 Measures of prediction accuracy

The strength of the prediction, mentioned above, is calculated internally by the algorithm and the method of calculation varies according to the algorithm. This parameter is however not a measure of the accuracy of the prediction. There are several ways to estimate the correctness of the prediction. These techniques usually compare the predicted structure for a sequence with the experimental (usually crystallographic) one. Thus, it is possible to test a prediction algorithm by using it to make predictions for a set of proteins whose structures are already known, and then comparing them. If the test set is properly chosen and the test correctly carried out, the measure of accuracy so derived for the prediction method may be expected to apply even to new sequences, where, of course, the experimental structure is unknown. An international effort to subject both secondary and tertiary structure prediction algorithms to uniform, unbiased assessment is carried out biennially by the CASP (Comparative Assessment of Structure Prediction) programme. A set of protein sequences is made available over the Internet to every group that has a prediction method and wishes it to be assessed. The set of sequences is obtained by approaching experimental groups that expect to have the crystallographic (or NMR) structures available for publication within a few months. These groups also agree not to publicise the structures until the CASP deadline is over. There are of course other criteria for the sequences, such as their length, their novelty, etc. However, the only information available to the participants is the sequence of the proteins. All participants are required to submit their predictions, before the deadline, in a specified format, of as many sequences as they wish, and at any level (i.e. secondary or tertiary). The predictions are then compared with the experimental structures by an independent team of assessors and the results of the assessment are made public. Among the parameters used to measure the accuracy of the prediction are the following.

(i) $Q_i^{pred} = P_i / (P_i + O_i)$. P_i is the number of residues correctly predicted to occur in the state i, and O_i is the number of residues incorrectly predicted to occur in the state i. These are the 'over-predicted' residues. $P_i + O_i$ therefore represents the total number of residues predicted to occur in state i. For example, if, out of a total of 100 residues in the protein, a given method predicts 20 to occur in helices, and if in the experimental structure, only 15 actually occur in helices, then the index

$$Q_\alpha^{pred} = P_\alpha / (P_\alpha + O_\alpha) = 20/35 = 0.57 \text{ or } 57\%.$$

Similar quantities are calculated for each of the three (or four) states considered. The final accuracy of the prediction method is an appropriate combination of the indices.

(ii) $Q_i^{obs} = P_i / (P_i + U_i)$. Here P_i has the same meaning as above. U_i is the number of residues actually in the state i in the experimental structure, but not predicted as such. These are the 'under predicted' residues. $P_i + U_i$ therefore represents the total number of residues in the state i in the experimental structure.

(iii) $Q_3 = (P_\alpha + P_\beta + P_{coil}) / T$, where P_α, P_β, and P_{coil} are the numbers of residues correctly predicted to be in these specific states, and T is the total number of residues. Q_3 is a global measure of accuracy, and is the one most frequently used in assessment. If we assume that the three states occur randomly, or, what is equivalent, that the prediction method is no different from a random choice of one of the three states, the theoretical random value for the Q_3 is 33%. In fact, due to bias in the actual occurrence of the three states, the random value is about 38%. A meaningful prediction scheme should therefore have a Q_3 value much higher than this.

(iv) The Matthews coefficients

$$C_i = (P_iN_i - U_iO_i) / [(P_i + U_i)(P_i + O_i)(N_i + U_i)(N_i + O_i)]^{1/2}.$$

Here the N_i are the total number of residues that occur in the state i. C_i assumes a value between 0 (a completely inaccurate prediction) and 1 (a perfect prediction), and is calculated for each state i. The final accuracy of the prediction is assessed based on all the coefficients. Q_i^{pred} and Q_i^{obs} are called single-state single residue coefficients of accuracy. Q_3 is a three-state single residue coefficient of accuracy. Too much reliance should not be placed on any one score, and it is best if the final accuracy of the prediction method is decided on the basis of a combination of scores.

7.1.2 Statistical techniques – the CF algorithm

This is one of the earliest (first generation) statistical methods. The CF (Chou-Fasman) method works on the basis that each of the twenty amino acids seen in proteins has a definite preference to exist in one of the three conformational states, viz. helix, strand and turn. Information regarding these was obtained by observing such preferences in the fifteen or so protein structures that were then known. First, the relative frequency f_i^h of occurrence of an amino acid residue i in a helix was calculated by taking the quotient (n_i^h/n^h), where n_i^h is the number of times the residue i occurs in helices, and n^h is the total number of residues that occur in helices. This was then converted to a 'conformational parameter' (or 'conformational preference' or 'conformational propensity') P_i^h by taking the quotient $(f_i^h/\langle f_i^h \rangle)$, where $\langle f_i^h \rangle$ is the average of the relative frequency for all the residues. Similar numbers were calculated for the other two conformational states considered, viz. P_i^b for strands and P_i^t for turns. The propensity values currently used in structure prediction are given in Table 7.1. These are calculated from the currently available protein structures. Not only are the values different from those originally used, but also the propensity value for 'coil' is replaced by one for 'turn'. Also there are specific propensities for the occurrence of the residues at particular positions i, i+1, i+2 and i+3 in a β turn, which are explained later in the text. All the same, the basic idea underlying these values, and the method of using them in remains the same as suggested originally. On the basis of such values the residues were classified into helix or strand breakers and formers. Residues that positively contribute to the formation of a structural element are called formers and those that prevent or stop its formation are called breakers. Table 7.1 displays this classification, and uses the following symbols. H_a: strong helix former. h_a: helix former. I_a: weak helix former. i_a: indifferent, in other words the residue may or may not be a part of a helix and it shows no preference either way. b_a: weak helix breaker. B_a: strong helix breaker. The symbols H_b, h_b, I_b, i_b, b_b, and B_b are used for the beta strands. To use these parameters in secondary structure prediction we apply the algorithm given below, which follows the original faithfully, but is not identical to it.

Step 1: Assign appropriate values from the table to all the residues in the peptide.

Step 2: Scan through the peptide and identify regions where 4 out of 6 contiguous residues have P(h) > 100. In other words if there are at least four contiguous helix formers, that region is declared a helix.

Step 3: Extend the helix in both directions until a set of four contiguous residues that have an average P(h) < 100 is reached. This means the residues are helix breakers. That is declared the end of the helix.

Table 7.1 Secondary structural propensities (x100) for the twenty amino acids.

Name	P(h)		P(b)		P(turn)	f(i)	f(i+1)	f(i+2)	f(i+3)
Alanine	142	H_a	83	I_b	66	0.06	0.076	0.035	0.058
Arginine	98	i_a	93	i_b	95	0.070	0.106	0.099	0.085
Aspartic Acid	101	i_a	54	i_b	146	0.147	0.110	0.179	0.081
Asparagine	67	b_a	89	b_b	156	0.161	0.083	0.191	0.091
Cysteine	70	i_a	119	h_b	119	0.149	0.050	0.117	0.128
Glutamic Acid	151	H_a	037	B_b	74	0.056	0.060	0.077	0.064
Glutamine	111	h_a	110	h_b	98	0.074	0.098	0.037	0.098
Glycine	57	B_a	75	i_b	156	0.102	0.085	0.190	0.152
Histidine	100	h_a	87	b_b	95	0.140	0.047	0.093	0.054
Isoleucine	108	I_a	160	H_b	47	0.043	0.034	0.013	0.056
Leucine	121	H_a	130	h_b	59	0.061	0.025	0.036	0.070
Lysine	114	I_a	74	b_b	101	0.055	0.115	0.072	0.095
Methionine	145	h_a	105	h_b	60	0.068	0.082	0.014	0.055
Phenylalanine	113	h_a	138	h_b	60	0.059	0.041	0.065	0.065
Proline	57	B_a	55	b_b	152	0.102	0.301	0.034	0.068
Serine	77	i_a	75	b_b	143	0.120	0.139	0.125	0.106
Threonine	83	i_a	119	h_b	96	0.086	0.108	0.065	0.079
Tryptophan	108	h_a	137	h_b	96	0.077	0.013	0.064	0.167
Tyrosine	69	b_a	147	h_b	114	0.082	0.065	0.114	0.125
Valine	106	h_a	170	H_b	50	0.062	0.048	0.028	0.053

Step 4: If the segment defined by this procedure is longer than 5 residues and the average P(h) > P(b) for that segment, the segment can be assigned as a helix.
Step 5: Repeat this procedure to locate all of the helical regions in the sequence.
Step 6: Scan through the peptide and identify a region where 3 out of 5 of the residues have a value of P(b) > 100. This will identify a portion of the sequence rich in beta formers. That region is declared as a beta strand.
Step 7: Extend the strand in both directions until a set of four contiguous residues that have an average P(b) < 100 (i.e. four strand breakers) is reached. That is declared the end of the beta strand.
Step 8: Any segment of the region located by this procedure is assigned as a beta strand if the average P(b) > 105 and the average P(b) > P(h) for that region.
Step 9: Any region containing overlapping helical and strand assignments are taken to be helical if the average P(h) > P(b) for that region. It is a strand if the average P(b) > P(h) for that region.
The algorithm also has a special step for identifying turns, (more specifically called reverse turns or β turns) in proteins. A turn is a specific arrangement of four consecutive amino acid residues in a polypeptide chain that reverses the direction of progression of the chain, i.e. turns it around by 180°. The last four columns in Table 7.1 give the propensity values for the occurrences of each residues at positions 1, 2, 3 and 4 of a turn.
Step 10: To identify a bend at residue number j, the following value is calculated.

$$p(t) = f(j)f(j+1)f(j+2)f(j+3)$$

where the f(j+1) value is used for the residue j+1, and similarly for f(j+2) and f(j+3). If p(t) > 0.000075, and if the average value for P(turn) > 1.00 in the tetrapeptide, and finally if the averages for the tetrapeptide obey the inequality P(h) < P(turn) > P(b), then a beta-turn is predicted at that location.

The CF algorithm not only can locate secondary structural regions but also detects regions with the potential for conformational changes. A wide range of prediction accuracies (58% - 86%) has been reported. However more recent comparative studies have shown an average Q_3 value of about 48% for this method. In general, prediction accuracy is better for a protein with a single type of secondary

structure (all helix or all strand) than for a mixed type protein. Helices were predicted better than strands and turns.

7.1.3 Statistical techniques – the GOR I algorithm

This is another first generation method based on single residue statistics. (GOR stands for Garnier-Osguthorpe-Robson). It has a strong theoretical basis in information theory. If S represents one of the four possible conformations (S = alpha, beta, turn or coil), and R is one of twenty possible amino acids, then we can obtain a measure of 'conformational information' for each residue for each conformational state as

$$I(S;R) = \log_e[P(S/R)/P(S)]$$

where P(S/R) is a conditional probability. In other words, it is the probability of observing the conformational state S when the residue present is R. P(S) is just the probability of observing the state S. Note that the emphasis here is the inverse of what it is in the CF algorithm. There the question was, given a particular amino acid what is the conformational state? In the GOR method we ask, given a conformational state what is the likely residue? (This inversion of emphasis, however, is only in the way in which the database is used to generate the required statistics. The final results in both methods give a residue-by-residue prediction of the secondary structure). These probabilities P(S/R) for each of the twenty residues in each of four conformational states are estimated from the database of known structures, by counting the frequency of occurrence of each residue in each conformation. The probabilities P(S) of the occurrence of the states is given by counting the total number of residues in each state, and dividing by the total number of residues in all states. The calculations thus require the estimation of a total of 20 (residues) x 4 (conformational states) = 80 parameters. The above equation for single residue information was later modified to include the so-called directional information regarding neighbouring residues. Simply stated, the directional information indicates the effect of the neighbouring residues on the conformational state of the residue under consideration. Normally we would have to consider the effect of all the residues in the protein sequence, since in principle all of them would affect the conformation at every residue. This however is not practicable. In the GOR I method a window of eight residues on either side of the reference residue has been considered, on the basis of calculations that showed that this would sufficiently well represent the effect of all neighbouring residues. Thus the information regarding the conformational state at the position j in the protein sequence is given by the following expression.

$$I(S_j, R_1 \ldots R_N) \approx I(S_j, R_j) + \Sigma_{m=-8 \text{ to } +8} I(S_j, R_{j+m})$$

The second term on the right hand side of the equation is the directional information and is calculated from the database by counting the frequency of each of the twenty residues at the 16 adjacent positions, eight positions on the N terminal side and eight on the C terminal side, for a given conformation of the central residue. The amino acid type of the central residue is irrelevant, and only its conformation is considered. Thus there will a total of 16 (positions) x 20 (amino acids) x 4 (conformational states) = 1280 numbers to represent the directional information. Together with the 80 informational parameters defined earlier and used to estimate the first term in the above equation, this gives a total of 1360 parameters that are estimated from the database of known structures. In utilising these parameters for secondary structure prediction, the GOR I method simply estimates the information according to the equation above for each of the four conformational states at each residue position. The state with the highest value for the information is assigned to that position. The prediction, of course, has to be interpreted correctly. For example, it is possible that in certain stretches of the chain, alternate residues are predicted to be in different conformational states. This is best interpreted as an uncertain prediction. The GOR I method has had a success rate of $Q_3 = 55\%$.

7.1.4 Use of segment statistics – the GOR III algorithm

The GOR II algorithm that in 1989 followed GOR I was an update of the database including new experimental protein structure data that then became available, is thus still a first generation technique. In the GOR III version, however there was a change in the concept, which justified calling it a second-generation method. One of the relatively minor changes made was to define and use only three conformational states, namely helix, sheet and coil. The difficulty of distinguishing between turns and coils, especially since there were so many possible turns, lead to this decision. A more important change was in the introduction of the so-called pair information. This took into consideration the correlation between the type of residue at different positions in the window and the residue at the centre, i.e. the reference residue for which the prediction is being made. The expression for the information is now given as

$$I(S_j, R_1 \ldots R_N) \approx I(S_j, R_j) + \Sigma_{m = -8 \text{ to } +8} I(S_j, R_{j+m} | R_j)$$

Now the second term on the right hand side includes the conditional probability of a particular residue at position j+m, given the occurrence of the residue R at position j, i.e. the reference position. To calculate these information parameters we need to estimate the frequencies of simultaneous occurrence of all possible pairs of residues at specific positions within the window. The database of protein structures available at the time GOR III was developed did not allow the calculation of statistically significant values for many pairs of residues. Thus one had to make use of so-called dummy frequencies, particularly in the case of residues such as Trp and Met, which are relatively scarce in protein sequences. Basically, the idea was to have a fallback and calculate the frequencies for the scarce residues on the assumption that their occurrence is uncorrelated with the residue and the conformational state at the position j at the centre of the window. The use of such dummy frequencies increased the success rate of the predictions, as compared to the use of the actual frequencies of correlated occurrence. The Q_3 value for the best implementations of the algorithm reached 60%. A further improvement in the method has been since made, by taking into consideration the frequency of the correlated occurrence of all possible pairs of residues at all possible pairs of positions in the window. This was made possible by the large database that is now available, leading to statistically reliable estimation of the frequencies. This is called the GOR IV algorithm and was published in 1996. Just as for the CF method, web based servers implementing the GOR algorithm are available.

7.1.5 Neural networks in secondary structure prediction – PHD

Neural networks are algorithmic devices constructed in a manner loosely analogous to the animal brain. In the brain neurons are connected to one another in a dense network. The processing of information, in particular, the recognition of complex underlying patterns from subtle visible signals, is thought to arise out of the network structure. Computer-based artificial neural networks (ANNs) thus consist of units analogous to neurons that receive input information from other units and send output signals to yet others. The connections are many-to-one and one-to-many. Therefore a given unit in an ANN receives input signals from several other units, and processes them using an appropriate a mathematical function (as, for example, a simple summation). Based on the result, the unit sends an output signal to several more units. There are various categories of connections possible, for example networks in which loops are allowed, others where the output signal from an unit is redirected to itself, and so on. There are also several categories of mathematical functions that may be used in generating the output signal based on the input. This variety allows the use of ANNs in several different fields including, computational linguistics, speech recognition, recognising faces, speech synthesis, etc.

Neural networks used in secondary structure prediction are the so-called multi-layered, feed-forward networks (Figure 7.4). The units are arranged in layers and the connections are always in the forward direction from the units in the lower layer to the ones in the upper layer. The mathematical

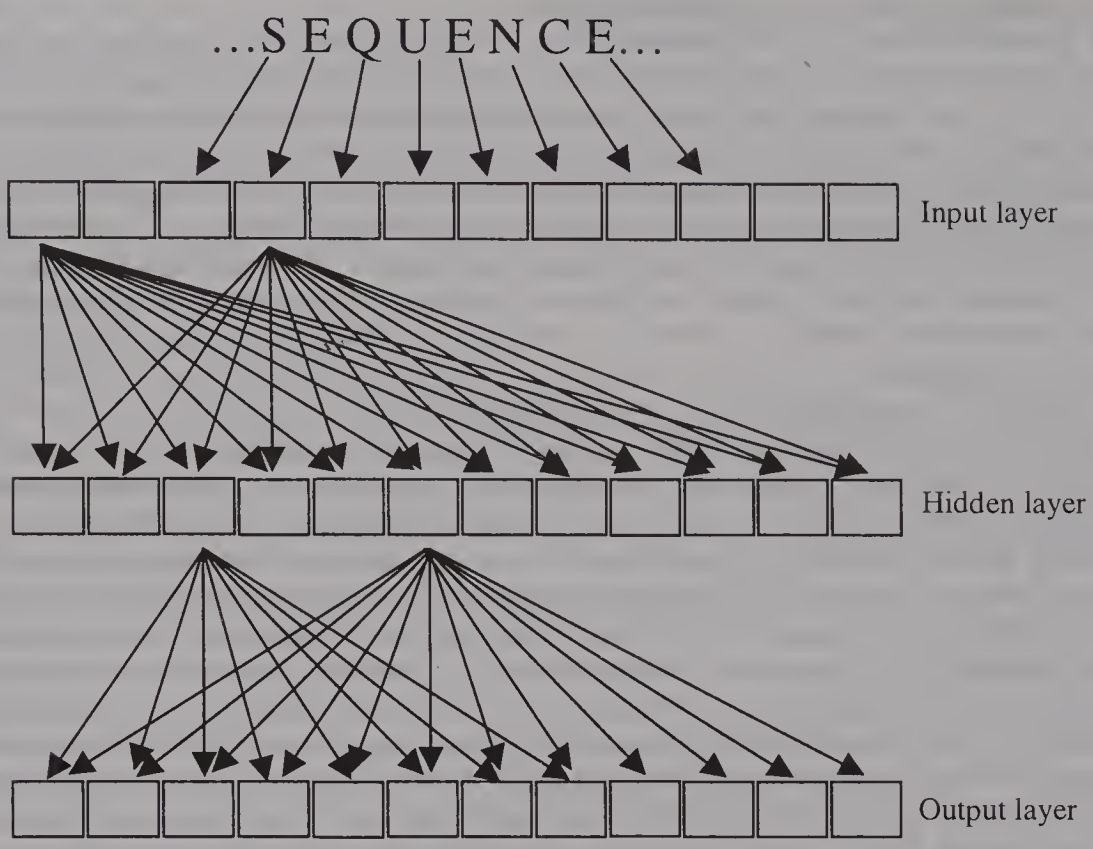


Figure 7.4. A schematic diagram of a feed-forward multi layered artificial neural network (ANN). For clarity, only some of the interconnections between the layers are shown. But, in fact, every element of a layer is connected to every element of the next layer.

function used to process the input at each unit is such as to produce an output signal that is close to one of two values, i.e. 0 or 1, (or sometimes –1 or +1). This allows the ANN to be configured as a classification or recognition system. In the present case, the ANN is used to classify the amino acids as belonging to a specific secondary structure type. Figure 7.4 shows a three-layered ANN, with the first input layer accepting information from each residue in the sequence of the protein. Each unit in this layer applies the processing function to all the input it receives and, according to the result, sends out an output signal to each unit in the second hidden layer. This procedure is again repeated in this layer and an output signal is sent to each layer in the third or output layer. In this example, the output layer consists of three units, corresponding to the three structural types, helix, sheet and coil, and each of these three units in turn sends an output signal. Since the output signals are always adjusted to be close to 0 or 1, a value of 1 from one of the units and 0 from the other two would imply a corresponding prediction for the residue under consideration. The procedure is applied in turn to every residue in the sequence.

The use of an ANN involves three steps, viz. construction, training and utilisation. The first step involves decisions regarding the number of layers, the number of units in each layer, and the mathematical function, called the gating function, that is to be used in each layer to connect the input to the output. These decisions, in particular the last one, would give rise to a number of weights and constants that have to adjusted in order to obtain the correct performance from the ANN. For example the gating function is usually a weighted sum of all the inputs, multiplied by a sigmoidal function that

results in a number close to either 0 or to 1. The adjustment of the weights is carried out by 'training' the ANN by using a set of data where the answers are already known. In the present context, this means a set of sequences for which the secondary structure is already known. This training set of sequences is fed into the newly constructed ANN and for each residue in each sequence the prediction by the ANN is compared with the known structure. If the prediction is not perfectly correct, the weights and constants are adjusted gradually by a method called the back-propagation algorithm (or some similar technique) to bring the prediction closer to the correct one. This is repeated for all the residues and all the sequences, through several cycles, until the weights do not change and the predictions are as close as possible to the correct structures. During the training process it is important to keep several points in mind so as to obtain the best values for the weights, in other words to obtain the best ANN for the purpose. Firstly it is necessary to choose the training set of sequences to be truly representative of the sequences that the ANN would encounter during its utilisation phase, even though this may not be very easy since only a limited number of structures are known. Secondly not all the sequences available are to be used in the training. A set of sequences with known structures, called the test set, is usually kept apart, so that the trained ANN may be tested, and further trained if necessary – i.e. the test set is not to be used for obtaining the weights, but only to test the ANN. Again this is sometimes difficult to carry out in practise.

For the purpose of secondary structure prediction, several groups have implemented neural networks in different ways, with varying degrees of success. Here we describe one of most successful and commonly used implementations, namely PHD. This is a third generation method that incorporates evolutionary information as well as sequence information in the input to the ANN. PHD consists of several steps other than the ANN. First the input sequence is compared with available sequences in the database and a multiple sequence alignment with all similar sequences is constructed. In order to obtain the structure prediction at each residue position, i.e. each column in this multiple alignment, a window of thirteen residues, six residues on either side, is considered and the following information is extracted and fed into the ANN: the profile of amino acid substitutions; the weights for each amino acid type compiled for all the columns in the alignment; the numbers of insertions/deletions (indels) in each column; the position of the window with respect to the entire sequence; and the amino acid composition and length of the protein. All this information has been incorporated into the gating function and the ANN has been constructed and trained with four units in the hidden layer and three in the output layer. The input layer of course has to contain enough units to incorporate all the above information, one unit for each piece. The results of PHD have been very good with Q_3 values of greater than 70% for different sets of test sequences. The programme is available over the Internet as a server.

There are of course other, equally successful implementations of ANNs in secondary structure prediction, one of which is NNPredict.

7.1.6 Nearest neighbour methods – PREDATOR and NNSSP

Like the methods mentioned above, these methods also make residue-by-residue predictions. The general philosophy of this set of methods is to base the prediction of the secondary structure of a given amino acid residue on the effect of its neighbouring residues in the sequence. To do this, a window is defined in the query sequence centred on the residue under consideration. The size of this window is commonly 16, with eight residues on either side of the reference point. The sequence within this window is then aligned with a 'training' set of protein sequences chosen because their secondary structures are already known. There are about 100 to 400 sequences in this set, and a further criterion for their choice is the lack of significant sequence similarity among the members of the set. A number (say 50) of fragments that match best with the query sequence are then chosen and, based on the secondary structure of the central residue in them, an assignment of the structure of the reference residue in the query sequence is made. Since all the best matching sequences will not have the same

structure, necessarily the prediction will be a probabilistic one, with different probabilities assigned for every residue for each of the three structural types, viz. helix, sheet and coil.

Thus far the different implementations of the nearest neighbour algorithm are more or less the same. (Note, that the term 'nearest neighbour' does not refer to residues adjacent to the reference residue. Rather it refers to sequences in the training set that are the most similar, or 'nearest' to the query sequence.) The only small differences lie in the choice of database, or the sequence matching method, etc. But after this point the strategies diverge. The PREDATOR program additionally uses amino acid pair statistics to predict hydrogen bonds between neighbouring β-strands and between residue i and residue i + 4 in helices. This information is incorporated into the method as a set of rules, and along with the information derived from the nearest neighbour sequences, is finally used to make an unambiguous prediction of the secondary structure at each residue. PREDATOR may thus be called a combination of nearest neighbour and rule based methods. It has a reported a Q_3 up to 75% and is thus one of the most successful secondary structure prediction programs.

NNSSP is another implementation of nearest neighbour methods. It uses the probabilistic prediction described above as input to a neural network. The ANN is then used to make a clear residue-by-residue prediction. The success rate of this method is also high with reported Q_3 value of up to 73%.

7.1.7 Consensus methods – JPRED, NPS

To take advantage of all the different methods of protein secondary structure prediction, and not rely on just one technique, some groups have devised consensus methods. Constructing such a combination prediction for a sequence, or for a multiple sequence alignment, can be a complicated task owing to the different input and output formats and parameters that the different methods use. JPRED is secondary structure prediction server that has been designed to accept a sequence or an alignment in standard format and run a prediction using six different algorithms – NNSSP, DSC, PREDATOR, ZPRED, MULPRED and PHD. These were chosen as being the current state-of-the-art. In particular all these methods exploit evolutionary information and perform their prediction on multiple sequence alignments. Thus even if the user is interested only in a single sequence, the JPRED server extracts other similar sequences, and performs a multiple sequence alignment, which is then used as input to the six prediction programs. The input requirements and parameters are standardised so that the user inputs the information only once in a standard format. The output is also standardised and only a three-state prediction – helix, sheet and coil – is considered. The six predictions are then combined and a simple majority decides the consensus prediction for each residue. For example, at a given residue position in the input sequence, if four or more of the predictions correspond to one structural type, say helix, and the others to another type, say sheet, the consensus prediction is helix. If the prediction is a tie, with three programs predicting one structure, and three another, the prediction made by PHD is considered the correct one. This approach gave good results, significantly better than each of the six programs individually. A Q_3 value of 73% has been reported over all type of sequences.

NPS stands for Network Protein Server, and is another consensus-based technique. The user may chose up to twelve different methods, including first generation methods such as GOR, of which three versions are included. Other programs in the list are PHD, DSC, PREDATOR, HNN and SOPMA. The user is allowed to set the parameters individually for each program. The input to the server is a single query sequence. The server creates multiple alignments where required. The output consists of the prediction made by each of the chosen methods, and a consensus prediction based on the majority.

7.1.8 What is the best accuracy attainable – pitfalls, precautions and prognosis.

The steady improvement in the methods of secondary structure prediction make it appear that a final solution of the problem is not far away. Already accuracy levels of $Q_3 = 80\%$ are attainable by some

methods in specific sets of sequences, though not consistently. Many of these methods, such as nearest neighbour methods depend on existing known databases of sequences and structures. Such databases are growing in size at a very rapid rate, and therefore the future performance of these methods is bound to improve significantly. Furthermore, algorithms such as PREDATOR incorporate long-range interactions, and these, again, are bound to improve in quality as more experimental information becomes available. Thus secondary structure prediction can be expected to surpass 70-75% accuracy. However the best possible will not, on average, be 100%. It has been seen experimentally that highly related sequences do not always share identical structures. Even when a query sequence can be aligned confidently to a sequence of known structure, the alignment will produce a secondary structure prediction with Q_3 of only 88% on average. Predictions based on multiple sequence alignments cannot be expected to perform better than this. Some of the other points to be noted in interpreting the results of secondary structure prediction for a new sequence are given below.

- Most methods predict alpha helices far more accurately than beta strands. The accuracy value is for the overall prediction, and has to be interpreted differently while considering different sections of the sequence. In this context, third-generation methods such as PHD and Predator are clearly superior, in that the beta strands also have a high level of accuracy.
- Accuracy measures are derived from test sequences and are averages over distributions of sequences. In the case of specific prediction, the accuracy may not be the same. For example, if a prediction technique with a quoted accuracy of 70% ± 10% is applied to a new sequence, two-thirds of the time, the resulting prediction will have accuracy between 60 and 80%. Thus there are still chances that the specific prediction is better than 80% or worse than 60%.
- The reliability of a prediction is different from the accuracy (Q_3) of a prediction technique. A general rule of thumb is that two are correlated – the more accurate the prediction technique, the more reliable its prediction.
- Consensus methods appear to perform very well. Nevertheless, unless the prediction on which the consensus is based are all of high accuracy, there is a danger that a bad prediction may be chosen simply because it is the majority.
- A final caution applies to protein structure prediction in general, secondary or tertiary. The correlation between sequence and structure may not be 100%. There could be subtle non-linear effects coming into play. For example, small changes at some positions in the sequence, even of just a single residue, may lead to large unpredictable changes in the structure. As yet no methods exist that correctly predict the secondary structure in such 'pathological' cases.

7.2 Protein Tertiary Structure Prediction

The literature on methods to predict the three dimensional structure of a protein from its sequence alone is vast, and scores of techniques have been devised. None however are successful enough to replace experimental methods like X-ray crystallography or NMR, and *in silico* or purely computational protein structure determination remains the holy grail[20] of molecular biophysics. In general, the techniques may be divided into three broad categories: homology modelling, threading

[20] The term 'holy grail' refers to the great quest undertaken by the knights of King Arthur's Round Table to search for original goblet, or grail, from which Jesus Christ drank at the last supper. The quest involved superhuman efforts by the best of the knights. Finally, after several years of searching, only the noblest knight of them all could find it. In the present context, it is likely that the scientist who finds an accurate structure prediction method would be honoured with the Nobel prize.

techniques, and *ab initio* structure prediction (Figure 7.5). The first, namely homology modelling, depends on finding a previously determined protein structure with a high degree of sequence similarity

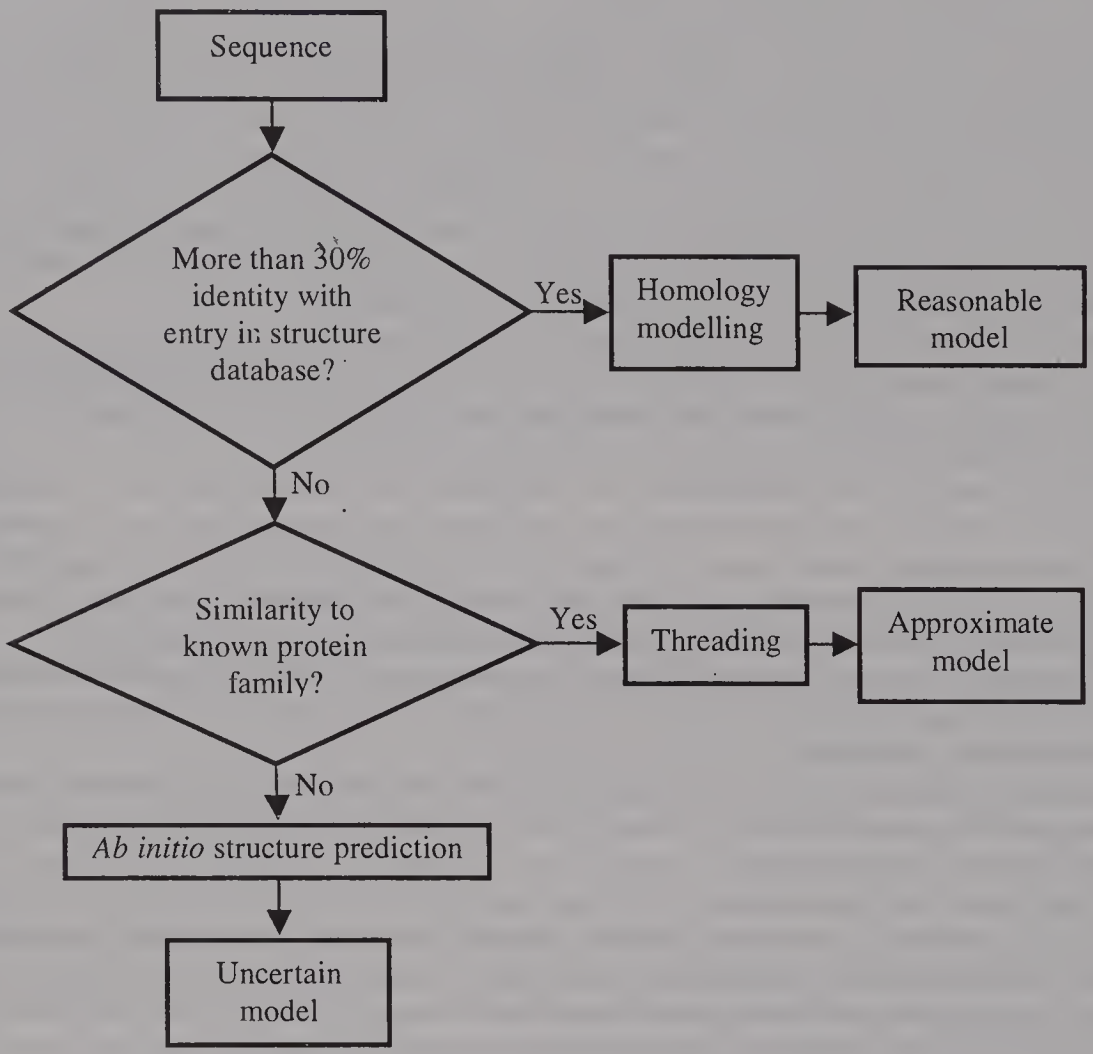


Figure 7.5 Flowchart for 3-D protein structure prediction

with the protein of unknown structure. This of course is not always possible. The second technique relies on recognising a core subsequence in the unknown sequence that can be recognised as belonging to a family of sequences with known common structure. This is also called fold recognition. It is very similar to homology modelling, but there are important differences, which will be discussed later. The third method relies only on knowledge of the sequence, and physics, chemistry and computation to arrive at the three-dimensional structure. This is obviously the most difficult of all and is therefore attempted only when the other methods have failed.

Under each category, there are several algorithms. All of them require a large amount of computational effort, and a high degree of user intervention. There are a few web servers that offer automatic three-dimensional structure prediction services, but the degree of success in using them depends largely on whether similar sequences already exist in the structure database. In this section we describe some these methods.

7.2.1 Homology modelling

This is also known as comparative protein modelling or knowledge based modelling, and the word homology, used in this context, does not necessarily imply a strong evolutionary relationship, though such a relationship may well be present. The prime requirement is the existence of a model structure, usually one that has been determined experimentally. The technique is used when the unknown sequence, called the target, bears a sufficiently strong sequence similarity with another sequence, called the template, for which the structure is already known. On the basis of the degree of similarity it may then be inferred that structure of the target is the same as or similar to that of the template. Following up on this idea results in a set of atomic positional coordinates for the target, i.e. the structure of the target. The increase in the number of experimental structures, and developments in modelling software has lead to this particular technique becoming more and more useful. It has been estimated that given current techniques, a valid structure can be determined *in silico* for a significant part of almost 50% of known protein sequences. Furthermore the accuracy of the predictions and the confidence that they are correct approaches that of a low resolution experimental structure determination. Broadly the technique consists of four steps: selecting the template, alignment of target with template, building the model, and evaluating the model.

The first step, selecting the template, is the most important one. A wrong selection at this stage cannot be corrected at a later stage in the modelling and careful attention to a proper choice of the template is amply repaid by the final results. It is best if the target and the template are actually homologous proteins, in the strict biological sense of sharing a common ancestor. One may then be confident that they share a common function and therefore a common structure. However, since homology, in this sense, is itself usually determined by the degree of similarity between the sequences, a more important concept to define is similarity. That is, how much similarity is required between target and template, and in how long a sequential run of residues, to accurately predict structure of the former from that of the latter? Studies have suggested that if pairs of sequences have residue-by-residue identity greater than 50%, then they have over 90% of these residues in regions where the structure is the same. Further, when the structures of two similar sequences are compared, the overall rms difference of corresponding C^{α} atoms increases as the sequence identity decreases. It ranges from about 0.3 Å when the identity is close to 100% to about 4 Å for 20% identity. Regions of the protein that normally have divergent structures, such as loops and turns have similar structures only when the sequence identity is greater than 50%. Also the number of insertions and/or deletions increases as sequence identity decreases.

Template selection is facilitated by the availability of a several sequence and structure databases and efficient software for matching the target sequence with these. Programs like BLAST, FASTA, etc., when used on databases such as the PDB, CATH, etc., swiftly identify possible templates. A refinement of this technique is the use of multiple sequence alignments. The target sequence is aligned with families of sequences that are already categorised as possessing similar structures and functions. The experimental structure of a single member of such a family would then serve as a template for all the others. Alternatively, such a family may be built up using a recursive program such as PSIBLAST, which creates profiles of sequences similar to the target and uses the profiles to search for more and more distant, but clearly identifiable homologues. There could be several sequences with known structures similar to the target and therefore several possible templates. One approach is to build models using all of these, and then to use the evaluation procedure (step 4) to select the best of them. However it is possible to avoid or at least reduce the enormous amount of work this would entail by applying the following criteria. Firstly that template is best that has the closest sequence similarity to the target. Secondly the template structure that we use should, as far as possible, have been determined in the same biochemical conditions of pH, ionic strength, etc., as those in which we desire the structure of the target. And finally the template structure should be a reliable one. For example, in the case of

X-ray crystallographic structures we look for the one at the highest resolution. It is has been observed, however, that instead of using just one template, the quality of the model is improved by using many templates, provided the corresponding sequences are all approximately at the same level of similarity with the target.

After selecting the template, the second step in the modelling procedure is to align the target sequence with the template sequence. While sequence alignment precedes template selection, the alignment needs to be repeated with greater rigour once the selection is made. This is especially true when the similarity score is low. We now use pair-wise global sequence alignment algorithms, such as the Needleman and Wunsch method. Again if possible, it is better to perform multiple sequence alignment using programmes such as CLUSTAL, or a variation of it. All possible templates are first multiply aligned and a profile constructed. Other sequences belonging to the same family may also be added to the profile, which is then aligned to the target. The best possible template is then chosen as the initial model for the target.

The third step is to build the model, based on the target-template alignment. There are conceptually two ways this can be achieved. The first is to use the structure of the template directly as the structure of the target, replacing the side chains of the residues that differ. A subsequent optimisation step would then take care of the side chain interactions. Another way to build the model is to use the template to calculate restraints to be applied on the target, such as inter-residue distances and angles, specific disulphide bonds, stacking interactions between aromatic residues, etc. A distance-geometry algorithm is then used to create a model of the target structure that would satisfy these restraints. Again a subsequent optimisation step is performed to ensure that the structure satisfies physical and chemical principles. When the target-template sequence similarity is low, or when the resolution of the template structure is low, the polypeptide backbone may be correctly placed, while the side chain atom positions are still unclear. Special algorithms have been devised to take of this problem incorporating optimisation techniques from operations research and physics such as 'branch and bound' method, or 'mean field' theory. Thus model building would consist of first building the backbone, then placing the side chains, and finally optimising the entire structure. Some of these techniques take a large amount of computational time and a high degree of user expertise. Clearly, automatic model building techniques are possible only in those rare cases where the sequence similarity is high and a good template structure is available.

The final step in the modelling process is evaluation of the model. We first ensure that the model satisfies physical constraints and principles, such as non-interpenetration of the atoms. Next in degree of importance comes the stereochemistry, such as bond lengths, angles, etc. The Ramachandran plot is a very good way of checking the geometry of the model and programs such as PROCHECK are available to carry out these tasks. Finally biological and biochemical principles are considered. The model should of course satisfy all such known concepts. However, the model would not really be of any great use unless it suggested some new ideas related to biology.

Despite passing all the evaluation tests, the model could still possess one or many of the following errors. There could be errors in the packing of side chain atoms in the core of the protein. Since residues that differ from the ones in the template have no model, we need to use optimisation techniques mentioned above to fix their positions. There is however no absolute way of knowing if the optimal arrangement is in fact the naturally correct one. There could be errors in the trace of the main chain polypeptide sequence as a consequence of incorrect placement of the side chain atoms. There could be errors that arise due to the insertions or deletions of a long stretch of residues. A model for such a stretch in the target becomes difficult to build. When the target-template identity is less than 30%, especially, a serious source of error is incorrect alignment of the two sequences. This implies that incorrect structures, chosen from the template are assigned to misaligned residues in the target. Using multiple sequences and/or structures to build the model may reduce alignment errors. When distantly

related proteins are used as templates, it is possible that an incorrect template is chosen. In this case the model would probably be completely wrong, but such errors are difficult to detect until the very end, when predictions made by the model do not find experimental or theoretical support.

Despite these problems comparative modelling is an extremely useful supplement to the experimental determination of protein structures. Homology models have been used in the following ways. They have been used to design mutants to test ideas regarding protein folding and function. Especially, single point mutants, in which only one residue is different between target and template, are relatively easy to build (if we ignore possible non-linear effects). Models have been used to identify active sites. A particular use of homology models is in drug design, where frequently small sequence changes in the certain crucial regions of the protein lead to loss of effect for the drug. Models of the accompanying structural changes may be built to understand the effects, and perhaps design more effective drugs. A practical application of modelling is to build models that may be used in molecular replacement methods of solving X-ray single crystal structures. Stated simply, this means that X-ray data is collected pertaining to a protein with a sequence similar to another whose structure already known. The latter structure then is used to interpret the data and arrive at a structure for the former. A similar use for the models is found in interpreting NMR data. From the point of view of genomics, one of the major uses of the models is to confirm structural and functional relationships between sequences. The usefulness of the models depends on their accuracy. Models that are based on less than 30% sequence identity and having less than 50% of their atoms within 3.5Å of their correct positions still have the correct fold. This is often sufficient to predict its approximate biochemical function and to establish evolutionary relationships. Models based on 30%-50% sequence identity, and having 85% of the atoms modelled within 3.5Å of their correct positions are used to characterize the active and binding sites, since these are usually better conserved than the rest of the structure. Such models of medium accuracy may be also used in NMR structure determination, molecular replacement in X-ray structure analysis, and in design of molecular biology experiments, such as site-directed mutagenesis. Highly accurate models based on more than 50% sequence identity have average accuracy nearly that of low resolution X-ray structures. Such models can be used for detailed studies, for example, of the docking of small ligands or to define and study antibody epitopes.

7.2.2 Threading

Threading generalises the technique of homology modelling, and aligns the unknown sequence, not to another sequence of known structure, but to a likely structure built from families of structures with sequences similar to the target. Thus, at its core, threading has a sequence-structure alignment algorithm. The development of this method arose from the observation that structures of proteins tend to be often much more similar to each other than their sequences. In other words dissimilar sequences frequently give rise to similar structures, with similar or different functions. This observation has led to the suggestion that there exists in nature only a relatively small finite number of protein 'folds' or overall topologies, estimates for which range from 1000 to about 10,000. Whatever the actual number, it is clear that every new sequences does not necessarily correspond to a new fold. Even if a similar sequence does not exist in the structural database, it is likely that a similar fold does. The problem then is to identify which particular fold in the database best suits the target sequence. Threading is therefore also known as 'fold recognition' algorithm, or 'inverse folding', since we have a library of folds, and are looking to see which one best fits or 'threads' the target sequence. There are however subtle differences in the meanings of the three terms, and here we will describe only threading, in the following meaning of the word. Threading is when we choose a backbone fold (structure) and find the best way to mount (or fit, or thread) the target sequence on to it, with gaps if necessary, so that the good interactions between the residues in the sequences are maximised.

With this definition, threading works under the following four assumptions. (i) The database of known protein structures may be used to provide a library of templates. (ii) A sequence-structure alignment or threading places the residues of the target sequence at positions defined by the template. (iii) Different candidate threadings arise from different templates, different target sequences and different alignments of the target and template. (iv) A scoring function (that maybe defined using statistics) is able to distinguish the correct threading among all these candidates.

Threading may be divided into the following four tasks. First it is necessary to construct a complete library of possible folds or structural templates. Second, a scoring or fitness function must be designed that will allow a particular threading to be distinguished as better than the rest, and therefore the correct one. Third an efficient algorithm must be designed to search all possible alignments of the target sequence with every possible fold in the library. And finally there must a way of arriving at the best possible template and the best possible alignment to the temple using the above scoring function and the alignment algorithm. We will now consider each of steps in greater detail.

Members of the library of folds are described by various terms, reflecting both the concepts underlying a particular way of defining a fold, as well as the method of specifying it, i.e. the data structure. Some of the more common terms include cores, topology fingerprints, three-dimensional profiles, contact profiles, adjacency graphs, etc. The data usually consists of an abstraction of the three-dimensional structure, with symbols replacing the coordinates. As an example of such an abstraction we consider a three-dimensional profile. The following is a commonly used construction of a 3D profile. A structure is described by a set of descriptors specifying the three-dimensional environment of each residue. Thus the residue may be on the surface of the protein (or exposed), or partially on the surface and partially buried, or fully buried. The letters E, P and B represent these three states. Another feature of the residue encoded is the number of polar residues that surround it. Thus a partially buried residue surrounded by a partially polar environment is said to be in the state P_1, and a fully buried residue with a highly polar environment in the state B_3. Altogether, the combination of these two features leads to the six states specified as E (exposed), P_1 (partially buried, partially polar environment), P_2 (partially buried, fully polar environment), B_1, B_2 and B_3 (fully buried, with increasing degree of polar environment). The local secondary structure, helix, sheet, coil, is also taken into consideration, giving a total of 18 environment classes. Every residue in the protein is classified as belonging to one of these 18 classes. An analysis of the structures in the PDB leads to the observations that each of the twenty residues shows a definite pattern of preference for each of the environmental classes. This preference is quantified by calculating

score(i,j) = $\log_e$[(probability of residue j in class i)/(probability of residue j in any class)]

This yields an 18 x 20 matrix of log odds scores. This table is then used to construct the library of folds. This may be done all at once and stored as a database, or better, may be created as and when candidate folds are required to thread the target sequence. When a particular structure is identified as suitable model for the target, a three-dimensional profile is created by first identifying the structural environment at each residue position. The profile then would contain the scores of all twenty residues, if they were to be placed in this position, as shown in Table 7.2. Also a gap penalty may be specified that reflects the structural effects of the occurrence of a gap at that position. For example, one would not expect a gap in a buried helix, and any such residue position carries a high gap penalty. On the other hand, a position in an exposed coil has a low gap penalty. For n positions in the template structure considered, there will be n x 21 numbers in the matrix representing its 3-D profile.

The second task in the threading algorithm is to specify a scoring function. If the folds are constructed as described above, and if a dynamic programming method as described below is used for sequence-structure alignment, the scoring function is already defined by the probability values and the gap penalties in the table. Other methods of performing the first and third tasks may require an explicit definition of the scoring function. Thus the score(i,j) defined above is not the only way of constructing

Table 7.2 Profile of a fold. Partial table for the 'globin' fold is shown.

Position in fold	Environment class	Amino acid					Gap penalty	
		A	C	...	W	Y	Opening	Extension
1	E	12	-46	...	-214	-94	2	0.02
2	B_2	-66	-5	...	102	112	2	0.02
3	Eα	46	-44	...	-135	-210	200	200
4	$P_2\alpha$	6	-93	...	-114	-79	200	200
...	...	...	...	...	...	...	...	...

the scoring function. In particular the probabilities may be replaced by pseudo-energy terms obtained by using Bayesian statistics to convert the frequencies of occurrence of each amino acid in a given environmental state, first into posterior probabilities. These posterior probabilities, together with an appropriate statistical distribution function, will yield the energy terms. Another very simple scoring function used by some workers is a simplified version of the semi-empirical potential used in conformational and molecular dynamics studies. This version of the scoring function assumes that the score is a sum of the pair-wise residue-residue interaction energies. The interactions are assumed to depend only on the identities of the residues concerned, and not on their relative positions along the polypeptide chain or the neighbouring residues. Such a scoring function is called a 'contact potential', and has been used together with a Monte Carlo sequence-structure alignment technique.

Sequence-structure alignment is the third task to be performed in the course of the threading algorithm. One of the most common techniques used is dynamic programming, particularly in combination with the 3-D profile specification of the template library. Dynamic programming, as explained in Chapter 3 provides a globally optimum solution for the alignment problem in the case where gaps (indels) are allowed. The drawback when applied to sequence-structure alignment is that it requires a specification of the structure as a linear string of symbols. The 3-D profile satisfies this requirement, but at the cost of ignoring information that is present, say, in contact potentials, or in more physically realistic energy terms. The application of dynamic programming to align a target sequence with a template structural profile is straightforward. The target sequence is written as the leftmost column of the dynamic programming matrix, and the profile of the template structure is written on the first row as a string of symbols specifying the structural class at each position along the template. Table 7.3 is then used to fill the elements of the matrix, by looking up the probabilities of the residue specified by the row index being in the structural state given by the column index. Standard dynamic programming is then used to find the best alignment between the sequence and the structure. The significance of a score for a particular alignment is obtained by aligning, one by one, a large number of sequences selected at random from a database, finding the mean score μ and its standard deviation σ. The significance is then quantitated by the z value as

$$z(\text{score}) = (\text{score} - \mu)/\sigma.$$

The fourth and last task to be performed is one of selection – which is the best possible template and the best possible alignment? In the above example dynamic programming already gives the best possible alignment. To find the best possible template in a perfectly unbiased way, the target is aligned against the entire library of templates, and the best one may be chosen by comparing the z values. There are two immediate caveats. Firstly comparisons of the z values may not be suitable. μ is calculated by considering a limited set of sequences, and this set may not be fully representative of the target sequence. Secondly alignment of the target against the entire library of templates means either alignment against all known structures (in the PDB) or against a subset of structures chosen to represent all of them. In either case, if the target sequence preferred to adopt a structure far from any

known, threading would not be able to arrive at the correct native fold. What may be worse, the method may suggest a wrong structure as suitable for the target, with no independent way to check.

There are several techniques that attempt to overcome these drawbacks. None however can be said to perform perfectly. Nevertheless threading programmes are quite commonly used to predict structures of new sequences. A new sequence is usually compared with a database of sequence families, such as Pfam. If a good match is found, and if a template structure exists for that family, then threading is sure to produce reasonable results. If no such match is found, 'blind' threading against all templates may be tried, but the results are uncertain.

7.2.3 Ab initio structure prediction

Both homology modelling and threading rely on databases of previously determined structures. While neither class of methods has been sufficiently developed to be a fully automatic tool for protein structure determination, they are more advanced, and more successful than the third class of methods that we shall discuss now, namely *ab initio* structure prediction. Under a strict definition of this class an *ab initio* algorithm uses only the sequence of the protein, and the well-established laws and principles of physics and chemistry, to determine its three-dimensional structure. Underlying this definition is the deterministic principle that biology does not entail any new fundamental laws, different from those applicable to non-living matter. However biological systems are so complex that no straightforward or simple application of physics and chemistry has so far been able to fully explain them, and the protein folding problem is a classic example of this predicament. From the principles of physics is it clear that the final folded form of the protein is its minimum energy state. It would appear that an application of this principle is sufficient to go computationally from sequence to structure. However two problems arise immediately. First, how is the energy of the protein to be defined? Second, how to find the conformation for which this energy is a minimum? The two questions are not independent of each other, and, as we will see below, the answer to one of them is often based on the answer to the other. There are several different methods that attack both or either one of these questions. Some of the methods even 'cheat' a bit on the definition of *ab initio* and make use of existing structural information to define pseudo energy functions.

Let us first consider the problem of defining the energy. In order to be useful in structure prediction, the chief property that this function should possess is that its global minimum should represent the native structure of the protein. By global minimum, we mean that for all possible allowed structures of the protein (of which there are astronomical numbers), and the corresponding energy for each, the energy for the (unique) native structure should be clearly calculated as the least of all. By consensus among the structural biology community, the X-ray or NMR structure of a protein is considered its native functional form. The minimum of the energy function should thus correspond to this experimental structure. When predicting a new structure, obviously the experimental structure is not already known (for that would render the whole exercise meaningless). Therefore, though the energy function may be constructed on the basis of known structures, it should accurately reflect the true physical situation for all proteins, and not just the particular set on the basis of which it was developed. It has proved extremely difficult to arrive at such a function.

The first obvious choice is the semi-empirical potential function that has proved so successful in studies of the physical chemistry of small peptides (and other biological molecules). Such a potential is usually written as the sum of several terms, each of which is calculated as the sum of all possible pair-wise interactions between the atoms in the molecule. An example of one such potential, is the so-called AMBER force field. Its calculation requires the evaluation of N(N-1)/2 atom-atom interactions for each semi-empirical term. Thus it requires computational time of the order of N^2 to obtain the value of the energy for any given conformation of the molecule. Modern day computers are fast enough to calculate many such energies per second for small proteins of the order of about 100 or so residues.

However since the computational time increases quadratically (as the square) as the size of the protein is increased, the time required soon becomes too large for even the fastest computers to handle. Besides this, there is another problem associated with the semi-empirical potential given above. We have stated that the global minimum of the energy function should correspond to the native structure. Most (if not all) search algorithms that look for the minimum of such a function will be able to identify it only when it is clearly separated both in magnitude and position from other minima (Figure 7.6a) Ideally the function should possess only one deep minimum. In such cases there are robust, simple algorithms to find it. The actual situation with respect to the semi-empirical energy function is that it contains many minima, with their magnitudes also being approximately the same (Figure 7.6b). Thus search algorithms that start at an initial structure and look for the structure corresponding to the global minimum in steps risk getting stuck at the nearest local minimum, far from the native structure, though the energy value is reasonably low. One way to overcome this problem of multiple minima, which also helps in reducing the computational complexity, is by reducing the resolution at which the potential function is calculated. This means that the energy function is not written in terms of atom-atom potentials, but in terms of the interactions between suitably chosen groups of atoms. The 'united atom' potential, which is also a part of the AMBER force field, is one the early attempts at making such an approximation. Here, well-defined chemical groups of atoms, such as the methyl moiety, are considered as a single 'pseudo-atom'. In fact in this model, most hydrogen atoms found in proteins are not treated separately but subsumed within a pseudo-atom defined to include the carbon, nitrogen or oxygen atom to which they are bonded. Semi-empirical parameters are calculated and tabulated for the pseudo-atoms, just as for the real atoms, and are used in the energy calculations to greatly reduce the computation time, as well as to smoothen the energy landscape (i.e. to reduce the number and magnitudes of false minima). The UNRES force field developed recently also includes terms arising from the interactions of the residue with the solvent, and terms calculated from the propensities for the amino acids to assume specific secondary structures.

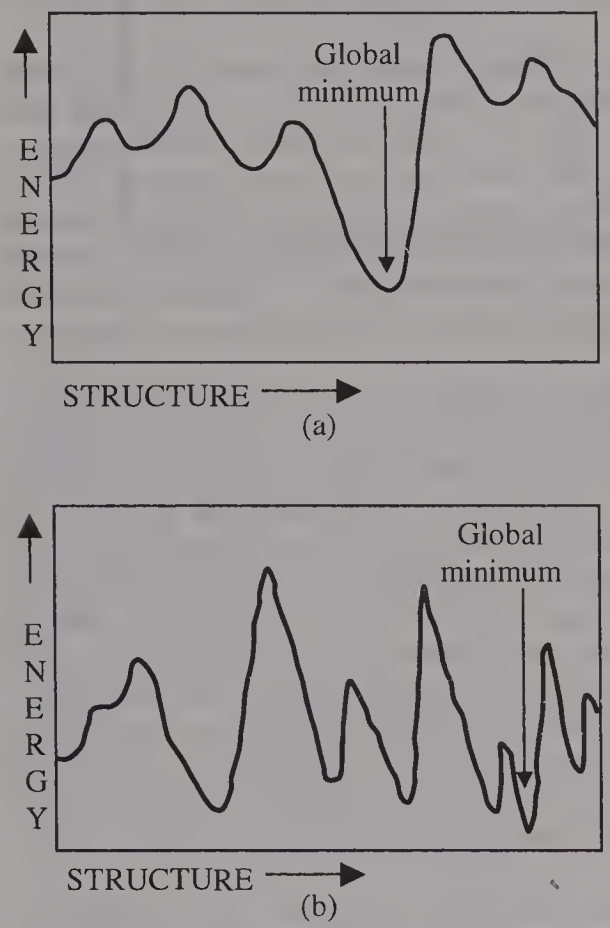


Figure 7.6 Schematic diagrams of different types of energy functions. Note that energy is plotted as a function of a single variable, when in fact it is a function of hundreds of variables even for small proteins.

Scoring of the structures is sometimes carried out using functions derived from analysis of known experimental results, not from physical chemistry. Typical of such functions are the so-called statistical potentials that use Boltzmann statistics derived from known structure databases, together with a Bayesian formalism to arrive at a function that describes the probability of any conformation. Such probability distribution functions are quite easily converted to energy functions. Sometimes constraints on the structures, determined for example from NMR experiments, are also used to

construct the potential function. The constraints may be in the form of a set of measured distances between pairs of atoms in the protein. In this case the protein structure prediction problem closely resembles the distance geometry problem commonly encountered in the NMR determination of structure. Simple stated, this problem arises when we are in possession of some of geometrical details of the structure, and we must use these to arrive at the coordinates of the atoms. In structure prediction, such constraints are used to calculate a pseudo-potential term, which is then added, with appropriate weighting, to the semi-empirical, or other, force field.

The second major problem in *ab initio* protein structure prediction is the design of an appropriate method to determine the structure corresponding to the global minimum of the scoring function used. Several methods have been suggested and have been shown to work to a greater or lesser degree. Broadly they can be divided into two classes. In the first class are those methods that use thermodynamic principles to start with an arbitrary conformation for the protein and work towards the global minimum. In the second are those that use methods that have been successfully applied in other areas of science, such as operations research to find the global optimum of a function, discrete or continuous. These latter methods do not attempt to mimic the physical process of protein folding, but merely to predict the correct structure, no matter how.

Molecular dynamics refers to the computation of the dynamics, or the motion, of a protein. In applying this methodology to the protein folding or protein structure prediction problem, the following assumptions are implicitly or explicitly made. First it is assumed that the actual physical forces influencing the folding process are sufficiently well represented by a semi-empirical force field, as discussed above, especially if solvent forces or other such forces are also properly incorporated into the model. The second assumption made is that the atoms of the proteins each move independently, influenced only by the force field modelled as above. Moreover this motion is completely described by Newton's laws of motion. The calculations are carried out as follows. The protein is placed at an arbitrary initial conformation, and the coordinates of every atom in this conformation are specified. A single atom is considered, and the semi-empirical force field is used to calculate the influence of every other atom on it. The reference atom is then allowed to 'move' under this influence, by assuming an arbitrary initial velocity. The new position of the atom after a very short interval of time (of the order of femtoseconds[21]) is computed simply by using Newton's second law of motion

$$F = ma$$

Now the force field is recalculated for the second atom, taking into account the new position of the first. The coordinates of the second atom, after one or two femtoseconds, are calculated, again by applying Newton's second law. This process is carried out atom by atom for the entire protein, until at the end of one round of calculations, all the atoms of the protein have shifted from their initial positions, but in a controlled and physically realistic way. The cycle of calculations is repeated over several hundreds or thousands femtoseconds. The motion of the atoms is influenced by the temperature factor used in the calculations. Obviously, at a higher temperature, the atoms would move a greater distance in that short period of time, than at lower temperature. This makes it possible to carry out the calculations in various interesting ways that simulate some physical processes. One protocol, usually referred to as 'simulated annealing', attempts to mimic the process by which physical objects like crystals find their global minimum. Here the calculations are first carried out at a very high temperature of about 3000 K or so, for say a few femtoseconds. The temperature is then lowered gradually, all the while taking care to see that the atoms do not fly apart. At the end of the simulation the temperature would have reached the 'room' temperature of about 300K. The idea is that even if the protein starts from a wrong structure, the initial high temperature pushes it to a loose, general structure, from where it slowly descends into the final correct conformation (Figure 7.7). Such simulations of

---

[21] 10^{-15} second

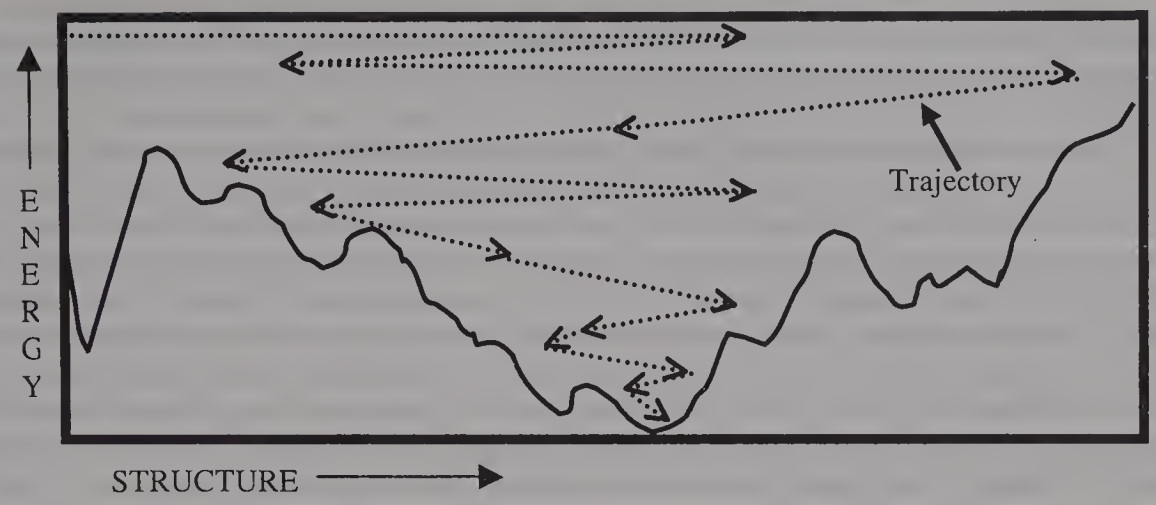


Figure 7.7 Molecular dynamics as a structure optimisation tool

protein motion have lead to interesting insights into the protein folding process, but their use in actually determining a new structure has been limited. The primary reasons for this are the imperfections in the force field, and the enormous computational time required for meaningful simulations of natural folding processes.

Several other more or less physically realistic simulations of protein folding have been suggested and implemented. Monte Carlo simulations use a random sampling technique to generate hundreds of thousands of protein conformations. The sampled structures are analysed using statistical thermodynamics to predict the correct structure. Techniques such as 'Conformational Space Annealing' perform similar calculations incorporating various ingenious methods of overcoming the problem of multiple minima. However the comments made above with regards to the success of molecular dynamics are applicable also to these methods.

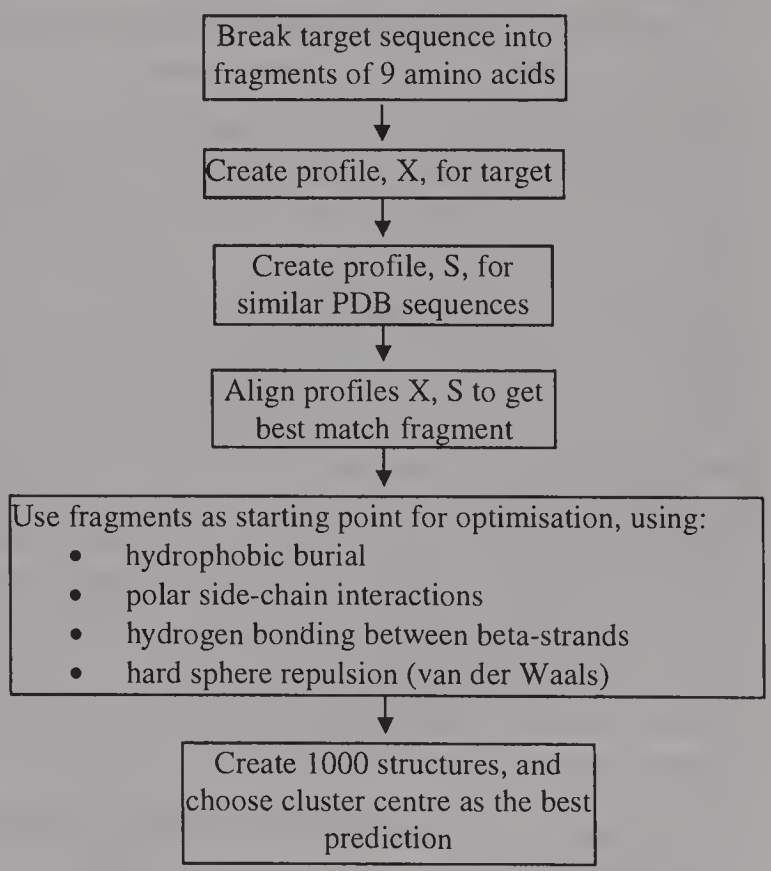


Figure 7.8. Flowchart of the ROSETTA algorithm

The second class of methods use 'unphysical' algorithms to arrive at the global optimum conformation. Methods normally in the realm of operations research, including 'branch and bound' techniques, evolutionary algorithms, etc., have been used to predict protein structure with varying degrees of success. One of the most successful of these has been incorporated in the program package ROSETTA. This method stretches the definition of *ab initio* structure prediction, and is sometimes classified under

threading methods. The method first scans the known structure database looking for occurrence of every possible peptide sequence five and nine residues long, respectively. There could be several occurrences of such peptides, with the same or with different structures. A library of structures for each possible peptide is thus built along with a weight corresponding to various parameters such as the number of times a particular member of the library has been found. For example to build a library of structure for the pentapeptide, say, ACDEF, where each letter represents an amino acid, the database is scanned for all occurrences of this peptide or others that are very similar to it by the criterion of well known substitution matrices. The structures of all these peptides are then culled out, arranged in clusters of similar structures and given a weight. This constitutes the library for this particular sequence (or set of sequences). Once the library is built, a new sequence is similarly split into short overlapping fragments. Each fragment is searched for its appropriate library of structures. The structures are spliced together to build up the complete sequence, which is optimised using Monte Carlo procedures. Figure 7.8 gives flowchart of this procedure. Several unknown structures have been obtained to a relatively (compared to other *ab initio* methods) high degree of accuracy. The best however is still not as good as even a low-resolution experimental structure.

7.3 CASP, Levinthal Paradox, Protein Folding

7.3.1 CASP

CASP stands for Critical Assessment of techniques for protein Structure Prediction. This is designed as a 'community-wide' experiment to use a common set of tools to independently assess the effectiveness of the various structure prediction methodologies available. The first experiment was run in 1996 and is now being run once every two years. Thus CASP1 was in 1996, and the latest one, CASP6, in 2004. The experiment is organised by John Moult and colleagues and is carried out as follows. Sometime in April of the year of the experiment, a set of protein sequences and other relevant information, except structure, is made available on the CASP website. These sequences are collected from X-ray crystallographers and NMR spectroscopists all over the world and represent proteins whose structure is about to be determined and refined in their laboratories. The participant experimentalists agree not to release their results until a specified deadline, which is usually set in August or September of that year. Before the expiry of this deadline, all participating structure predictors submit their predictions for as many of the sequences as they wish. Further, they may use any method, including ones that have not been published before. The prediction may be for secondary or tertiary structure. Tertiary structure prediction is made in the form of a list of atomic coordinates, at any resolution, from C^{α} atoms alone to a detailed specification of the position of every atom. After the deadline is over a set of independent assessors, chosen from among those not participating, compares the predictions with the experimental structures that are by then available. Several criteria are used in the assessment. For secondary structure prediction, some of the more important criteria used have been described in section 7.1.1. For tertiary structure prediction, an obvious criterion is the rms deviation in atomic positions between the experimental and predicted structures. Other criteria include the rms difference in the main-chain and side chain dihedral angles, as well as an overall estimate of the accuracy of the prediction based on the difference between the actual errors in the atomic coordinates and estimates of these errors provided by the participant. Using these criteria the assessors rank all predictions. The groups then meet at a conference in December to discuss the results. The proceedings of the conference are published and made available to the entire scientific community.

The goals of each CASP experiment are to assess the current state-of-the-art in protein structure prediction in an impartial way. The division into different categories of techniques is merely for convenience, and in fact has not been used in the more recent CASP experiments. The first aim of the

experiment is to compare the predicted structure with the experimental one, i.e. the true structure. Such comparison helps to assess the specific method used. More broadly it helps to establish which type of methods are best suited for further development. Since the experiment is carried out periodically, it also helps in the general assessment of the progress made in this area and to identify barriers to progress. This in turn provides a feeling for the direction in which the techniques are headed, and what could be future directions to explore, in theory, computation and experiment.

CASP1 made available 33 target sequences in three categories, namely comparative modelling, fold recognition and *ab initio* prediction. 35 groups of scientists submitted a total of 135 predictions in the various categories. In later experiments, the strict boundaries between the categories were blurred, and the evaluation process took only the scope of the model (i.e. secondary structure, C^{α} trace or all atomic coordinates) into consideration, and not the method by which they were arrived at. CASP4 had a total of 43 targets and attracted 163 different prediction groups. Each group attacked more than one target and sometimes submitted more than one model for a target, leading to a total of 11136 models. Most of these were three-dimensional coordinates, corresponding to ab initio predictions or homology models. 61 groups submitted threadings to structures in the PDB. There were also 35 groups that submitted only the secondary structure assignments.

This experiment has shown that there has been a small, but steady improvement in the prediction methodologies over the years. Structures of small proteins consisting largely of alpha helices are the most easily predicted. Structures that contain only beta strands and sheets are the most difficult to predict. As far as *ab initio* tertiary structure prediction is concerned, the best predictions for proteins of about 100 residues or more have an rmsd of greater than 5.0 A with the experimental structure.

7.3.2 Levinthal paradox

In 1968 Levinthal pointed out to a serious paradox that apparently existed at the centre of the protein folding problem. (We say 'apparently', because as we shall discuss later, there are reasons to believe that the statement of the paradox is based on fallacious arguments). The paradox follows from the fact that, under the usual and reasonable assumption that the bond lengths and angles are kept constant, the conformation of a protein is uniquely specified by the set of torsion angles corresponding to the rotatable bonds in the molecule. Such torsion angles tend to have three preferred values, and none of these may be ruled out *a priori*. Therefore if we have N rotatable bonds, there will be 3^N possible unique settings for the angles, and each would specify a possible conformation. To find the best one, a computer program would have to calculate some scoring function for each one of these. For a typical protein there would normally be greater than 100 rotatable torsion angles, even if we ignore the side chain angles and consider the main chain φ and ψ angles alone. Thus the program has to calculate the scoring function for 3^{100} conformations in the case of the smallest proteins. Even the best computers of the foreseeable future do not have the capacity to perform such calculations, and for larger proteins the computational requirement increases exponentially. In the language of computer science, such problems are termed NP. This acronym stands for Non-Polynomial, signifying that the computational cost increases exponentially with the size of the problem, and not as a polynomial of the size of the problem. To state it more explicitly, if C is the computational cost, N is the size of the problem and x is some number (finite, and often a small integer), then if

$$C = x^N$$

the problem, or more specifically the algorithm designed to solve the problem is termed NP. On the other hand, if

$$C = N^x$$

or some similar function in which N does not appear in the exponent, the algorithm is termed P or 'polynomial in N'. Though the latter type of algorithms also may take astronomical amounts of time, they essentially terminate in some finite time, even for the largest values of N. On the other hand, the

cost of solving a problem using a NP algorithm 'blows up' and very quickly reaches impossible magnitudes. Thus problems that are known to possess only NP algorithms to solve them are considered intractable. A brute force algorithm that examines every possible protein conformation before arriving at the correct one is an example of an NP algorithm in protein structure prediction. If this was the only algorithm available, the problem of protein structure prediction is intractable, incapable of being solved and will thus have to be abandoned. As the discussions in the previous sections have shown, thankfully this is not the case.

However, methods that do lead to correct structures in finite time are not physically realistic. It would be absurd, for example, to consider that a polypeptide chain that folds to its correct final structure in the cell or in a test tube does this by homology modelling! In naïve thermodynamic terms, the only physically reasonable method for the protein to fold is to more or less randomly sample every possible conformation, staying longer if the energy is low, and quickly escaping to another, if the energy is high. When it arrives at the global minimum energy conformation, it stays there for such a long time that we may consider it the native conformation[22].

The Levinthal paradox now becomes obvious. *How can a protein chain, that would take exponentially large amount of time to find the correct conformation, fold in finite amount of time, of the order of milliseconds to minutes?*

The basic fallacy in this paradox is the assumption that the polypeptide chain uses a brute force search in 'torsion angle space' to find the global minimum. But the statement of the paradox helps to focus attention on issues in the field of protein folding to be studied by experiment and theory. Further, such studies help to design better protein structure prediction algorithms. Information that could lead to a resolution of the paradox include the discovery of intermediate states for the protein, which are not in the final folded form, and yet possess sufficient amount of the native structure to be considered as a middle step in the folding process. This leads to the idea that the polypeptide chain folds in a hierarchical manner, first assuming an approximate global form, before soldifiying in steps to the final positions of all the atoms. Some of these findings are discussed briefly in the next section.

7.3.3 Protein folding

There has been an enormous amount of work carried out to understand protein folding. Experimental approaches include traditional techniques such as structure determination by crystallography and NMR, all forms of spectroscopy, particularly NMR, IR, UV/visible, Raman and fluorescence, and more recently single molecule studies using optical tweezers and atomic force microscopes. Theoretical studies include molecular dynamics, Langevin dynamics, lattice folding studies, database analysis and studies on the 'designability' of sequences. Despite all this work, however, protein folding is far from being fully understood, either in the scientific sense of knowing what the principles are, or in the practical sense of being able to compute a structure. However the work has provided a deep understanding of the process and has laid to rest some misconceptions that were earlier prevalent.

A current description of the protein folding process is the following. A polypeptide chain rapidly undergoes 'hydrophobic collapse' to form the 'molten globule' state. On a slower time scale, the molten globule intermediate condenses into the final folded form. In symbols,

$$\mathbf{U \leftrightarrow I \leftrightarrow N}$$

---

[22] There is a thermodynamically equivalent way of stating this, as follows. The sample consists of about 10^{23} copies of a polypeptide chain. The number of chains that exist in any given conformation is governed by its energy, with the overwhelming majority existing in the global minimum state. This however ignores the fact that experiments have seen single chains, or a small number of them, all fold to the correct structure.

where U represents the unfolded protein, I in the intermediate (molten globule) state and N is the native structure of the protein. The first reaction going from U to I is a rapid one, and I is the more stable state, tending to persist far longer than U. This intermediate state, like the unfolded state, and unlike the native state, is not a single structure, but a set of structures that share some characteristics. The set could be a discrete one with a large number of structures, distinctly different from each other, and capable of being characterised, to a small extent, by experiment; or it could be one with a continuous range of structures. Accordingly there are broadly two type of intermediates, the so-called 'framework intermediates' and the molten globule intermediates. Both have some amount of the secondary structure that is found in the native form. For example extensive studies on the folding of apomyoglobin have shown that at least three of the eight helices in the native structure also occur in the intermediate, with moderate stability. Evidence for the possible occurrence of native secondary structure in the intermediate also come from studies on peptides of length of about 13 or 14 residues clipped from the protein sequence that shown the same structure and contacts as in the final fold. In the framework intermediate, tertiary contacts between residues far apart in sequence are very similar to those found in the native structure. This is not so in the molten globule. The formation of the intermediate from the unfolded state occurs by either by hydrophobic collapse, or by the formation and later breakdown of non-native secondary structure. Hydrophobic collapse is when the set of hydrophobic residues in the protein chain, which usually occur at the core of the final globular protein structure (in water soluble proteins), come together to escape the aqueous medium, in the manner of a drop of oil. The other mechanism for the formation of the intermediate is when peptide segments first fold into secondary structures; then the amino acid residues interact further to form tertiary configurations. The secondary contacts are sometimes non-native, i.e. they do not occur in the final structure. Presumably, the presence of tertiary interactions in the intermediate leads to their disruption and to the formation of new, native secondary structures. It appears from the studies that in both methods of intermediate formation, the sequences present in the turn and loop portions of the native structure do not influence the folding in any drastic way. The important factor appears to be the packing of the groups in the hydrophobic core. Either, or both mechanisms of intermediate formation may occur, and may lead to either or both types of intermediates.

The intermediate and the native share many structural features, the chief one of them being a high level of common secondary structure. The intermediate however does not have the same level of tertiary contacts. It has a poorly defined hydrophobic core, which in turn leads to lower stability, as compared to the native. The native structure is additionally stabilised by backbone-backbone interactions, absent in the intermediate. According to one study, such interactions contribute about 65% of the total stabilisation energy, while backbone-side chain interactions contribute 25%. Side chain-side chain interactions contribute mainly to the specificity of the structure, and come into play only in the final moments of the native structure formation.

Among the many different observations that suggest a wide diversity in the details of protein folding mechanisms are the following.

- It has been observed that sometimes the native secondary structure is stabilised by tertiary contacts, and vice versa. This blurs to some extent the distinction between these two levels of protein structure, since the formation of local structure appears dependent of the formation of the global structure.
- Some proteins, like the chymotrypsin inhibitor protein, fold by two-state kinetics in which no intermediate structure has been observed, as opposed to the three-state kinetics described above.
- Medium to large proteins appear to have a long 'resting' period in the intermediate molten globule state, during which a relatively slow search for a 'native-like' conformation is undertaken. Once this additional intermediate step is taken and the required structure, close enough in conformation and energy to the native, is found, a rapid refinement takes place to finally arrive at the correct

native structure. The refinement includes condensation and rearrangement of hitherto 'floppy' surface residues.

- Protein folding is not entirely an un-assisted process. The hard hypothesis, that all the information required for folding is available in the sequence has been shown to require modification with the discovery of chaperones. These proteins assist other proteins in attaining the native conformation, and are therefore an integral part of the folding process. Theoretical studies have shown that the chaperones help by modifying the potential energy surface in such a way as to help the intermediate state or the unfolded state of the polypeptide chain to avoid folding pathways that lead to final misfolded forms.
- Experiments that study folding in single protein molecules indicate that there are often several pathways to protein folding, with varied intermediate states having different levels of native secondary structure, tertiary contacts etc., that all still lead to the same native structure.
- We may summarise this information as follows. An ensemble of unfolded states collapses into an ensemble of intermediate states, that, by different pathways, all fold into the same final native form.

7.4 Summary

- Protein structure is categorised into four levels: the primary structure is the amino acid sequence; the secondary structure is the local arrangement of atoms; the tertiary structure is the three-dimensional fold of the polypeptide chain; the quaternary structure is the arrangement of different polypeptide chains to make up the functional multimeric protein
- Secondary structure is commonly divided into three types – alpha helices, beta sheets and coils. Sometimes a fourth category, namely turns, is also included.
- The structure of a protein is encoded in its sequence.
- Structure prediction is carried out at the level of secondary and tertiary structure.
- A commonly used measure of accuracy in secondary structure prediction is Q_3, which evaluates the total percentage of residues correctly predicted in one of the three states, helix, sheet and coil.
- The CF technique is a first generation statistical method for secondary structure prediction.
- The CF method uses propensity values for each residues to exist in each of the three secondary structural states to predict the secondary structure of the sequence
- The propensities are calculated from analyses of the crystal structure database
- The GOR-I method of secondary structure prediction uses information theory together with the structure database to generate residue statistics
- The GOR-III method is an improvement of GOR-I and includes segment pair statistics, i.e. correlations between pairs of residues at different points in a stretch of secondary structure. This information improves the prediction.
- PHD is currently one of the most successful secondary structure prediction programs. It uses artificial neural networks to carry out the predictions.
- NNSSP and PREDATOR are again third generation programs for secondary structure prediction. These two algorithms base the prediction on the structure found in 'neighbour' sequences, i.e. sequence segments that are very closely similar to the reference sequence.
- Consensus methods use many different algorithms and programs. The final prediction comprises a consensus taken from all the resulting predictions.
- Best prediction accuracies are currently at the level of 85 to 90%. While improvements in the methods and in the reference database will increase this number, it is unlikely to reach 100%.

- Methods to predict the tertiary structure of proteins may be divided into three broad categories – homology modelling, threading and *ab initio* methods.
- Homology modelling is used when the unknown sequence, called the target, bears a sufficiently strong sequence similarity with another sequence, called the template, for which the structure is already known.
- The technique consists of four steps: selecting the template, alignment of target with template, building the model, and evaluating the model.
- Threading generalises the technique of homology modelling, and aligns the unknown sequence to a likely structure, which may be built from families of structures with sequences similar to the target.
- At its core, threading has a sequence-structure alignment algorithm.
- Threading may be divided into four tasks. First construct a complete library of possible folds or structural templates. Second, design a scoring or fitness function. Third, align the target sequence with every possible fold in the library. Fourth, evaluate the best possible alignment using the scoring function.
- An *ab initio* algorithm uses only the sequence of the protein, and the well-established laws and principles of physics and chemistry, to determine its three-dimensional structure.
- Since, according to physics, the global minimum energy conformation corresponds to the native structure of the protein, an *ab initio* algorithm must solve two problems. First, how is the energy of the protein to be defined? Second, how to find the conformation for which this energy is a minimum?
- The first problem has been addressed using semi-empirical force fields and statistical potentials.
- The second problem is addressed by using molecular dynamics simulations, or by adapting various global optimisation techniques, as well as by developing new ones.
- CASP stands for Critical Assessment of techniques for protein Structure Prediction. This is a 'community-wide' experiment to use a common set of tools to independently assess the effectiveness of the various structure prediction methodologies available.
- The Levinthal paradox asks -- how can a protein chain, that would take exponentially large amount of time to find the correct conformation if we consider only entropically driven thermodynamics, fold in finite amount of time, of the order of milliseconds to minutes?
- We may summarise the current understanding of the physical process of protein folding as follows. An ensemble of unfolded states collapses into an ensemble of intermediate states, that, by different pathways, all fold into the same final native form.

8

Nucleic Acid Structure

This chapter focuses on the structural bioinformatics of nucleic acids. As already described in Chapter 1, nucleic acids come in two major varieties that are relevant to biology – DNA and RNA. DNA stores genetic information and transmits it from one generation to the next. It is a quasi-periodic crystal in its three-dimensional structure. It is a double-stranded molecule, consisting of two nucleic acid strands that run in opposite directions. The two strands are wound around each other to form a double helix. The two strands stick to each other through hydrogen bonds that form between the bases in a highly specific manner. Of the four nucleotides A, T, G and C that make up the two DNA polynucleotide strands G always binds to C, and A always to T. This ensures that each strand of the double stranded molecule acts as a template for the other (Figure 8.1). Thus the biological function of DNA, viz. to store and transmit genetic information, is easily explained based on its double helical structure. In view of this, discerning readers may question the necessity for the present chapter, at least those portions of it dealing with DNA structure. What else is there to learn about its structure that Watson and Crick have not already spoken about, they may ask. The answer to that is, 'Quite a lot!' Modulations of the double helix and structures of DNA that are different from the double helix all have important biological roles to play, and these are all discussed in a later subsection.

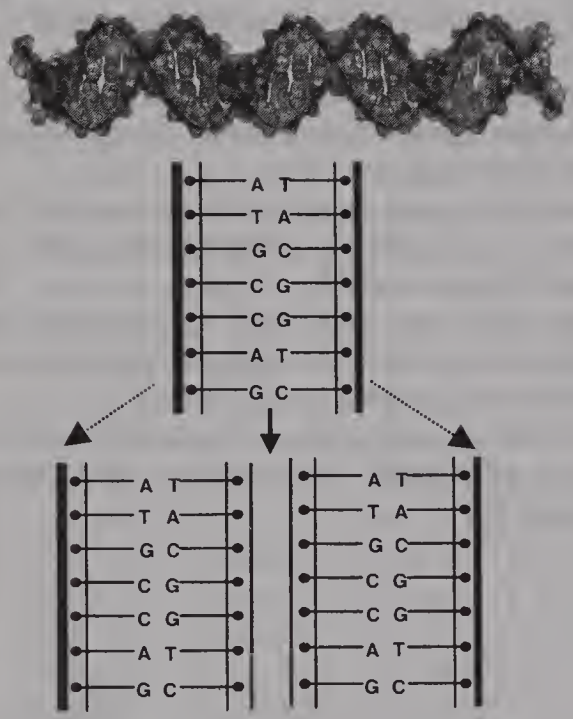


Figure 8.1. The two strands in the double helical structure of DNA act as templates for each other. This explains how information stored as the sequences of bases in the double helix can be transferred to the two daughter double helices.

The first part of the chapter deals with the other important nucleic acid – RNA. The change of the sugar moiety from deoxyribose to ribose, with its one extra oxygen atom, and the replacement of the base thymine by uracil convert the more stable DNA to the more flexible RNA. RNA can assume a

variety of structures, and this allows it to assume a variety of different roles. They act as messengers of the genetic information in the cell (mRNA), translators of the genetic code (tRNA) and even as enzymes with metabolic functions (ribozymes). RNA chains have secondary and tertiary structures that are dependent on their sequence. In this sense, as well in the sense of having different functions, they are very much like proteins. And, as in the case of proteins, a variety of different techniques have been developed to analyse RNA sequences, and predict secondary structure, tertiary structure and function.

8.1 RNA Structure Prediction

Most functional RNA molecules in living systems exist and function as single strands. There are exceptions to this, as for example the double-stranded genomes in some viruses. The secondary structure of double-stranded RNA is however better discussed in the context of double-stranded DNA, and here we will restrict ourselves to single stranded molecules. The lengths of the RNA strands range from about a few tens of nucleotides in the smallest to a few thousands in the largest. Not all of them assume secondary and tertiary structures that are essential for their function. mRNA is an example of an RNA molecule that does not take up any specific structure. However there are exceptions even here, and in some cases the secondary structure of mRNA plays an important regulatory role. In any case, for tRNA, rRNA, ribozymes (RNA enzymes) and a few other varieties of RNA, the secondary and tertiary folding is crucial for function. The secondary structure of RNA means the pattern in which portions of the strand fold back and form local, hydrogen-bonded (base-paired), double-stranded structures. With respect to the base pairs, apart from the well known Watson and Crick A-U and G-C base pairs, G-U base pairs also occur quite frequently in RNA molecules and these are also considered as part of the conventional secondary structure interactions. The cloverleaf[23] secondary structure of tRNA (Figure 8.2) is a well-known classic example. Note that the secondary structure of RNA is completely represented in a two-dimensional diagram, such as Figure 8.2. This is different from the secondary structure of proteins, by which we mean the local arrangement of atoms

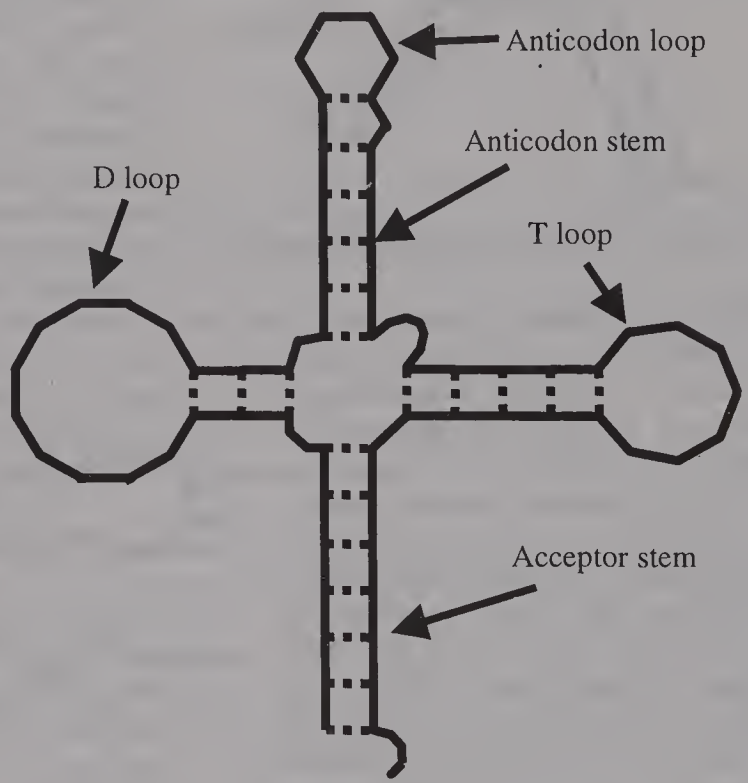


Figure 8.2. The clover-leaf structure of tRNA. Dotted lines represent base pairs.

[23] The clover is a small weed more common in European meadows than in Asian cities. Its leaf has a distinctive three-lobed shape. A stylised representation of this is found on playing cards in the suit 'Clubs', as for example in the 'Ace of Clubs'.

in three-dimensional space. The secondary structure is characterised by various motifs, which are important for the three-dimensional folding and the final functional structure of the molecule. Some of these motifs are pictured in Figure 8.3. They include stems, hairpin loops, bulges, internal loops and junctions. When the three-dimensional structure is formed, some of these elements come together to form higher-level base-paired structures, such as pseudoknots, kissing hairpins and hairpin-bulge contacts (Figure 8.4). Normally, however RNA secondary structure prediction does not include the latter types of structures, though often this also may be predicted by methods such as covariance analysis.

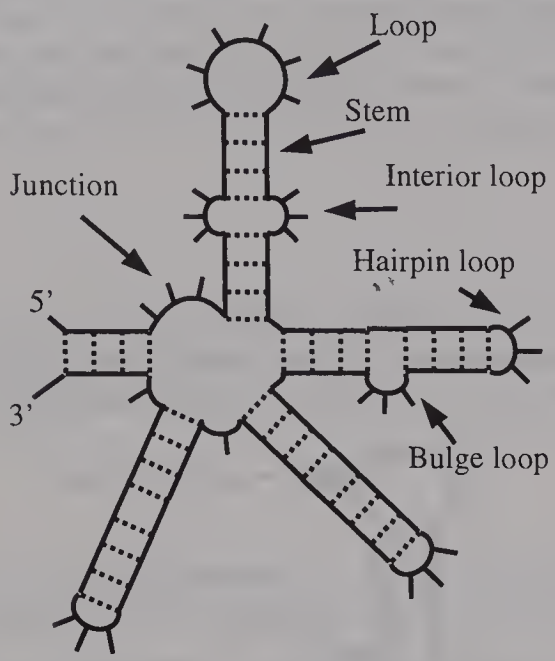


Figure 8.3 RNA secondary structure motifs

The secondary structure of RNA is easily represented in two dimensions (on paper) in a variety of ways. The most common method is to draw the secondary structure as a 'stem-loop' diagram (Figures 8.2, 8.3 and 8.4). This is satisfactory when the secondary structure is already known. Two other types of diagrams, the dot plot and the Nussinov circle (Figures 8.5a and b), serve not only as representations of a known secondary structure, but also help to arrive at the correct pattern when this is not known. Both these are diagrams of (potentially) base-paired regions in the molecule.

Dot plots have already been discussed in an earlier chapter in the context of sequence comparisons. In constructing a dot plot for the purpose of RNA secondary structure display, the initial steps are the same as in the previous case – the sequence is written on top from left to right, as well as at the left from top to bottom. This defines a matrix, each element of which is labelled by one residue from the sequence on top and one from the sequence at the left. In the present case, both are the same sequence. Each element of the matrix is filled with a dot when the two residues that label it are complementary to each other, and is left blank otherwise. In the completed dot plot, therefore, lines of dots perpendicular to the main diagonal represent regions of the sequences that may form a base paired stretch with another region of the same sequence. Such a dot plot is obviously symmetrical (mirror symmetry) about

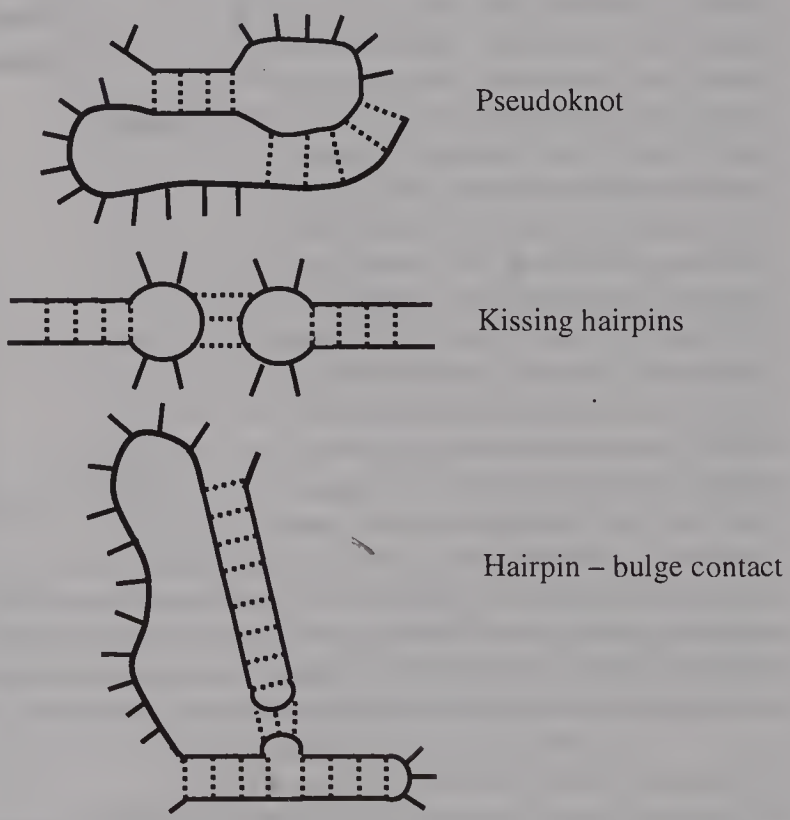


Figure 8.4 RNA tertiary contacts

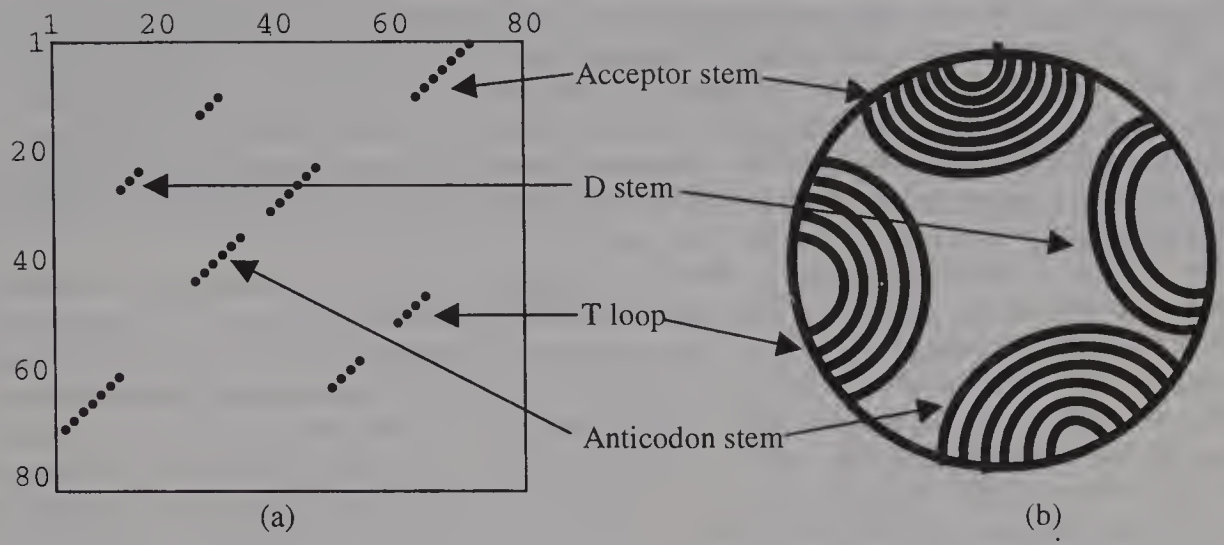


Figure 8.5 Representations of tRNA secondary structure. (a) Dot plot. (b) Nussinov circle.

the main diagonal and only half of it need be considered. The main diagonal itself is of course left blank. A further feature of such plots in the context of physically possible secondary structures arises from the fact that the two strands of base paired stem cannot follow one immediately behind the other in the RNA sequence – there needs to be a loop of at the very least three residues, usually four or more, before the strand can fold back to form base pairs. This means that in the dot plot, the lines of dots that represent possible stems need to be at least three residues away from the main diagonal. Lines that are any closer, or actually cross the diagonal, cannot be physically created in the RNA strand.

The Nussinov circle is the basis for the most commonly used dynamic programming method of secondary structure prediction. The concept of the representation is simple – the sequence is written around the circumference of a circle starting at an arbitrary point and ending at the same point (or rather, one just previous to it) after traversing all around the circle. Lines joining any two points on the circumference represent base pairs between the corresponding bases. Both dot plots and Nussinov circles can be used to represent all possible base pairs, but the latter provides an immediate tool to specify pairs of mutually incompatible base pairing patterns. When the pair of patterns is represented on the same Nussinov diagram, the lines joining the paired bases intersect. Therefore, one may state the rule that the Nussinov diagram of physically meaningful RNA secondary structure will not have intersecting lines. Of course, the rule does not include pseudoknots or other tertiary contacts, which will have intersecting lines when represented on a Nussinov circle.

8.1.1 Dynamic programming – the Nussinov algorithm

Finding the best secondary structure for a sequence, just like finding the best sequence alignment, is an NP problem in terms of computational complexity. This means that for an exhaustive solution of the problem, there is no algorithm that will complete it in an amount of time that can be expressed as a polynomial function of the length of the sequence. To appreciate this better, consider a sequence of length N. To find the best secondary structure, defined simply as one that contains the maximum number of base pairs, we need to consider pairing the first base with every other base in the sequence. This means N calculations. For each such pairing, we need to consider the next base as pairing with every other base left out of the first pairing. The two together involve a total of approximately N x (N-1) calculations. Continuing in this fashion we see that, crudely speaking, an exhaustive search would require N! calculations, which of course is a much large number than a polynomial function of

N. Even if we argue more realistically and eliminate pairings that lead to knots and other unrealistic features, we still end up with calculations growing as a^N, where a is some constant number. In other words, the computation time required grows exponentially. As in the case of sequence alignment, dynamic programming can reduce this substantially, though at a cost. The search is not guaranteed to produce the very best secondary structure, only one of the very best. This trade-off however is completely acceptable, and in practise, structures arrived at from applications of dynamic programming have turned out to be very close to the experimentally determined ones when these are finally made available.

The Nussinov method is a simple dynamic programming algorithm to predict RNA secondary structure. The method has since been expanded and further developed, and now it is a reliable and automatic computer program. We will first discuss the original algorithm, and afterwards explain later developments to it. The algorithm itself is best explained by working out an example. Before we do so however, we will briefly discuss the theory that lies behind it. As originally developed, the method searches for a secondary structure scheme that maximizes the number of base pairs. It does not consider other factors, such as base stacking or the strength of the base pairs. This search is carried out on the recursive principle of looking for a plausible first solution, and then building on it and refining it in steps. Thus, we start with an assumption that we have already found a secondary structure for a small portion of the sequence that has a total of N residues, labelled serially with the indices from 1 to N. We consider that the portion of the sequence whose secondary structure is known lies between residue i and residue j (i is less than j), and we now have to extend the structure by adding these two residues to it. There are four (and only four) ways that this can be done, as shown in Figure 8.6 – residue i and residue j may pair with each other and add this new base pair to the existing structure (Figure 8.6a); or, if in the substructure, residue j-1 is unpaired, residue i may pair with it and residue j may remain unpaired (Figure 8.6b); or, if in the substructure, residue i+1 is unpaired, residue j may pair with it and residue j may remain unpaired (Figure 8.6c); or finally, in the case where two residues k1 and k2 lying between i and j are both unpaired, they may form base pairs with i and j, thus introducing two optimal substructures (Figure 8.6d).

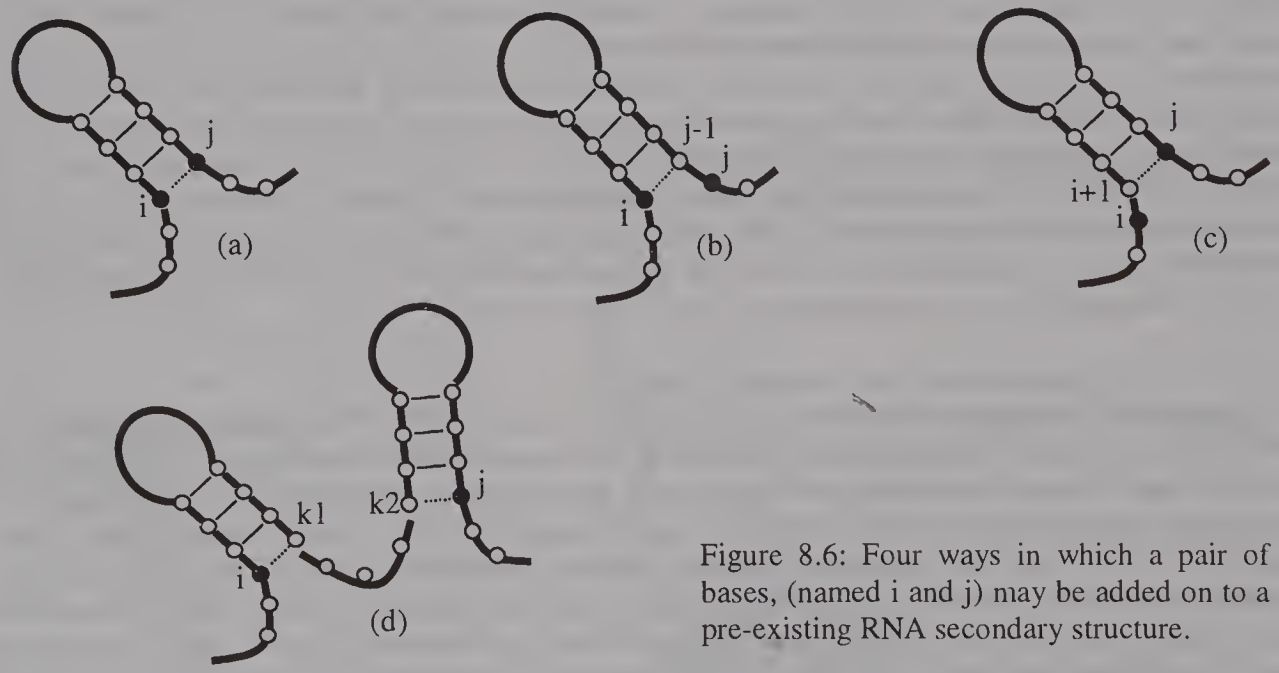


Figure 8.6: Four ways in which a pair of bases, (named i and j) may be added on to a pre-existing RNA secondary structure.

The Nussinov dynamic programming algorithm, which uses the above idea, is illustrated on the following example (Figure 8.7). The sequence chosen for the example is AAAGCCCUU. This is a

simple sequence with length L = 9, and can have many secondary structures, all with the same number of base pairs. The task is to automatically find one (or all) of them. The algorithm has three main steps: constructing and initialising the matrix; filling up the elements of the matrix using a recursion algorithm; and finally constructing the trace back.

The first step is simple. The sequence is written both horizontally across the top and vertically at the left. This defines a matrix, called G(i,j), each element of which is labelled by one residue i from the top and one residue j from the left (Figure 8.7a). Since this matrix is a comparison of a sequence with itself, it is always symmetrical across the main diagonal (see Chapter 3) and only half the matrix need

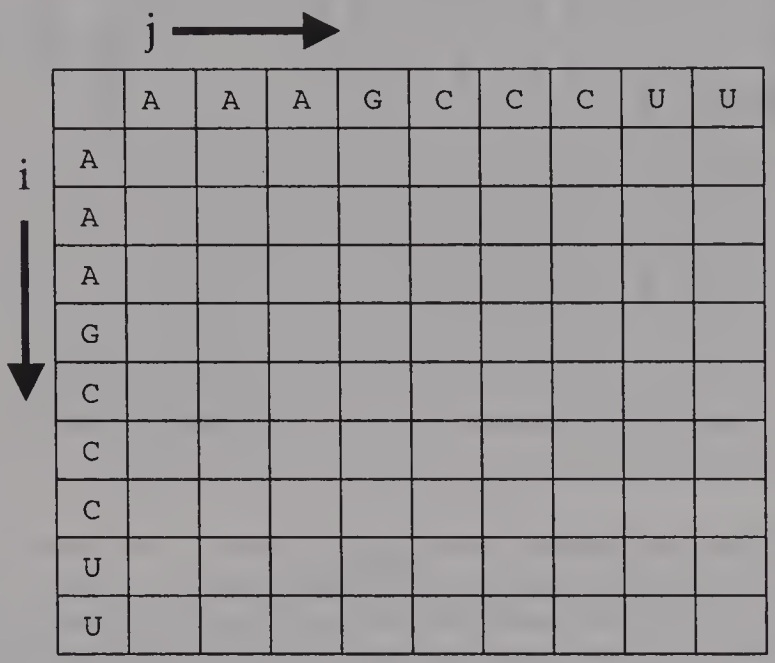


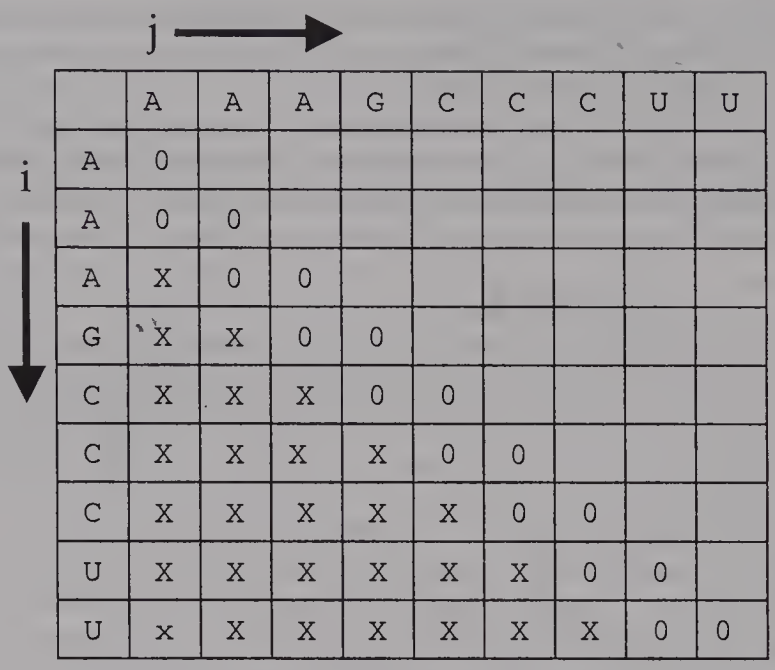
Figure 8.7a The matrix G(i,j)

be considered. Conventionally this is chosen to be the upper right half. The initialisation of the matrix is carried out as follows: Set G(i,i-1) = 0 for i = 2 to L, and set G(i,i) = 0 for i = 1 to L. This sets the main diagonal and the diagonal just below, both to zero, thereby ensuring that base pairs do not occur between a residue and the immediate next one. In addition, the entire lower left half of the matrix is blanked out, since we do not use this. The result of this operation is seen in Figure 8.7b.

The next step is to recursively fill up the matrix. We start from (i = 1, j = 2), and apply the following set of operations to each cell, incrementing both i and j by 1 each time. When we come to (i = L-1, j = L), we start again by setting (i = 2, j = 3), and so on until the matrix is filled. The operation at each cell is as follows: For all subsequences of length 2, to length L, we set

$$G(i,j) = \text{Maximum of} \begin{cases} G(i+1,j) \\ G(i,j-1) \\ G(i+1,j-1) + d(i,j) \\ \text{Maximum of } [G(i,k) + G(k+1,j)] \text{ for all values of k} \\ \quad \text{between i+1 and j-1.} \end{cases}$$

In the above expression, d(i,j) = 1.0 if the residue at sequence position i makes a base pair with the residue at sequence position j. Otherwise d(i,j) = 0.0. In words, to find the value of the matrix element G(i,j) we choose the largest of four different numbers. The first number is just the value of the matrix element G(i+1,j), which has already been determined. The second number is the value of the matrix element G(i,j-1), which likewise has already been determined. The third number is the value of the matrix element G(i+1,j-1) plus the value of any extra base pair between residue i and residue j added to



	A	A	A	G	C	C	C	U	U
A	0								
A	0	0							
A	X	0	0						
G	X	X	0	0					
C	X	X	X	0	0				
C	X	X	X	X	0	0			
C	X	X	X	X	X	0	0		
U	X	X	X	X	X	X	0	0	
U	x	X	X	X	X	X	X	0	0

Figure 8.7b The matrix G(i,j) after the initialisation step.

the existing base pairs. The fourth number involves the calculation of the expression given in square brackets for all values of the auxiliary index k, which ranges between i+1 and j-1. Thus if k is set to i+1, the value [G(i,i+1) + G(i+2,j)] is calculated. Next k is set to i+2 and the value [G(i,i+2) + G(k+3,j)] is calculated. This is repeated until k reaches the value j-1. Obviously, the number of times this expression is evaluated depends on the range between i and j. When the range has been exhausted, we compare all these numbers and set the largest among them as the fourth possible value for G(i,j).

The final part of this operation is to compare the four numbers calculated for G(i,j) and choose the largest of them. This is used the update the matrix cell. (In order to facilitate the operation of the third and final step, viz. the traceback procedure, we also store an indication as to which of the four possibilities considered gave rise to the value in the current cell). This completes the operations for this cell, and the next cell corresponding to the next pair of values (i,j) is taken up for calculation. Figure 8.7c shows various steps as the matrix is filled up. It may be noted that the four numbers calculated at each step correspond, respectively, to the four possible ways in which the pair of residues (i,j) may be added on to the already existing substructure. Thus the first number corresponds to the new score if the pair (i,j) is added such that residue i forms a pair with residue j-1 and residue j remains unpaired. The second number corresponds to the situation when residue i remains unpaired and j pairs with i-1. The third number corresponds to the situation when a base pair formed between residue i and residue j is added to the secondary structure. Finally, the fourth number corresponds to the situation when the existing sub-structures are combined, with i and j making base pairs with k and k+1, respectively.

Once the matrix has been filled by recursively applying the algorithm in step two, the score of the best possible secondary structure is easily obtained by inspection, as the highest valued in the matrix. Then actual structure itself is obtained in the third and final step by a simple traceback procedure, as follows. We start at the cell at the top right hand corner, which will have largest possible score. The indices of this cell are (1,L), where L is the length of the sequence. In order to find the next cell along the traceback route, we used the information stored in the previous step to find out which of

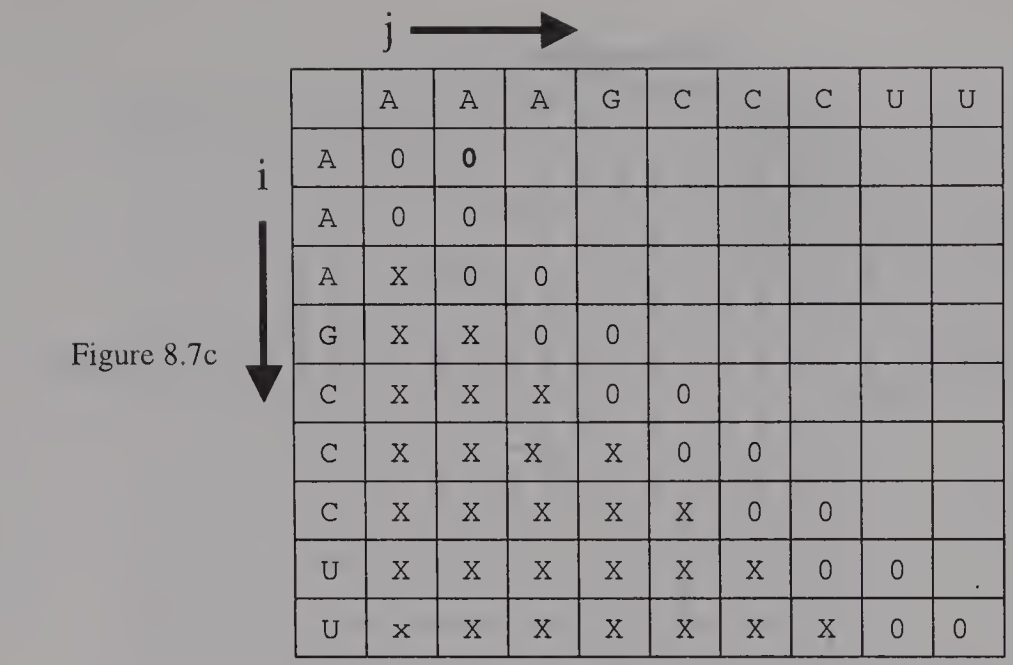


Figure 8.7c

G(1,2) = Maximum of
- G(2,2) = 0
- G(1,1) = 0
- G(2,1) + d(1,2) = 0 + 0 = 0
- Fourth term = 0, since k cannot take any value between 1 and 2

} = 0

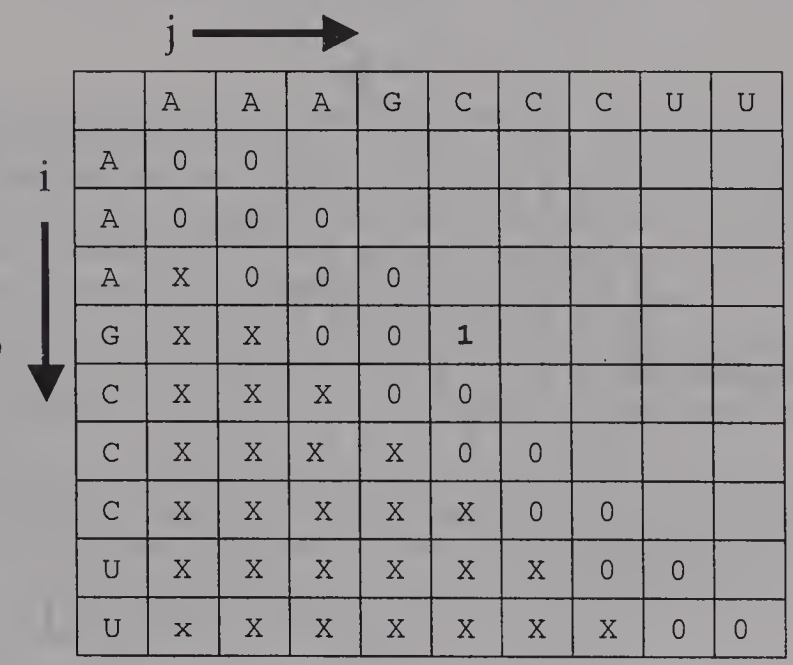
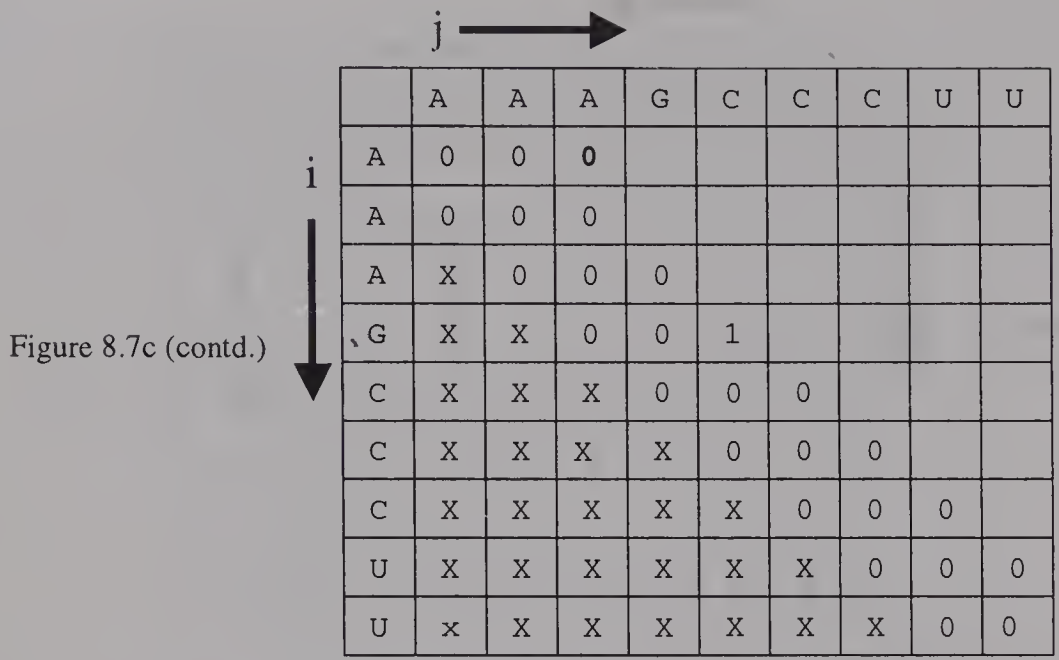


Figure 8.7c (contd.)

G(4,5) = Maximum of
- G(5,5) = 0
- G(4,4) = 0
- G(5,4) + d(4,5) = 0 + 1 = 1
- Fourth term = 0, since k cannot take any value between 4 and 5

} = 1

Figure 8.7c (contd.)

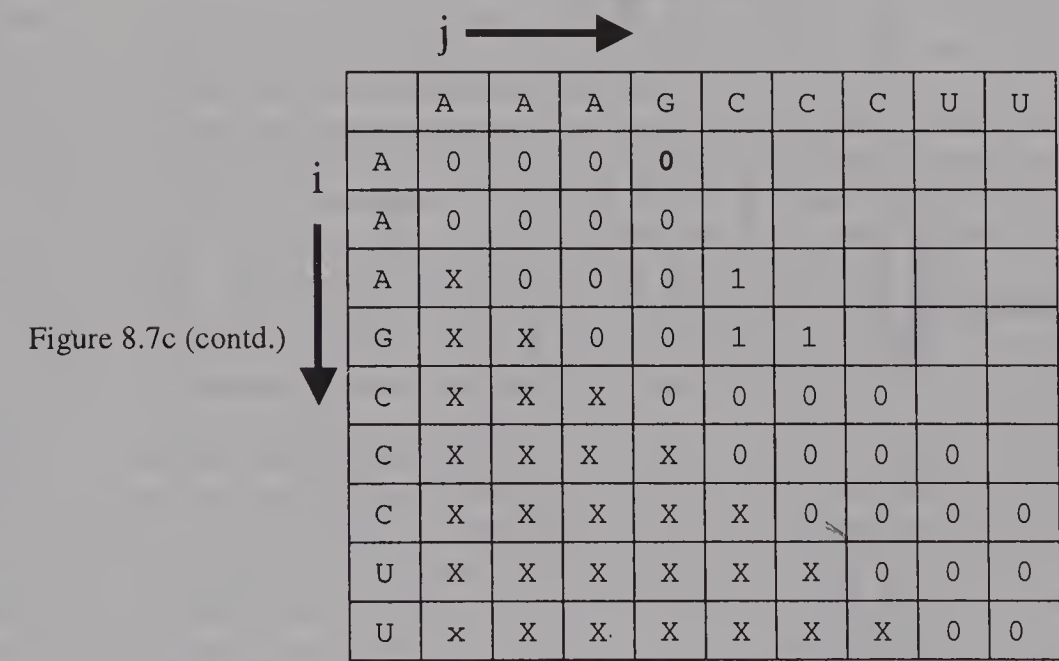


		A	A	A	G	C	C	C	U	U
i →	A	0	0	**0**						
	A	0	0	0						
	A	X	0	0	0					
	G	X	X	0	0	1				
	C	X	X	X	0	0	0			
	C	X	X	X	X	0	0	0		
	C	X	X	X	X	X	0	0	0	
	U	X	X	X	X	X	X	0	0	0
	U	x	X	X	X	X	X	X	0	0

j →

$$G(1,3) = \text{Maximum of} \begin{cases} G(2,3) = 0 \\ G(1,2) = 0 \\ G(2,2) + d(1,3) = 0 + 0 = 0 \\ \text{Maximum of } \{G(1,2) + G(3,3) = 0 + 0 = 0\} = 0 \end{cases} = 0$$

Figure 8.7c (contd.)

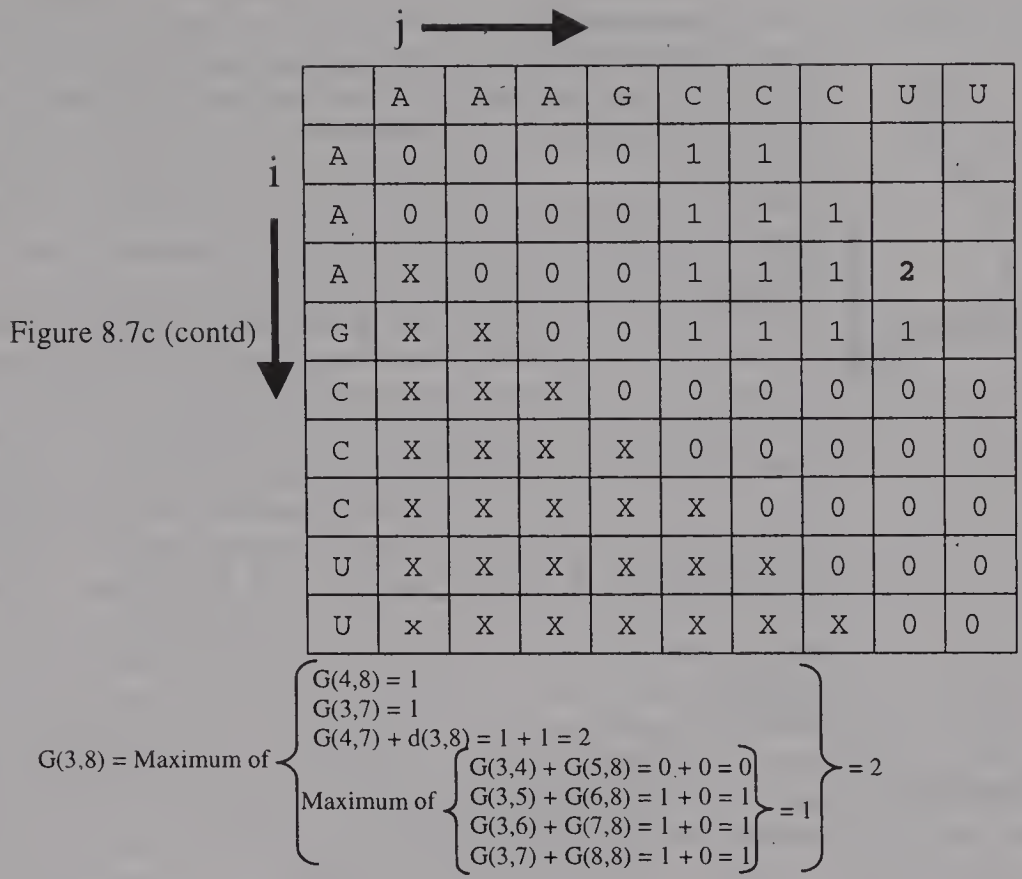


		A	A	A	G	C	C	C	U	U
i →	A	0	0	0	**0**					
	A	0	0	0	0					
	A	X	0	0	0	1				
	G	X	X	0	0	1	1			
	C	X	X	X	0	0	0	0		
	C	X	X	X	X	0	0	0	0	
	C	X	X	X	X	X	0	0	0	0
	U	X	X	X	X	X	X	0	0	0
	U	x	X	X	X	X	X	X	0	0

j →

$$G(1,4) = \text{Maximum of} \begin{cases} G(2,4) = 0 \\ G(1,3) = 0 \\ G(2,3) + d(1,4) = 0 + 0 = 0 \\ \text{Maximum of} \begin{cases} G(1,2) + G(3,4) = 0 + 0 = 0 \\ G(1,3) + G(4,4) = 0 + 0 = 0 \end{cases} = 0 \end{cases} = 0$$

Figure 8.7c (contd)



the four possibilities was used to arrive at the current position. If it were one of the first three numbers, the situation is straightforward and the corresponding cell is added to the traceback route. For instance if it were the first number that was the maximum, then an (i,j-1) base pair is added to the secondary structure. Similarly, if it were one of the next two numbers, an appropriate pair of bases is added to the structure. However, if it were the fourth number, then this corresponds to a bifurcation or a branching. The traceback route then splits into two branches and the above procedure has to be repeated along each branch, until the end of every branch is reached. This may, of course, happen several times, if there were several branches. (The illustration in Figure 8.7c does not have any branching and shows a simple case). This concludes the algorithm. The best secondary structure for the sequence has been determined (Figure 8.8).

We note that, as shown in Figure 8.8, it is sometimes possible to find other, equally optimal, secondary structures starting from the same cell. In fact this depends on the order in which we search for the appropriate next cell in the traceback algorithm. The algorithm, as it stands, does not distinguish between these different structures. Other constraints and criteria have to be included in the procedure in order to overcome this kind of degeneracy.

8.1.2 Dynamic programming – the Zuker algorithm

In the Nussinov algorithm, the correctness of a given secondary structure was evaluated simply by counting the number of base pairs it had. The best structure would have the largest possible the number of total base pairs. It is obvious this is too simplified an approach. It could lead, in some cases, to absurd and incorrect results, some of which are detailed below.

Figure 8.7c (concluded)

		A	A	A	G	C	C	C	U	U
i	A	0	0	0	0	1	1	1	2	**3**
	A	0	0	0	0	1	1	1	2	3
	A	X	0	0	0	1	1	1	2	2
	G	X	X	0	0	1	1	1	1	2
	C	X	X	X	0	0	0	0	0	0
	C	X	X	X	X	0	0	0	0	0
	C	X	X	X	X	X	0	0	0	0
	U	X	X	X	X	X	X	0	0	0
	U	X	X	X	X	X	X	X	0	0

G(1,9) = Maximum of {
- G(2,9) = 3
- G(1,8) = 2
- G(2,8) + d(3,8) = 2 + 1 = 3
- Maximum of { G(1,2) + G(3,9) = 0 + 2 = 2; G(1,3) + G(4,9) = 0 + 1 = 1; G(1,4) + G(5,9) = 0 + 0 = 0; G(1,5) + G(6,9) = 1 + 0 = 1; G(1,6) + G(7,9) = 1 + 0 = 1; G(1,7) + G(8,9) = 1 + 0 = 1; G(1,8) + G(9,9) = 2 + 0 = 2 } = 2

} = 3

	A	A	A	G	C	C	C	U	U
A	0	0	0	0	1	1	1	2	***3***
A	0	0	0	0	1	1	1	2	***3***
A	X	0	0	0	1	1	1	***2***	2
G	X	X	0	0	1	1	***1***	1	2
C	X	X	X	0	0	***0***	0	0	0
C	X	X	X	X	0	***0***	0	0	0
C	X	X	X	X	X	0	0	0	0
U	X	X	X	X	X	X	0	0	0
U	X	X	X	X	X	X	X	0	0

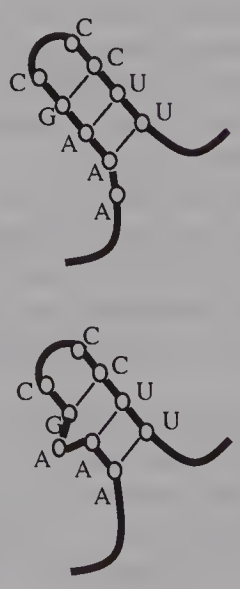


Figure 8.8 The traceback, and the two possible secondary structures for this matrix

- An example of an absurd structure is shown in Figure 8.9, where a loop with 'zero' residues is formed if we were to simply maximize the base pairs. This is physically impossible, and experimental and modelling studies have shown that a RNA loop must consist of a minimum of three bases. (In that sense, the sequence shown above in Figures 8.7 and 8.8 cannot form a stable stem-loop structure.)
- There is a difference in energy between an A.T base pair and a C.G base pair, the latter being more stable than the former. Given a choice between the two, with all other things being equal, a well designed algorithm must chose a C.G base pair. The Nussinov algorithm does not make any provision for this.
- If a situation arose where we must choose between two sets of base pairs with equivalent total energy, e.g. three A.T base pairs versus two C.G base pairs, other criteria are used, such a base stacking energy. The Nussinov algorithm does not incorporate these.
- In general, the stability of particular secondary structure also depends on a number of other factors such as number and size of loops, type of loops, number of branch points, possible tertiary interactions, and so on.

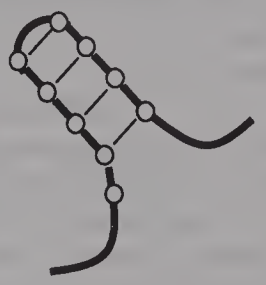


Figure 8.9. An 'impossible' secondary structure.

It is of course difficult, if not impossible to incorporate all these factors into any secondary structure prediction algorithm. The quickest and easiest way to do this, at least in part, is using simple energy rules. This is the track followed by the Zuker algorithm. This involves an extensive modification of the both the filling step (step 2 above) and the traceback step (step 3 above). One modification of the filling step is to assign an energy values, more specifically a value for the Gibb's free energy, to each base pair in a secondary structure. That is, there is a function e such that e(i,j) is the energy of a base pair. The Gibb's free energy of the entire structure, is then given by:

$$E = \Sigma_{i,j}\, e(i,j)$$

for all base pairs(i,j) in the structure. Reasonable values of e at 37° C are -3, -2 and -1 kcal/mole for GC, AU and GU base pairs, respectively. Unfortunately, such simple rules are insufficient to capture the destabilizing effects of various loops, or the nearest neighbour interactions in helices and loops.

These were taken into consideration by means of loop-dependent energy rules. In this context, a 'k-loop' is a specific term used to describe a sub-structure that consists of one or more base pairs, and at least one single stranded section (Figure 8.10). A formal definition of a k-loop is the following. If (i,j) is a base pair in the secondary structure, and $i < k < j$, we say that k is 'accessible' from (i,j) if there is no base pair (i',j') such that $i < i' < k < j' < j$. Similarly, we say that the base pair (k,l) is accessible if both k and l are accessible. The set of (k-l) base pairs and k' single-stranded bases accessible from (i,j) is called the k-loop closed by (i,j). A 'null' k-loop, consists of those single- and double-stranded bases accessible from no base pair. This is usually referred to as the 'exterior loop', and its bases and base pairs are called 'free' bases and 'free' base pairs, respectively. Then any secondary structure partitions the sequence, uniquely into m number of k-loops. Note that m is the number of base pairs in the secondary structure. Energies are assigned to the k-loops, and the energy of any particular secondary structure is given by:

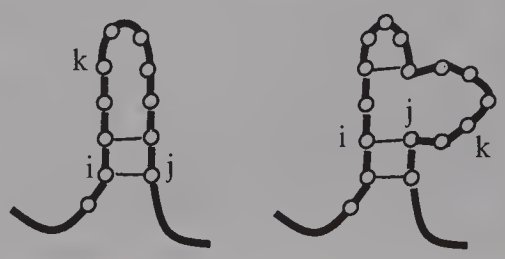


Figure 8.10. k loops

$$E = \Sigma_i\, e(L_i), \text{ for i ranging from 0 to m.}$$

e is now a function of k-loops instead of a function of base pairs. The k-loop concept is just a more formal way of defining the loops described previously. Thus, a 1-loop is just a hairpin loop, and a 2-

loop may a stacked pair, bulge loop or exterior loop. The values $e(L_i)$ have been calculated for each type of loop and for different sizes of each loop and tabulated. These values have been obtained from analysis of experiments and from theoretical physical chemistry of polymers. For example the increment in the energy due to a hairpin loop of size L (i.e. L single stranded bases in the loop) is calculated to be

$$\Delta E = 1.75 \times RT \times \ln(L)$$

where R is the gas constant and T is the absolute temperature. This expression is in addition to the energy of the base pairs, which again depend not only on the base pair type, i.e. AT, CG or GU, but also on how there are juxtaposed, i.e. on the stacking energies. Such numbers have also been calculated and tabulated and can be used to estimate the increase in energy at every step of the secondary structure building process. Other features of the secondary structure are similarly scored in terms of their contribution to the energy. They include stacked pairs, helices (more than 2 consecutive stacked pairs), bulge loops and interior loops. Terms are also tabulated as destabilising features, such as the case where the end of the loop is not a GC base pair, the case of a highly asymmetric interior loop, and the case of multiply branched loops. The fill and traceback algorithms mentioned above cannot be used directly for loop-dependent energies. The steps have to be modified to take the different types of loops and their energies into consideration. This requires a complete reformulation of the steps.

MFOLD is the program that incorporates all the above features and is widely used for RNA secondary structure prediction. The method is suitable not just for predicting the structure, but also to calculate various thermodynamic quantities for RNA sequences, such as melting temperature and heat capacity. Figure 8.11 gives the heat capacity as a function of temperature calculated using the above methodology for the 5S ribosomal RNA sequence from rat. The peaks and troughs in the curve represent the transitions that occur in the heat capacity due to secondary structural transitions. The curve can of course be verified experimentally, and a close match between theory and experiment has been observed.

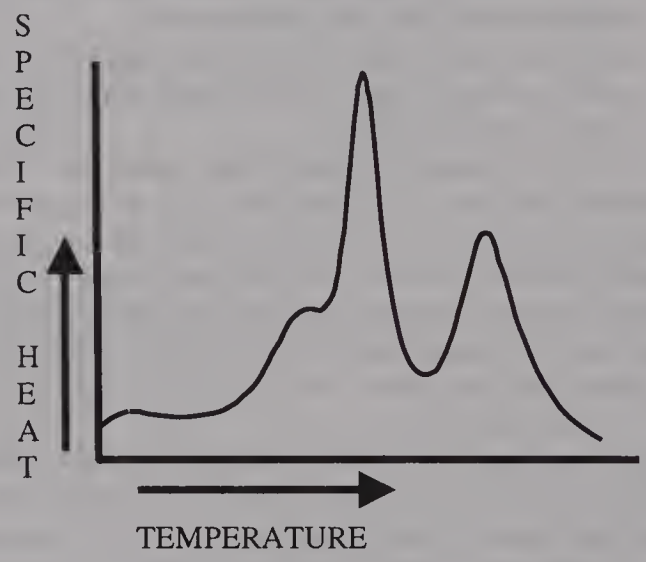


Figure 8.11 The heat capacity curve for the 5S ribosomal RNA of rat.

8.1.3 Structure prediction by analysis of sequence covariance

This is a powerful method of secondary structure prediction, based on analysis of RNA sequences from multiple sources. If we know that the functions of a set of sequences are all the same, then we are sure that their tertiary structures, and even more so, their secondary structures are also the same. Therefore, multiple alignment of the entire set of sequences would enable us to identify those bases that change so that a particular base pairing scheme is conserved. In other words, we may identify covariance in pairs of bases that may be far apart in the sequence, but that would probably pair with each other when the secondary structure is formed. Figure 8.12 shows a portion of a set of six tRNA sequences. This is the anticodon stem portion, and the correlated variation in the bases that form the base paired stem is clearly seen. The principle underlying such covariance is that even though the sequence may change,

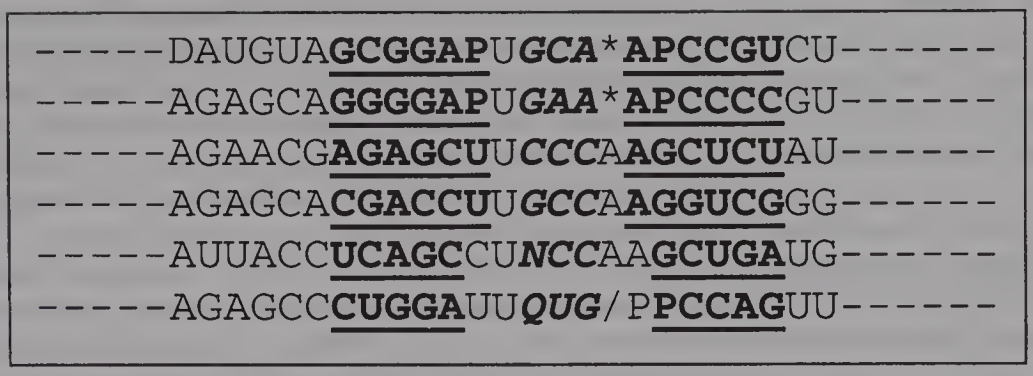


Figure 8.12. The anticodon portion of the alignment of six tRNA sequences. The covarying bases are indicated in 'bold and underline'. The anticodon is in italics. The characters D, P, *, N, Q and / stand for special nucleoside bases found in tRNA. In particular, P stands for pseudouridine, which base pairs with adenine. Note that though the six sequences are different, the variations are such that the anticodon stem has at least five base pairs.

the secondary structure is retained between sequences, and therefore when one base of a hydrogen-bonded pair changes, its partner would also change in order to retain the pairing.

Several formal methods of covariance analysis use visual or numerical calculations that lead to representations of the secondary structure. We discuss one such method, which is not currently in widespread use, but is simple and illustrative of this class of algorithms. The method used dots of two different colours to highlight differences between two sequences to assist in the search for base paired stems. It involved first aligning two sequences to maximize their primary structure similarity. Then, coloured dots were used to indicate any changes at a given position for the two aligned sequences. Each position falls into one of three categories. The nucleotide at that position could be the same in the two sequences. In this case, the position was not marked by any dot. The second possibility is that a 'transition' could occur, i.e. a purine in one sequence could be substituted by a different purine in the other sequence, or a pyrimidine could be replaced by another pyrimidine. This was indicated by a dot of one particular colour, say red. The third type of change was called a 'transversion' and was indicated by a dot of a different colour (say green). This occurred when a purine in one sequence was substituted by a pyrimidine in the other. The patterns of red and green dots so obtained could be used to match different regions of the aligned sequences and map out covariances that indicated secondary structure regions. A large number of pairs of aligned sequences were used before conclusively arriving at the final secondary structure prediction. There are several obvious drawbacks to this method. In the first place, it is difficult to automate, requiring human intervention (though this may actually be an advantage!). Thus, it can only analyse two sequences at a time. Secondly, it allows the identification only of secondary structure stems containing consecutive, antiparallel, nested base pairs. It requires strict covariation, and at least two compensatory base changes as proof for a particular base paired region.

Another method involves the calculation of a term called the mutual information content. The calculation starts with a rigorous multiple sequence alignment of entire set of RNA sequences being considered. (We note in passing that such multiple sequence alignment is itself often carried out based on known secondary structure, or based on known covariance. However this will complicate the discussion, and we will not consider the methods of multiple sequence alignment here, but in a separate chapter, viz. Chapter 4.) The alignment is then analysed to extract the mutual information content between the columns. This is a measure of the covariance between the bases in any two

columns. It is calculated as a number between 0 (no covariance at all) and 2 (perfect covariance) bits, depending on whether the presence of a particular base in one column contains no information at all or perfectly predicts what base will be present in the other column. The expression for the calculation is derived from information theory and uses the frequencies of occurrence of each base at each column in the alignment to obtain the value of the mutual information content for every possible pair of bases. Thus if the length of each sequence is L. there will a total of L*(L-1) such numbers, and they could be displayed as matrix. It is then simple to identify regions that would probably be involved in strong secondary interactions, perhaps forming base paired stems.

The mutual information content is often supported by another measure that counts the total number of changes that occur across a multiple sequence alignment. Correlated changes may be the consequence of the need to conserve structure or function in different organisms at every position. The measure, called 'event count', is derived from the ratio of the number of correlated changes to the total number of changes that occur between sequences from different organisms. The score that is generated is then used to evaluate possible interactions. The method assumes that a phylogenetic tree can be constructed for the multiple sequence alignment, which can be used to identify 'neighbouring' sequences in which changes occur. Under these conditions, multiple changes between closely related sequences contribute more to the score than a few changes between distantly related species.

A more recent implementation of covariance analysis makes use of the so-called Stochastic Context Free Grammars (SCFG). The method is used more to identify sequences that have a secondary folding similar to one already known, rather than to 'predict' the secondary structure of a new sequence in the manner of methods discussed above. As the name implies these methods were originally devised to analyse languages. Their use now however extends far beyond linguistics, and they may be applied to any system that consists of discrete strings of symbols. RNA sequences fit this description thoroughly – they are strings of symbols chosen from four different possibilities, namely A, U, C and G. In the terminology of SCFG, these symbols that represent the final sequence, are called 'terminal' symbols. The non-terminal symbols are those used in the 'production' runs to generate the final sequence. Conventionally these are written down as S_0, S_1, S_2, etc. A set of 'production rules' are defined, which when applied recursively to the non-terminal symbols, lead to the final correct sequence. For example one may write down one possible production rule as; $S_0 \rightarrow S_1$; $S_1 \rightarrow CS_2G$. Here we still have not defined S_2, and we may need other production rules to define these and other symbols that we may use. The production rule written above, simply means that, no matter what the rule that generates S_2, (or no matter what the context), we always write down two nucleotide bases, one at each end of the 'structure' defined by S_2. (This is how the concept of covariance is incorporated into the analysis. To use SCFG for other types of sequence analyses, the production rules may involve only one base or even many more bases, instead of exactly a pair of them.) Of course we may write down any one of the 16 possible pairs of bases in this rule, or in other words, we may write down sixteen different production rules for this step alone. For example, in addition to the one above, we may write $S_1 \rightarrow AS_2U$, $S_1 \rightarrow GS_2U$, and so on. Each of this has a different level of probability. (That is why the method is 'stochastic'). For example if the pair of bases in consideration were to form part of a secondary structural stem, the rules $S_1 \rightarrow CS_2G$, $S_1 \rightarrow AS_2U$, $S_1 \rightarrow CS_2G$, $S_1 \rightarrow US_2A$, $S_1 \rightarrow GS_2U$ and $S_1 \rightarrow US_2G$ would be much more probable than the remaining ten, which do not lead to hydrogen bonded base pairs. The probabilities for the production rules are calculated from a set of sequences whose secondary structure is known correctly. Once the production rules have been written down and all the probabilities calculated, then we have at hand a good probabilistic model for the secondary structure. We may then use this to identify possible structures in new sequences that are presented to us.

8.1.4 RNA tertiary structure prediction and RNA folding

The prediction and theoretical understanding of the three-dimensional structure of RNA molecules has not been the subject, so far, of wide-spread research, at least not to the extent that protein folding and structure prediction has been investigated. If we consider the problem superficially, it may appear that RNA tertiary structure prediction ought to be simpler than protein structure prediction, purely because the number of different groups that interact are only the four bases (as compared to the twenty amino acids for protein) with the strongest interaction between them being the formation of Watson-Crick base pairs. The reality however is somewhat different. In the first place, as we have seen above, Watson-Crick base pairs, together with the 'wobble' G.U base pair decide the secondary structure of the molecule. Other types of interactions between the bases, and between the bases and the backbone atoms, decide the tertiary folding. Secondly, this folding is usually mediated by ions that neutralise the negative charge on the phosphate groups, and it is difficult to incorporate these effects into an automatic algorithm for RNA tertiary structure prediction. Early attempts at prediction concentrated on the structure of transfer RNA or tRNA. The discovery of its cloverleaf secondary structure, both by sequence analysis and by biochemical studies, lead to many workers attempting to predict its tertiary structure. When the X-ray crystal structure of the molecule was finally determined, it was seen that the predicted structures were partly true, but nevertheless could not used as models for any further analysis. In most of the predicted models the four stems in the cloverleaf were arranged so that they formed two more or less continuous columns of helices, each column consisting of two helices arranged one on top of the other. The columns themselves were arranged parallel to one another. The crystal structure of the molecule indeed showed that the four stems were arranged in two columns of helices. However, the two columns were not parallel, but almost perpendicular to each other in the well-known 'L' shaped arrangement. The structure was stabilised by several tertiary interactions between the bases, such as the formation of hydrogen bonded base triplets, and stacking interactions between the bases in different loops. These tertiary interactions could not be anticipated purely from the secondary structure. Given this experience, and given the fact that experimentally determined RNA structures are much fewer than protein structures, it is only in recent times that efforts are being made to understand and predict RNA structure.

One of the major efforts is the construction of an RNA motif database called SCOR (scor.lbl.gov), identifying repeated patterns of tertiary structure in RNA. Such patterns include the following:

- Co-axial helices, in which two separate helical stems in the RNA secondary structure stack on top of each other to form a pseudo-continuous helix, as for example the D stem and the anti-codon stem of tRNA.
- Tetraloop-receptor, in which adenine bases that are adjacent along the sequence form a pseudo base pair (and thus form a structure known as the AA platform) and open up one side for stacking or pairing with bases from a non-contiguous RNA strand. Another sequence motif that forms a similar type of structure is the GNRA loop (G = guanine, N = any nucleotide, R = any purine and A = adenine).
- Ribose Zippers, in which the ribose hydroxyl group on one strand makes a bifurcated hydrogen bond with a base atom as well as the ribose group on the other strand.
- A-minor motif, in which the minor groove edges of adenine bases, which do not have any exocyclic atoms, are inserted into the minor grooves of neighbouring helices, particularly where there are CG base pairs.
- Kissing hairpin loops, in which the bases of two loops that are far apart in the sequence, but close together in the structure, form base pairs with each other, forming a double-stranded helix with one strand from each loop.

- Triple stranded structures, in which a double stranded stem accommodates a third strand from elsewhere in the sequences in either of the grooves to form a three-stranded helix. Pseudo knots, in general, and the interaction between the D loop and the T stem and loop of tRNA, have this motif.

Each motif has some characteristic sequence features, which makes it sometimes possible to identify it from sequence alone, and may thus be used in RNA tertiary structure prediction.

Current methods of computer modelling of RNA structure depends a great deal on phylogenetic and covariation analysis using RNA sequences. In these efforts, sequence and motif databases play a major role. We have already described how covariation analysis is used in modelling the secondary structure. Non Watson-Crick base pairs are very important for tertiary interactions, just as Watson-Crick base pairs dictate secondary structure. Thus, it is important to study covariation of bases that may form non Watson-Crick interactions. Nucleic acid bases have three edges that can participate in hydrogen bonding interactions with other bases (Figure 8.13). These are the Watson-Crick edge, the Hoogsteen edge and the shallow (or minor) groove edge. Tertiary interactions usually occur in the Hoogsteen edge and in the shallow groove edge, and the Watson-Crick edge is usually reserved for secondary structure interactions, but even this may participate in tertiary interactions. The identification of co-varying bases that are clearly not a part of the secondary structure interactions is the first step towards building a model three-dimensional structure for a RNA sequence. In one of the first efforts in 1990, when only the experimental structure of tRNA was known, this was the method followed to build a computer model of the three-dimensional architecture of Group I catalytic introns. Note that the model did not pertain just to one specific sequence, but to the whole lot of them used for sequence analysis. Thus the model was necessarily at low resolution, and was considered a heuristic tool, rather than an accurate representation of the structure. The sequence analysis in this case was carried out as follows. A total of 87 group I introns sequences were analysed by sequence alignment and correlation analysis, and divided into 4 main groups and 10 subgroups. One or two sequences fell outside this classification, as they were far longer and possessed extra loops. In such cases, only the relevant information was included. The multiple sequence alignment was used to identify covariances. These covariances were separated into two groups – those that clearly arose from secondary structure interactions, and those that pointed to tertiary interactions. The latter group was further divided into a set of core interactions that occurred with a high probability in the sequences, and peripheral interactions that occurred in some of sequences, but not all. These sets of covariations were considered constraints on the structure, with appropriate weights. For building the model on the computer, certain reasonable assumptions were made. It was assumed that all the secondary structure stems were all A-type RNA double helices with standard geometry. It was also assumed that the major interaction stabilising the tertiary structure was

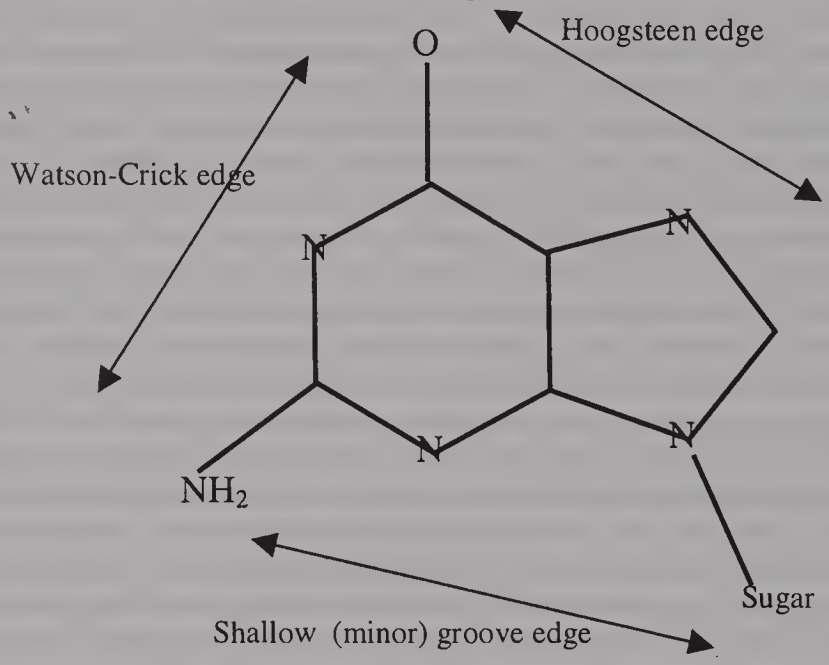


Figure 8.13. The three interacting faces of a purine base.

base stacking, followed by tertiary hydrogen bonds between the bases. The other assumptions were those that followed from comparative analyses. All the sequences should adopt the same general three-dimensional fold. And finally, the invariant (and covariant) residues should be involved in maintaining the tertiary structure or the catalytic activity (or both). Coordinates for the secondary structure helices were then obtained in the standard geometry (as described in Chapter 5). Fragments of loops and tertiary interactions were extracted as appropriate from the structure of tRNA or previous models. The fragments were then assembled and manipulated using interactive computer graphics. And these resulting structures were refined to ensure correct geometry, i.e. correct bond lengths, angles, etc. The method was iterative, with several rounds of manipulation and geometrical refinement. The final model obtained in shown in Figure 8.14, alongside the crystal structure obtained much later (in 1998). Overall, the model is similar to the experimental structure. In particular, the relative positions of some of the helices are correct. This method however is far from being an automatic method and requires a great deal of expertise and effort.

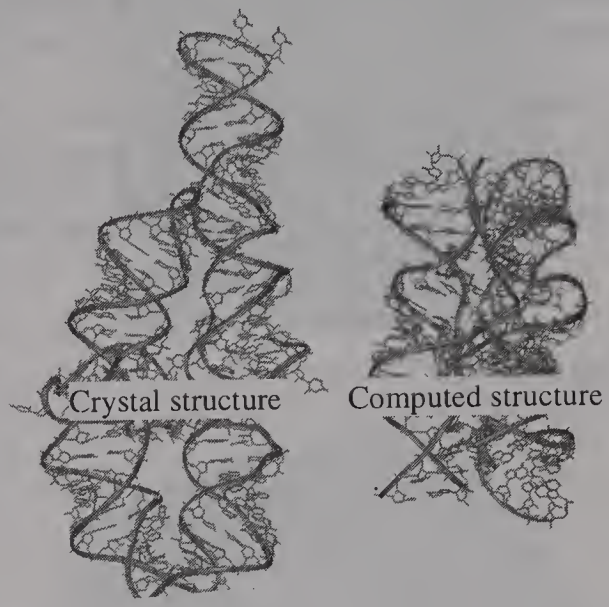


Figure 8.14 Comparison of the crystal structure (left) of a Group I intron with the computed model (right).

8.2 DNA Structural Polymorphism

About fifty years ago James Watson and Francis Crick published a paper in the journal Nature describing a model for the three-dimensional structure of DNA. Several experiments had already established the fact that DNA transfers hereditary information from one generation to another in all living organisms, be it bacteria or complex mammals. It was clear that knowledge of its structure was an essential first step towards understanding its function. The structure proposed by Watson and Crick fully lived up to this promise. The base-pairing scheme they proposed immediately made it clear that the two intertwined strands of the molecule acted as a template for each other, and knowledge of each was fully contained in the other. DNA replication was then easily explained. Work soon followed to show how DNA stored biological information as the one-dimensional sequence of nucleotide bases. This seminal discovery has transformed biology and is directly responsible for the new disciplines of Biotechnology and Bioinformatics, and therefore this book! The structure of DNA is easily described as 'double helix'. This however is not the entire story. There is more than one type of double helix, and these are described in the first part of this section. Also, the simple description of the structure hides some of the complex sequence-dependent modulations that occur in the helix. Some of the extreme modulations lead to structures that are not double helical at all, being instead three or four-stranded. Such structures are described in the latter part of this section. The reason for including these descriptions in a book given over to bioinformatics stems from the wide spread understanding among the scientists that apart from the sequence, the structure of DNA also plays a large role in storage and transfer of information. In other words the essentially one-dimensional representation of biological information in the DNA sequence is complemented by the three dimensional structure. A common

metaphor for the DNA molecule is that of a library. This metaphor implies a passive role for the molecule, with only the sequence being important. The present section however indicates that in fact DNA probably has a more active role, especially in its interactions with proteins in a sequence specific manner. Thus, DNA is not just the script of the 'Drama of Life', but a prominent actor.

8.2.1 Regular DNA helices, A, B and Z types

The first structures of DNA suggested were the right-handed models called the A and B type helices. The general double helical structure of these helices is well known and we will only give a brief description of it here, as follows. Dave Cates has noted that the chemical constitution of DNA is same as that of Coca Cola! Both consist of sugar, phosphates, and planar bases – A, T, G and C in the case of DNA, and caffeine in the case of Coca Cola. And both sets of compounds attain their full 'functional' forms only in the presence of huge quantities of water. However, the arrangement of these chemical groups is quite different in DNA compared to Coca Cola. DNA is a polymer, the monomer of which is a nucleotide. Each nucleotide consists of a phosphate group, a sugar moiety and a base. The base is attached to the sugar, which is in turn bonded to the phosphate at the other end. On polymerisation, the monomers bond together by phosphodiester bonds, each pair of such bonds connecting two adjacent sugar molecules through the phosphate group (Figure 8.15). The bases attached to the sugar on the other side decide the 'sequence' of this dinucleotide. A large number (up to 10^9) of such connections form one of the two polynucleotide chains that make up DNA. The two chains wrap around each other with the sugar-phosphate backbone on the outside and the bases on the inside (Figure 8.16). The bases that face each other on the inside of the helix form complementary base pairs, with adenine pairing with thymine, and guanine pairing with cytosine. The base pairs have nearly the same overall shapes, especially if we ignore the exocyclic atoms. This uniformity makes it possible to arrange the base pairs as flat rungs of the DNA 'ladder', except that this ladder is twisted about its long axis. The phosphate-sugar sides of the ladder are arranged in two anti-parallel helices.

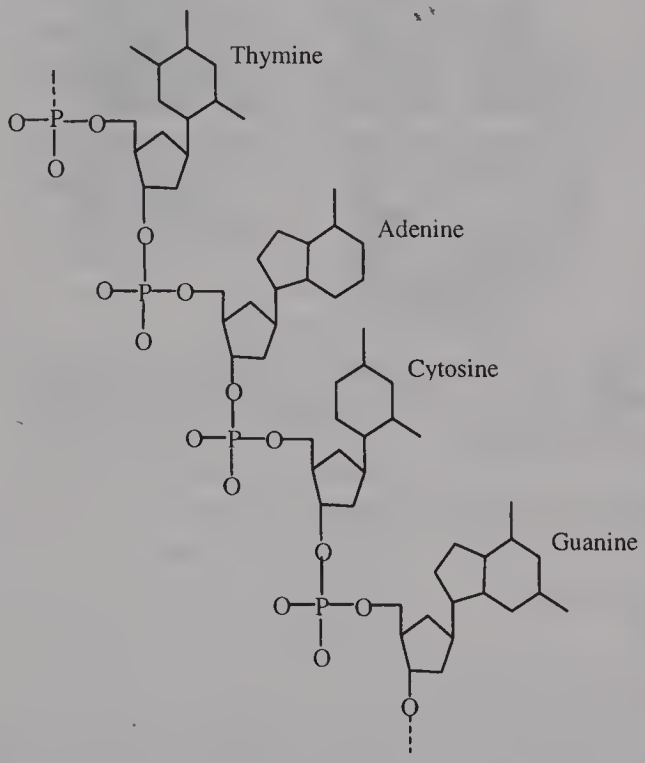


Figure 8.15. Schematic representation of a DNA tetranucleotide.

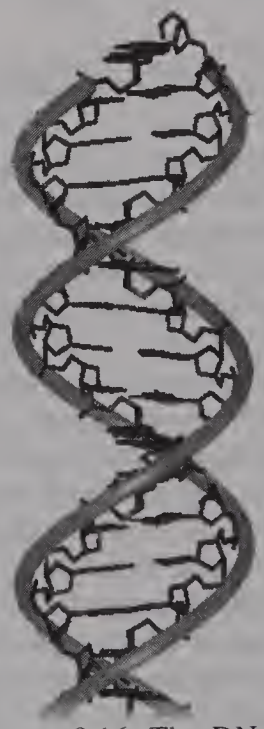


Figure 8.16. The DNA double-helix (B-type)

A rationalisation of this double helical structure, at least in gross terms, may be developed from the chemical nature of the polynucleotide chains. The first point to note is the hydrophobic nature of the bases. In aqueous solution, therefore, the arrangement of the bases would be such as to minimise the exposed surface area of these groups.

Clearly this is possible if, firstly, there are two strands with the bases arranged inwards, away from the solvent, and secondly if the planar bases are stacked on top of each, like a pile of coins, such that only their sides are open to the solvent. Given the highly polar nature of the sugar-phosphate backbone, additional forces to drive the structure towards the double helix come from the propensity for these groups to be solvated. However, the question remains, why a helix? Why not, say, a structure that resembles an untwisted ladder, rather than a twisted one? The answer to this is clear when we look at the distance between two adjoining phosphate groups, approximately 6 A, and the thickness of each base pair, approximately 3.5 A. If the ladder were to be untwisted, the distance between the base pairs would also be 6 A, and this would leave a space between them that may then be exposed to solvent, an unfavourable situation (Figure 8.17a). In order to bring the base pairs into contact, it is necessary to twist the ladder so that the projected distance between the phosphates is about the same as the thickness of the bases (Figure 8.17b). Thus the way

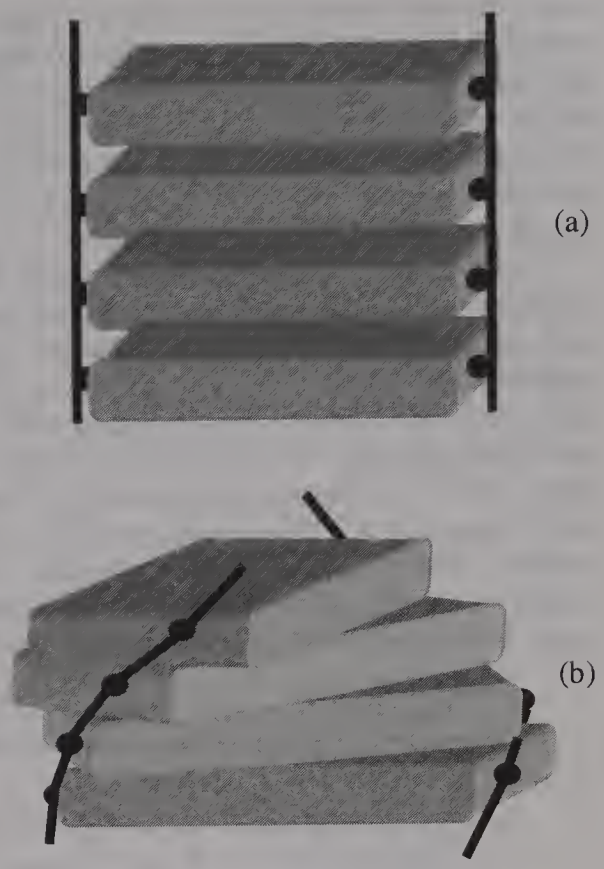


Figure 8.17. Why a helix?

in which the chemical nature of the various components of DNA decide the double helical structure of DNA is clear, given the fact of base pairing. The idea that the interactions between the base pairs contribute rather heavily to DNA structure is an important one and we will return to it later.

While the simple ladder model of DNA is able to explain why its structure should be helical, a more detailed understanding requires consideration of atomic details. Figure 8.18 shows that for such a model there are essentially nine independent parameters per nucleotide to describe the structure. These are the six backbone torsion angles, the two parameters that describe the conformation of the sugar ring and the glycosidic torsion angle that describes the orientation of the base with respect to the sugar ring. In the case of a regular helix, in which the structure does not change from one helical step to the next, we require only one set of nine parameters to describe the conformation of each nucleotide, together with

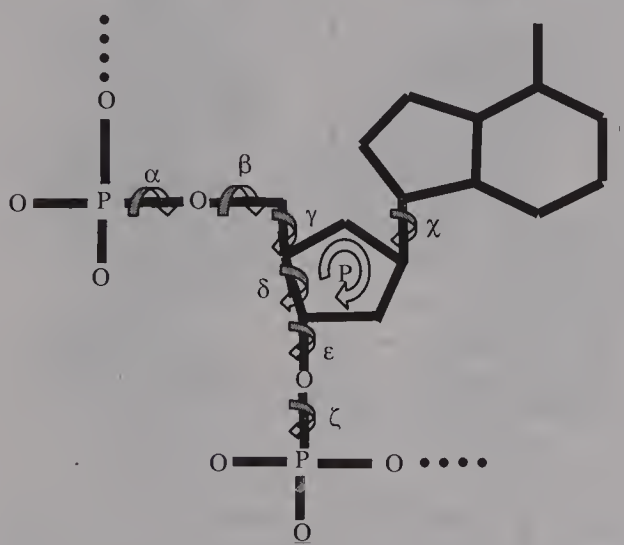


Figure 8.18. The nine degrees of freedom per nucleotide in a polynucleotide chain The nine degrees of freedom are the six backbone torsion angles, the glycosidic torsion between sugar and base, the phase angle of pseudorotation of the sugar ring, and the sugar ring amplitude of pucker. Of these, the first eight are indicated in the figure. (The amplitude of pucker is not shown).

a set of helical parameters that explains relation between one nucleotide and the next along the same strand. The nucleotides on the opposite, complementary strand are related always by the two-fold axes that exist in all regular double helices, perpendicular to the helix axis and in between two base pairs In Figure 8.19 the two-fold axis is the horizontal arrow perpendicular to the helix axis, which is indicated by the vertical arrow. Only one two-fold axis is indicated in the figure, though it exists between every base pair.

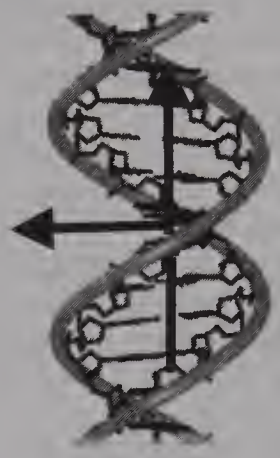


Figure 8.19

The A and B type helices described originally by Watson and Crick are regular helices. Their names recall the fact that they were first built as models to explain two different types of X-ray diffraction patterns obtained. The B type pattern was obtained from 'wet' DNA fibres (relative humidity approximately 80%) and the A type pattern was obtained from 'dry' DNA fibres (relative humidity approximately 60%). One set of parameters defined in Figure 8.18, describes each helix, respectively. These parameters are given in Table 8.1. These numbers, however, do not sufficiently help to see the differences in the shapes of the two types of helices. An additional set of parameters, called helical parameters, is therefore also given.

Table 8.1

	A	B	Z	
			pG	pC
α (°)	-50	-46	52	110
β (°)	172	-147	-153	-168
γ (°)	41	36	178	54
δ (°)	79	157	76	147
ε (°)	-146	155	-72	-103
ζ (°)	-78	-96	102	-91
χ (°) (Glycosidic torsion)	-154 (*anti*)	-98 (*anti*)	89 (*syn*)	-159 (*anti*)
Pseudo rotation phase angle (°)	162	18	162	18
Bases per turn	11	10	12	
Pitch (Å)	28.2	33.7	43.7	
Axial rise/residue (Å)	2.56	3.37	3.62	
Rotation/ residue (twist °)	32.7	36	-10 (CpG step)	-50 (GpC step)
Tilt (°)	20.0	-5.9	-5	
Sugar pucker	C3' *endo*	C2' *endo*	C3' *endo*	C2' *endo*

These describe the structure in terms of the overall geometry of helix. Thus the rise (distance between two base pairs as measured along the helix axis), the twist (the angle between one base pair and the next), the tilt (the angle made by the base plane with the helix axis), the width of the helix, and the size of the two grooves, all these are immediately recognisable from three-dimensional plastic or wooden models or diagrams of the two helical types. One of the most easily recognisable differences is the way

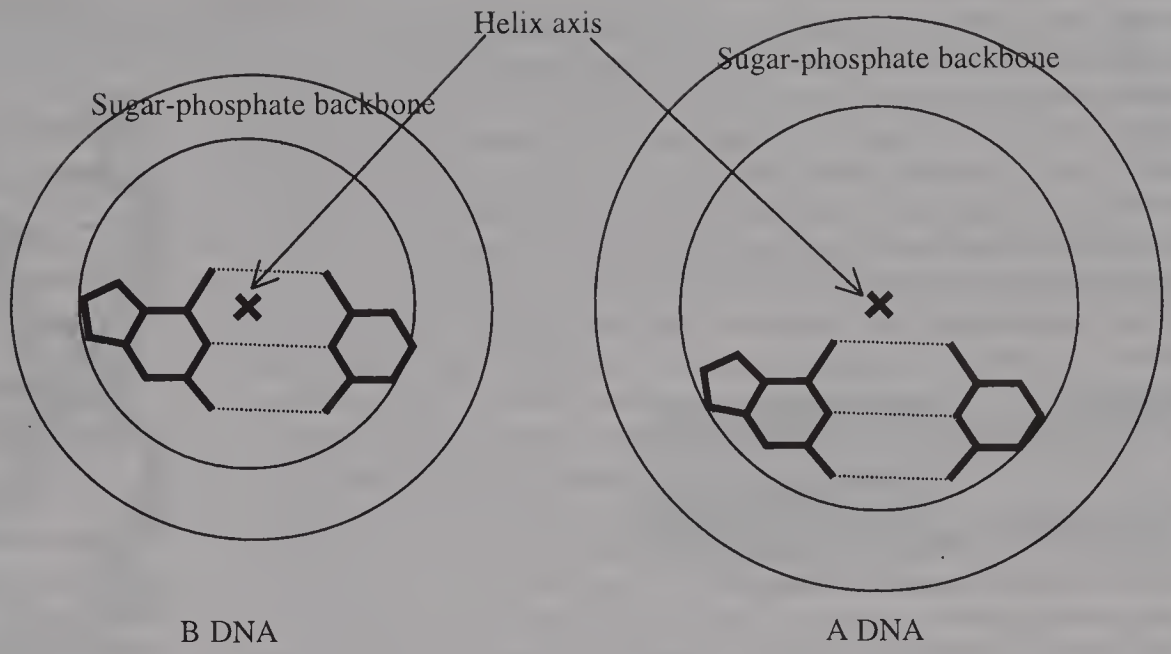


Figure 8.20 The difference between the A type helix and the B type helix. View down the helix axis.

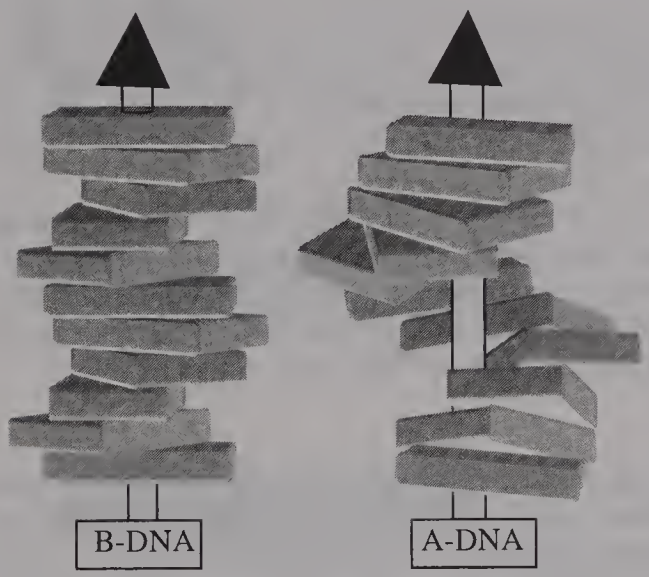


Figure 8.21 Another view of the two helical types.

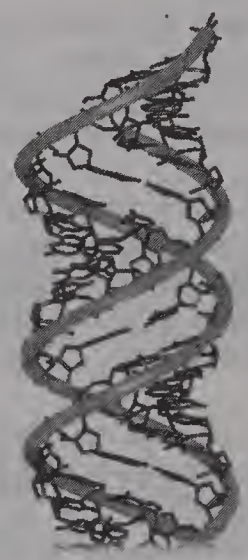


Figure 8.22. The A type helix.

in which the imaginary helical axis passes through the base pairs in B type DNA, but is away from the bases in A type DNA (Figure 8.20). Another way of appreciating this is to go back to our simple model of the double helix as a twisted ladder. As shown in Figure 8.21, in B type DNA the ladder twists about its own long axis, whereas in A type DNA the ladders wraps around a thick imaginary pole at the centre of which the helix axis runs Figure 8.22 gives another view of the A-type helix.

For all their differences, A and B type helices are fundamentally similar to each other, particularly in two aspects. Firstly, in both types of helices, the structure is uniform and regular, and does not vary from one base pair or base step to the next. Secondly, and more prominently, both are right-handed helices. Thus if the fingers of the right hand are used to show the curvature of the

backbone around the helix axis, the thumb points in the direction of progression of the strands. Only the right hand will serve the purpose for right-handed helices. In 1978, single crystal X-ray diffraction revealed the structure of a DNA fragment (specifically with the sequence CGCGCG) that differed from A and B type helices in both these aspects (besides others). This form of double helix was dubbed Z DNA (Figure 8.23). Z DNA is left-handed. In other words, we have to use the fingers of the left hand to indicate the relation between the curvature of the backbone and its forward movement. Also, the structure is not uniform as we proceed along the sequence, but alternates between one set of values for the twist, the sugar conformation, the backbone angles and the glycosidic angles, and another set (Table 8.1). This alternation is correlated with the sequence. Nucleotides with the base guanine have one set of values, and nucleotides with the base cytosine have another set of values. This was considered the one of first examples of sequence specific structure of DNA. Other studies indicated that Z DNA occurred, or could occur, when the following conditions were met. The sequence should be of the alternating pyrimidine-purine type; there should be a stretch of at least four CGCG bases in the sequence to nucleate the helix; and the ionic strength of the DNA solution should be very high. Besides these, there are other features that assist in forming and maintaining the Z type structure, including methylation of the cytosine bases, the presence of cobalt hexamine chloride, etc.

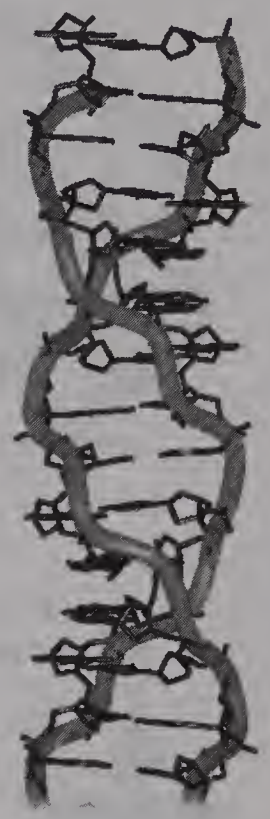


Figure 8.23 Z type DNA

Besides these three clearly defined helical types there are a few other helical models built to explain X-ray fibre diffraction and other low-resolution biophysical experiments. Most of these models are right handed double helices and are dubbed with other letters of the English alphabet, such as C DNA, D DNA, H DNA, etc. The exact biological relevance of these different helical types is still unclear. There is a great deal of speculation and some experimental evidence that structural types could play a part in gene regulation. The transition from right handed to left handed helix is thought to occur when torsional stress in the helix needs to be relieved, such as when the helix is supercoiled. Helical types are thought also to be important for DNA binding proteins to recognise specific sequences. This aspect will be explored further in the next section.

8.2.2 Sequence specific DNA structure

We saw in the previous section that one of the major ways in which Z DNA differs from A and B DNA is that the former is not a completely regular helix, while the latter two are. In Z DNA, each nucleotide has a slightly different structure. The sugar pucker, the glycosidic torsion angle, and the backbone torsion angles for cytidine[24] are all different from the values in guanosine. Some of the overall helical parameters also vary from base step to base step. These include the twist, which alternates between -10 and -50, and the groove widths, which are alternatively narrow and wide. (Other helical parameters remain the same at each base step, for instance the rise or the distance along the helix axis between one base pair and the next). A and B type helices as described above are completely regular and both the nucleotide conformational parameters and the helical parameters do not show any change from one base pair to the next, no matter what the sequence.

---

[24] Cytidine is the nucleoside formed by the condensation of the cytosine base with the sugar moiety. Likewise guanosine is the combination of guanine and the sugar group. The addition of a phophate group to each yields a nucleotide.

The discovery of sequence-specific features in DNA helices was made when the structure was determined from single crystals. DNA fibres are ordered only in one or two dimensions, and their X-ray diffraction patterns contain only information averaged over billions of base pairs. Consequently, detailed information on possible sequence-dependent effects had to wait for chemical technology to improve until it became possible to synthesize large quantities of pure, homogeneous oligonucleotides. This in turn made it possible to grow large single crystals, well-ordered in three dimensions, which could be used to obtained detailed structural information at the atomic level. Among the first such structures determined was that of the hexanucleotide CGCGCG, which, as described above, very surprisingly was a left handed helix with sequence dependent features. Other sequences subsequently solved by single crystal X-ray crystallography showed that such sequence-dependent microheterogeneity was a feature not only of Z DNA but also of the other helical types. Some oligonucleotides crystallised as recognisable B type helices and others as A type helices. Despite a clear identification as one or the other helical type, at each base step the detailed structure of the molecule varied. The description of the molecule could not now be made in terms of one set of helical parameters, but required the specification of a set of parameters for each base step and base pair. Moreover, certain other features were observed that were not noticeable in the average structures obtained from fibre diffraction. A clear example of this is the so-called 'propeller twist' in a base pair (Figure 8.24). This and other similar features showed sequence-dependent variation from base pair to base pair. A new set of parameters was therefore devised to describe DNA structure, called the set of 'base-centred' parameters (as opposed to the set of 'backbone-centred' parameters defined earlier). Figure 8.25 illustrates some of these parameters.

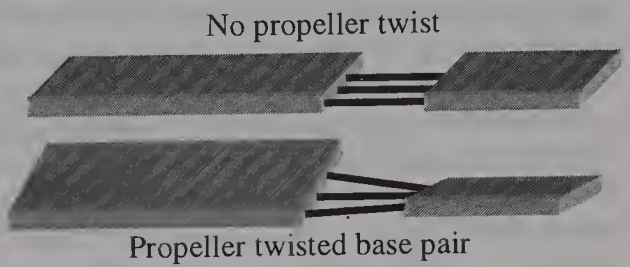


Figure 8.24 Propeller twist in a Watson-Crick base pair.

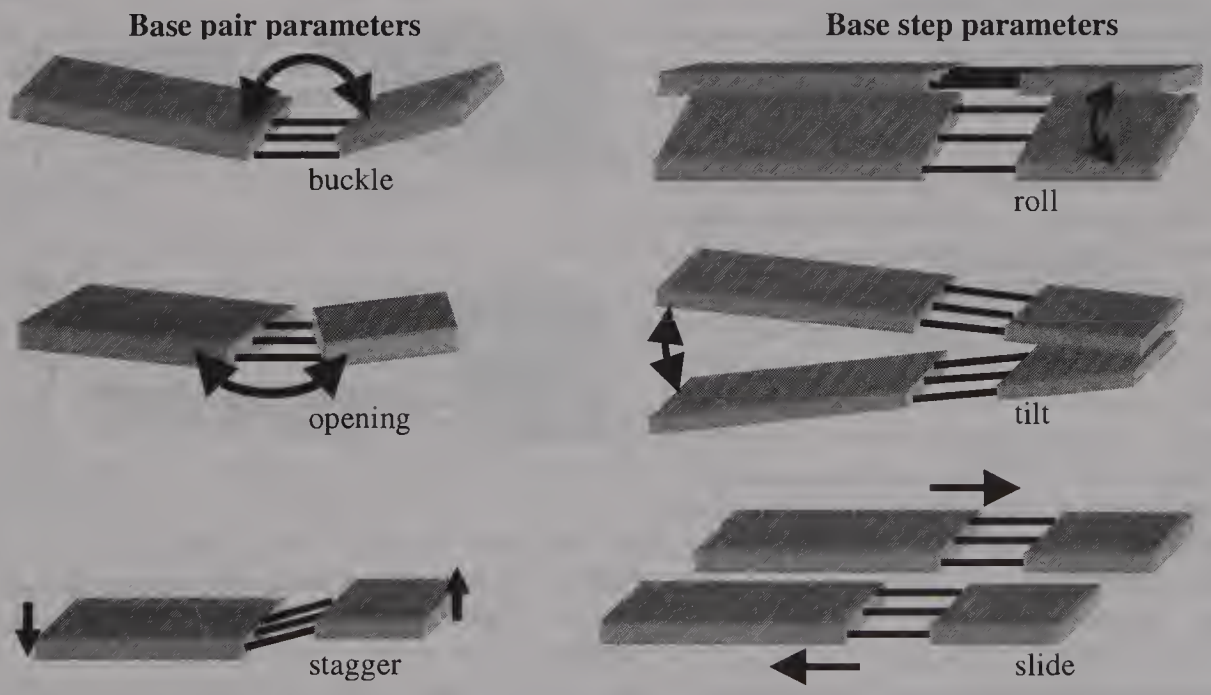


Figure 8.25 Some of the base-centred parameters according to the Cambridge convention.

While the single-crystal X-ray crystallography experiments clearly established the existence of sequence-dependent structure, the exact relation between sequence and structure remains unknown. In a sense, this is similar to the protein structure prediction problem. We know that a particular sequence (probably) leads to a particular type of helix, and a particular microstructure. However, given a sequence we do not yet possess any algorithm or indeed any rationale to build the correct structure. There are some pointers, and these come from analysis of the sequence dependent effects in the interactions of bases and base pairs with each other along the helix. Some examples of these effects are described in the following.

Consider 'propeller twist'. To explain its occurrence, we may build a structural argument that also takes into consideration the chemical and stereochemical properties of the bases. We start with the highly hydrophobic nature of the planar base moieties and the consequent need to expose as little as possible of their surface to the aqueous environment. The best way this can be accomplished is to stack the bases (or base pairs) on top of one another like a stack of cards. The thickness of the bases is about 3.4 A, and therefore the distance along the stack from one base pair to the next should also be 3.4 A. However, the sugar phosphate backbone is so constituted that the distance between one phosphorous atom and the next is about 6 A. Thus, a straight, parallel, non-helical stack of the base pairs, as illustrated in Figure 8.17a, leads to 'holes' between consecutive base pairs, which the solvent molecules could access. Energetically therefore it would be more favourable if consecutive base pairs were either displaced with respect to each other along their short axis, or if the stack was twisted into a helix (Figure 8.17b). Either of these arrangements would satisfy the contradictory requirements that the distance between the base pairs be 3.4 A while the distance between the phosphates be 6 A. For long-range stability, clearly, the helical arrangement is better and this is the one chosen by nature. This solution to the stacking problem however opens up another difficulty, for, in sliding the adjacent base pairs with respect to each other, some part of the bases are now exposed to the solvent. Now if the bases are 'rolled' with respect to the vertical axis, the exposed surface area is once again minimised. This is a satisfactory solution if the first displacement is along the short axis. However, if the displacement is a helical twist, as is the case in DNA, then the best ways to minimise the exposed surface is for each base of a base pair to 'roll' in opposite directions leading to propeller twist. Thus, we see that propeller twist follows as a structural consequence of the chemical requirement to minimize exposed hydrophobic surface area. With hindsight, one may even argue that without such propeller twisting the double helical structure of DNA would be an inefficient way of arranging a stack of flat hydrophobic chemical groups, and therefore not be possible! Of course, Watson and Crick were not deterred by such considerations. Their models do not have propeller twist.

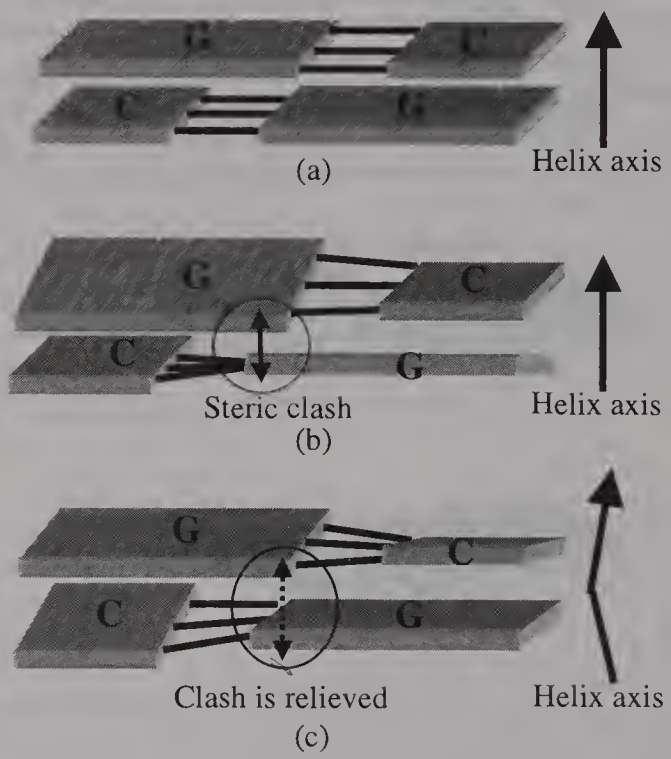


Figure 8.26. Propeller twisting leads to roll, and thence to sequence-specific bending of the DNA helix.

Propeller twisting, in turn, may lead to certain sequence-specific structural effects, as follows. Consider a dinucleotide sequence such as CG, i.e. a pyrimidine-purine sequence. Figure 8.26a shows an idealised view of the mini-duplex of this sequence, in which, for simplicity, the twist between consecutive base pairs has not been indicated. Neither is there any 'roll' angle between them. Also, at this stage, the propeller twisting that is normally present has not been introduced. In may be seen in the figure that the two purine bases overlap across the strands. If now we introduce a propeller twist in each base pair, the exocyclic oxygen or nitrogen atoms belonging to the guanine bases would experience steric clashes, as in Figure 8.26b. One way to relieve these clashes is to introduce a roll angle between the two base pairs (Figure 8.26c). If we further note that in so introducing a roll angle between two base pairs, we bend the DNA helix at that point, we may conclude that bends will probably occur more favourably at pyrimidine-purine sequences. Indeed experiments have shown that, where there are continuous runs of purines or pyrimidines, the DNA helix is stiff, and where the sequence alternates, the helix is more flexible.

Other sequence-specific effects are not so straightforward to analyse. In fact, even the above analysis becomes far more complicated when the duplex is twisted into its natural helical shape. However, the lesson we draw from the above is that the structure of DNA depends, at least to some extent, on its sequence. In analysing DNA sequences, therefore we need to consider these aspects, particularly in relation to the bending and supercoiling of the helix.

8.2.3 DNA bending

DNA helices persist as long straight rods for as much as 150 to 200 base pairs before they bend into largely curved shapes. However, gel migration experiments have shown that certain specific sequences of DNA may exist in a bent shape even when the total length is smaller than the so-called persistence length of 150 to 200 base pairs. Further experimentation revealed that the bent sequences were of the form $(XAAAAX)_n$, where A is adenine, and X represents any one of guanine, cytosine or thymine, and n is 3 or 4. These 'A tracts' were placed about 10 base pairs apart on the sequence and there were three or four of them placed in this manner at the point of bending. These discoveries lead to several proposals for sequence-specific bending of DNA. One of models suggested is the 'junction' model (Figure 8.27). In this model the helix bends by roll or tilt at the junctions between the A-tracts and the other parts of the sequence. We note that by introducing a roll between two consecutive base pairs, as illustrated in the previous paragraph, the helix axis is induced to bend towards either the major or minor grooves. By introducing a tilt angle between two base pairs, the helix bends in a perpendicular direction, stretching the phosphate-sugar strand on one side and compressing it on the other. The placement of the A-tracts 10 base pairs apart ensures that the total bending is in the same direction. If the phasing were, say, five base pairs apart, then, due to the helical twist, the bending due to the first tract would be cancelled out by the bending due to the second. The resulting DNA rod would not be bent but more or less straight.

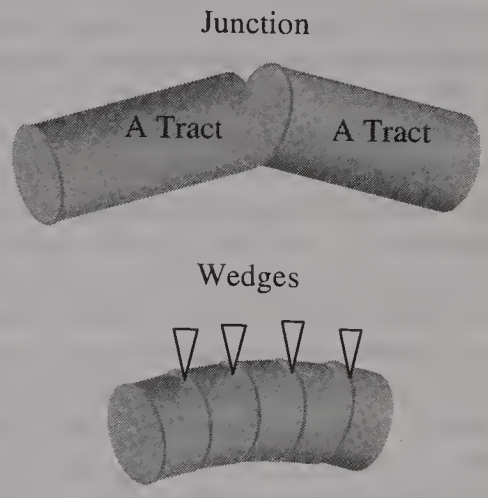


Figure 8.27

The second mechanism, the so-called 'wedge' model, proposed for DNA bending also takes advantage of the phasing of the A tracts. In this model the bending (by a roll or tilt angle) occurs at every AA base step and not just at the ends of the A tracts. Again, the phasing ensures that all the bending is in the same direction. Recent crystallographic experiments have indicated that in DNA

sequences with several consecutive adenine bases, the AT base pairs have a rather large propeller twist. This however does not lead to roll or tilt at the base step. On the contrary, steric clashes are accommodated in different ways, by say, slide or shift. Interestingly, such sequences have cross-strand hydrogen bonds. These bonds form in addition to the normal Watson Crick hydrogen bonds between an A.T base pair, and are formed between adenine and the next thymine along the sequence on the other strand (Figure 8.28). Such cross-strand bonds apparently serve to stiffen the A-tract, leading to the conclusion that the junction model may probably the more correct model to describe the bending of A-tract DNA.

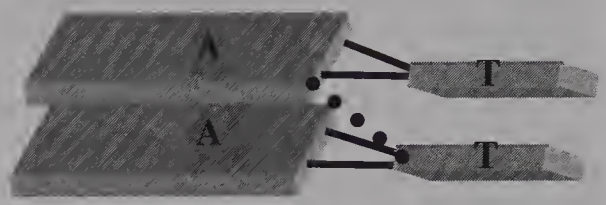


Figure 8.28 Cross strand hydrogen bonds (dotted line) in AA steps

Genome sequences have regions consisting of A tracts, and more generally purine tracts. In particular, the DNA sequences that wrap around histones to form the nucleosome core particle contain many A tracts with a phasing of 10 base pairs. This leads to the speculation that such sequences are chosen for their ability to bend properly, and that nature selects for DNA sequences also based upon the structural possibilities.

8.2.4 DNA supercoiling

The best and easiest way to appreciate DNA supercoiling is to take a rubber band in your hands and carry out the following very simple experiment. First, allow the rubber band to relax to its natural, roughly circular, untwisted shape and hold it loosely with both hands between the thumb and forefinger. Now introduce extra twists into the band by rolling it between the fingers of one hand while keeping its other side tightly clamped in the other hand. After a sufficient number of twists, you will notice that two twisted parts of the rubber band further twist around each other to form a 'supercoils' (Figure 8.29).

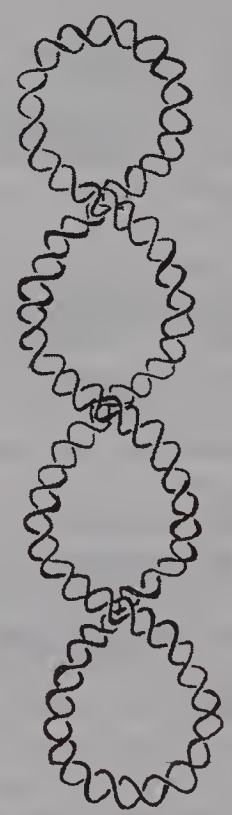


Figure 8.29 Supercoiled DNA

Supercoiling is an everyday occurrence noticeable in telephone cables and the like. It is a way of relieving the extra strain to which the strand is subjected when it is twisted more than the amount that is natural to it. A DNA double-helical strand has a natural twist of about one turn every 30 to 35 A, depending on the type of helix. This is of course also called the pitch of the helix. If the double helix is now twisted by a greater amount that this, it will develop a strain, which may be relieved by supercoiling. Such supercoils have been seen in electron micrographs of DNA subjected to torsional stress. More recently, DNA supercoiling is being studied using the techniques of single molecule biophysics.

Three measurable parameters of the twisted double helix are relevant to supercoiling. The first is the 'twist', which has been described earlier as the rotation angle along the helix axis made by one base pair with respect to the next. In the present context, twist is better derived from the pitch of the helix – the smaller the pitch, the larger the value of twist. The twist is measured as the number of times the two strands of the helix are wound around the helix axis. Thus, for B type DNA a short fragment of 10 base pairs would have a twist value of Tw = +1. Left-handed helices have negative values for twist. If there were no other feature that winds the two strands around each other, as it happens in B type DNA, the measured twist has the same valuc as the second of the three parameters, namely 'linking

number'. This is defined as the number of times the two strands of the helix wind around each other. In a short 10 base pair fragment of B type DNA the value of linking number is the same as the value of the twist, Lk = +1. The third parameter is called 'writhe', and this measures the number of times both strands of the helix together wind around the helix axis, without winding around themselves. In B type DNA this does not happen at all, and effectively the value of writhe Wr = 0. However, writhe has a non-zero value in other types of DNA, for example A type DNA. We recall our description of B type DNA as a thick ribbon twisted about its own axis, and A type DNA as a similar thick ribbon, but now wrapped around an imaginary central pole. (See Figure 8.21). In the latter case, the description illustrates the definition of writhe. Clearly, A type DNA has a relatively large value for writhe. There is a relationship between the three parameters, as follows.

$$Tw + Wr = Lk$$

For paired and twisted strands (such as the two strands of DNA) that are further not allowed to fray or unwrap at the ends by either circularisation or by clamping the ends, the linking number is constant. Any change in the twist, either over-winding or under-winding, is reflected as appropriate changes in the writhe, and is visible as supercoiling. Changes in writhe that make it more negative (or less positive) appear as left handed supercoils also called negative supercoiling. Likewise, changes in writhe towards more positive values appear as positive or right-handed supercoils. In other words, over-winding of the double helix leads to negative supercoiling and under-winding to positive supercoiling. The biological consequences of this are interesting. During the process of DNA replication, especially, the two strands of the DNA are unwound. If this happens without any mechanism to relax the torsional strain, supercoils would form, perhaps leading to a disruption of the replication process. (This process may be easily modelled by using a long length of a 'twisted-pair' electrical cable. If the two wires at one end of the cable are pulled apart, supercoils immediately develop in the other portions of the cable.) Topoisomerases are a special class of enzymes that exist particularly in order to take care of this problem, by snipping one or both strands of the DNA at the point of strain, turning one strand around the other, and ligating the broken strand back, having thus taken care of one extra twist and having reduced one unit of writhe. It has also been speculated that if some regions of the DNA duplex were to be in the left handed helical form, while other regions were right handed, then this combination may relieve the strain. This is because over-winding of a right-handed helix is the same as under-winding of a left handed one. The two effects would cancel out, leaving a duplex with no strain.

8.2.5 Unusual DNA structures

In this section we give a brief outline of some biologically relevant structures of DNA that are not double helices. Most of them occur only in specific DNA sequences, though the origin of such specificity is not clear.

- Figure 8.30 shows a model of 'slipped DNA', in which the amount of slippage is one base pair. Such a structure could arise if the DNA sequence were to suffer a point mutation in one strand, either an insertion, or a deletion. Note that the overall shape of the helix remains more or less unchanged, and the extra base at one end is simply extended out of the helix. Another way in which the extra base could be accommodated is shown at the other end of the helix. Here the extra base is stacked into the regular helix, leaving the opposite strand empty, in which there would have ordinarily been another base. Again, the shape

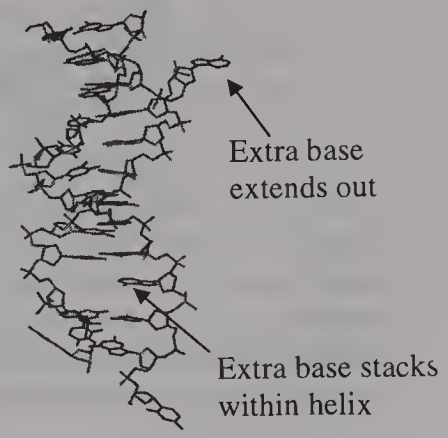


Figure 8.30 Slipped DNA (crystal structure)

of the helix remains almost undisturbed. In both cases, however, DNA repair enzymes would have sufficient means to recognise the problem and correct it by either excising the extra base or inserting its pair on the opposite strand. Certain sequences, such as triplet repeat sequences, are highly prone to slippage even in the absence of any mutations.

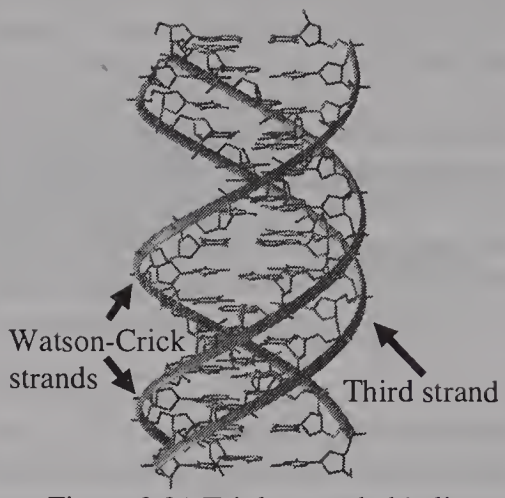
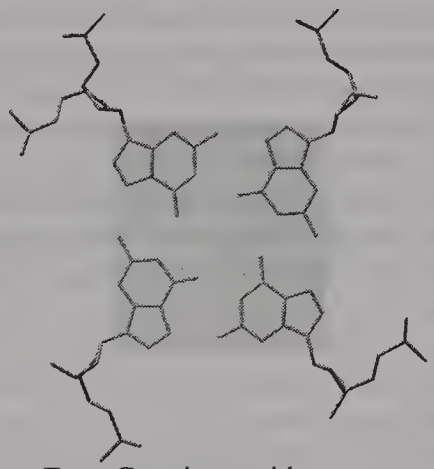
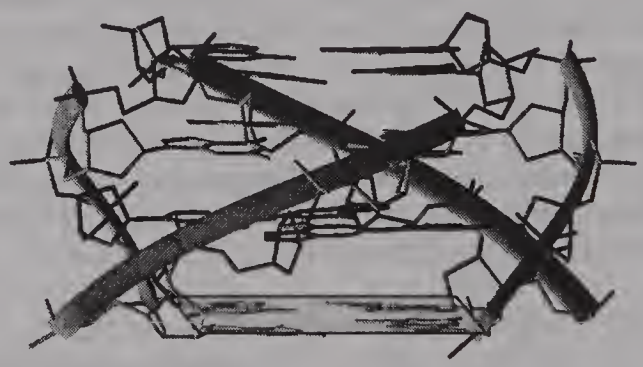


Figure 8.31 Triple stranded helix (model).

- Figure 8.31 shows the structure of triple-stranded DNA. Two of the strands form the usual Watson-Crick base paired B type double helix. The third strand fits into the major groove of this double helix, and its bases make the so-called Hoogsteen hydrogen bonds with the bases of the duplex. Such structures there are highly sequence specific. They occur especially in polypurine or polypyrimidine stretches in the sequence.
- Figure 8.32 shows an example a 'G-plate' structure. This is a four-stranded structure that occurs when four guanine bases come together to form a planar, cyclic, hydrogen-bonded motif called a G-plate. These plates stack on top of each other, with a slight twist to form the quadruplex. The guanine bases may come from different strands, or from one folded strand. Sequences that have G tracts, as found for example in telomeres[25], are well suited to adopt this structure.



Four Guanine residues come together to form a 'G-plate'



The G-plates are stacked to form a DNA tetraplex

Figure 8.32 The 'G-plate' DNA tetraplex (crystal structure)

- Figure 8.33 shows another four-stranded DNA structure. This is called an 'i' (for 'intercalated') motif. Again, as for G-plates, the base pairing is different from the usual Watson-Crick scheme. At slightly acidic pH about half the cytosine residues are protonated at the N3 position. They can therefore easily form cytosine-cytosine base pairs bringing two stands together. Two such duplexes then merge by intercalation of the base pairs to form the four-stranded i-motif structure. Such structures are formed by sequences that have C tracts. Like G tracts, such patterns obviously

[25] The outer ends of chromosomes are called telomeres

occur at telomeres, since G on one strand would imply C on the other. It has been speculated that both G-plate and i-motif structures play an important role in homologous recombination.

- The final figure in this section (Figure 8.34) shows the structure of a Holliday recombination junction. Here four self-complementary strands have chosen not to form two duplexes, as may have been expected, but instead have combined to form the junction. Such structures may be expected not only at recombination junctions, but also, for example, at cruciform structures that may be adopted by palindromic DNA sequences.

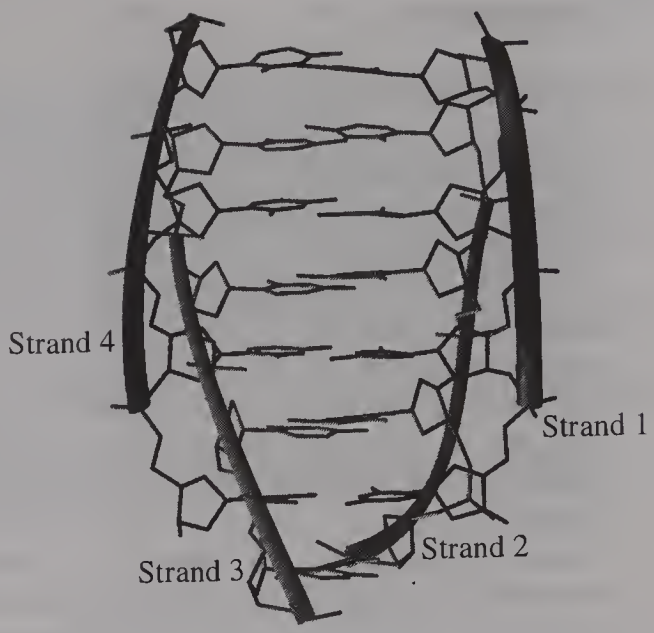


Figure 8.33 The intercalated DNA tetraplex, or 'i' motif structure (crystal structure).

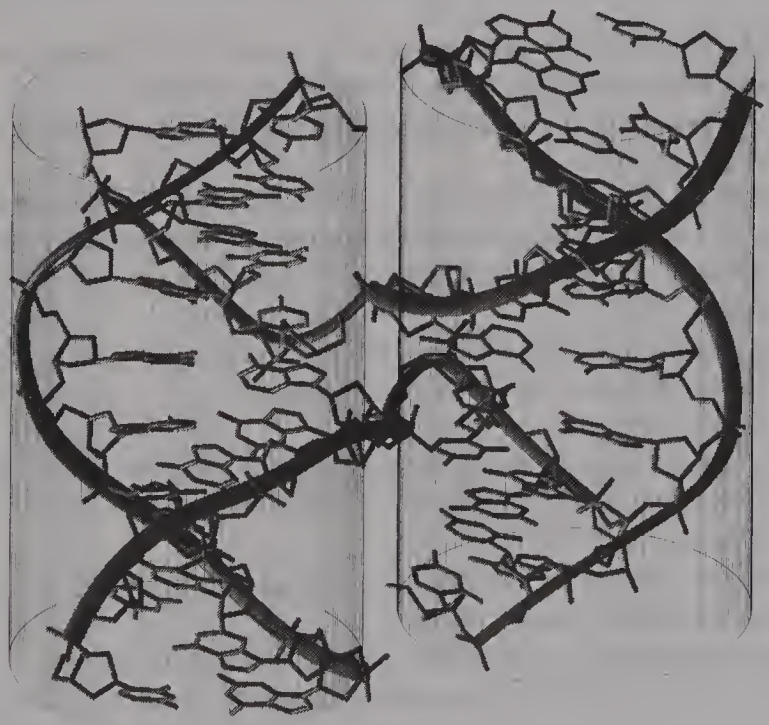


Figure 8.34 The Holliday junction or recombination junction (crystal structure). The wire-frame cylinders outline the two helices.

8.3 Summary

- RNA chains have secondary and tertiary structures that are dependent on their sequence.
- A variety of different techniques have been developed to analyse RNA sequences, and predict secondary structure, tertiary structure and function.
- The secondary structure of RNA specifies the pattern in which portions of the strand fold back and form local, hydrogen-bonded (base-paired), double-stranded structures.
- G-U base pairs also occur quite frequently in RNA molecules.
- The secondary structure is characterised by various motifs, which are important for the three-dimensional folding, including stems, hairpin loops, bulges, internal loops and junctions.
- The secondary structure of RNA is easily represented in two dimensions (on paper) in a variety of ways, such as 'stem-loop' diagrams, Nussinov circles and dot plots.
- Finding the best secondary structure for a sequence is an NP problem in terms of computational complexity.
- Dynamic programming can reduce the complexity substantially, though the search is not guaranteed to produce the very best secondary structure, only one of the very best.
- A dynamic programming algorithm has three main steps: constructing and initialisation of the matrix; filling up the elements of the matrix using a recursion algorithm; and finally constructing the trace back.
- In this algorithm, the correctness of a given secondary structure is evaluated simply by counting the number of base pairs it has. The best structure has the largest possible the number of total base pairs.
- The quickest and easiest way to incorporate other factors such as stacking interactions into the secondary structure prediction algorithm is to use simple energy rules.
- MFOLD is the program that incorporates all the above features and is widely used for RNA secondary structure prediction.
- Structure prediction by analysis of sequence covariance is another powerful method of secondary structure prediction, based on analysis of RNA sequences from multiple sources.
- Several formal methods of covariance analysis use visual or numerical calculations that lead to the secondary structure.
- Another method involves the calculation of a term called the mutual information content.
- A more recent implementation of covariance analysis makes use of Stochastic Context Free Grammars.
- The prediction of the three-dimensional structure of RNA molecules has not been the subject of widespread research to the extent that protein structure prediction has been investigated.
- The structure of DNA is a double helix.
- A rationalisation of the double helical structure, at least in gross terms, may be developed from the chemical nature of the polynucleotide chains.
- Complex sequence-dependent structural modulations occur in the helix.
- The first structures of DNA suggested were the right-handed double helices called A and B type.
- Based on a simple model of the double helix as a twisted ladder, we may say that in B type DNA the ladder twists about its own long axis, whereas in A type DNA the ladders wraps around a thick imaginary pole at the centre of which the helix axis runs.
- Z DNA is a left-handed double helix. Also, the structure is not uniform as we proceed along the sequence, but alternates between one set of values for the twist, the sugar conformation, the backbone angles and the glycosidic angles, and another set.

- The discovery of sequence-specific features in DNA helices was made when the structure was determined from single crystals.
- In the crystals, despite a clear identification of the oligonucleotides structure as one or the other helical type, at each base step the detailed structure of the molecule varied.
- Based of the experimental results, a new set of parameters was devised to describe DNA structure, called 'base-centred' parameters (as opposed to 'backbone-centred' parameters).
- Analyses of the sequence dependent effects in the interactions of bases and base pairs with each other along the helix leads to some understanding of the sequence dependent micro-structure.
- Experimental discoveries lead to several proposals for sequence-specific bending of DNA, such as the wedge model and the junction model.
- If a DNA double helix is twisted by a greater amount than what is natural to it, it will develop a strain, and this may be relieved by supercoiling.
- Triplexes, G-plates, i-motifs, junctions, etc. are some of the unusual DNA structures that are not double helices, and have been observed by experiment.

Further Reading

I have not included any primary material in this list. There are thus no references here to any of the research papers, Ph.D. theses or research reports that have been the source of much information when writing this book. However, many of the secondary materials, i.e. textbooks and monographs referred to below contain most of the required references. Also I strongly advise the reader to make full use of the Internet. Simple 'Google' searches using any of the words or phrases on which more information is desired will yield all the references required. The problem usually is one of plenty – it becomes difficult to know what to download and read. I hope this book and the references given here would help the serious student to deal with this problem.

General:

1. *Sequence Analysis Primer.* M. Gribskov and J. Devereaux (eds). (1991). Stockton Press, New York, USA
2. *Bioinformatics: A Practical Guide to the Analysis of Genes and Proteins.* A.D. Baxevanis and B.F. Francis Oulette (eds.) (1998). John Wiley and Sons, Inc. New York, USA.
3. *Bioinformatics: Sequences, Structures and Databanks.* D. Higgins and W. Taylor. (eds.) (2000). Oxford University Press, Oxford, UK.
4. *Post-Genome Informatics.* M. Kanehisa (2000). Oxford University Press, Oxford, UK.
5. *Bioinformatics: Methods and Protocols.* S. Misener and S.A. Kravetz (eds.) (2000). Humana Press, New Jersey, USA.
6. *Bioinformatics: Sequence and Genome Analysis.* D. Mount. (2000). Cold Spring Harbor Laboratory Press, Cold Spring Harbor, USA.
7. *Developing Bioinformatics Computer Skills.* C. Gibas and P Jambeck. (2001). Shroff Publishers and Distributors Pvt. Ltd. Mumbai, India.
8. *Introduction to Bioinformatics.* A.M. Lesk (2002). Oxford University Press, Oxford, UK.
9. *Bioinformatics for Beginners.* K. Mani and N. Vijayraj (2002). Kalaikathir Achchagam, Coimbatore, India.
10. *Biophysics.* Vasantha Pattabhi and N. Gautham (2002). Narosa Publishing House, New Delhi, India
11. *Bioinformatics for Dummies.* J.M. Claverie and C. Notredame (2003). Wiley Publishing, Inc., New York, USA.

12. *Bioinformatics Computing.* B. Bergeron (2003). Prentice-Hall of India Pvt. Ltd., New Delhi, India
13. *Bioinformatics: Genes, Proteins and Computers.* C.A. Orengo, D.T. Jones and J.M. Thornton (2003). BIOS Scientific Publishers, Ltd. Oxford, UK.
14. *Instant Notes: Bioinformatics.* D.R. Westhead and J. Howard Parish (2003). Viva Books Pvt. Ltd., New Delhi, India.
15. *Basic Bioinformatics.* S. Ignacimuthu, s.j., (2005). Narosa Publishing House, New Delhi, India

Chapter 1:

16. *Computers Today.* S.K. Basandra (1997). Galgotia Publishers, New Delhi
17. *Database System Concepts.* A Silberschatz, H.F. Korth and S. Sudarshan. (1997). The McGraw-Hill Companies Inc., New York, USA.
18. *Genes VII.* B. Lewin (2000). Oxford University Press, Oxford, U.K.
19. *Molecular Cell Biology.* H. Lodish, A. Berk, S.L. Zipursky, P. Mastudaira, D. Baltimore and J. Darnell (2000). W.H. Freeman and Company, New York, USA.

Chapter 2:

20. *Nucleic Acids Research* (January 2005) Volume 33, Number 1. Database Issue. (This issue is available online for free and open access at the following url http://nar.oupjournals.org/content/vol33/suppl_1/
21. *Introduction to Bioinformatics.* T.K. Attwood and D.J. Parry-Smith (1999). Addison Wesley Longman Limited, England.
22. *Bioinformatics Sequences, Structures and Databanks.* D. Higgins and W. Taylor (eds.) (2000). Oxford University Press, Oxford, UK.

Chapter 3:

23. *Algorithms on Strings, Trees and Sequences.* D. Gusfield (1997). Cambridge University Press, Cambridge, UK.
24. *Biological Sequence Analysis.* R. Durbin, S. Eddy, A. Krogh and G. Michison. (1998). Cambridge University Press, Cambridge, UK.
25. *Computational Molecular Biology.* P.A. Pevzner. (2000). The MIT press, Cambridge, USA.
26. *Bioinformatics.* P. Baldi and S. Brunak. (2003). Affiliated East-West Press, New Delhi, India

Chapter 4:

27. *Atlas of Protein Sequence and Structure.* M.O. Dayhoff (1978). National Biomedical Research Foundation, Maryland, USA.
28. *Molecular Evolution.* W-H Li (1997). Sinauer Associates Inc., Publishers. Massachusetts, USA.
29. *Biological Sequence Analysis.* R. Durbin, S. Eddy, A. Krogh and G. Michison. (1998). Cambridge University Press, Cambridge, UK.

Chapter 5:

30. *Biophysical Chemistry, Vol I, II and III.* C.R. Cantor and P.R. Schimmel (1980). W.H. Freeman and Company, New York, USA.
31. *Protein Structure: New Approaches to Disease and Therapy.* M. Perutz. (1992). W.H. Freeman and Company, New York, USA.
32. *Molecular Modelling: Principles and Applications.* A.R. Leach (2001). Prentice-Hall, Harlow, UK.
33. *Introduction to Macromolecular Crystallography.* A. McPherson (2003). Wiley-Liss, Inc. New Jersey, USA

Chapter 6:

34. *Biological Sequence Analysis.* R. Durbin, S. Eddy, A. Krogh and G. Michison. (1998). Cambridge University Press, Cambridge, UK.
35. *Pattern Discovery in Biomolecular Data.* J.T.L. Wang, B.A. Shapiro and D. Sasha. (eds.) (1999). Oxford University Press, Oxford, UK.
36. *Bioinformatics Sequences, Structures and Databanks.* D. Higgins and W. Taylor. (eds.) (2000). Oxford University Press, Oxford, UK.
37. *Bioinformatics.* P. Baldi and S. Brunak. (2003). Affiliated East-West Press, New Delhi, India

Chapter 7:

38. *Principles of Protein Structure.* G. Schultz and R.H. Schirmer. (1984). Springer Verlag, New York, USA.
39. *The Protein Folding Problem and Tertiary Structure Prediction.* K.M. Merz, Jr. and S.M. Le Grand. (eds) (1994). Birkhauser Boston, Boston, USA.
40. *Protein Structure Prediction: Methods and Protocols.* D.M. Webster. (ed). (2000). Humana Press Inc., New Jersey, USA.
41. *Introduction to Protein Architecture.* A.M. Lesk. (2001). Oxford University Press, Oxford, UK.

Chapter 8:

42. *Principles of Nucleic Acid Structure.* W. Saenger. (1984). Springer Verlag, New York, USA.
43. *DNA Structure and Function.* R.R. Sinden. (1994). Academic Press, San Diego, USA.
44. *Understanding DNA: The Molecule and How It Works.* C.R. Calladine and H.R. Drew. (1997). Academic Press, San Diego, USA.

Index

3D-COFFEE, 99
3DNA, 164

Aaindex, 32
ab initio, 134, 192, 198, 200, 203, 207
accessible surface area, 155
ACeDB, 40
Active Server Pages, 32
ALFRED, 31
Allelic Variant Database, 31
alpha (α) helix, 180
AMBER force field, 198
aminoacyl-tRNA synthetases, 11
analogs, 58
Anfinsen, 181
Anifinsen's experiment, 12
anomalous scattering technique, 146
antisense strand, 10
Application Service Providers, 22
artificial neural networks, 128, 141, 187, 206
Asthma Gene Database, 31
ATP, 4, 37, 44
ATP synthase, 3

back-propagation, 137, 189
Baum-Welch algorithm, 130
Bayesian statistics, 126, 197
beta (β) strand, 181
BIND, 29
Biosym suite, 16
BLAST, 16, 21, 23, 42, 46, 55, 68, 83, 87, 90, 92, 193
blastn, 84
blastp, 84
blastx, 85
BLOCKS, 45, 53, 107
BLOSUM matrices, 24, 101, 108, 121
Bobscript, 157
Boltzmann distribution, 148
bond angles, 158
bond lengths, 158
branch and bound, 118, 120, 194, 201

C score, 128
C value paradox, 12
C++, 16
Carillo-Lipman algorithm, 96, 120
CASP, 183, 202, 207
CATH, 51, 54, 193
Central Dogma, 5, 9, 25
channel proteins, 4
chaperone proteins, 12
charge coupled devices, 146
chi, 2
chloroplast, 3
Chou-Fasman method, 184
chromosomes, 9, 236
CLUSTAL, 97, 120, 128, 194
cluster computers, 16
clustering, 175
CluSTr, 43

COGS, 43
combinatorial explosion, 57, 89, 172
comparative protein modelling, 193
computational linguistics, 1, 187
Compute pI/MW, 132
computer graphics, 146
conformational propensity, 184
conformational space annealing, 201
ConQuest, 51
contact potential, 197
contrast, 150
convex hull, 162
coordinate transformations, 157
cosmic radiation, 5
COSY, 149
crystal, 6, 145, 147, 151, 153, 161, 195, 231, 232
CSD, 46, 50
CURVES, 164
CUTG, 41
cytosol, 4

DALI, 169, 172
Darwinian principles of evolution, 2
Database Management Systems, 17
dbSNP, 30
Delaunay triangulation, 162
depth cueing, 153, 178
detector, 145
deterministic patterns, 123
diffraction, 7, 145, 151
DIP, 30
direct methods, 146
discrete solid state devices, 14
distance matrix, 112, 120, 122, 167, 177
distances geometry, 149
disulphide bonds, 48, 194
DNA fingerprinting, 22
DNA supercoiling, 234
dot matrix, 62
double helix, 6, 9, 225
double resonance, 148
drug design, 195
drugs, 46, 195
DSC, 190
DSSP, 53
Dynamic programming, 68, 89, 92, 95, 172, 197, 211, 217, 238

electron diffraction, 151
EMBL, 33, 38, 40, 53
EMBOSS, 16
endocytosis, 4
endoplasmic reticulum, 3
endosymbionts, 4
energy landscape, 199
enhancers, 7, 10, 23, 133, 140
ENZYME database, 29
EPD, 41
eubacteria, 4
Eulerian angles, 165
E-values, 84, 87
evolutionary algorithms, 201
excitation, 148
ExInt, 42
exons, 10, 12, 23, 36, 42, 133, 137
Expressed Sequence Tags, 137
extreme value distribution, 62

FASTA, 23, 41, 55, 68, 83, 90, 193
FASTF, 88
FASTS, 88
FASTX, 88
FASTY, 88
feed-forward networks, 187
Feng-Dolittle algorithm, 97, 120
F_{hkl}, 146
FID, 148
Fitch-Margoliash clustering, 98
FlyBase, 40, 53
fold recognition, 192, 195, 203
FORTRAN, 16
Fourier analysis, 151
Fourier transform, 138, 146, 151
free induction decay, 148
FREEHELIX, 164
FRET, 30
FSSP, 52
FTNMR, 148

gap, 108, 121
Gap penalties, 121
GCG package, 16
GDB, 32
GenBank, 20, 26, 33, 36, 40, 48, 53, 137
gene, 10, 12
Gene discovery, 133, 135, 139, 142

Gene Expression Omnibus, 27
GeneMark, 135, 142
GeneScan, 137, 142
genetic code, 10, 101, 133, 140, 209
genetic maps, 31
genome, 12, 21
genotype, 5
geometrical optics, 151
Gibbs sampling, 177
GOBASE, 41
Golgi apparatus, 3
GOR I algorithm, 186
GOR II algorithm, 187
GOR III algorithm, 187
GPCRDB, 42
G-plate, 236
GRAIL, 136, 142
graph theory, 176
graphical user interface, 16
Grasp, 157

hairpin-bulge, 210
hardware z buffer, 153
hash table, 66, 68, 89, 176
heavy atom, 146
heterologs, 58
heuristics, 83
HGBASE, 31
hidden Markov models, 45, 129, 141
hidden surface removal, 152, 178
hierarchical alignment, 97
hierarchical model, 17
highlighting, 152, 178
High-Scoring segment Pair, 84
HNN, 190
Holliday recombination junction, 237
holy grail, 191
homologous recombination, 9, 57, 237
homologs, 58
homology modelling, 52, 58, 191, 195, 198, 204, 207
Hoogsteen edge, 224
HSSP, 52
HUGE, 42
hydrogen bonds, 48, 53, 158
hydrophilic heads, 3
hydrophobic collapse, 204
hydrophobic interactions, 150
hydrophobic tails, 3

Image reconstruction, 150
i-motif, 236
InBase, 42
inclination, 163
indels, 76
INE, 32
information theory, 125
Insight II, 157
integrated circuits, 14
InterPro, 46
introns, 10, 12, 23, 36, 42, 133
inverse folding, 195
isoaccepting tRNAs, 140
isoelectric point, 132

JPRED, 190
Jukes Cantor model, 119

KEGG, 28
Kimura model of evolution, 99
KinMutBase, 31
kissing hairpins, 210
knowledge based modelling, 193

Lamarck's theory, 5
Langevin dynamics, 204
lattice, 145, 151
lattice folding, 204
Laurence Bragg, 144
least squares, 146
Levinthal paradox, 202, 207
lysosomes, 4

machine learning, 1
magnetic lenses, 150
magnetic resonance imaging, 147
magnetisation, 148
Markov model, 102, 129, 130, 135, 140
matrix, 149
maximal segment pairs, 83
maximum likelihood, 118
maximum parsimony, 115, 122
mean field, 194
MEDLINE, 29
megablast, 85

metabolic pathways, 1, 28
metabolism, 2
MFOLD, 220, 238
MHCPep, 43
microarrays, 27
microprocessor, 15
Miller indices, 146
MIPS, 36
mitochondria, 4
molecular biology, 2
molecular dynamics, 150, 200
molecular replacement technique, 146
Molscript, 157
Monte Carlo simulations, 201
Moore's law, 15
motifs, 23, 37, 42, 44, 51, 53, 91, 123, 126, 128, 131, 133, 142, 181, 210, 238
MOWSE, 132
MULPRED, 190
multicellular organism, 2
multi-dimensional NMR, 148
multiple isomorphous replacement, 146
multiple sequence alignment, 91
mutation data matrix, 104
mutual information content, 221, 238

n of m filter, 64
natural language processing, 1
NDB, 51, 54
Needleman-Wunsch algorithm, 69, 76, 95, 173
neighbour joining, 114, 122
network model, 17, 18
Network Protein Server, 190
NMR, 147
NNPredict, 189
NNSSP, 189, 190, 206
NOESY, 149
NP-complete, 92
NTDB, 32
nuclear magnetic resonance, 147
nuclear membrane, 4
nuclear Overhauser effect, 148
nucleus, 3, 147, 158
NUPARM, 164
Nussinov algorithm, 25, 211, 217, 219
Nussinov circle, 210

Occam's Razor, 116
Okazaki fragments, 9
OMIM, 30, 32
Omniome, 40
Open Database Connectivity, 32
opening, 163
operating system, 16
Operational Taxonomic Units, 109
operons, 7, 29
orthologs, 59

PALI, 51
PAM matrices, 36, 101, 103, 105, 107, 121
paralogs, 59
PARST, 164
peptaibol database, 43
peptide bond, 12, 161
Peroxisomes, 4
perspective, 13
Pfam, 42, 45, 53, 131, 198
phase problem, 146
PHD, 187, 189, 206
phenotype, 5
phylogenetic tree, 98, 109, 119, 122, 139, 222
phylogeny, 109, 121
PIR-PSD, 36, 40
PMD, 31
polymerisation, 226
positive predictive value, 127
POSSUM, 177
potential, 150
prana, 2
prebiotic systems, 2
PREDATOR, 189, 206
primary structure of the protein, 161
printed circuit board, 15
PRINTS, 44, 53
probabilistic patterns, 124
PROCHECK, 164, 194
ProDom, 46
production rule, 222
profile, 87
prokaryotic, 3
promoter, 7, 10, 23, 36, 41, 53, 92, 133, 137, 139
propeller, 163, 232
PROPSEARCH, 132
PROSITE, 44, 53, 107, 121, 123, 131, 141
protein folding, 10, 12, 182, 195, 198, 200, 223
PROTOMAT, 107

pseudogenes, 13
pseudoknots, 210
pseudorotation, 159
PSI-BLAST, 87
PubMed, 29
p-value, 177

quasi-periodic crystal, 6, 208
quaternion, 167

R factor, 146
radioactivity, 5
Ramachandran map, 144, 161, 178
RASMOL, 157
ray tracing, 152
REBASE, 32
reductionist programme, 2
regular expressions, 44, 53, 93, 141
RegulonDB, 29
relational database management system, 18
relational model, 17, 18
replication, 5, 9, 108
resolution, 146, 150
resonance frequency, 148
reverse transcriptase, 5
ribosomal RNA, 9
ribosome, 10
ribosomes, 3
rise, 58, 108, 131, 133, 138, 149, 163
RNA, 11, 65
RNA polymerase, 10, 41
rocking, 153
roll, 163
rooted trees, 110
ROSETTA, 201
RPS-BLAST, 87

S score, 128
salt bridges, 48, 101
SARF2, 177
scanning electron microscope, 150
SCOP, 51, 54
SCOR, 223
scoring function, 196, 200, 203, 207
scoring schemes, 76
SEM, 150
semi-empirical, 150
semi-empirical potential, 197
sense strand, 10
sequence logos, 124, 128
SETOR, 157
shading, 152, 178
shadowing, 153, 178
shallow groove edge, 224
shear, 163
signal transducers, 3
SignalP, 128
simulated annealing, 175, 200
slipped DNA, 235
Smith-Waterman algorithm, 81, 83, 90, 92, 173
SOPMA, 190
SP score, 95
spectrum, 148
spin state, 147
spliceosomes, 10
splicing, 10
SQL, 32
SSAP, 172
stagger, 163
Stanford Microarray Database, 28
stereo projections, 154
stereochemistry, 146
Stochastic Context Free Grammars, 222, 238
stretch, 163
Structural Distance Metric, 52
structure amplitude, 146
structure of NaCl, 144
sub-graph isomorphism, 177
substitution matrices, 24, 45, 66, 91, 100, 107, 120, 202
surfaces, 152, 154, 163, 178
SWISS-2DPAGE, 32
SWISS-PROT, 36, 40, 43, 53

TATA box, 10, 93
tblastn, 85
tblastx, 85
T-COFFEE, 99
telomerase, 9
telomeres, 9, 236
TEM, 150
TFASTF, 88
TFASTS, 88
TFASTX, 88
TFASTY, 88

thermal ellipsoid plots, 155
threading, 195, 207
tilt, 163
tip, 163
torsion angles, 158
transcription, 9, 10, 41, 44
transfer RNA, 11
transition, 148
translocation factors, 12
transmission electron microscope, 150
TrEMBL, 40
triple-stranded, 236
TRIPOS, 157
twist, 163, 232

UM-BBD, 29
UNRES force field, 199
UPGMA, 112, 114, 122

van der Waals interactions, 158
Vector Alignment Search Tool, 177
Vector NTI suite, 16
vis vitalis, 2
Viterbi algorithm, 130
VLSI, 15
von Neumann model, 15

Watson-Crick edge, 224
wedge model, 233
WormBase, 28, 40

xenologs, 59
X-ray crystallography, 144
X-rays, 6, 145, 150

z value, 62, 153, 197
ZPRED, 190
Zuker algorithm, 217, 219



http://nihlibrary.nih.gov

10 Center Drive
Bethesda, MD 20892-1150
301-496-1080

NIH LIBRARY
3 1496 00993 7692

Bioinformatics
Databases and Algorithms

N Gautham

Bioinformatics: Databases and Algorithms has two features that distinguish it from others in this genre. First, it seeks to explain some of the basic algorithms in bioinformatics to readers who have little exposure to computer science and mathematics. Second, it deals with structural bioinformatics in greater depth than most other books. The emphasis of this book is on algorithms, though databases have not been entirely neglected, and they have a whole chapter to themselves. Also much attention is given to structural bioinformatics.

In the hierarchy of textbooks, it falls between an elementary introduction to the subject, and an advanced monograph on the topic, and serves as a bridge between the two.

Key Features:

- Describes methods of biomolecular sequence and structure analyses, of interest to students and practitioners of bioinformatics, both in the corporate and academic sectors
- Explains many of the crucial algorithms, making them intelligible to those without any background in computer science
- Attention to structural bioinformatics

OCT 2 2007
NOV 17 2008
SEP -1 2008
JUL -7 2008
FEB 21 2008

ISBN 1-84265-300-8
9 781842 653005

Alpha Science International Ltd.